



# Wing-M130 Processor Technical Reference Manual

**Version r1p0, 2024.05.31**

# Contents

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| Revision History                                                                 | 1  |
| 1. Overview                                                                      | 2  |
| 1.1. Block Diagram                                                               | 2  |
| 1.2. Features                                                                    | 2  |
| 1.2.1. Instruction Set                                                           | 2  |
| 1.2.2. Privilege Mode and Physical Memory Protection                             | 3  |
| 1.2.3. Memory Subsystem                                                          | 3  |
| 1.2.4. Miscellaneous                                                             | 3  |
| 1.3. Specification Compliance                                                    | 3  |
| 1.4. Configuration Options                                                       | 4  |
| 2. Wing-M130 Core                                                                | 6  |
| 2.1. Instruction Fetch Unit (IFU)                                                | 6  |
| 2.1.1. Features                                                                  | 6  |
| 2.2. Pipeline Control Unit                                                       | 6  |
| 2.2.1. Instruction Decode Unit (DEC)                                             | 6  |
| 2.2.2. Issue Unit                                                                | 6  |
| 2.2.3. Commit                                                                    | 6  |
| 2.3. Execution Unit (EXU)                                                        | 7  |
| 2.3.1. Features                                                                  | 7  |
| 2.4. Exceptions                                                                  | 7  |
| 2.4.1. Exceptions and the Priority                                               | 7  |
| 2.4.2. Enter Exception handler                                                   | 8  |
| 2.4.3. Return from an Exception                                                  | 9  |
| 2.5. CSR Registers                                                               | 10 |
| 2.5.1. CSR Address Mapping Conventions                                           | 10 |
| 2.5.2. Register Field Conventions                                                | 10 |
| 2.5.3. CSR Listing                                                               | 10 |
| 2.5.4. Vendor ID Register, <code>mvendorid</code>                                | 12 |
| 2.5.5. Architecture ID Register, <code>marchid</code>                            | 13 |
| 2.5.6. Implementation ID Register, <code>mimpid</code>                           | 14 |
| 2.5.7. Hardware thread ID Register, <code>mhartid</code>                         | 15 |
| 2.5.8. Pointer to Configuration Data Structure Register, <code>mconfigptr</code> | 16 |
| 2.5.9. Machine Status Register, <code>mstatus</code>                             | 17 |
| 2.5.10. ISA and Extensions Register, <code>misa</code>                           | 20 |
| 2.5.11. Machine Interrupt-Enable Register, <code>mie</code>                      | 22 |
| 2.5.12. Machine Trap-Handler Base Address Register, <code>mtvec</code>           | 24 |
| 2.5.13. Additional Machine Status Register, <code>mstatush</code>                | 25 |
| 2.5.14. Scratch Register for Machine Trap Handlers, <code>mscratch</code>        | 26 |
| 2.5.15. Machine Exception Program Counter Register, <code>mepc</code>            | 27 |
| 2.5.16. Machine Cause Register, <code>mcause</code>                              | 28 |
| 2.5.17. Machine Bad Address or Instruction Register, <code>mtval</code>          | 30 |
| 2.5.18. Machine Interrupt-pending Register, <code>mip</code>                     | 31 |
| 2.5.19. Machine Low-Power Configuration Register, <code>m1pcfg</code>            | 33 |

|                                                 |    |
|-------------------------------------------------|----|
| 2.5.20. Table Jump Base Vector and Control, jvt | 34 |
| 3. Instruction Set                              | 35 |
| 3.1. Assembler Syntax Rules                     | 35 |
| 3.1.1. Format of Instruction Syntax             | 35 |
| 3.1.2. Address Mode                             | 35 |
| 3.2. RV32I Base Integer Instruction Set         | 36 |
| 3.2.1. Programmers' Model for Base Integer ISA  | 36 |
| 3.2.2. Base Instruction Formats                 | 36 |
| 3.2.3. ADD                                      | 37 |
| 3.2.4. ADDI                                     | 38 |
| 3.2.5. AND                                      | 39 |
| 3.2.6. ANDI                                     | 40 |
| 3.2.7. AUIPC                                    | 41 |
| 3.2.8. BEQ                                      | 42 |
| 3.2.9. BGE                                      | 43 |
| 3.2.10. BGEU                                    | 44 |
| 3.2.11. BLT                                     | 45 |
| 3.2.12. BLTU                                    | 46 |
| 3.2.13. BNE                                     | 47 |
| 3.2.14. EBREAK                                  | 48 |
| 3.2.15. ECALL                                   | 49 |
| 3.2.16. FENCE                                   | 50 |
| 3.2.17. JAL                                     | 51 |
| 3.2.18. JALR                                    | 52 |
| 3.2.19. LB                                      | 54 |
| 3.2.20. LBU                                     | 55 |
| 3.2.21. LH                                      | 56 |
| 3.2.22. LHU                                     | 57 |
| 3.2.23. LUI                                     | 58 |
| 3.2.24. LW                                      | 59 |
| 3.2.25. MRET                                    | 60 |
| 3.2.26. OR                                      | 61 |
| 3.2.27. ORI                                     | 62 |
| 3.2.28. SB                                      | 63 |
| 3.2.29. SH                                      | 64 |
| 3.2.30. SL                                      | 65 |
| 3.2.31. SLLI                                    | 66 |
| 3.2.32. SLT                                     | 67 |
| 3.2.33. SLTI                                    | 68 |
| 3.2.34. SLTIU                                   | 69 |
| 3.2.35. SLTU                                    | 70 |
| 3.2.36. SRA                                     | 71 |
| 3.2.37. SRAI                                    | 72 |
| 3.2.38. SRL                                     | 73 |
| 3.2.39. SRLI                                    | 74 |
| 3.2.40. SUB                                     | 75 |
| 3.2.41. SW                                      | 76 |

|                                                                     |     |
|---------------------------------------------------------------------|-----|
| 3.2.42. WFI                                                         | 77  |
| 3.2.43. XOR                                                         | 78  |
| 3.2.44. XORI                                                        | 79  |
| 3.3. RV32E Base Integer Instruction Set                             | 80  |
| 3.3.1. RV32E Programmers's Model                                    | 80  |
| 3.3.2. RV32E Instruction Set Encoding                               | 80  |
| 3.4. "Zifencei" Instruction-Fetch Fence                             | 81  |
| 3.4.1. FENCE.I                                                      | 81  |
| 3.5. "M" Standard Extension for Integer Multiplication and Division | 82  |
| 3.5.1. DIV                                                          | 83  |
| 3.5.2. DIVU                                                         | 84  |
| 3.5.3. REM                                                          | 85  |
| 3.5.4. REMU                                                         | 86  |
| 3.5.5. MUL                                                          | 87  |
| 3.5.6. MULH                                                         | 88  |
| 3.5.7. MULHSU                                                       | 89  |
| 3.5.8. MULHU                                                        | 90  |
| 3.6. "C" Standard Extension for Compressed Instructions             | 91  |
| 3.6.1. C.ADD                                                        | 92  |
| 3.6.2. C.ADDI                                                       | 93  |
| 3.6.3. C.ADDI16SP                                                   | 94  |
| 3.6.4. C.ADDI4SPN                                                   | 95  |
| 3.6.5. C.AND                                                        | 96  |
| 3.6.6. C.ANDI                                                       | 97  |
| 3.6.7. C.BEQZ                                                       | 98  |
| 3.6.8. C.BNEZ                                                       | 99  |
| 3.6.9. C.EBREAK                                                     | 100 |
| 3.6.10. C.J                                                         | 101 |
| 3.6.11. C.JAL                                                       | 102 |
| 3.6.12. C.JALR                                                      | 103 |
| 3.6.13. C.JR                                                        | 104 |
| 3.6.14. C.LI                                                        | 105 |
| 3.6.15. C.LUI                                                       | 106 |
| 3.6.16. C.LW                                                        | 107 |
| 3.6.17. C.LWSP                                                      | 108 |
| 3.6.18. C.MV                                                        | 109 |
| 3.6.19. C.NOP                                                       | 110 |
| 3.6.20. C.OR                                                        | 111 |
| 3.6.21. C.SLLI                                                      | 112 |
| 3.6.22. C.SRAI                                                      | 113 |
| 3.6.23. C.SRLI                                                      | 114 |
| 3.6.24. C.SUB                                                       | 115 |
| 3.6.25. C.SW                                                        | 116 |
| 3.6.26. C.SWSP                                                      | 117 |
| 3.6.27. C.XOR                                                       | 118 |
| 3.7. "Zicsr" Control and Status Register (CSR) Instructions         | 119 |
| 3.7.1. CSRRC                                                        | 120 |

|                                                  |     |
|--------------------------------------------------|-----|
| 3.7.2. CSRRCI                                    | 121 |
| 3.7.3. CSRRS                                     | 122 |
| 3.7.4. CSRRSI                                    | 123 |
| 3.7.5. CSRRW                                     | 124 |
| 3.7.6. CSRRWI                                    | 125 |
| 3.8. "B" Standard Extension for Bit Manipulation | 126 |
| 3.8.1. ANDN                                      | 128 |
| 3.8.2. BCLR                                      | 129 |
| 3.8.3. BCLRI                                     | 130 |
| 3.8.4. BEXT                                      | 131 |
| 3.8.5. BEXTI                                     | 132 |
| 3.8.6. BINV                                      | 133 |
| 3.8.7. BINVI                                     | 134 |
| 3.8.8. BSET                                      | 135 |
| 3.8.9. BSETI                                     | 136 |
| 3.8.10. CLMUL                                    | 137 |
| 3.8.11. CLMULH                                   | 138 |
| 3.8.12. CLMULR                                   | 139 |
| 3.8.13. CLZ                                      | 140 |
| 3.8.14. CPOP                                     | 141 |
| 3.8.15. CTZ                                      | 142 |
| 3.8.16. MAX                                      | 143 |
| 3.8.17. MAXU                                     | 144 |
| 3.8.18. MIN                                      | 145 |
| 3.8.19. MINU                                     | 146 |
| 3.8.20. ORC.B                                    | 147 |
| 3.8.21. ORN                                      | 148 |
| 3.8.22. REV8                                     | 149 |
| 3.8.23. ROL                                      | 150 |
| 3.8.24. ROR                                      | 151 |
| 3.8.25. RORI                                     | 152 |
| 3.8.26. SEXT.B                                   | 153 |
| 3.8.27. SEXT.H                                   | 154 |
| 3.8.28. SH1ADD                                   | 155 |
| 3.8.29. SH2ADD                                   | 156 |
| 3.8.30. SH3ADD                                   | 157 |
| 3.8.31. XNOR                                     | 158 |
| 3.8.32. ZEXT.H                                   | 159 |
| 3.9. "Zc" code-size reduction                    | 160 |
| 3.9.1. PUSH/POP Register Instructions            | 160 |
| 3.9.2. Table Jump Overview                       | 161 |
| 3.9.3. C.LBU                                     | 163 |
| 3.9.4. C.LH                                      | 164 |
| 3.9.5. C.LHU                                     | 165 |
| 3.9.6. C.SB                                      | 166 |
| 3.9.7. C.SH                                      | 167 |
| 3.9.8. C.ZEXT.B                                  | 168 |

|                                                                        |     |
|------------------------------------------------------------------------|-----|
| 3.9.9. C.SEXT.B                                                        | 169 |
| 3.9.10. C.ZEXT.H                                                       | 170 |
| 3.9.11. C.SEXT.H                                                       | 171 |
| 3.9.12. C.NOT                                                          | 172 |
| 3.9.13. C.MUL                                                          | 173 |
| 3.9.14. CM.PUSH                                                        | 174 |
| 3.9.15. CM.POP                                                         | 177 |
| 3.9.16. CM.POPRET                                                      | 180 |
| 3.9.17. CM.POPRETZ                                                     | 183 |
| 3.9.18. CM.MVA01S                                                      | 186 |
| 3.9.19. CM.MVSA01                                                      | 187 |
| 3.9.20. CM.JALT                                                        | 188 |
| 3.9.21. CM.JT                                                          | 189 |
| 3.10. RISC-V Integer Conditional (Zicond) Operations Extension         | 190 |
| 3.10.1. Usage Examples                                                 | 190 |
| 3.10.2. CZERO.EQZ                                                      | 192 |
| 3.10.3. CZERO.NEZ                                                      | 193 |
| 3.11. “Smnmi” Standard Extension for Resumable Non-Maskable Interrupts | 194 |
| 3.11.1. MNRET                                                          | 194 |
| 3.12. Wingsemi Instruction Extensions                                  | 195 |
| 3.12.1. ADDSHF                                                         | 198 |
| 3.12.2. ANDSHF                                                         | 199 |
| 3.12.3. BEQI                                                           | 200 |
| 3.12.4. BGEI                                                           | 201 |
| 3.12.5. BGEUI                                                          | 202 |
| 3.12.6. BLTI                                                           | 203 |
| 3.12.7. BLTUI                                                          | 204 |
| 3.12.8. BNEI                                                           | 205 |
| 3.12.9. LB.POST                                                        | 206 |
| 3.12.10. LB.PRE                                                        | 207 |
| 3.12.11. LBR                                                           | 208 |
| 3.12.12. LBR.POST                                                      | 209 |
| 3.12.13. LBR.PRE                                                       | 210 |
| 3.12.14. LBU.POST                                                      | 211 |
| 3.12.15. LBU.PRE                                                       | 212 |
| 3.12.16. LBUR                                                          | 213 |
| 3.12.17. LBUR.POST                                                     | 214 |
| 3.12.18. LBUR.PRE                                                      | 215 |
| 3.12.19. LH.POST                                                       | 216 |
| 3.12.20. LH.PRE                                                        | 217 |
| 3.12.21. LHR                                                           | 218 |
| 3.12.22. LHR.POST                                                      | 219 |
| 3.12.23. LHR.PRE                                                       | 220 |
| 3.12.24. LHU.POST                                                      | 221 |
| 3.12.25. LHU.PRE                                                       | 222 |
| 3.12.26. LHUR                                                          | 223 |
| 3.12.27. LHUR.POST                                                     | 224 |

|                                                                                                                                    |     |
|------------------------------------------------------------------------------------------------------------------------------------|-----|
| 3.12.28. LHUR.PRE                                                                                                                  | 225 |
| 3.12.29. LW.POST                                                                                                                   | 226 |
| 3.12.30. LW.PRE                                                                                                                    | 227 |
| 3.12.31. LWR                                                                                                                       | 228 |
| 3.12.32. LWR.POST                                                                                                                  | 229 |
| 3.12.33. LWR.PRE                                                                                                                   | 230 |
| 3.12.34. ORSHF                                                                                                                     | 231 |
| 3.12.35. SB.POST                                                                                                                   | 232 |
| 3.12.36. SB.PRE                                                                                                                    | 233 |
| 3.12.37. SBR                                                                                                                       | 234 |
| 3.12.38. SBR.POST                                                                                                                  | 235 |
| 3.12.39. SBR.PRE                                                                                                                   | 236 |
| 3.12.40. SH.POST                                                                                                                   | 237 |
| 3.12.41. SH.PRE                                                                                                                    | 238 |
| 3.12.42. SHR                                                                                                                       | 239 |
| 3.12.43. SHR.POST                                                                                                                  | 240 |
| 3.12.44. SHR.PRE                                                                                                                   | 241 |
| 3.12.45. SUBSHF                                                                                                                    | 242 |
| 3.12.46. SW.POST                                                                                                                   | 243 |
| 3.12.47. SW.PRE                                                                                                                    | 244 |
| 3.12.48. SWR                                                                                                                       | 245 |
| 3.12.49. SWR.POST                                                                                                                  | 246 |
| 3.12.50. SWR.PRE                                                                                                                   | 247 |
| 3.12.51. REV8HW                                                                                                                    | 248 |
| 3.12.52. REV8SLHW                                                                                                                  | 249 |
| 3.12.53. XORSHF                                                                                                                    | 250 |
| 4. Physical Memory Protection (PMP)                                                                                                | 251 |
| 4.1. Features                                                                                                                      | 251 |
| 4.2. Memory Access Permission                                                                                                      | 251 |
| 4.2.1. Instruction fetches permission check                                                                                        | 251 |
| 4.2.2. Data accesses permission check                                                                                              | 251 |
| 4.3. Address Matching Mode                                                                                                         | 252 |
| 4.3.1. NAPOT                                                                                                                       | 252 |
| 4.4. PMP Locking                                                                                                                   | 252 |
| 4.5. Matching Logic                                                                                                                | 252 |
| 4.5.1. Address Matching Priority                                                                                                   | 253 |
| 4.5.2. Multiple accesses                                                                                                           | 253 |
| 4.6. CSR registers                                                                                                                 | 254 |
| 4.6.1. PMP configuration register, <i>pmpcfg</i> < <i>n</i> >                                                                      | 255 |
| 4.6.2. PMP address register, <i>pmpaddr</i> < <i>i</i> > ( <i>i</i> =4* <i>n</i> , 4* <i>n</i> +1, 4* <i>n</i> +2, 4* <i>n</i> +3) | 257 |
| 5. Physical Memory Attribute (PMA)                                                                                                 | 258 |
| 5.1. Features                                                                                                                      | 258 |
| 5.2. Address Match                                                                                                                 | 258 |
| 5.3. Vacant holes                                                                                                                  | 258 |
| 5.4. Accessibility                                                                                                                 | 258 |
| 5.5. Memory Attribute                                                                                                              | 259 |
| 5.5.1. Idempotent                                                                                                                  | 259 |



|                                                |     |
|------------------------------------------------|-----|
| 5.5.2. Cacheable                               | 259 |
| 5.6. Configuration of PMA Regions              | 259 |
| 6. Memory Subsystem (MSS)                      | 260 |
| 6.1. Features                                  | 260 |
| 6.2. Memory Model                              | 260 |
| 6.2.1. Memory Type                             | 260 |
| 6.2.2. Normal Memory                           | 260 |
| 6.2.3. Device Memory                           | 260 |
| 6.3. Alignment                                 | 260 |
| 6.4. Endianness                                | 261 |
| 6.4.1. Endianness Format                       | 261 |
| 6.4.2. Endianness Configuration                | 262 |
| 6.4.3. Instructions to Perform Byte Reverse    | 262 |
| 6.5. Memory-Map                                | 263 |
| 6.5.1. System Memory-mapped Register Space     | 263 |
| 6.5.2. External Interfaces                     | 263 |
| 7. Debug System                                | 265 |
| 7.1. Overview                                  | 265 |
| 7.2. Debug Module                              | 265 |
| 7.2.1. Features                                | 265 |
| 7.2.2. Debug Authentication                    | 266 |
| 7.2.3. Reset Control                           | 266 |
| 7.2.4. Hart DM States                          | 267 |
| 7.2.5. Run Control                             | 268 |
| 7.2.6. Program Buffer                          | 268 |
| 7.2.7. Abstract Commands                       | 268 |
| 7.2.8. Debug Module Interface                  | 274 |
| 7.2.9. Debug Module Registers                  | 274 |
| 7.3. Debug Transport Module                    | 287 |
| 7.3.1. JTAG Debug Transport Module             | 287 |
| 7.3.2. JTAG DTM Registers                      | 287 |
| 7.4. CSR Registers                             | 294 |
| 7.4.1. Debug Control and Status Register, dcsr | 295 |
| 7.4.2. Debug Program Counter, dpc              | 299 |
| 7.4.3. Debug Scratch Register 0, dscratch0     | 300 |
| 7.4.4. Debug Scratch Register 2, dscratch2     | 301 |
| 7.4.5. Debug Scratch Register 3, dscratch3     | 302 |
| 8. Trigger Module                              | 303 |
| 8.1. Features                                  | 303 |
| 8.2. Trigger Type                              | 303 |
| 8.2.1. Memory Access Trigger                   | 303 |
| 8.3. Native Debug Triggers                     | 305 |
| 8.4. Trigger Access Concept                    | 305 |
| 8.5. CSR Registers                             | 306 |
| 8.5.1. Trigger Select, tselect                 | 307 |
| 8.5.2. Trigger Data 1, tdata1                  | 308 |



|                                                              |     |
|--------------------------------------------------------------|-----|
| 8.5.3. Match Control Type 6, mcontrol6                       | 309 |
| 8.5.4. Trigger Data 2, tdata2                                | 312 |
| 8.5.5. Trigger Info, tinfo                                   | 313 |
| 8.5.6. Trigger Control, tcontrol                             | 314 |
| 9. Performance Monitor Unit (PMU)                            | 315 |
| 9.1. Features                                                | 315 |
| 9.2. Counters Availability to the Next-Lower Privileged Mode | 315 |
| 9.3. Counters' Increment Control                             | 315 |
| 9.4. Basic Counters and Timers                               | 315 |
| 9.5. CSR Registers                                           | 317 |
| 9.5.1. Machine Counter Enable Register, mcounteren           | 318 |
| 9.5.2. Machine Counter-Inhibit Register, mcountinhibit       | 319 |
| 9.5.3. Machine Cycle Counter, mcycle                         | 320 |
| 9.5.4. Machine Instruction Retired Counter, minstret         | 321 |
| 9.5.5. Cycle Counter, cycle                                  | 322 |
| 9.5.6. Timer, time                                           | 323 |
| 9.5.7. Instruction Retired Counter, instret                  | 324 |
| 10. Interrupt Controller                                     | 325 |
| 10.1. Features                                               | 325 |
| 10.2. Core-Local Interrupt Controller (CLIC)                 | 325 |
| 10.2.1. CLIC Interrupt ID Mapping                            | 325 |
| 10.2.2. Interrupt Enable and Pending                         | 326 |
| 10.2.3. Interrupt Trigger Type and Polarity                  | 326 |
| 10.2.4. Interrupt Level and Interrupt Priority               | 327 |
| 10.2.5. Interrupt Arbitration                                | 329 |
| 10.2.6. Vectored Mode and None-Vectored Mode                 | 329 |
| 10.2.7. Enter Interrupt handler                              | 330 |
| 10.2.8. Return from an Interrupt                             | 331 |
| 10.2.9. Interrupt Preemption and Nested Interrupts           | 332 |
| 10.2.10. Interrupt Tail-Biting                               | 332 |
| 10.2.11. Conditional Swap mscratch on Privilege Mode Change  | 334 |
| 10.2.12. Conditional Swap mscratch on Interrupt Level Change | 335 |
| 10.3. Cases on Interrupt Entry                               | 335 |
| 10.3.1. Multiple Instruction on Interrupt Entry              | 336 |
| 10.4. Non-Maskable Interrupts (NMI)                          | 336 |
| 10.4.1. Resumable Non-Maskable Interrupts (RNMI)             | 336 |
| 10.5. CSR Registers                                          | 338 |
| 10.5.1. CLIC CSRs                                            | 339 |
| 10.5.2. NMI CSRs                                             | 346 |
| 10.6. Memory-Mapped Registers                                | 352 |
| 10.6.1. Machine CLIC Configuration Register, mcliccfg        | 353 |
| 10.6.2. CLIC Interrupt Pending Register, clicintip<n>        | 354 |
| 10.6.3. CLIC Interrupt Enable Register, clicintie<n>         | 355 |
| 10.6.4. CLIC Interrupt Attribute Register, clicintattr<n>    | 356 |
| 10.6.5. CLIC Interrupt Input Control Register, clicintctl<n> | 358 |
| 11. Timer                                                    | 359 |

|                                                                             |     |
|-----------------------------------------------------------------------------|-----|
| 11.1. Features                                                              | 359 |
| 11.2. Timer Control                                                         | 359 |
| 11.3. Timer Interrupt                                                       | 359 |
| 11.4. Timer Calibration                                                     | 359 |
| 11.5. Memory-Mapped Registers                                               | 361 |
| 11.5.1. Machine Timer Register, lower part, mtime_lo                        | 362 |
| 11.5.2. Machine Timer Compare Register, lower part, mtimecmp_lo             | 363 |
| 11.5.3. Machine Timer Control Register, mtimectl                            | 364 |
| 11.5.4. Machine Software Interrupt Pending, msip                            | 366 |
| 11.5.5. Machine Timer Calibration Value Register, lower part, mtimecalib_lo | 367 |
| 11.5.6. Software Reset Control Register, srcr                               | 368 |
| 12. Reliability, Availability, and Serviceability (RAS)                     | 369 |
| 12.1. Feature                                                               | 369 |
| 12.2. Error Information Record                                              | 369 |
| 12.2.1. Error Detection and Correction                                      | 369 |
| 12.2.2. Error Information                                                   | 369 |
| 12.2.3. Error Record Enable and Disable                                     | 370 |
| 12.2.4. Error Record Invalidate                                             | 370 |
| 12.3. Error Priority and Error Overwrite Rules                              | 370 |
| 12.3.1. The Severity of Errors                                              | 370 |
| 12.3.2. Overwrite Rules                                                     | 370 |
| 12.4. Error Report and Signaling                                            | 371 |
| 12.5. Corrected Error Counting                                              | 371 |
| 12.6. Memory-Mapped Registers                                               | 372 |
| 12.6.1. Vendor and Implementation ID, vendor_n_imp_id                       | 373 |
| 12.6.2. Error Bank Information, bank_info                                   | 374 |
| 12.6.3. Summary of Valid Error Records, valid_summary                       | 375 |
| 12.6.4. Control Register, control_0                                         | 376 |
| 12.6.5. Status Register, status_0                                           | 378 |
| 12.6.6. Address Register, addr_0                                            | 381 |
| 12.6.7. Information Register, info_0                                        | 382 |

# Revision History

| Date       | Version | Description          |
|------------|---------|----------------------|
| 2023.10.31 | v0p1    | The initial version. |
| 2024.05.31 | v1p0    | The release version. |
| -          | -       | -                    |

# 1. Overview

Wing-M130 processor is a RISC-V compatible low-power processor with an in-order, single-issue, three-stage pipeline design. It is designed to fulfill cost-sensitive, power-constrained solutions, targeting at areas like MCU controller, IoT, sensors etc.

## 1.1. Block Diagram

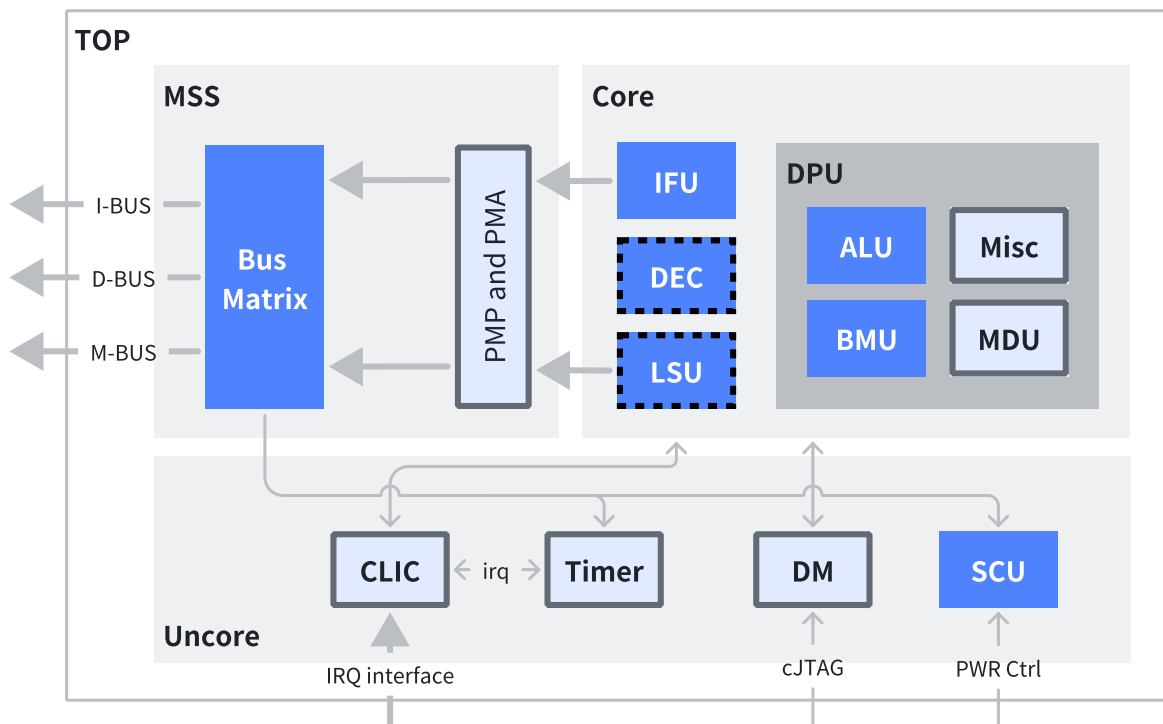


Figure 1. Wing-M130 Processor Block Diagram

Figure 1 shows the block diagram of Wing-M130. It consists of Core, Memory SubSystem (MSS), and Uncore. Wing-M130 core fetches instructions from MSS, decodes it and performs instruction execution. MSS is the bridge of Wing-M130 core and the external system. Uncore handles components like interrupt controller, debugger, etc.

## 1.2. Features

The features of Wing-M130 are summarized in the following

### 1.2.1. Instruction Set

- Support base integer ISA RV32E with 16 general purpose registers.
- Support Instruction-Fetch Fence (zifencei)
- Support Control and Status Register (CSR) Instructions (zicsr)
- Support for Multiplication and Division operations (M-extension)
  - An option to choose a multiplier from fast (default) or slow implementation (xmuls1).

- Support multiply instructions only (zmmul)
  - An option to choose a multiplier from fast (default) or slow implementation (xmulsi).
- Optimized for code-size reduction.
  - Standard Extension for compressed instructions (C-extension).
  - Code-size reduction enhancement (zce).
  - Support integer conditional operations (zicnd).
  - Wingsemi code-size reduction (xwingcs).
- Support Bit Manipulation (B-extension).

## 1.2.2. Privilege Mode and Physical Memory Protection

- Support two privilege level: Machine-Mode and User-Mode.
- Support Physical Memory Protection mechanism.
  - Configurable PMP regions and granularity.
- Support Physical Memory Attribute.
  - Configurable PMA regions.

## 1.2.3. Memory Subsystem

- Support little-endian by default
  - Optionally support big-endian in data-side.

## 1.2.4. Miscellaneous

- RISC-V Debug compatibal implementation
  - Configurable debug interface from the following: cJTAG/JTAG
  - Configurable number of Triggers.
- Compatible with Core-Local Interrupt Controller (smcllc).
  - Configurable interrupt inputs.
  - Configurable priority levels of a interrupt inputs.
- Performance Monitor Unit (PMU)

## 1.3. Specification Compliance

Wing-M130 processor is compliance with a series of RISC-V specifications, including basic ISA specification and its extensions, non-ISA specifications like debug, trace, and interrupt. It also implements AMBA-based interconnect.

The following table shows the version of features that are supported on Wing-M130 processor.

| Feature | Version | Status   | Description                          | Link                             |
|---------|---------|----------|--------------------------------------|----------------------------------|
| RV32I   | 2.1     | Ratified | 32-bit base integer instruction set. | <a href="#">riscv-isa-manual</a> |
| M       | 2.0     | Ratified | Integer multiplication and Division. | <a href="#">riscv-isa-manual</a> |

| Feature  | Version | Status   | Description                                              | Link                                      |
|----------|---------|----------|----------------------------------------------------------|-------------------------------------------|
| C        | 2.0     | Ratified | Compressed instructions.                                 | <a href="#">riscv-isa-manual</a>          |
| B        | 1.0     | Frozen   | Bit manipulation instructions.                           | <a href="#">riscv-bitmanip</a>            |
| Zicsr    | 2.0     | Ratified | Control and Status Register (CSR) instructions.          | <a href="#">riscv-isa-manual</a>          |
| Zifencei | 2.0     | Ratified | Instruction-Fetch fence.                                 | <a href="#">riscv-isa-manual</a>          |
| Zfinx    | 1.0     | Ratified | Floating-point in Integer registers.                     | <a href="#">riscv-isa-manual</a>          |
| Zmmul    | 1.0     | Ratified | Implements the multiplication subset of the M extension. | <a href="#">riscv-isa-manual</a>          |
| Zicond   | 1.0     | Draft    | Integer conditional operations.                          | <a href="#">riscv-zicond</a>              |
| Zce      | 1.0     | Frozen   | Code-size reduction for microcontrollers.                | <a href="#">riscv-code-size-reduction</a> |

Table 1. RISC-V instruction set related specifications support on the Wing-M130 processor

| Feature     | Version | Status   | Description                                   | Link                                 |
|-------------|---------|----------|-----------------------------------------------|--------------------------------------|
| Machine-ISA | 1.12    | Ratified | Machine-level operations available in M-mode. | <a href="#">riscv-isa-manual</a>     |
| Debug       | 1.0     | Frozen   | RISC-V debug support.                         | <a href="#">riscv-debug</a>          |
| Smclic      | 0.9     | Draft    | Core-Local Interrupt Controller               | <a href="#">riscv-fast-interrupt</a> |

Table 2. RISC-V non-instruction set related specifications support on the Wing-M130 processor

| Feature | Version | Description                             | Link                                            |
|---------|---------|-----------------------------------------|-------------------------------------------------|
| AHB     | -       | The AMBA Advanced High-performance Bus. | <a href="#">AMBA AHB Protocol Specification</a> |
| APB     | APB5    | The AMBA Advanced Peripheral Bus.       | <a href="#">AMBA APB Protocol Specification</a> |

Table 3. Arm interconnect support on the Wing-M130 processor

## 1.4. Configuration Options

Wing-M130 supports a wide range of configuration options which enables customers to specify their own configuration during implementation time to trade off their PPA result and match the functional requirements.

The following table shows the supported configuration options available at implementation time.

| Feature                                | Options                           | Dependency               |
|----------------------------------------|-----------------------------------|--------------------------|
| Instruction Set                        |                                   |                          |
| Multiply and Divide                    | Support Multiply only (zmmul)     | -                        |
|                                        | Multiply and Divide (M-extension) |                          |
| Multiply implementation                | Fast implementation (default)     | zmmul or M<br>-extension |
|                                        | Slow implementation (xmulsi)      |                          |
| Privileged Level and Memory Protection |                                   |                          |



| Feature                     | Options                                                       | Dependency |
|-----------------------------|---------------------------------------------------------------|------------|
| Physical Memory Protection  | Not support                                                   | U-mode     |
|                             | Support Physical Memory Protection (PMP)                      |            |
| Number of PMP regions       | 0 (Equal to PMP "Not support"), 4, 8 (default), 16, 32        | PMP        |
| Minimum PMP granularity     | 16 (default), 32, 64, 128, 256, 512, 1024, 2048, 4096 in byte | PMP        |
| PMP address matching mode   | Naturally aligned power-of-two region (NAPOT) only            | PMP        |
|                             | NAPOT and Top of Range (TOR)                                  |            |
| Physical Memory Attribute   | User customized.                                              |            |
| Memory Subsystem            |                                                               |            |
| Debug functions             |                                                               |            |
| Debug                       | Not support                                                   | -          |
|                             | Debug Module (sdext)                                          |            |
| Debug Interface             | cJTAG/JTAG                                                    | sdext      |
| Number of triggers (sdtrig) | 2, 4, 6, 8, 10, 12, 14, 16                                    | sdext      |
| Interrupt                   |                                                               |            |
| Interrupt Controller        | Core-Local INTerrupt (CLINT)                                  | -          |
|                             | Core-Local Interrupt Controller (smcllic)                     |            |
| Number of interrupts        | 1 to 4080                                                     | smcllic    |
| Non-Maskable Interrupts     | Non-Maskable Interrupts (smunmi)                              | -          |
|                             | Resumable Non-Maskable Interrupts (smrnmi)                    |            |
| Timer                       |                                                               |            |
| Interface                   |                                                               |            |
| External Master Interface   | User customized three buses from I-Bus, D-Bus, and M-Bus      |            |

Table 4. Wing-M130 Configuration Options

## 2. Wing-M130 Core

### 2.1. Instruction Fetch Unit (IFU)

Instruction Fetch Unit (IFU) generates the next PC address, and sends instruction fetch request to the Memory SubSystem (MSS). When MSS returns the fetch result, IFU aligns the fetched instructions and sends them into the instruction buffer. After instructions are returned from MSS, branch instructions are identified and predicted.

#### 2.1.1. Features

- Generate the next PC for instruction fetch.
- Fetch instruction bundle and interact with MSS.
- Instruction alignment and Branch instruction pre-decode.
- Handle back-to-back redirect request from the backend. The request comes from either the interrupt in vector mode or the table jump instructions.
- Support a static branch predictor with Forward Not-Taken, Backward Taken (FNTBT).
- Support a Return Address Stack (RAS) to predict function return.

### 2.2. Pipeline Control Unit

The pipeline control unit controls the execution of the processor. It consists of instruction decode, instruction issue, and instruction commit logics.

#### 2.2.1. Instruction Decode Unit (DEC)

Instruction Decode Unit (DEC) decodes instructions from instruction buffer.

##### 2.2.1.1. Features

- Decode instructions into control signals
- Identify illegal instructions

#### 2.2.2. Issue Unit

Under certain conditions, issue instructions to execution units.

##### 2.2.2.1. Features

- Issue instructions to execution units.
- Split Macro instruction to uops.
- Forwarding and hazards check.
- Handle general purpose register-file accessing.

#### 2.2.3. Commit

This logic decides whether an instruction could be committed.

### 2.2.3.1. Features

- Commit the instruction when it triggers no exceptions.
- Record instructions with exceptions.
- Execute special purpose instructions, like `ecall`, `ebreak`, `wfi`, etc.
- Response to interrupts.
- Response to debug requests.

## 2.3. Execution Unit (EXU)

Execution Unit (EXU) handles the execution of instructions, including basic scalar arithmetic, logic, shift instructions, and supports the address calculation of load/store instructions. Branch instructions are also handled here.

### 2.3.1. Features

- An arithmetic calculation unit (ALU) for scalar operations such as arithmetic, logic, shift, etc.
- An additional ALU executed on WB stage (LALU), supporting basic scalar operations.
- Support an optional Bit-Manipulation Unit (BMU) for bit-manipulation operations (B-extension).
- Configurable Multiplication and Division Unit (MDU)
  - With support for Multiplication and Division (M-extension) or Multiplication Only (Zmmul).
  - An option to choose a multiplier from fast or slow implementation.
- Support a User-Defined Block (UDB) for user-customized instruction extension.
- Support address calculation for Load/Store instructions.
- Support the calculation of branch condition and resolved PC for branch instructions.

## 2.4. Exceptions

Exceptions are defined as unusual condition occurring *synchronously* to the instruction execution flow.

### 2.4.1. Exceptions and the Priority

Table 5 lists the possible exceptions occurred in Wing-M130 processor.

| Interrupt | Exception Code | Description                    |
|-----------|----------------|--------------------------------|
| 0         | 0              | Instruction address misaligned |
| 0         | 1              | Instruction access fault       |
| 0         | 2              | Illegal instruction            |
| 0         | 3              | Breakpoint                     |
| 0         | 4              | Load address misaligned        |
| 0         | 5              | Load access fault              |

| Interrupt | Exception Code | Description                   |
|-----------|----------------|-------------------------------|
| 0         | 6              | Store/AMO address misaligned  |
| 0         | 7              | Store/AMO access fault        |
| 0         | 8              | Environment call from U-mode  |
| 0         | 11             | Environment call from M-mode  |
| 0         | 31             | Machine synchronous RAS fault |

Table 5. Exception Code

If an instruction may raise multiple *synchronous* exceptions, the descreasing priority order of Table 6 indicates which exception is taken and reported in mcause.

| Priority       | Exception Code | Description                                          |
|----------------|----------------|------------------------------------------------------|
| <i>Highest</i> | 3              | Instruction address breakpoint                       |
|                | 1              | Instruction access fault                             |
|                | 31             | Machine synchronous RAS fault (by instruction fetch) |
|                | 2              | Illegal instruction                                  |
|                | 0              | Instruction address misaligned                       |
|                | 8,11           | Environment call                                     |
|                | 3              | Environment break                                    |
|                | 3              | Load/Store/AMO address breakpoint                    |
|                | 5,7            | Load/Store/AMO access fault                          |
|                | 4,6            | Load/Store/AMO address misaligned                    |
| <i>Lowest</i>  | 31             | Machine synchronous RAS fault (by data accesses)     |

Table 6. Exception Priority in Decreasing Priority Order

## 2.4.2. Enter Exception handler

In Wing-M130 processor, the instruction with an exception is pipelined to WB-stage. The instructions before it are committed, while the instructions after it are all flushed. The exception instruction would not be committed, and the information about this exception will be recorded.

### 2.4.2.1. Update CSRs

#### 2.4.2.1.1. Update mepc

The register mepc holds the return address when the hart returns from exception handler. When a trap is taken into M-mode, mepc is written with the address of the instruction that encountered the exception.

#### 2.4.2.1.2. Update mstatus

mstatus.mpie is set to the value of mstatus.mie, and mstatus.mie is set to 0.

mstatus.mpp is set to the previous privilege mode. Therefore, when a trap is taken from privilege mode y

into privilege mode `x`, `mstatus.mpp` is set to `y`.

#### 2.4.2.1.3. Update `mcause`

The register `mcause` is written with a code indicating the event that caused the trap. Refer to [Table 5](#).

#### 2.4.2.1.4. Update `mtval`

The register `mtval` is either set to zero or written with exception-specific information to assist software in handling the trap. Refer to [Table 7](#).

| Exception Code | Description                    | Trap Value                              |
|----------------|--------------------------------|-----------------------------------------|
| 0              | Instruction address misaligned | The address that causes the fault       |
| 1              | Instruction access fault       | The address that causes the fault       |
| 2              | Illegal instruction            | The instruction's binary representation |
| 3              | Breakpoint                     | The address that causes the fault       |
| 4              | Load address misaligned        | The address that causes the fault       |
| 5              | Load access fault              | The address that causes the fault       |
| 6              | Store/AMO address misaligned   | The address that causes the fault       |
| 7              | Store/AMO access fault         | The address that causes the fault       |
| 8              | Environment call from U-mode   | Zero                                    |
| 11             | Environment call from M-mode   | Zero                                    |
| 31             | Machine synchronous RAS fault  | The address that causes the fault       |

*Table 7. Trap Value*

Note that,

- When a misaligned load or store causes an access-fault, `mtval` will contain the address of the portion of the access that caused the fault.
- When an instruction access-fault occurs, `mtval` will contain the address of the portion of the instruction that caused the fault.

#### 2.4.2.2. Redirect the Program Counter

The PC will be redirected to the entry of exception handler specified in `mtvec`.

#### 2.4.3. Return from an Exception

To return from the exception handler, an `mret` instruction is to be executed

## 2.5. CSR Registers

### 2.5.1. CSR Address Mapping Conventions

The standard RISC-V ISA sets aside a 12-bit encoding space (csr[11:0]) for up to 4096 CSRs. By convention, the upper 4 bits of the CSR address (csr[11:8]) are used to encode the read and write accessibility of the CSRs according to privilege level. The top two bits (csr[11:10]) indicate whether the register is read/write (00, 01 or 10) or read-only (11). The next two bits (csr[9:8]) encode the lowest privilege level that can access the CSR.

#### NOTE

Machine-mode standard read-write CSRs with 0x7B0-0x7BF are only visible to debug mode. Implementation should raise illegal-instruction exception on machine-mode access to the set of registers.

- Attempts to access a non-existent CSR raise an illegal-instruction exception.
- Attempts to access a CSR without appropriate privilege level raise an illegal-instruction exception.
- Attempts to write a read-only register raise an illegal-instruction exception.

### 2.5.2. Register Field Conventions

| Abbreviation | Description                                                                                                                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| R            | Read-only. Write with no effect.                                                                                                                  |
| R/W1C        | Read/Write Ones to Clear. Writing 0 to every bit has no effect. Writing 1 to every bit clears the field. The result of other writes is UNDEFINED. |
| RW           | Read-Write.                                                                                                                                       |
| W1           | Write-only. Only writing one has an effect, writing zero is ignored. Read access returns zero.                                                    |
| WARL         | Write Any value, Read Legal value. If a value written is unsupported, the implementation converts the value to one that is supported.             |
| WARZ         | Write-only. Read access returns zero.                                                                                                             |
| WLRL         | Write/Read Only Legal Values. Write illegal values have no effect.                                                                                |

Table 8. Register Access Abbreviations

### 2.5.3. CSR Listing

The following lists the CSRs that have currently been allocated in Wing-M130 processor.

| CSR Address | Privilege | Register   | Description                             |
|-------------|-----------|------------|-----------------------------------------|
| 0xF11       | MRO       | mvendorid  | Vendor ID                               |
| 0xF12       | MRO       | marchid    | Architecture ID                         |
| 0xF13       | MRO       | mimpid     | Implementation ID                       |
| 0xF14       | MRO       | mhartid    | Hardware thread ID                      |
| 0xF15       | MRO       | mconfigptr | Pointer to configuration data structure |



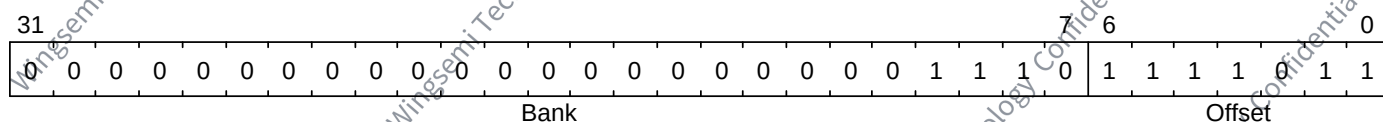
| CSR Address | Privilege | Register | Description                                 |
|-------------|-----------|----------|---------------------------------------------|
| 0x300       | MRW       | mstatus  | Machine status register                     |
| 0x301       | MRW       | misa     | ISA and extensions                          |
| 0x304       | MRW       | mie      | Machine interrupt-enable register           |
| 0x305       | MRW       | mtvec    | Machine trap-handler base address           |
| 0x310       | MRW       | mstatush | Additional machine status register          |
| 0x340       | MRW       | mscratch | Scratch register for machine trap handlers  |
| 0x341       | MRW       | mepc     | Machine exception program counter           |
| 0x342       | MRW       | mcause   | Machine trap cause                          |
| 0x343       | MRW       | mtval    | Machine bad address or instruction          |
| 0x344       | MRW       | mip      | Machine interrupt-pending register          |
| 0x7C4       | MRW       | m1pcfg   | Machine Low-Power Configuration             |
| 0x017       | URW       | jvt      | Table jump base vector and control register |

## 2.5.4. Vendor ID Register, mvendorid

### Description

The mvendorid is a 32-bit read-only register providing the JEDEC manufacturer ID of the provider of the processor. This register is read as 0x0000077B.

JEDEC manufacturer IDs are ordinarily encoded as a sequence of one-byte continuation codes 0x7f, terminated by a one-byte ID not equal to 0x7f, with an odd parity bit in the most-significant bit of each byte. mvendorid encodes the number of one-byte continuation codes in the Bank field, and encodes the final byte in the offset field, discarding the parity bit.



### Configuration

This register is mandatory implemented.

### Field Description

#### Bank, Bits [31:7]

The number of one-byte continuation codes. This field is encoded as 14.

#### Offset, Bits [6:0]

The final byte of JEDEC ID, the parity bit is discarded. This field is encoded as 123.

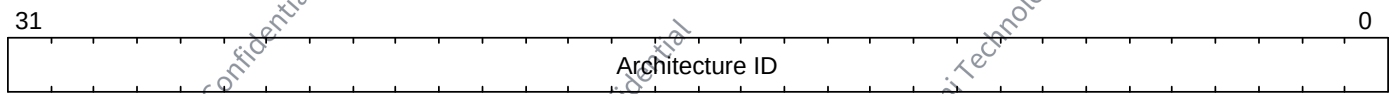
### CSR Address

0xF11

## 2.5.5. Architecture ID Register, marchid

### Description

The marchid is a 32-bit read-only register encoding the base microarchitecture of the hart.



### Configuration

This register is mandatory implemented.

### Field Description

#### Architecture ID, Bits [31:0]

The hart microarchitecture ID. This field is encoded as 0xE1545130.

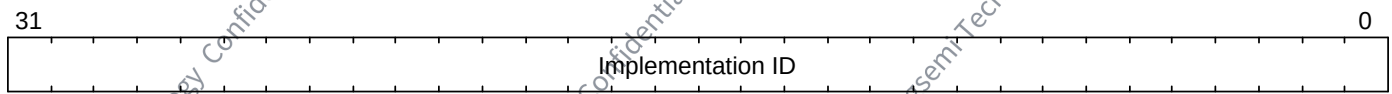
### CSR Address

0xF12

## 2.5.6. Implementation ID Register, mimpid

### Description

The mimpid is a 32-bit read-only register that provides a unique encoding of the version of the processor implementation.



### Configuration

This register is mandatory implemented.

### Field Description

#### Implementation ID, Bits [31:0]

The hart implementation ID. This field is encoded as 0x13002404.

### CSR Address

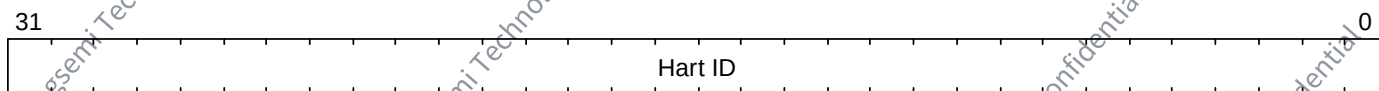
0xF13

## 2.5.7. Hardware thread ID Register, mhartid

### Description

The mhartid is a 32-bit read-only register containing the integer ID of the hardware thread running the code. Hard IDs are not necessarily be numbered contiguously in a multiprocessor system, but must be unique within the execution environment.

Wing-M130 processor offers external pins to SoC for the configuration of each hart's ID in its own execution environment.



### Configuration

This register is mandatory implemented.

### Field Description

#### Hart ID, Bits [31:0]

The integer ID of the hardware thread running the code. The encoding of this field depends on the external pins.

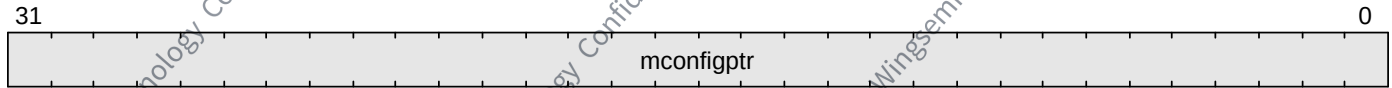
### CSR Address

0xF14

## 2.5.8. Pointer to Configuration Data Structure Register, mconfigptr

### Description

The mconfigptr is a 32-bit read-only register that holds the physical address of a configuration data structure. Software can traverse this data structure to discover information about the harts, the platform, and their configuration.



### Configuration

This register is mandatory implemented.

### Field Description

#### mconfigptr, Bits [31:0]

This field is read-as-zero, indicating the configuration data structure does not exist.

### CSR Address

0xF15



### Description

|    |    |   |   |   |   |   |   |   |   |     |    |     |     |     |      |    |    |    |    |    |     |    |    |   |     |      |     |      |   |     |   |     |   |
|----|----|---|---|---|---|---|---|---|---|-----|----|-----|-----|-----|------|----|----|----|----|----|-----|----|----|---|-----|------|-----|------|---|-----|---|-----|---|
| 31 | 30 |   |   |   |   |   |   |   |   | 23  | 22 | 21  | 20  | 19  | 18   | 17 | 16 | 15 | 14 | 13 | 12  | 11 | 10 | 9 | 8   | 7    | 6   | 5    | 4 | 3   | 2 | 1   | 0 |
| SD | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | TSR | TW | TVM | MXR | SUM | MPRV |    | XS |    | FS |    | MPP |    | VS |   | SPP | MPIE | UBE | SPIE | 0 | MIE | 0 | SIE | 0 |

This register is mandatory implemented.

## SD, Bit [31]

**RES0, Bits [30:23]**

TSR, Bit [22]

**TW, Bit [21]**

**0b1** When WFI instruction is executed in U-mode, it causes an illegal instruction exception.

## TVM, Bit [20]

**MXR, Bit [19]**

**SUM, Bit [18]**

**MPRV, Bit [17]**

**0b0** Loads and stores behave as normal, using the translation and protection mechanisms of the current privilege mode.

**0b1** Load and store memory addresses are translated and protected, and endianness is applied, as though the current privilege mode were set to `mstatus.MPP`.

An MRET instruction that changes the privilege mode to a mode less privileged than M-mode also sets MPRV to 0.

Refer to [Section 4.2.2](#) for more details.

The behavior is also managed by `mnstatus.nmie`, if `smrnmi` is configured. Refer to [Section 10.4.1.1](#)

for more details.

This field is reset to 0.

#### **XS, Bits [16:15]**

**RES0**, Reserved, **WARL** field.

#### **FS, Bits [14:13]**

**RES0**, Reserved, **WARL** field.

#### **MPP, Bits [12:11]**

The MPP bits hold the previous privilege mode in M-mode. This field is **WARL**. Write any value other than 0b00 will read as 0b11.

**0b00** The previous privilege mode is U-mode.

**0b11** The previous privilege mode is M-mode.

This field is reset to 0b11.

#### **VS, Bits [10:9]**

**RES0**, Reserved, **WARL** field.

#### **SPP, Bits [8]**

**RES0**, Reserved, **WARL** field.

#### **MPIE, Bit [7]**

Pervious Global Interrupt Enable bit (M-mode). When a trap is taken from privilege mode y into the privilege mode x, it is set to MIE.

This field is reset to 1.

#### **UBE, Bit [6]**

U-mode Big-Endian. This field is **R**, and has the following encoding. The value of this field is the same with `mstatush.mbe`.

**0b0** The endianness is little-endian.

**0b1** The endianness is big-endian.

#### **SPIE, Bit [5]**

**RES0**, Reserved, **WARL** field.

#### **RES0, Bit [4]**

Reserved, **WPRI** field.

#### **MIE, Bit [3]**

Global interrupt-enable bit for M-mode.

**0b0** Interrupts are globally disabled when the hart is executing in M-mode.

**0b1** Interrupts are globally enabled when the hart is executing in M-mode.

This field is reset to 0.

**RES0, Bit [2]**

Reserved, **WPRI** field.

**SIE, Bit [1]**

**RES0**, Reserved, **WARL** field.

**RES0, Bit [0]**

Reserved, **WPRI** field.

**CSR Address**

0x300

## 2.5.10. ISA and Extensions Register, *misa*

### Description

The *misa* is a **WARL** read-only register reporting the ISA supported by the hart.

|     |    |    |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |
|-----|----|----|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|
| 31  | 30 | 29 |   | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| MXL | 0  | 0  | 0 | 0  | Z  | Y  | X  | W  | V  | U  | T  | S  | R  | Q  | P  | O  | N  | M  | L  | K  | J | I | H | G | F | E | D | C | B | A |

### Configuration

This register is mandatory implemented.

### Field Description

#### MXL, Bits [31:30]

Machine XLEN field encodes the native base integer ISA width. It is a **WARL** field encoded as

**0b01** Support XLEN=32.

#### RES0, Bits [29:26]

Reserved, **WARL** field.

#### Z, Bit [25]

**RES0**, Reserved, **WARL** field.

#### Y, Bit [24]

**RES0**, Reserved, **WARL** field.

#### X, Bit [23]

**0b1** Support User-specified Extension.

#### W, Bit [22]

**RES0**, Reserved, **WARL** field.

#### V, Bit [21]

**RES0**, Reserved, **WARL** field.

#### U, Bit [20]

**0b1** Support User mode.

#### T, Bit [19]

**RES0**, Reserved, **WARL** field.

#### S, Bit [18]

**RES0**, Reserved, **WARL** field.

#### R, Bit [17]

**RES0**, Reserved, **WARL** field.

#### Q, Bit [16]

**RES0**, Reserved, **WARL** field.

#### P, Bit [15]

**RES0**, Reserved, **WARL** field.

**O, Bit [14]****RES0**, Reserved, **WARL** field.**N, Bit [13]****RES0**, Reserved, **WARL** field.**M, Bit [12]****0b1** Support Multiply and Divide extension.**L, Bit [11]****RES0**, Reserved, **WARL** field.**K, Bit [10]****RES0**, Reserved, **WARL** field.**J, Bit [9]****RES0**, Reserved, **WARL** field.**I, Bit [8]****0b0** Support RV32E base ISA.

This bit is always reads as the complement of the E bit.

**H, Bit [7]****RES0**, Reserved, **WARL** field.**G, Bit [6]****RES0**, Reserved, **WARL** field.**F, Bit [5]****RES0**, Reserved, **WARL** field.**E, Bit [4]****0b1** Support RV32E base ISA.

This bit is always reads as the complement of the I bit.

**D, Bit [3]****RES0**, Reserved, **WARL** field.**C, Bit [2]**

Compressed extension.

**0b1** Support compressed extension.**B, Bit [1]****0b1** Support Bit-Manipulation extension.**A, Bit [0]****RES0**, Reserved, **WARL** field.**CSR Address**

0x301

### Description

mie supports up to 32 interrupts, each bit of which enables/disables the corresponding interrupts. The bits [15:0] are allocated to standard interrupt causes only, while bit 16 and above are designated for platform or custom use.

[illegible]

This register is present only when `smclic` is not implemented. Otherwise, it is hardwired to zero.

**MRIE, Bit [31]**

**0b0** Disable machine asynchronous RAS fault.

**0b1** Enable machine asynchronous RAS fault.

**RES0, Bits [30:12]**

**MEIE, Bit [11]**

**0b0** Disable machine external interrupt.

**0b1** Enable machine external interrupt.

## RES0, Bit [10]

**SEIE, Bit [9]**

### RES0, Bit [8]

**MTIE, Bit [7]**

Machine timer interrupt enable.



**0b0** Disable machine timer interrupt.

**0b1** Enable machine timer interrupt.

This field is reset to 0.

**RES0, Bit [6]**

Reserved, **WARL** field.

**STIE, Bit [5]**

**RES0**, Reserved, **WARL** field.

**RES0, Bit [4]**

Reserved, **WARL** field.

**MSIE, Bit [3]**

Machine software interrupt enable.

**0b0** Disable machine software interrupt.

**0b1** Enable machine software interrupt.

This field is reset to 0.

**RES0, Bit [2]**

Reserved, **WARL** field.

**SSIE, Bit [1]**

**RES0**, Reserved, **WARL** field.

**RES0, Bit [0]**

Reserved, **WARL** field.

**CSR Address**

0x304

### Description

[illegible]

This register is mandatory implemented.

**base, Bits [31:6]**

This field is reset to an UNSPECIFIED value.

This field is **WARL** with the following encoding

**mode, Bits [1:0]**

This field is **WARL** with the following encoding

**0b11** The CLIC interrupt-handling mode.

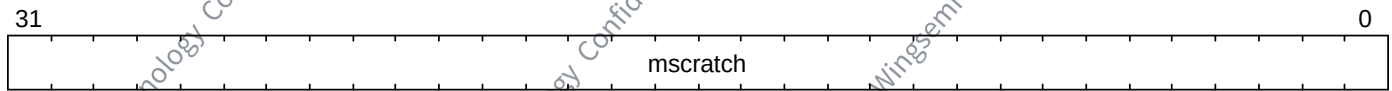
## 0x305



## 2.5.14. Scratch Register for Machine Trap Handlers, mscratch

### Description

The mscratch register is a read/write register dedicated for use by machine mode. Typically, it is used to hold a pointer to a machine-mode hart-local context space and swapped with a user register upon entry to an M-mode trap handler.



### Configuration

This register is mandatory implemented.

### Field Description

#### mscratch, Bits [31:0]

Software dedicated content.

This field is reset to an UNSPECIFIED value.

### CSR Address

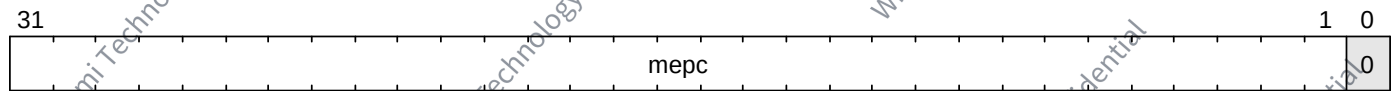
0x340

### 2.5.15. Machine Exception Program Counter Register, mepc

#### Description

The mepc register is a read/write register which holds the address where a trap return to.

When a trap is taken into M-mode, mepc is written with the address of the instruction that was interrupted or that encountered the exception. Otherwise, mepc is never written by the implementation, though it may be explicitly written by software.



#### Configuration

This register is mandatory implemented.

#### Field Description

##### mepc, Bits [31:1]

The address where a trap return to.

This field is reset to an UNSPECIFIED value.

##### RES0, Bit [0]

Reserved, WARL field.

#### CSR Address

0x341

## 2.5.16. Machine Cause Register, mcause

### Description

The mcause register is a 32-bit read-write register, when a trap is taken into, mcause is written with information indicating the event that caused the trap.



### Configuration

This register is mandatoroty implemented.

### Field Description

#### Interrupt, Bit [31]

**0b0** Exception

**0b1** Interrupt

This field is reset to 0.

#### minhv, Bit [30]

**0b0** mepc is the address of an instruction.

**0b1** mepc is the address of a table entry.

This field is reset to 0.

#### mpp, Bits [29:28]

Previous privilege mode, it mirrors mstatus.mpp field. Reading or writing this field is equivalent to reading or writing mstatus.mpp.

#### mpie, Bit [27]

Previous interrupt enable, it mirrors mstatus.mpie field. Reading or writing this field is equivalent to reading or writing mstatus.mpie.

#### RES0, Bits [26:24]

Reserved, **WARL** field.

#### mpil, Bits [23:16]

Previous interrupt level.

This field is reset to 0.

#### RES0, Bits [15:12]

Rserved, **WARL** field.

#### Exception code, Bits [11:0]

Exception/Interrupt code. Refer to [Table 5](#) and [Table 16](#). This field is **WLRL** field.

This field is reset to 0.

0x342



### Description

If `mtval` is written with a nonzero value when a breakpoint, address-misaligned, or access-fault exception occurs on an instruction fetch, load, or store, then `mtval` will contain the faulting address.

If `mtval` is written with a nonzero value when an instruction access-fault exception occurs on a system with variable-length instructions, then `mtval` will contain the address of the portion of the instruction that caused the fault, while `mepc` will point to the beginning of the instruction.

31 0

mtval

This register is mandatory implemented.

**mtval, Bits [31:0]**

This field is reset to an UNSPECIFIED value.

0x343



**RES0, Bit [6]**

Reserved, **WARL** field.

**STIP, Bit [5]**

**RES0**, Reserved, **WARL** field.

**RES0, Bit [4]**

Reserved, **WARL** field.

**MSIP, Bit [3]**

Machine software interrupt pending.

**0b0** Machine software interrupt is not in pending state.

**0b1** Machine software interrupt is in pending state.

This field is reset to 0.

**RES0, Bit [2]**

Reserved, **WARL** field.

**SSIP, Bit [1]**

**RES0**, Reserved, **WARL** field.

**RES0, Bit [0]**

Reserved, **WARL** field.

**CSR Address**

0x344

## 2.5.19. Machine Low-Power Configuration Register, m1pcfg

### Description

The m1pcfg is a 32-bit read-write register that provides low-power control of the hart.



### Configuration

This register is mandatory implemented.

### Field Description

#### RES0, Bits [31:1]

Reserved, **WARL** field.

#### ds, Bit [0]

Deep Sleep. Indicate whether the hart could enter into deep sleep or not. Waking from deep sleep might takes longer time.

**0** The sleep state is not deep sleep.

**1** The sleep state is deep sleep.

### CSR Address

0x7C4

## 2.5.20. Table Jump Base Vector and Control, jvt

### Description

The jvt register is a 32-bit read/write register which holds the jump table configuration, consisting of the jump table base address and the jump table mode.



### Configuration

This register is present only when zcmt is configured.

### Field Description

#### base, Bits [31:6]

This field contains the base address of the jump table, which is aligned to 64-byte.

This field is reset to an UNSPECIFIED value.

#### mode, Bits [5:0]

It is a **WARL** field with the following encoding.

**0b000000**      Jump table mode.

### CSR Address

0x017

## 3. Instruction Set

### 3.1. Assembler Syntax Rules

#### 3.1.1. Format of Instruction Syntax

{ } - Brace brackets. Any field in these brackets is optional. For this optional field, the encoding of its presence or absence is specified.

#### 3.1.2. Address Mode

The following addressing modes are supported for memory access instructions.

##### Offset addressing

The memory access address is obtained by adding <offset> and <base>.

The assembly syntax is illustrated as:

```
// <base> : should always be a register value from GPR
// <offset> : could be either immedaite or register value from GPR
<offset>(<base>)
```

##### Pre-increment addressing

The memory access address is obtained by adding <offset> and <base>. The addition result is written back into the base register.

The assembly syntax is illustrated as:

```
// <base> : should always be a register value from GPR
// <offset> : could be either immedaite or register value from GPR
[<offset>(<base>)]!
```

##### Post-increment addressing

The memory access address is obtained by applying <base> directly. The offset, implied in <offset>, is added to the base register, and written back into the base register.

The assembly syntax is illustrated as:

```
// <base> : should always be a register value from GPR
// <offset> : could be either immedaite or register value from GPR
<offset>(<base>!)
```

## 3.2. RV32I Base Integer Instruction Set

RV32I base integer instruction set is designed to be sufficient to form a compiler target and to support modern operating system environments. The ISA was also designed to reduce the hardware required in a minimal implementation.

### 3.2.1. Programmers' Model for Base Integer ISA

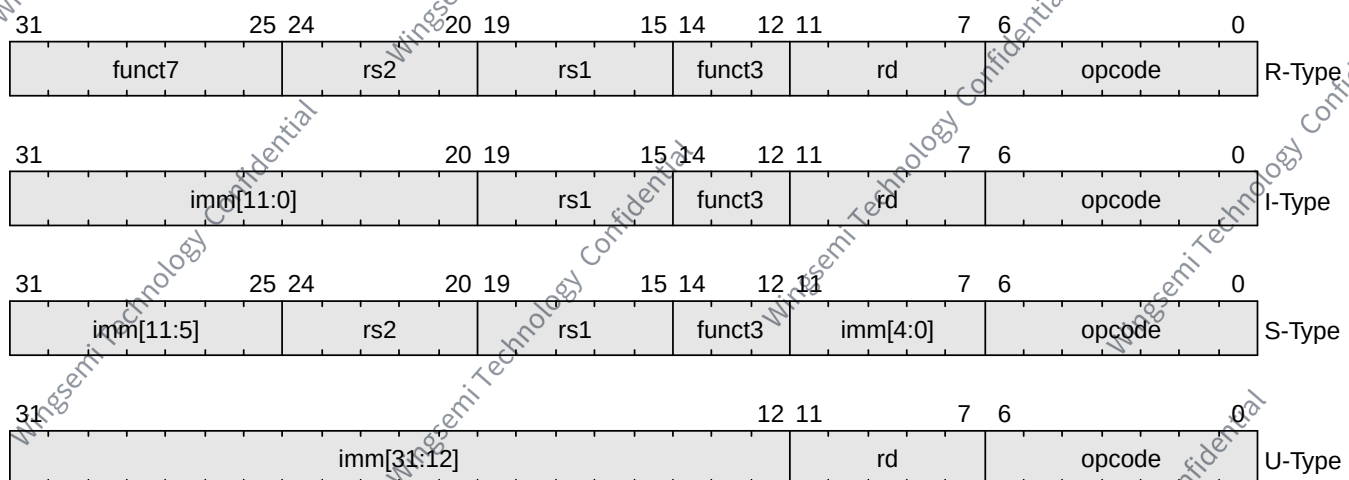
For RV32I, the 32 x registers are each 32-bits width, i.e., XLEN=32. Register x0 is hardwired with all bits equal to 0. General purpose registers x1-x31 hold values that various instructions interpret as a collection of Boolean values, or as two's complement signed binary integers or unsigned binary integers.

There is one additional unprivileged register: the program counter pc holds the address of the current instruction.

The standard software calling convention uses register x1 to hold the return address for a call, with register x5 available as an alternate link register. The standard calling convention uses register x2 as the stack pointer.

### 3.2.2. Base Instruction Formats

In the base RV32I ISA, there are four core instruction formats (R/I/S/U), as shown in the following,



The RISC-V ISA keeps the source (*rs1* and *rs2*) and destination (*rd*) registers at the same position in all formats to simplify decoding. Except for the 5-bit immediates used in CSR instructions, immediates are always sign-extended, and are generally packed towards the leftmost available bits in the instruction and have been allocated to reduce hardware complexity. In particular, the sign bit for all immediates is always in bit 31 of the instruction to speed sign-extension circuitry.



### 3.2.3. ADD

#### Description

ADD performs the addition of *rs1* and *rs2*. Overflows are ignored and the low XLEN bits of results are written to the destination *rd*.

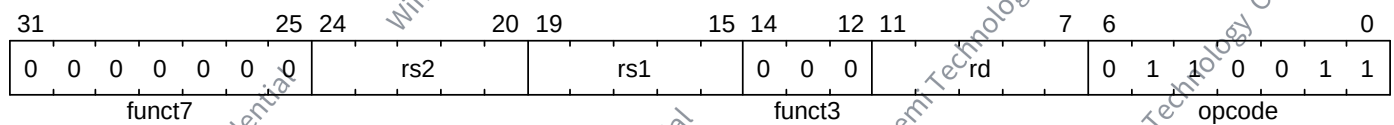
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

add *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
int<XLEN> result;

result = X[rs1] + X[rs2];
X[rd] = result;
```



### 3.2.5. AND

#### Description

AND performs bitwise AND on register *rs1* and the register *rs2*. The result is placed in in *rd*.

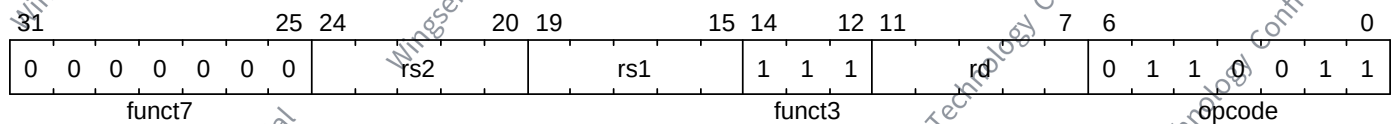
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

and *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
X[rd] = X[rs1] & X[rs2];
```

### Description

## Configuration

### Mnemonic

## Encoding



```
X[rd] = result;
```

### Description

## Configuration

### Mnemonic

## Encoding



Page 41

### 3.2.8. BEQ

#### Description

BEQ takes the branch if registers *rs1* and *rs2* are equal. The target address is calculated by adding sign-extended 12-bit B-immediate in multiples of 2 bytes to the address of the branch instruction. This instruction is used by the Alias BEQZ *rs1*, <label>.

#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

beq *rs1*, *rs2*, <label>

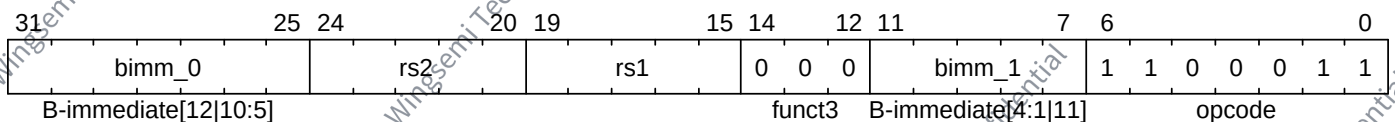
<label> - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int13 offset_t;
```

```
offset_t = bimm_0[6] :: bimm_1[6] :: bimm_0[5:0] :: bimm_1[3:0] :: 1'b0;
```

```
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Alias Condition

| Alias                     | Condition          |
|---------------------------|--------------------|
| BEQZ <i>rs1</i> , <label> | <i>rs2</i> == 5'd0 |

#### Pseudo-code

```
int13 offset_t;
```

```
uint<XLEN> target_pc;
```

```
offset_t = bimm_0[6] :: bimm_1[6] :: bimm_0[5:0] :: bimm_1[3:0] :: 1'b0;
```

```
target_pc = PC + SignExt(offset_t, XLEN);
```

```
#ifndef SUPPORT_RV32C
```

```
    CheckInsnAddrMisalign(target_pc);
```

```
#endif
```

```
if (X[rs1] == X[rs2]) {
```

```
    BranchTo(target_pc, BranchType_DIR, Conditional);
```

```
}
```

### 3.2.9. BGE

#### Description

BGE takes the branch if the signed value in register *rs1* is greater than or equal to that signed value in *rs2*. The target address is calculated by adding sign-extended 12-bit B-immediate in multiples of 2 bytes to the address of the branch instruction. This instruction is used by the Alias BGEZ *rs1*, <label>.

#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

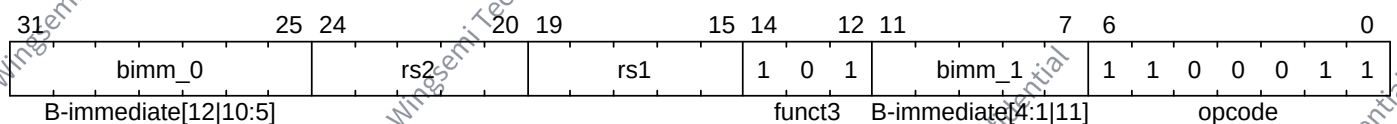
bge *rs1*, *rs2*, <label>

<label> - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int13 offset_t;
```

```
offset_t = bimm_0[6] :: bimm_1[6] :: bimm_0[5:0] :: bimm_1[3:0] :: 1'b0;
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Alias Condition

| Alias                     | Condition          |
|---------------------------|--------------------|
| BGEZ <i>rs1</i> , <label> | <i>rs2</i> == 5'd0 |

#### Pseudo-code

```
int13 offset_t;
int<XLEN> src0, src1;
uint<XLEN> target_pc;

offset_t = bimm_0[6] :: bimm_1[6] :: bimm_0[5:0] :: bimm_1[3:0] :: 1'b0;
target_pc = PC + SignExt(offset_t, XLEN);

if (!HaveCExt()) {
    CheckInsnAddrMisalign(target_pc);
}

src0 = X[rs1];
src1 = X[rs2];
if (src0 >= src1) {
    BranchTo(target_pc, BranchType_DIR, Conditional);
}
```



### 3.2.10. BGEU

#### Description

BGEU takes the branch if the unsigned value in register *rs1* is greater than or equal to that unsigned value in *rs2*. The target address is calculated by adding sign-extended 12-bit B-immediate in multiples of 2 bytes to the address of the branch instruction.

#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

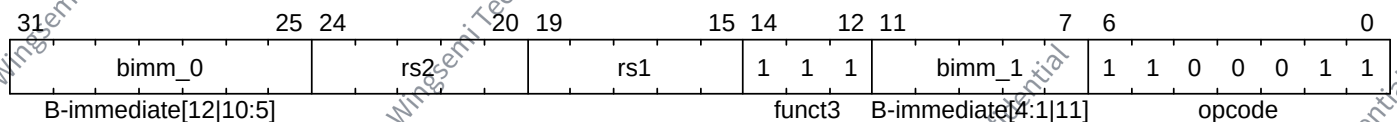
**bgeu** *rs1*, *rs2*, <*label*>

<**label**> - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int13 offset_t;
```

```
offset_t = bimm_0[6] :: bimm_1[6] :: bimm_0[5:0] :: bimm_1[3:0] :: 1'b0;
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Pseudo-code

```
int13 offset_t;
uint<XLEN> src0, src1;
uint<XLEN> target_pc;

offset_t = bimm_0[6] :: bimm_1[6] :: bimm_0[5:0] :: bimm_1[3:0] :: 1'b0;
target_pc = PC + SignExt(offset_t, XLEN);

if (!HaveCExt()) {
    CheckInsnAddrMisalign(target_pc);
}

src0 = X[rs1];
src1 = X[rs2];
if (src0 >= src1) {
    BranchTo(target_pc, BranchType_DIR, Conditional);
}
```

### 3.2.11. BLT

#### Description

BLT takes the branch if the signed value in register *rs1* is less than that signed value in *rs2*. The target address is calculated by adding sign-extended 12-bit B-immediate in multiples of 2 bytes to the address of the branch instruction. This instruction is used by the Alias BLTZ *rs1*, *<label>*.

#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

**blt** *rs1*, *rs2*, *<label>*

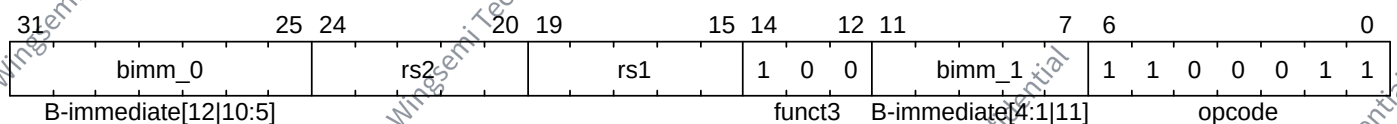
**<label>** - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int13 offset_t;
```

```
offset_t = bimm_0[6] :: bimm_1[6] :: bimm_0[5:0] :: bimm_1[3:0] :: 1'b0;
```

```
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Alias Condition

| Alias                                  | Condition          |
|----------------------------------------|--------------------|
| BLTZ <i>rs1</i> , <i>&lt;label&gt;</i> | <i>rs2</i> == 5'd0 |

#### Pseudo-code

```
int13 offset_t;
int<XLEN> src0, src1;
uint<XLEN> target_pc;
```

```
offset_t = bimm_0[6] :: bimm_1[6] :: bimm_0[5:0] :: bimm_1[3:0] :: 1'b0;
target_pc = PC + SignExt(offset_t, XLEN);
```

```
if (!HaveCExt()) {
    CheckInsnAddrMisalign(target_pc);
}
```

```
src0 = X[rs1];
src1 = X[rs2];
if (src0 < src1) {
    BranchTo(target_pc, BranchType_DIR, Conditional);
}
```

### 3.2.12. BLTU

#### Description

BLTU takes the branch if the unsigned value in register *rs1* is less than that unsigned value in *rs2*. The target address is calculated by adding sign-extended 12-bit B-immediate in multiples of 2 bytes to the address of the branch instruction.

#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

`bltu rs1, rs2, <label>`

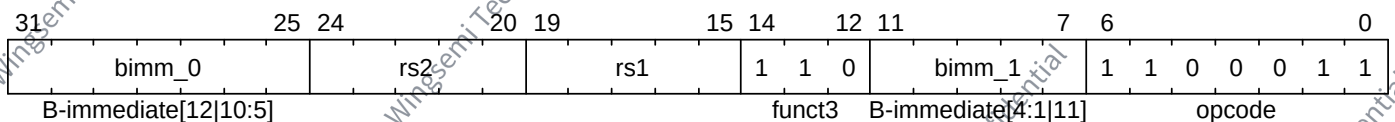
**<label>** - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int13 offset_t;
```

```
offset_t = bimm_0[6] :: bimm_1[6] :: bimm_0[5:0] :: bimm_1[3:0] :: 1'b0;
```

```
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Pseudo-code

```
int13 offset_t;
```

```
uint<XLEN> src0, src1;
```

```
uint<XLEN> target_pc;
```

```
offset_t = bimm_0[6] :: bimm_1[6] :: bimm_0[5:0] :: bimm_1[3:0] :: 1'b0;
```

```
target_pc = PC + SignExt(offset_t, XLEN);
```

```
if (!HaveCExt()) {
```

```
    CheckInsnAddrMisalign(target_pc);
```

```
}
```

```
src0 = X[rs1];
```

```
src1 = X[rs2];
```

```
if (src0 < src1) {
```

```
    BranchTo(target_pc, BranchType_DIR, Conditional);
```

```
}
```

### 3.2.13. BNE

#### Description

BNE takes the branch if registers *rs1* and *rs2* are unequal. The target address is calculated by adding sign-extended 12-bit B-immediate in multiples of 2 bytes to the address of the branch instruction. This instruction is used by the Alias BNEZ *rs1*, <label>.

#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

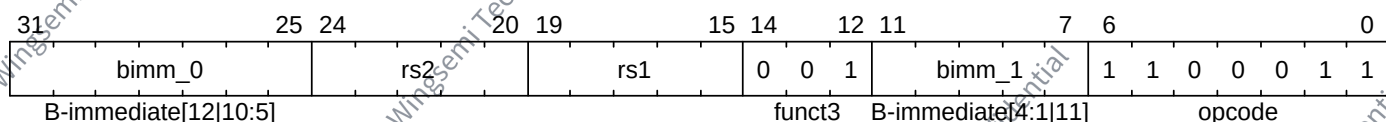
bne *rs1*, *rs2*, <label>

<label> - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int13 offset_t;
```

```
offset_t = bimm_0[6] :: bimm_1[6] :: bimm_0[5:0] :: bimm_1[3:0] :: 1'b0;
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Alias Condition

| Alias                     | Condition          |
|---------------------------|--------------------|
| BNEZ <i>rs1</i> , <label> | <i>rs2</i> == 5'd0 |

#### Pseudo-code

```
int13 offset_t;
uint<XLEN> target_pc;

offset_t = bimm_0[6] :: bimm_1[6] :: bimm_0[5:0] :: bimm_1[3:0] :: 1'b0;

target_pc = PC + SignExt(offset_t, XLEN);

#ifdef SUPPORT_RV32C
    CheckInsnAddrMisalign(target_pc);
#endif

if (X[rs1] != X[rs2]) {
    BranchTo(target_pc, BranchType_DIR, Conditional);
}
```

### Description

## Configuration

### Mnemonic

## Encoding



Page 48

### 3.2.15. ECALL

#### Description

ECALL makes a service request trap taken to the Machine-Mode. When executed in U-mode or M-mode, it generates an *enviroment-call-from-U-mode* or *enviroment-call-from-M-mode* exception.

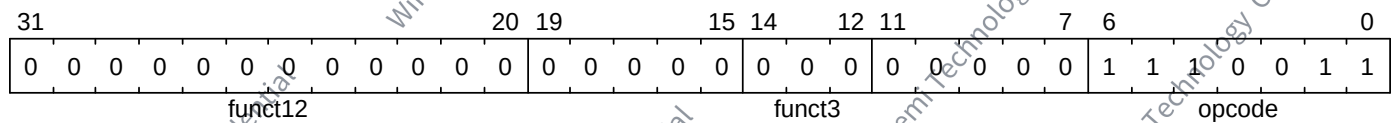
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

ecall

#### Encoding



#### Pseudo-code

```
if (current_priv == PRIV_U) {
    ExceptionTaken(ECALL_FROM_U-MODE);
} else {
    ExceptionTaken(ECALL_FROM_M-MODE);
}
```

### 3.2.16. FENCE

#### Description

The FENCE instruction is used to order device I/O and memory accesses as viewed by other RISC-V harts and external devices or coprocessors. Any combination of device input (I), device output (O), memory reads (R), and memory writes (W) may be ordered with respect to any combination of the same. Informally, no other RISC-V hart or external device can observe any operation in the successor set following a FENCE before any operation in the predecessor set preceding the FENCE.

In Wing-M130 processor, at least one bit in both successor set and predecessor set shall be set, otherwise this instruction is treated as a NOP.

FENCE orders all memory operations in its predecessor set before all memory operations in its successor set.

Additionally, FENCE.TSO orders all load operations in its predecessor set before all memory operations in its successor set, and all store operations in its predecessor set before all store operations in its successor set.

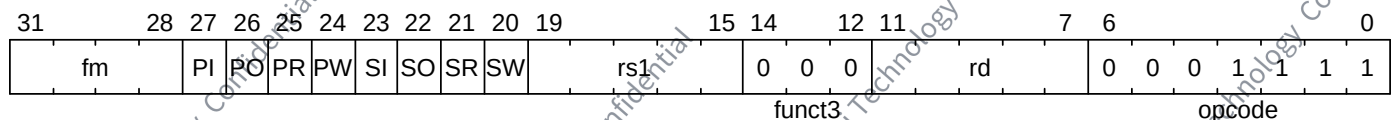
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

fence<.tso>

#### Encoding



#### Pseudo-code

```
DFTYPE type;
uint4 predecessor = PI :: PO :: PR :: PW;
uint4 successor = SI :: SO :: SR :: SW;

if (fm!=4'd0 && fm!=4'd8) {
    ExceptionTaken(ILLEGAL_INSTRUCTION);
} else {
    if (successor==0 || predecessor==0) {
        nop();
    } else {
        if (fm==4'd0) {
            type = DFTYPE_RW;
        } else if (fm==4'd8) {
            type = DFTYPE_R;
        }
        DataFence(type);
    }
}
```



### 3.2.17. JAL

#### Description

JAL uses the J-type format, where the J-immediate encodes a signed offset in multiples of 2 bytes. The offset is sign-extended and added to the address of the jump instruction to form the jump target address. Jumps can therefore target a  $\pm 1\text{MiB}$  range. JAL stores the address of the instruction following the jump (pc+4) into register *rd*. The standard software calling convention uses *x1* as the return register and *x5* as an alternate link register. This instruction is used by the Alias J <label>.

#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

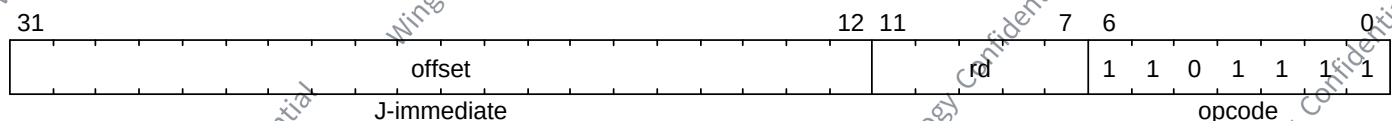
jal *rd*, <label>

<label> - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int21 offset_t;
```

```
offset_t = offset[19] :: offset[7:0] :: offset[8] :: offset[18:9] :: 1'b0;
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Alias Condition

| Alias     | Condition         |
|-----------|-------------------|
| J <label> | <i>rd</i> == 5'd0 |

#### Pseudo-code

```
uint<xlen> target_pc;
int21 offset_t;

offset_t = offset[19] :: offset[7:0] :: offset[8] :: offset[18:9] :: 1'b0;
target_pc = PC + SignExt(offset_t, XLEN);

if (!HaveCExt()) {
    CheckInsnAddrMisalign(target_pc);
}

X[rd] = PC + 4;
if (rd == 5'd1 || rd == 5'd5) {
    BranchTo(target_pc, BranchType_DIRCALL, Unconditional);
} else {
    BranchTo(target_pc, BranchType_DIR, Unconditional);
}
```



```
} else if ((rd==5'd1 && rs1==5'd1) || (rd==5'd5 && rs1==5'd5)) {  
    // Call far-away subroutine using a two-instruction sequence:  
    // lui ra, imm20; jalr ra, imm12(ra), and auipc ra, imm20; jalr ra, imm12(ra)  
    BranchTo(target_pc, BranchType_DIRCALL, Unconditional);  
}  
} else {  
    if (rs1==5'd1 || rs1==5'd5) {  
        // Subroutine return  
        BranchTo(target_pc, BranchType_RET, Unconditional);  
    } else {  
        BranchTo(target_pc, BranchType_INDIR, Unconditional);  
    }  
}  
}
```



### Description

## Configuration

### Mnemonic

## Encoding



Page 55







### Description

## Configuration

### Mnemonic

## Encoding



Page 58

### Description

## Configuration

### Mnemonic

## Encoding



Page 59

### 3.2.25. MRET

#### Description

The `mret` instruction is used to return from a trap taken into M-mode. This instruction can be executed in M-mode only. Executing in U-mode will raise an illegal instruction exception.

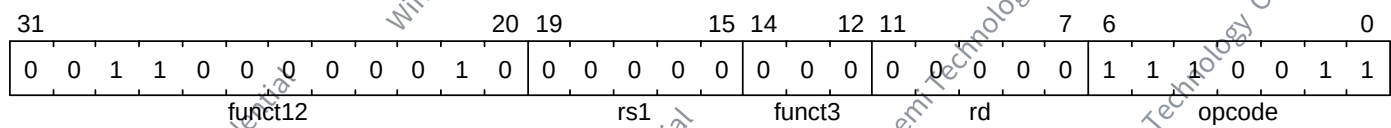
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

`mret`

#### Encoding



#### Pseudo-code

`Return_From_TrapHandler();`

### 3.2.26. OR

**Description**

OR performs bitwise OR on register *rs1* and the register *rs2*. The result is placed in *rd*.

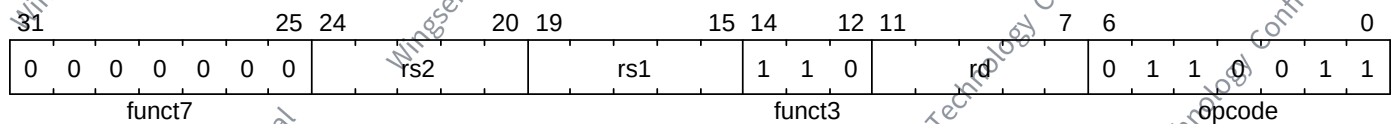
**Configuration**

This instruction is mandatory implemented.

**Mnemonic**

or *rd, rs1, rs2*

**Encoding**



**Pseudo-code**

```
X[rd] = X[rs1] | X[rs2];
```

### Description

## Configuration

### Mnemonic

## Encoding



```
X[rd] = result;
```

### 3.2.28. SB

#### Description

SB stores the lower 8-bit value of *rs2* to memory. The effective address is obtained by adding register *rs1* to the sign-extended 12-bit offset.

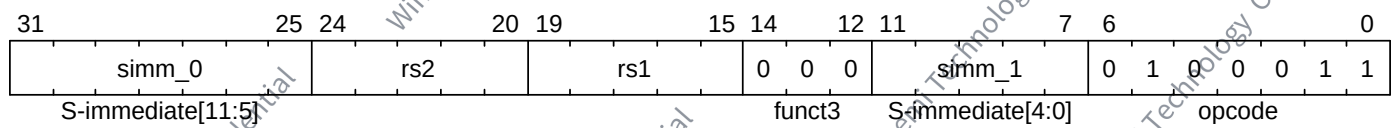
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

sb *rs2*, *rs1*, *simm*

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + SignExt((simm0::simm1), XLEN);
MemStore(X[rs2][7:0], addr_t, Byte, AccType_Normal);
```

### 3.2.29. SH

#### Description

SH stores the lower 16-bit value of *rs2* to memory. The effective address is obtained by adding register *rs1* to the sign-extended 12-bit offset.

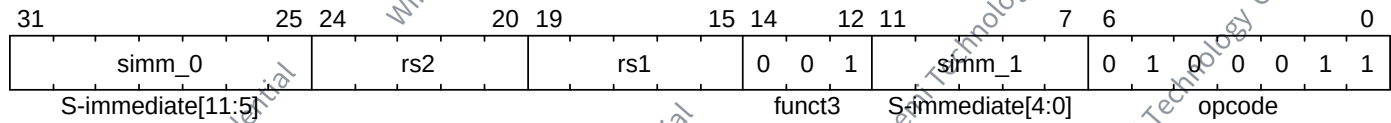
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

sh *rs2*, *rs1*, *simm*

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + SignExt((simm0::simm1), XLEN);
MemStore(X[rs2][15:0], addr_t, HalfWord, AccType_Normal);
```



### 3.2.30. SLL

#### Description

SLL performs a logical left shift on the register *rs1*, the shift amount is held in the lower 5-bits of register *rs2*.

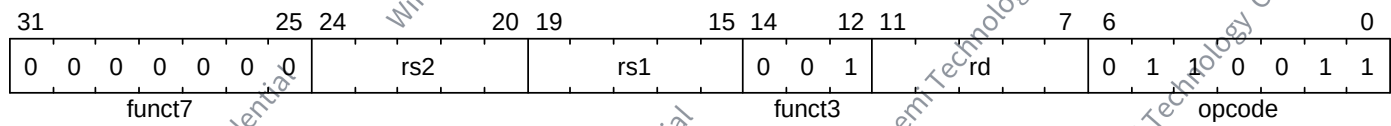
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

sll *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
int5 shamt;  
  
shamt = X[rs2][4:0];  
X[rd] = X[rs1] << shamt;
```

### 3.2.31. SLLI

#### Description

SLLI performs a logical left shift on the register *rs1*, the shift amount is encoded in the *shamt*.

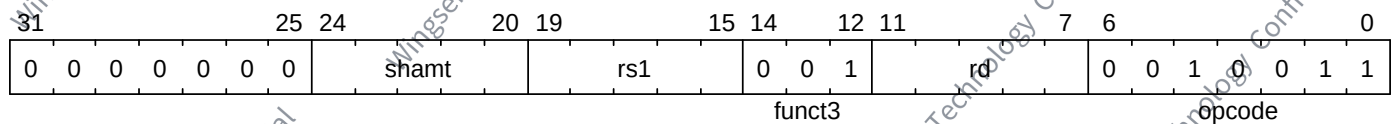
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

`slli rd, rs1, shamt`

#### Encoding



#### Pseudo-code

```
X[rd] = X[rs1] << shamt;
```

### 3.2.32. SLT

#### Description

SLT places the value 1 in register *rd* if register *rs1* is less than register *rs2* when both are treated as signed numbers, else 0 is written to *rd*.

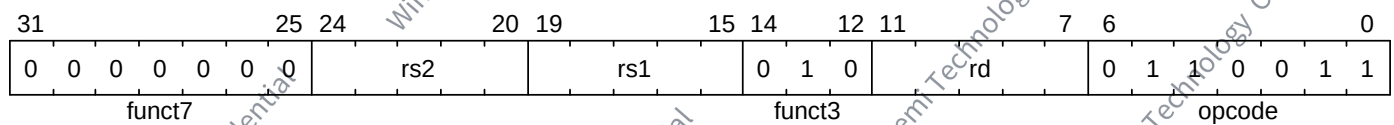
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

slt *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
int<XLEN> src0, src1;

src0 = X[rs1];
src1 = X[rs2];

X[rd] = (src0 < src1) ? 1 : 0;
```

### Description

## Configuration

### Mnemonic

## Encoding



Page 68



### 3.2.35. SLTU

#### Description

SLTU places the value 1 in register *rd* if register *rs1* is less than register *rs2* when both are treated as unsigned numbers, else 0 is written to *rd*. This instruction is used by the Alias SNEZ *rd, rs*.

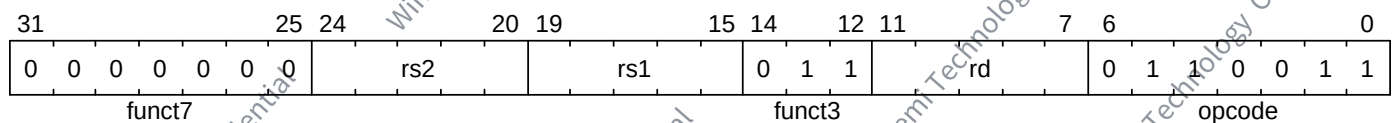
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

sltu *rd, rs1, rs2*

#### Encoding



#### Alias Condition

| Alias              | Condition          |
|--------------------|--------------------|
| SNEZ <i>rd, rs</i> | <i>rs1</i> == 5'd0 |

#### Pseudo-code

```
uint<XLEN> src0, src1;

src0 = X[rs1];
src1 = X[rs2];

X[rd] = (src0 < src1) ? 1 : 0;
```

### 3.2.36. SRA

#### Description

SRA performs an arithmetic right shift on the register *rs1*, the shift amount is held in the lower 5-bits of register *rs2*.

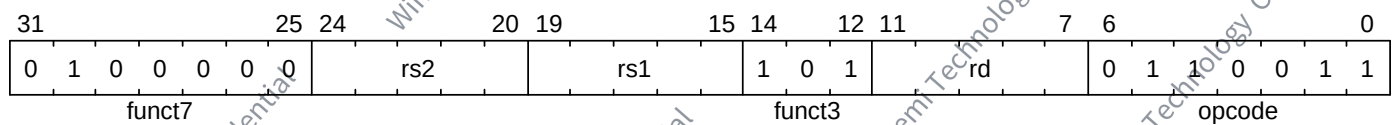
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

*sra rd, rs1, rs2*

#### Encoding



#### Pseudo-code

```
int5 shamt;
int<XLEN> src0;

shamt = X[rs2][4:0];
src0 = X[rs1];
X[rd] = X[rs1] >> shamt;
```

### 3.2.37. SRAI

#### Description

SRAI performs an arithmetic right shift on the register *rs1*, the shift amount is encoded in the *shamt*.

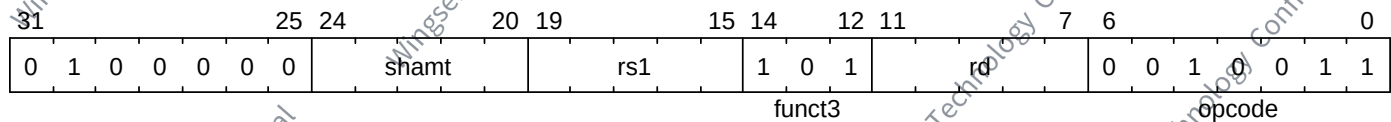
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

srai *rd*, *rs1*, *shamt*

#### Encoding



#### Pseudo-code

```
int<XLEN> src0;

src0 = X[rs1];
X[rd] = src0 >> shamt;
```



### 3.2.38. SRL

#### Description

SRL performs a logical right shift on the register *rs1*, the shift amount is held in the lower 5-bits of register *rs2*.

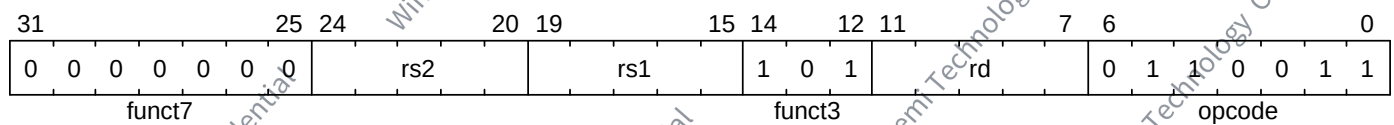
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

srl *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
int5 shamt;
uint<XLEN> src0;

shamt = X[rs2][4:0];
src0 = X[rs1];
X[rd] = X[rs1] >> shamt;
```

### 3.2.39. SRLI

#### Description

SRLI performs a logical right shift on the register *rs1*, the shift amount is encoded in the *shamt*.

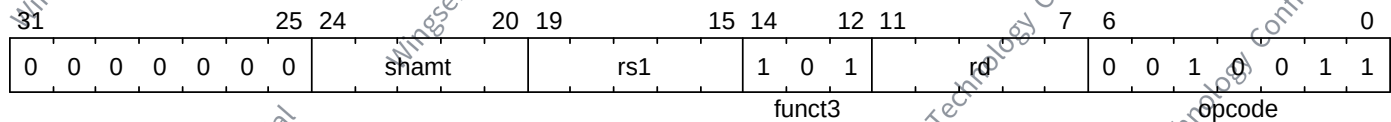
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

*srl* *rd*, *rs1*, *shamt*

#### Encoding



#### Pseudo-code

```
uint<XLEN> src0;

src0 = X[rs1];
X[rd] = src0 >> shamt;
```

### 3.2.40. SUB

#### Description

SUB performs the subtraction of *rs2* from *rs1*. Overflows are ignored and the low XLEN bits of results are written to the destination *rd*.

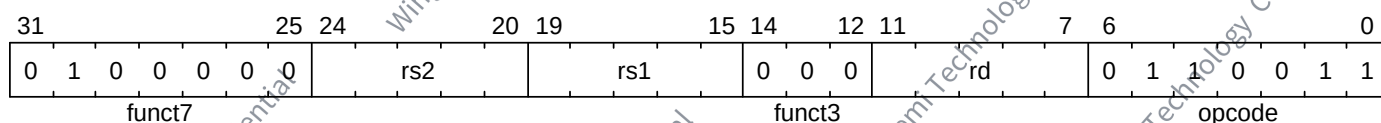
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

sub *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
int<XLEN> result;

result = (int<XLEN>) X[rs1] - (int<XLEN>) X[rs2];
X[rd] = result;
```

### 3.2.41. SW

#### Description

SW stores the 32-bit value of *rs2* to memory. The effective address is obtained by adding register *rs1* to the sign-extended 12-bit offset.

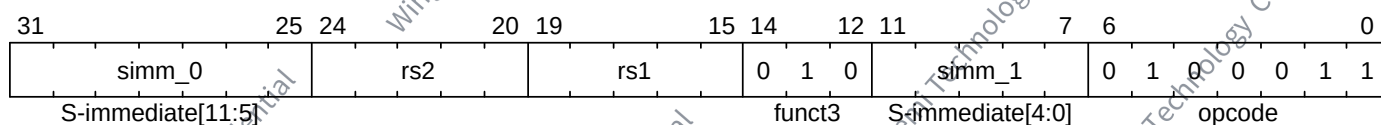
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

sw *rs2*, *rs1*, *simm*

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + SignExt((simm0::simm1), XLEN);
MemStore(X[rs2][31:0], addr_t, Word, AccType_Normal);
```

### 3.2.42. WFI

#### Description

The Wait For Interrupt instruction informs the implementation that the current hart can be stalled until an interrupt might need servicing.

#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

wfi

#### Encoding

|    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |    |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|----|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|----|----|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|
| 31 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | 20 | 19 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|----|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|----|----|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|

#### Pseudo-code

```
if (current_priv == PRIV_U) {
    ExceptionTaken(ILLEGAL_INSTRUCTION);
}
```

```
WaitForInt();
```

### 3.2.43. XOR

#### Description

XOR performs bitwise XOR on register *rs1* and the register *rs2*. The result is placed in *rd*.

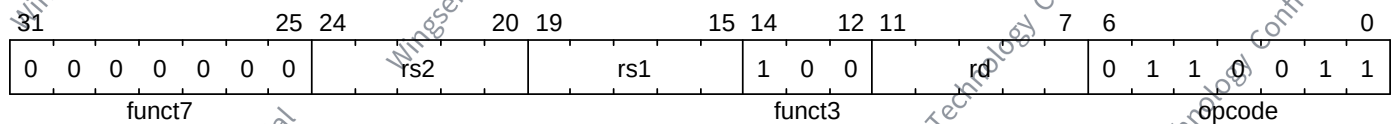
#### Configuration

This instruction is mandatory implemented.

#### Mnemonic

xor *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
X[rd] = X[rs1] ^ X[rs2];
```



### 3.3. RV32E Base Integer Instruction Set

RV32E base integer instruction set is designed for microcontrollers in embedded systems. RV32E is reduced versions of RV32I: the only change is to reduce the number of integer registers to 16.

#### 3.3.1. RV32E Programmers's Model

RV32E reduces the integer register count to 16 general-purpose registers (x0-x15), where x0 is a dedicated zero register.

#### 3.3.2. RV32E Instruction Set Encoding

RV32E uses the same instruction-set encoding as RV32I, except that only register x0-x15 are provided. All encodings specifying the other registers x16-31 are reserved. When used, an illegal instruction exception is raised.





### 3.5. "M" Standard Extension for Integer Multiplication and Division

This chapter describes the standard integer multiplication and division instruction extension, which contains instructions that multiply or divide values held in two integer registers.

In addition, RISC-V provides the zmmul extension which implements the multiplication subset of the M-extension. It adds all of the multiplication instructions. The encodings are identical to those of the corresponding M-extension instructions.

### 3.5.1. DIV

#### Description

DIV performs an XLEN bits by XLEN bits signed integer division of *rs1* by *rs2*, rounding towards zero.

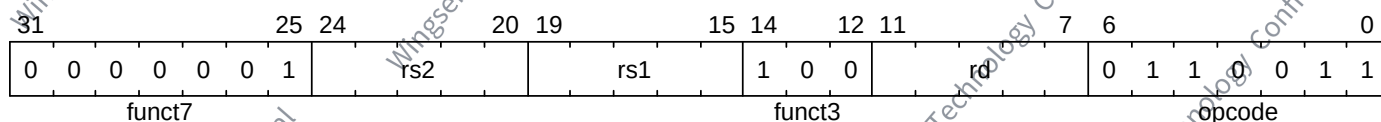
#### Configuration

This instruction is present only when M-extension is configured.

#### Mnemonic

div *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
int32 src0, src1;

src0 = X[rs1];
src1 = X[rs2];

if (src1 == 32'd0) {
    X[rd] = 32'hffffffff;
} else if (src0==32'h80000000 && src1==32'hffffffff) {
    X[rd] = 32'h80000000;
} else {
    X[rd] = src0 / src1;
}
```

### 3.5.2. DIVU

#### Description

DIVU performs an XLEN bits by XLEN bits unsigned integer division of *rs1* by *rs2*, rounding towards zero.

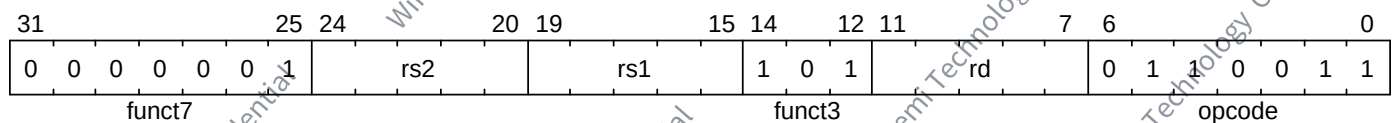
#### Configuration

This instruction is present only when M-extension is configured.

#### Mnemonic

divu *rd, rs1, rs2*

#### Encoding



#### Pseudo-code

```
uint32 src0, src1;

src0 = X[rs1];
src1 = X[rs2];

if (src1 == 32'd0) {
    X[rd] = 32'hffffffff;
} else {
    X[rd] = src0 / src1;
}
```

### 3.5.3. REM

#### Description

REM performs an XLEN bits by XLEN bits signed integer division of *rs1* by *rs2*. The remainder is stored into *rd*.

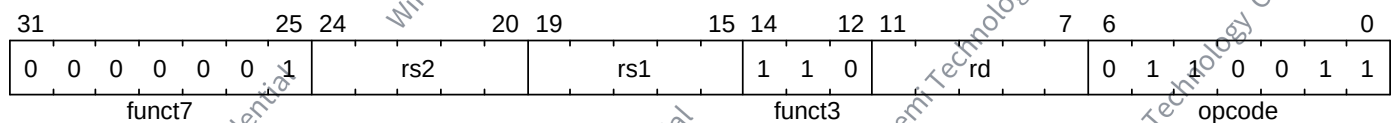
#### Configuration

This instruction is present only when M-extension is configured.

#### Mnemonic

rem *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
int32 src0, src1;

src0 = X[rs1];
src1 = X[rs2];

if (src1 == 32'd0) {
    X[rd] = src0;
} else if (src0==32'h80000000 && src1==32'hffffffff) {
    X[rd] = 32'd0;
} else {
    X[rd] = src0 % src1;
}
```

### 3.5.4. REMU

#### Description

REMU performs an XLEN bits by XLEN bits unsigned integer division of *rs1* by *rs2*. The remainder is stored into *rd*.

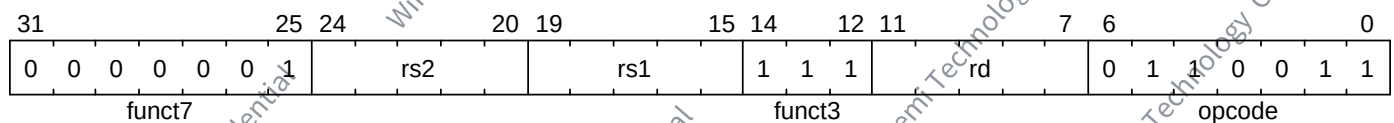
#### Configuration

This instruction is present only when M-extension is configured.

#### Mnemonic

`remu rd, rs1, rs2`

#### Encoding



#### Pseudo-code

```
uint32 src0, src1;

src0 = X[rs1];
src1 = X[rs2];

if (src1 == 32'd0) {
    X[rd] = src0;
} else {
    X[rd] = src0 % src1;
}
```

### 3.5.5. MUL

#### Description

MUL performs an XLEN-bit  $\times$  XLEN-bit multiplication of *rs1* by *rs2* and places the lower XLEN bits in the destination register *rd*.

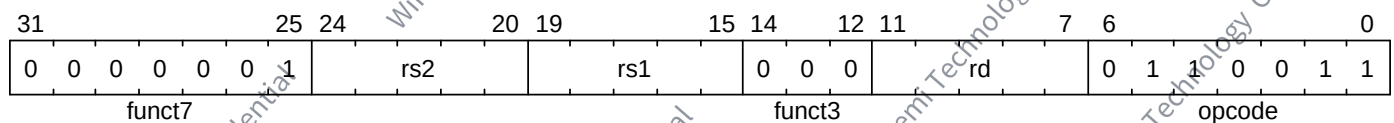
#### Configuration

This instruction is present only when M-extension or zmmul is configured.

#### Mnemonic

mul *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
int<XLEN> src0, src1;
int<2*XLEN> result;

src0 = X[rs1];
src1 = X[rs2];
result = src0 * src1;

X[rd] = result[XLEN-1:0];
```

### 3.5.6. MULH

#### Description

MULH performs a signed XLEN-bit  $\times$  signed XLEN-bit multiplication of *rs1* by *rs2* and places the higher XLEN bits in the destination register *rd*.

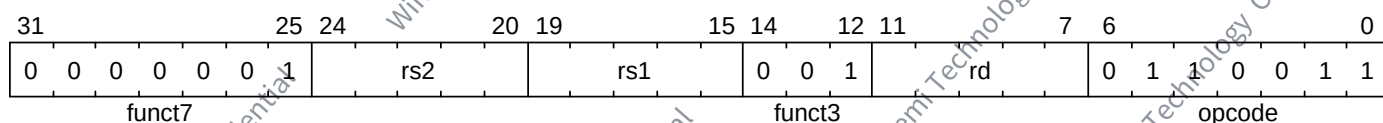
#### Configuration

This instruction is present only when M-extension or zmmul is configured.

#### Mnemonic

`mulh rd, rs1, rs2`

#### Encoding



#### Pseudo-code

```
int<XLEN> src0, src1;
int<2*XLEN> result;

src0 = X[rs1];
src1 = X[rs2];
result = src0 * src1;

X[rd] = result[2*XLEN-1:XLEN];
```



### 3.5.7. MULHSU

#### Description

MULHSU performs a signed XLEN-bit  $\times$  an unsigned XLEN-bit multiplication of *rs1* by *rs2* and places the higher XLEN bits in the destination register *rd*.

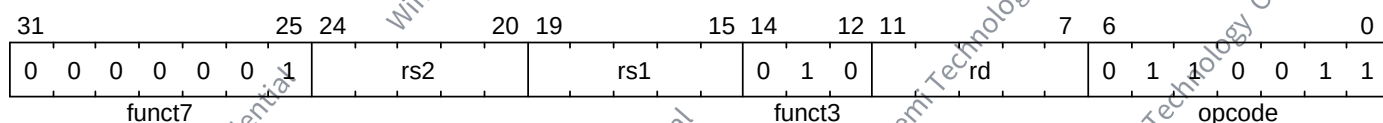
#### Configuration

This instruction is present only when M-extension or zmmul is configured.

#### Mnemonic

`mulhsu rd, rs1, rs2`

#### Encoding



#### Pseudo-code

```
int<XLEN> src0;
uint<XLEN> src1;
int<2*XLEN> result;

src0 = X[rs1];
src1 = X[rs2];
result = src0 * src1;

X[rd] = result[2*XLEN-1:XLEN];
```

### 3.5.8. MULHU

#### Description

MULHU performs an unsigned XLEN-bit  $\times$  an unsigned XLEN-bit multiplication of *rs1* by *rs2* and places the higher XLEN bits in the destination register *rd*.

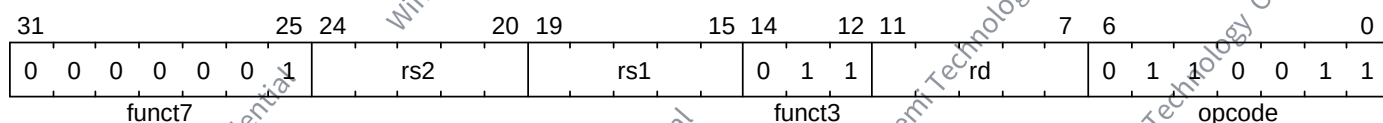
#### Configuration

This instruction is present only when M-extension or zmmu1 is configured.

#### Mnemonic

`mulhu rd, rs1, rs2`

#### Encoding



#### Pseudo-code

```
uint<XLEN> src0, src1;
uint<2*XLEN> result;

src0 = X[rs1];
src1 = X[rs2];
result = src0 * src1;

X[rd] = result[2*XLEN-1:XLEN];
```

### 3.6. "C" Standard Extension for Compressed Instructions

The RISC-V standard compressed instruction-set extension, named "C", which reduces static and dynamic code size by adding short 16-bit instruction encodings for common operations.

RVC uses a simple compression scheme that offers shorter 16-bit version of common 32-bit RISC-V instructions when:

- the immediate or address offset is small, or
- one of the registers is the zero register ( $x0$ ), the ABI link register ( $x1$ ), or the ABI stack pointer ( $x2$ ), or
- the destination register and the first source register are identical, or
- the registers used are the 8 most popular ones.

The C-extension is compatible with all other standard instruction extensions. The C-extension allows 16-bit instructions to be freely intermixed with 32-bit instructions, with the latter now able to start on any 16-bit boundary. With the addition of the C-extension, no instructions can raise instruction-address-misaligned exceptions.

C-extension specifies additional three-bit register operands, namely  $rs1'$ ,  $rs2'$  and  $rd'$ , which encodes 8 most popular register from  $x8$  to  $x15$  respectively.

### 3.6.1. C.ADD

#### Description

This instruction adds the values in register *rd* and *rs2* (*rs2* ≠ *x0*) and writes the result to register *rd*.

#### NOTE

The code points with *rs2* = *x0* correspond to the [C.JALR](#) and [C.EBREAK](#) instructions.

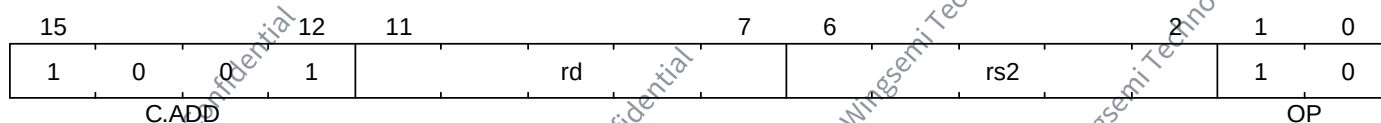
#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

**cadd** *rd, rs2*

#### Encoding



#### Pseudo-code

```
X[rd] = X[rd] + X[rs2];
```

### 3.6.2. C.ADDI

#### Description

This instruction adds the non-zero sign-extended 6-bit immediate to the value in register *rd* (*rd* ≠ 0) then writes the result to *rd*.

#### NOTE

The code point with *rd* = 0 corresponds to the [C.NOP](#) instruction.

#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

c.addi *rd*, *imm*

#### Encoding

| 15     | 13 | 12 | 11     |  | 7  | 6 |  | 2        | 1  | 0 |
|--------|----|----|--------|--|----|---|--|----------|----|---|
| 0      | 0  | 0  | imm[5] |  | rd |   |  | imm[4:0] | 0  | 1 |
| C.ADDI |    |    |        |  |    |   |  |          | OP |   |

#### Pseudo-code

```
X[rd] = X[rd] + SignExt(imm, XLEN);
```

### 3.6.3. C.ADDI16SP

#### Description

This instruction adds the non-zero sign-extended 6-bit immediate to the value in the stack pointer (x2), where the immediate is scaled to represent multiples of 16 in the range (-512, 496). When *nzimm* = 0, an illegal instruction fault will issue.

#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

`caddi16sp nzimm`

**<nzimm>** - nzimm is encoded as 16 times of the immediate in the range (-512, 496), i.e.

```
nzimm = imm[5] :: imm[2:1] :: imm[3] :: imm[0] :: imm[4] :: 4'b0;
```

#### Encoding

|          |    |    |        |    |   |   |   |                  |          |   |   |    |  |
|----------|----|----|--------|----|---|---|---|------------------|----------|---|---|----|--|
| 15       | 13 | 12 | 11     | 7  | 6 | 2 | 1 | 0                |          |   |   |    |  |
| 0        | 1  | 1  | imm[5] | 0  | 0 | 0 | 1 | 0                | imm[4:0] | 0 | 1 |    |  |
| nzimm[9] |    |    |        | rd |   |   |   | nzimm[4 6 8:7 5] |          |   |   | OP |  |

#### Pseudo-code

```
int10 nzimm;
nzimm = imm[5] :: imm[2:1] :: imm[3] :: imm[0] :: imm[4] :: 4'b0;

if (nzimm==0) {
    ExceptionTaken(ILLEGAL_INSTRUCTION);
} else {
    X[2] = X[2] + SignExt(nzimm, XLEN);
}
```

### 3.6.4. C.ADDI4SPN

#### Description

This instruction adds a zero-extended non-zero immediate, scaled by 4, to the stack pointer x2, and writes the result to *rd'*. This instruction is used to generate pointers to stack-allocated variables. When *nzuimm* = 0, an illegal instruction fault will issue.

#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

`c.addi4spn rd', nzuimm`

**<nzuimm>** - nzuimm is encoded as 4 times of the immediate, i.e.

```
nzuimm = imm[5:2] :: imm[7:6] :: imm[0] :: imm[1] :: 2'b0;
```

#### Encoding

| 15         | 13 | 12 |                     | 5   | 4 | 2   | 1 | 0 |
|------------|----|----|---------------------|-----|---|-----|---|---|
| 0          | 0  | 0  |                     | imm |   | rd' | 0 | 0 |
| C.ADDI4SPN |    |    | nzuimm[5:4 9:6 2 3] |     |   | OP  |   |   |

#### Pseudo-code

```
int10 nzuimm;
nzuimm = imm[5:2] :: imm[7:6] :: imm[0] :: imm[1] :: 2'b0;

if (nzuimm==0) {
    ExceptionTaken(ILLEGAL_INSTRUCTION);
} else {
    X[rd'+8] = X[2] + ZeroExt(nzuimm, XLEN);
}
```

### 3.6.5. C.AND

**Description**

This instruction computes the bitwise AND of the values in registers *rd'* and *rs2'*, the writes the result to register *rd'*.

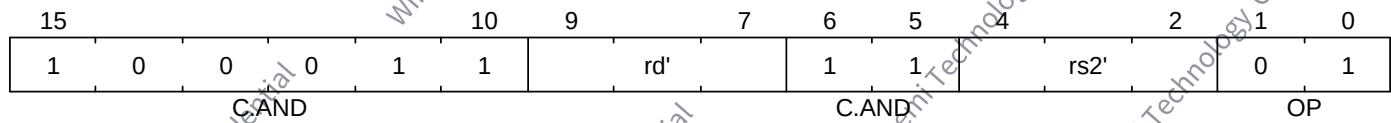
**Configuration**

This instruction is present only when C-extension is configured.

**Mnemonic**

c.and *rd'*, *rs2'*

**Encoding**



**Pseudo-code**

```
X[rd'+8] = X[rd'+8] & X[rs2'+8];
```



### 3.6.6. C.ANDI

#### Description

This instruction computes the bitwise AND of the value in register *rd'* and the sign-extended 6-bit immediate, then writes the result to *rd'*.

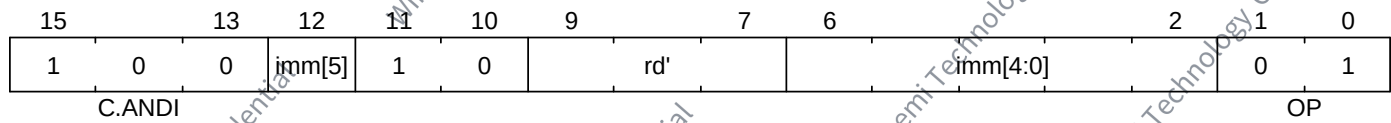
#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

*c.andi rd', imm*

#### Encoding



#### Pseudo-code

```
X[rd'+8] = X[rd'+8] & SignExt(imm, XLEN);
```

### 3.6.7. C.BEQZ

#### Description

This instruction performs conditional control transfer. The offset is sign-extended and added to PC to form the branch target address. It can therefore target a  $\pm 256B$  range. C.BEQZ takes the branch if the value in register *rs1'* is zero.

#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

`c.beqz rs1', <label>`

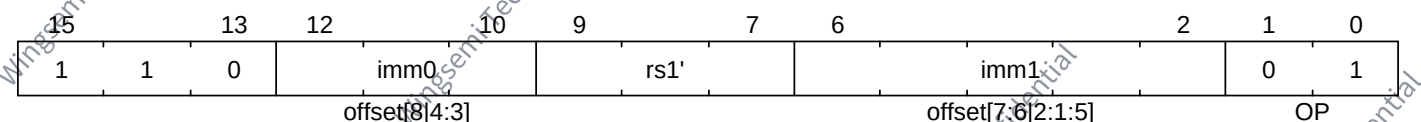
**<label>** - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int9 offset_t;
```

```
offset_t = imm0[2] :: imm1[4:3] :: imm1[0] :: imm0[1:0] :: imm1[2:1] :: 1'b0;
```

```
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Pseudo-code

```
int9 offset_t;
```

```
uint<XLEN> target_pc;
```

```
offset_t = imm0[2] :: imm1[4:3] :: imm1[0] :: imm0[1:0] :: imm1[2:1] :: 1'b0;
```

```
target_pc = PC + SignExt(offset_t, XLEN);
```

```
if (X[rs1'+8] == 0) {
```

```
    BranchTo(target_pc, BranchType_DIR, Conditional);
```

```
}
```

### 3.6.8. C.BNEZ

#### Description

This instruction performs conditional control transfer. The offset is sign-extended and added to PC to form the branch target address. It can therefore target a  $\pm 256B$  range. C.BNEZ takes the branch if the value in register *rs1'* is not zero.

#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

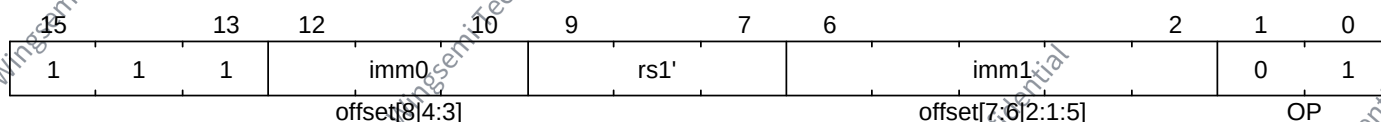
`c.bnez rs1', <label>`

**<label>** - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int9 offset_t;
```

```
offset_t = imm0[2] :: imm1[4:3] :: imm1[0] :: imm0[1:0] :: imm1[2:1] :: 1'b0;
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Pseudo-code

```
int9 offset_t;
uint<XLEN> target_pc;

offset_t = imm0[2] :: imm1[4:3] :: imm1[0] :: imm0[1:0] :: imm1[2:1] :: 1'b0;

target_pc = PC + SignExt(offset_t, XLEN);

if (X[rs1'+8] != 0) {
    BranchTo(target_pc, BranchType_DIR, Conditional);
}
```

### 3.6.9. C.EBREAK

**Description**

This instruction causes control to be transferred back to the debugging environment.

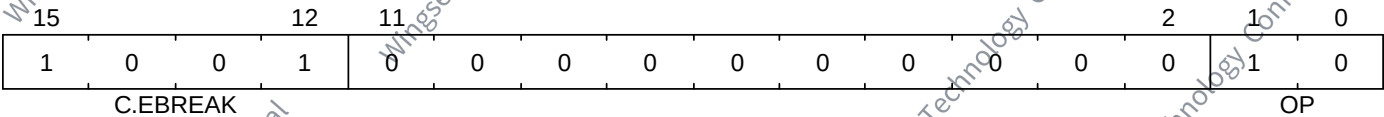
**Configuration**

This instruction is present only when C-extension is configured.

**Mnemonic**

c.ebreak

**Encoding**



**Pseudo-code**

```
uint<XLEN> exception_return = PC;

ExceptionTaken(SOFTWARE_BREAKPOINT, exception_return);
```

### 3.6.10. C.J

#### Description

This instruction performs an unconditional control transfer. The offset is sign-extended and added to the PC to form the jump target address. C.J can therefore target a  $\pm 2\text{KiB}$  range.

#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

c.j <label>

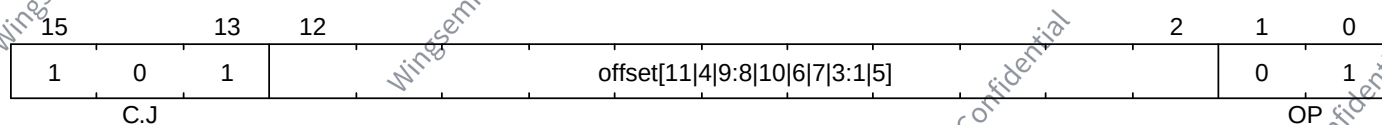
<label> - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int12 offset_t;

offset_t = offset[10]::offset[6]::offset[8:7]::offset[4]::
          offset[5]::offset[0]::offset[9]::offset[3:1]::1'b0;

<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Pseudo-code

```
uint<xlen> target_pc;
int12 offset_t;

offset_t = offset[10]::offset[6]::offset[8:7]::offset[4]::
          offset[5]::offset[0]::offset[9]::offset[3:1]::1'b0;
target_pc = PC + SignExt(offset_t);

BranchTo(target_pc, BranchType_DIR, Unconditional);
```

### 3.6.11. C.JAL

#### Description

This instruction performs an unconditional control transfer. The offset is sign-extended and added to the PC to form the jump target address. C.JAL can therefore target a  $\pm 2\text{KiB}$  range. Additionally, the address of the instruction following the jump (PC+2) is written to the link register  $x1$ .

#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

`cjal <label>`

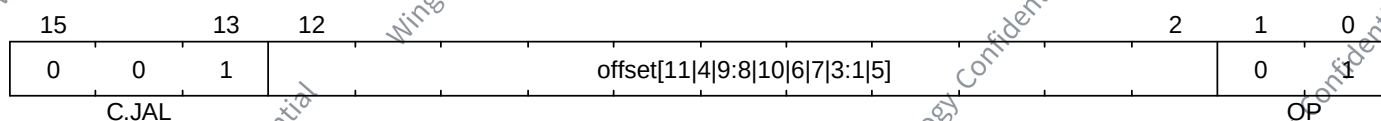
**<label>** - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int12 offset_t;

offset_t = offset[10]::offset[6]::offset[8:7]::offset[4]::
          offset[5]::offset[0]::offset[9]::offset[3:1]::1'b0;

<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Pseudo-code

```
uint<xlen> target_pc;
int12 offset_t;

offset_t = offset[10]::offset[6]::offset[8:7]::offset[4]::
          offset[5]::offset[0]::offset[9]::offset[3:1]::1'b0;
target_pc = PC + SignExt(offset_t);

X[1] = PC + 2;
BranchTo(target_pc, BranchType_DIRCALL, Unconditional);
```

### 3.6.12. C.JALR

#### Description

This instruction performs an unconditional control transfer to the address in register *rs1*. C.JALR is only valid when *rs1* ≠ x0. Additionally, the address of the instruction following the jump (PC+2) is written to the link register x1.

**NOTE** The code point with *rs1* = x0 corresponds to the [C.EBREAK](#) instruction.

#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

c.jalr *rs1*

#### Encoding

| 15     |  |   |  | 12 |  |   |  | 11  |  |  |  | 7 |  |   |  | 6  |  |   |  | 2 |  |   |  | 1 |  | 0 |  |
|--------|--|---|--|----|--|---|--|-----|--|--|--|---|--|---|--|----|--|---|--|---|--|---|--|---|--|---|--|
| 1      |  | 0 |  | 0  |  | 1 |  | rs1 |  |  |  | 0 |  | 0 |  | 0  |  | 0 |  | 0 |  | 1 |  | 0 |  |   |  |
| C.JALR |  |   |  |    |  |   |  |     |  |  |  |   |  |   |  | OP |  |   |  |   |  |   |  |   |  |   |  |

#### Pseudo-code

```
uint<xlen> target_pc = X[rs1];
```

```
X[1] = PC + 2;
```

```
if (rs1==5'd5) {
    // Indirect subroutine return and call (coroutines)
    BranchTo(target_pc, BranchType_RETCALL, Unconditional);
} else {
    // Indirect subroutine call to an address in rs1
    BranchTo(target_pc, BranchType_INDIRCALL, Unconditional);
}
```

### 3.6.13. C.JR

#### Description

This instruction performs an unconditional control transfer to the address in register *rs1*. C.JR is only valid when *rs1*  $\neq$  x0. Otherwise, an illegal instruction fault will issue.

#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

c.jr *rs1*

#### Encoding

|     |  |   |  |    |  |   |  |     |  |  |  |   |  |  |  |    |  |   |  |   |  |   |  |   |  |   |  |
|-----|--|---|--|----|--|---|--|-----|--|--|--|---|--|--|--|----|--|---|--|---|--|---|--|---|--|---|--|
| 15  |  |   |  | 12 |  |   |  | 11  |  |  |  | 7 |  |  |  | 6  |  | 2 |  |   |  | 1 |  | 0 |  |   |  |
| 1   |  | 0 |  | 0  |  | 0 |  | rs1 |  |  |  |   |  |  |  | 0  |  | 0 |  | 0 |  | 0 |  | 1 |  | 0 |  |
| CJR |  |   |  |    |  |   |  |     |  |  |  |   |  |  |  | OP |  |   |  |   |  |   |  |   |  |   |  |

#### Pseudo-code

```
uint<xlen> target_pc = X[rs1];

if (rs1==5'd0) {
    RaiseException(IllegalInstruction);
} else {
    if (rs1==5'd1 || rs1==5'd5) {
        // Subroutine return
        BranchTo(target_pc, BranchType_RET, Unconditional);
    } else {
        BranchTo(target_pc, BranchType_INDIR, Unconditional);
    }
}
```



### 3.6.14. C.LI

#### Description

This instruction loads the sign-extended 6-bit immediate into the register *rd*.

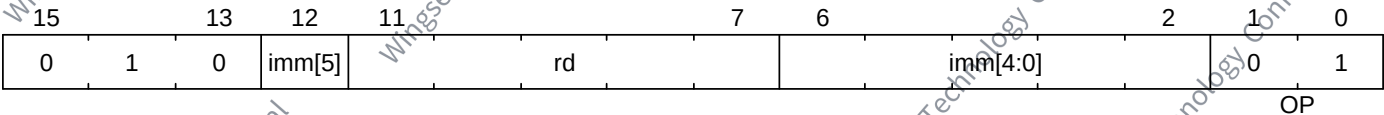
#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

*c.li rd, imm*

#### Encoding



#### Pseudo-code

```
X[rd] = SignExt(imm, XLEN);
```

### 3.6.15. C.LUI

#### Description

This instruction loads the non-zero 6-bit immediate field into bits 17-12 of the destination register *rd* (*rd* ≠ x2), clears the bottom bits, and sign-extends bit 17 into all higher bits of the destination. When *imm* = 0, an illegal instruction fault will issue.

**NOTE** The code point with *rd* = x2 corresponds to the [C.ADDI16SP](#) instruction.

#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

c.lui *rd*, *imm*

#### Encoding

|    |    |    |        |    |   |          |   |     |
|----|----|----|--------|----|---|----------|---|-----|
| 15 | 13 | 12 | 11     | 7  | 6 | 2        | 1 | 0   |
| 0  | 1  | 1  | imm[5] | rd |   | imm[4:0] |   | 0 1 |
| OP |    |    |        |    |   |          |   |     |

#### Pseudo-code

```
int18 imm_t = imm :: 12'b0;

if (imm==6'b0) {
    ExceptionTaken(ILLEGAL_INSTRUCTION);
} else {
    X[rd] = SignExt(imm_t, XLEN);
}
```

### 3.6.16. C.LW

#### Description

This instruction loads a 32-bit value from memory into register *rd'*. It computes an effective address by adding the zero-extended offset, scaled by 4, to the base address in register *rs1'*.

#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

`c.lw rd', offset(rs1')`

#### Encoding

|      |    |    |             |      |          |     |             |    |   |   |   |
|------|----|----|-------------|------|----------|-----|-------------|----|---|---|---|
| 15   | 13 | 12 | 10          | 9    | 7        | 6   | 5           | 4  | 2 | 1 | 0 |
| 0    | 1  | 0  | imm[4:2]    | rs1' | imm[1:0] | rd' | 0           | 0  |   |   |   |
| C.LW |    |    | offset[5:3] |      |          |     | offset[2:6] | OP |   |   |   |

#### Pseudo-code

```
uint<XLEN> addr_t;
uint7 offset_t;
offset_t = imm[0] :: imm[4:2] :: imm[1] :: 2'b0;

addr_t = X[rs1'+8] + ZeroExt(offset_t, XLEN);
X[rd'+8] = MemLoad(addr_t, Word, AccType_Normal);
```

### 3.6.17. C.LWSP

#### Description

This instruction loads a 32-bit value from memory into register *rd*. It computes an effective address by adding the zero-extended offset, scaled by 4, to the stack pointer, *x2* (or *sp*). When *rd* = *x0*, an illegal instruction fault will issue.

#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

`c.lwsp rd, offset(sp)`

#### Encoding

| 15     | 13 | 12 | 11        | 7  | 6 | 2               | 1  | 0 |
|--------|----|----|-----------|----|---|-----------------|----|---|
| 0      | 1  | 0  | imm[5]    | rd |   | imm[4:0]        | 1  | 0 |
| C.LWSP |    |    | offset[5] |    |   | offset[4:2]7:6] | OP |   |

#### Pseudo-code

```
uint<XLEN> addr_t;
uint8 offset_t;
offset_t = imm[1:0] :: imm[5] :: imm[4:2] :: 2'b0;

addr_t = X[2] + ZeroExt(offset_t, XLEN);
X[rd] = MemLoad(addr_t, Word, AccType_Normal);
```

### 3.6.18. C.MV

#### Description

This instruction copies the value in register *rs2* (*rs2* ≠ *x0*) into register *rd*.

**NOTE** The code points with *rs2* = *x0* correspond to the [C.JR](#) instruction.

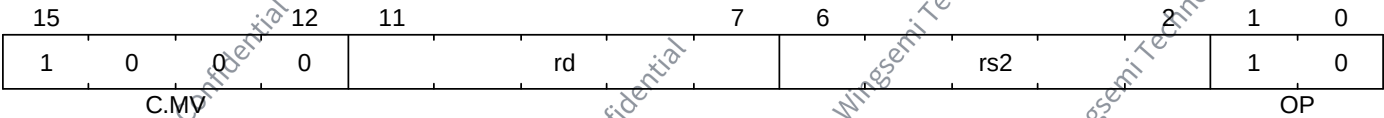
#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

**c**mv *rd*, *rs2*

#### Encoding



#### Pseudo-code

```
X[rd] = X[rs2];
```

### 3.6.19. C.NOP

#### Description

This instruction does not change any user-visible state, except for advancing the PC and incrementing any applicable performance counters.

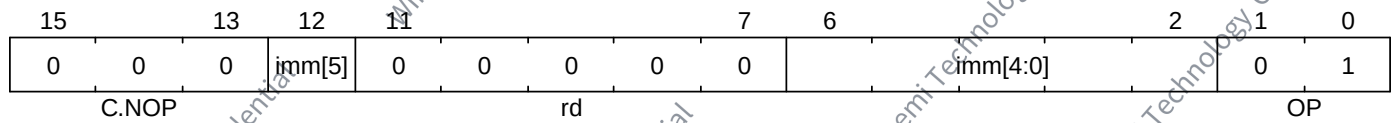
#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

c.nop

#### Encoding



#### Pseudo-code

```
// Do nothing but advancing PC
nop();
```

### 3.6.20. C.OR

#### Description

This instruction computes the bitwise OR of the values in registers *rd'* and *rs2'*, then writes the result to register *rd'*.

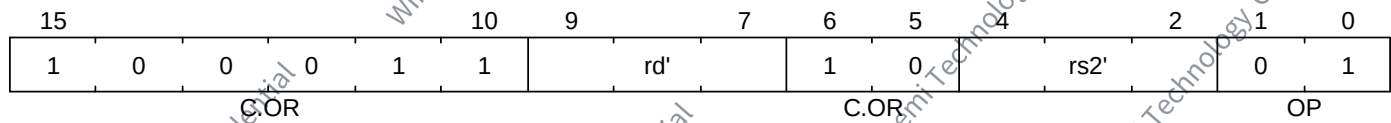
#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

c.or *rd'*, *rs2'*

#### Encoding



#### Pseudo-code

```
X[rd'+8] = X[rd'+8] | X[rs2'+8];
```

### 3.6.21. C.SLLI

#### Description

This instruction performs a logical left shift of the value in register *rd* then writes the result to *rd*. The shift amount is encoded in the *shamt* field.

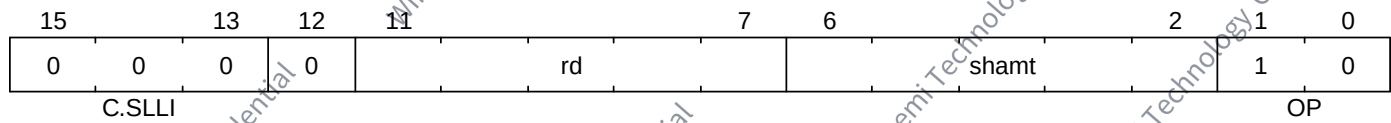
#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

`c.slli rd, shamt`

#### Encoding



#### Pseudo-code

```
X[rd] = X[rd] << shamt;
```



### 3.6.22. C.SRAI

**Description**

This instruction performs an arithmetic right shift of the value in register *rd'* then writes the result to *rd'*. The shift amout is encoded in the *shamt* field.

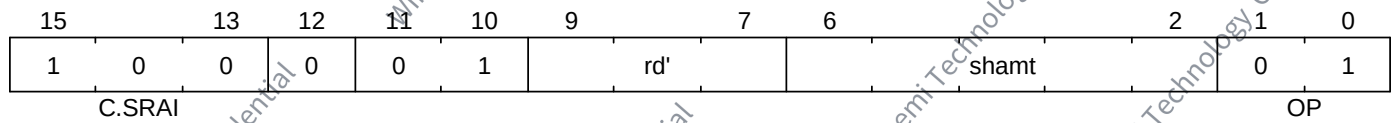
**Configuration**

This instruction is present only when C-extension is configured.

**Mnemonic**

c.srai *rd'*, *shamt*

**Encoding**



**Pseudo-code**

```
X[rd'+8] = (int<XLEN>) X[rd'+8] >> shamt;
```

### 3.6.23. C.SRLI

**Description**

This instruction performs a logical right shift of the value in register *rd'* then writes the result to *rd'*. The shift amount is encoded in the *shamt* field.

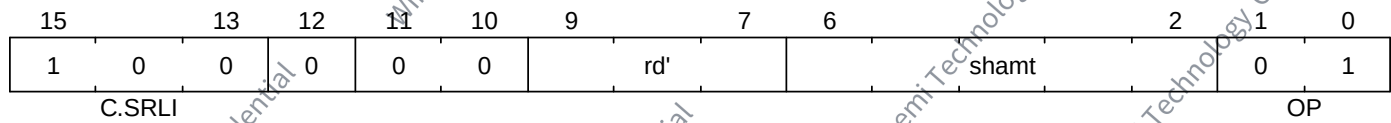
**Configuration**

This instruction is present only when C-extension is configured.

**Mnemonic**

c.srli *rd'*, *shamt*

**Encoding**



**Pseudo-code**

```
X[rd'+8] = (uint<XLEN>) X[rd'+8] >> shamt;
```

### 3.6.24. C.SUB

**Description**

This instruction subtracts the value in register *rs2'* from the value in register *rd'*, then writes the result to register *rd'*.

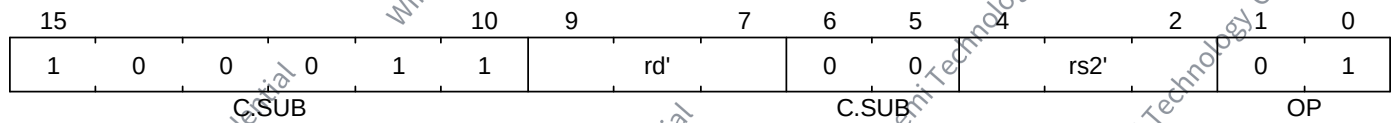
**Configuration**

This instruction is present only when C-extension is configured.

**Mnemonic**

c.sub *rd', rs2'*

**Encoding**



**Pseudo-code**

```
X[rd'+8] = X[rd'+8] - X[rs2'+8];
```

### 3.6.25. C.SW

#### Description

This instruction stores a 32-bit value in register *rs2'* to memory. It computes an effective address by adding the zero-extended offset, scaled by 4, to the base address in register *rs1'*.

#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

c.sw *rs2'*, *offset(rs1')*

#### Encoding

|      |    |    |             |      |          |             |   |   |   |   |    |
|------|----|----|-------------|------|----------|-------------|---|---|---|---|----|
| 15   | 13 | 12 | 10          | 9    | 7        | 6           | 5 | 4 | 2 | 1 | 0  |
| 1    | 1  | 0  | imm[4:2]    | rs1' | imm[1:0] | rs2'        | 0 | 0 |   |   |    |
| C.SW |    |    | offset[5:3] |      |          | offset[2:6] |   |   |   |   | OP |

#### Pseudo-code

```
uint<XLEN> addr_t;
uint7 offset_t;
offset_t = imm[0] :: imm[4:2] :: imm[1] :: 2'b0;

addr_t = X[rs1'+8] + ZeroExt(offset_t, XLEN);
MemStore(X[rs2'+8], addr_t, Word, AccType_Normal);
```

### 3.6.26. C.SWSP

#### Description

This instruction stores a 32-bit value in register *rs2* to memory. It computes an effective address by adding the zero-extended offset, scaled by 4, to the stack pointer, *x2* (or *sp*).

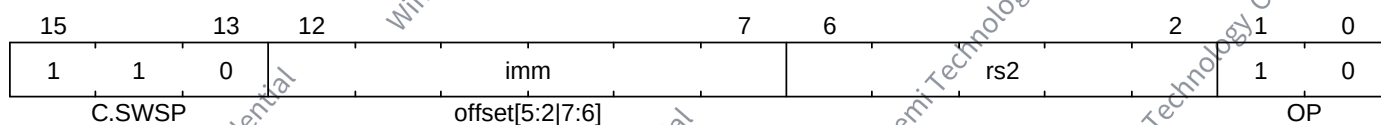
#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

`c.swsp rs2, offset(sp)`

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;
uint8 offset_t;
offset_t = imm[1:0] :: imm[5:2] :: 2'b0;

addr_t = X[2] + ZeroExt(offset_t, XLEN);
MemStore(X[rs2], addr_t, Word, AccType_Normal);
```

### 3.6.27. C.XOR

#### Description

This instruction computes the bitwise XOR of the values in registers *rd'* and *rs2'*, the writes the result to register *rd'*.

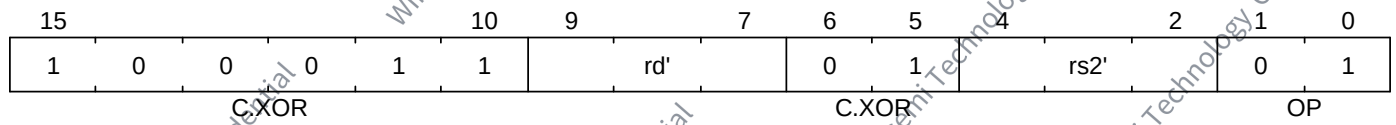
#### Configuration

This instruction is present only when C-extension is configured.

#### Mnemonic

c.xor *rd'*, *rs2'*

#### Encoding



#### Pseudo-code

```
X[rd'+8] = X[rd'+8] ^ X[rs2'+8];
```

### 3.7. "Zicsr" Control and Status Register (CSR) Instructions

RISC-V defines a separate address space of 4096 Control and Status registers associated with each hart. This chapter defines the full set of CSR instructions that operate on these CSRs.

All CSR instructions atomically read-modify-write a single CSR.

For any event or consequence that occurs due to a CSR having a particular value, if a write to the CSR gives it that value, the resulting event or consequence is said to be an indirect effect of the write. Indirect effects of a CSR write are not considered by the RISC-V ISA to be side effects of that write.

Some CSRs, such as the instructions-retired counter, `instret`, may be modified as side effects of instruction execution. In these cases, if a CSR access instruction reads a CSR, it reads the value prior to the execution of the instruction. If a CSR access instruction writes such a CSR, the write is done instead of the increment. In particular, a value written to `instret` by one instruction will be the value read by the following instruction.





### 3.7.2. CSRRCI

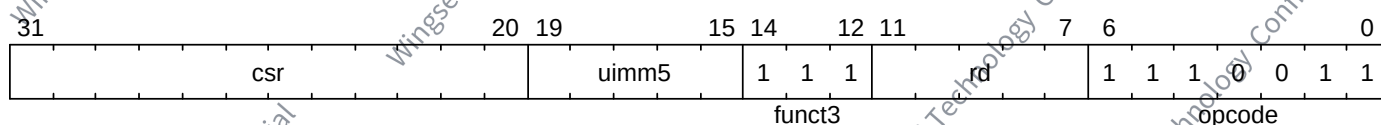
#### Description

This instruction reads the value of the CSR, zero-extendeds the value to XLEN bits, and writes it to integer register *rd*. The 5-bit unsigned immediate is zero-extended to XLEN and is treated as a bit mask that specifies bit positions to be cleared in the CSR. Any bit that is high will cause the corresponding bit to be cleared in the CSR, if that CSR bit is writable. Other bits in the CSR are not explicitly written.

#### Mnemonic

csrrci *rd, csr, uimm5*

#### Encoding



#### Alias Condition

| Alias                    | Condition         |
|--------------------------|-------------------|
| csrrci <i>csr, uimm5</i> | <i>rd</i> == 5'd0 |

#### Pseudo-code

```
uint<32> csr_t;

// Read CSR
csr_t = CSR[csr];

// Write CSR
if (uimm5 != 0) {
    CSR[csr] = (~ZeroExt(uimm5, XLEN)) & csr_t;
}

X[rd] = csr_t;
```

### 3.7.3. CSRRS

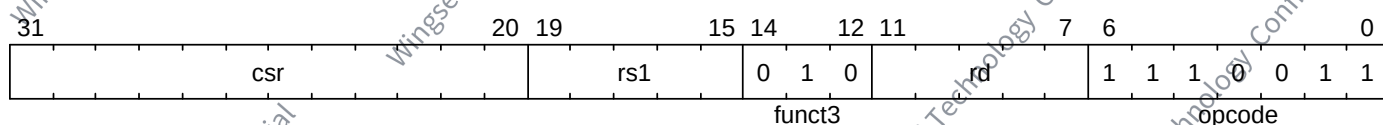
#### Description

This instruction reads the value of the CSR, zero-extends the value to XLEN bits, and writes it to integer register *rd*. The initial value in integer register *rs1* is treated as a bit mask that specifies bit positions to be set in the CSR. Any bit that is high in *rs1* will cause the corresponding bit to be set in the CSR, if that CSR bit is writable. Other bits in the CSR are not explicitly written.

#### Mnemonic

csrrs *rd, csr, rs1*

#### Encoding



#### Alias Condition

| Alias                 | Condition          |
|-----------------------|--------------------|
| csrr <i>rd, csr</i>   | <i>rs1</i> == 5'd0 |
| csrrs <i>csr, rs1</i> | <i>rd</i> == 5'd0  |

#### Pseudo-code

```
uint<XLEN> csr_t;

// Read CSR
csr_t = CSR[csr];

// Write CSR
if (rs1 != 0) {
    CSR[csr] = X[rs1] | CSR[csr];
}

X[rd] = csr_t;
```

### 3.7.4. CSRRSI

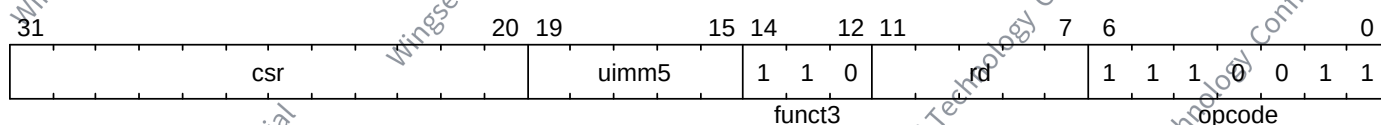
#### Description

This instruction reads the value of the CSR, zero-extended the value to XLEN bits, and writes it to integer register *rd*. The 5-bit unsigned immediate is zero-extended to XLEN and is treated as a bit mask that specifies bit positions to be set in the CSR. Any bit that is high will cause the corresponding bit to be set in the CSR, if that CSR bit is writable. Other bits in the CSR are not explicitly written.

#### Mnemonic

csrrsi *rd*, *csr*, *uimm5*

#### Encoding



#### Alias Condition

| Alias                            | Condition         |
|----------------------------------|-------------------|
| csrrsi <i>csr</i> , <i>uimm5</i> | <i>rd</i> == 5'd0 |

#### Pseudo-code

```
uint<XLEN> csr_t;

// Read CSR
csr_t = CSR[csr];

// Write CSR
if (uimm5 != 0) {
    CSR[csr] = csr_t | ZeroExt(uimm5, XLEN);
}

X[rd] = csr_t;
```



### 3.7.6. CSRRWI

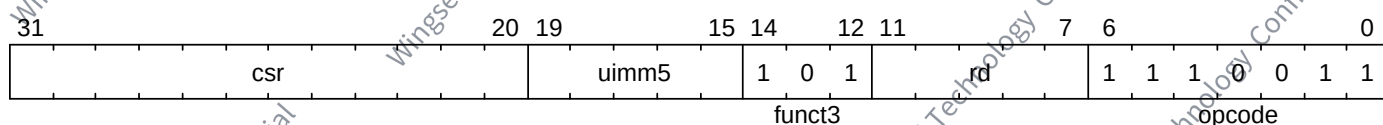
#### Description

This instruction reads the old value of the CSR, zero-extends the value to XLEN-bits, then writes it to integer register *rd*. The 5-bit unsigned immediate is zero-extended to XLEN and is written into *csr*. If *rd* = *x0*, then the instruction shall not read the CSR and shall not cause any of the side effects that might occur on a CSR read.

#### Mnemonic

`csrrwi rd, csr, uimm5`

#### Encoding



#### Alias Condition

| Alias                          | Condition               |
|--------------------------------|-------------------------|
| <code>csrrwi csr, uimm5</code> | <code>rd == 5'd0</code> |

#### Pseudo-code

```
uint<XLEN> csr_t;

// Read CSR
if (rd != 0) {
    csr_t = CSR[csr];
}

// Write CSR
CSR[csr] = ZeroExt(uimm5, XLEN);

X[rd] = csr_t;
```

### 3.8. "B" Standard Extension for Bit Manipulation

The bit-manipulation (bitmanip) extension collection is comprised of several component extensions to the base RISC-V architecture that are intended to provide some combination of code size reduction, performance improvement, and energy reduction. While the instructions are intended to have general use, some instructions are more useful in some domains than others. Hence, several smaller bitmanip extensions are provided, rather than one large extension. Each of these smaller extensions is grouped by common function and use case, and each has its own Zb\*-extension name.

The groups of bitmanip extensions are

- Address generation instructions zba
- Basic bit-manipulation zbb
- Carry-less multiplication zbc
- Single-bit instructions zbs

| Mnemonic                    | Instruction                     | zba | zbb | zbc | zbs |
|-----------------------------|---------------------------------|-----|-----|-----|-----|
| andn <i>rd, rs1, rs2</i>    | AND with inverted operand       |     | ✓   |     |     |
| bclr <i>rd, rs1, rs2</i>    | Single-Bit Clear (Register)     |     |     |     | ✓   |
| bcrl <i>rd, rs1, shamt</i>  | Single-Bit Clear (Immediate)    |     |     |     | ✓   |
| bext <i>rd, rs1, rs2</i>    | Single-Bit Extract (Register)   |     |     |     | ✓   |
| bexti <i>rd, rs1, shamt</i> | Single-Bit Extract (Immediate)  |     |     |     | ✓   |
| binv <i>rd, rs1, rs2</i>    | Single-Bit Invert (Register)    |     |     |     | ✓   |
| binvi <i>rd, rs1, shamt</i> | Single-Bit Invert (Immediate)   |     |     |     | ✓   |
| bset <i>rd, rs1, rs2</i>    | Single-Bit Set (Register)       |     |     |     | ✓   |
| bseti <i>rd, rs1, shamt</i> | Single-Bit Set (Immediate)      |     |     |     | ✓   |
| clmul <i>rd, rs1, rs2</i>   | Carry-less multiply (low-part)  |     |     | ✓   |     |
| clmulh <i>rd, rs1, rs2</i>  | Carry-less multiply (high-part) |     |     | ✓   |     |
| clmulr <i>rd, rs1, rs2</i>  | Carry-less multiply (reversed)  |     |     | ✓   |     |
| clz <i>rd, rs</i>           | Count leading zero bits         |     | ✓   |     |     |
| cpop <i>rd, rs</i>          | Count set bits                  |     | ✓   |     |     |
| ctz <i>rd, rs</i>           | Count trailing zero bits        |     | ✓   |     |     |
| max <i>rd, rs1, rs2</i>     | Maximum                         |     | ✓   |     |     |
| maxu <i>rd, rs1, rs2</i>    | Unsigned maximum                |     | ✓   |     |     |
| min <i>rd, rs1, rs2</i>     | Minimum                         |     | ✓   |     |     |
| minu <i>rd, rs1, rs2</i>    | Unsigned minimum                |     | ✓   |     |     |
| orc.b <i>rd, rs</i>         | Bitwise OR-Combine              |     | ✓   |     |     |
| orn <i>rd, rs1, rs2</i>     | OR with inverted operand        |     | ✓   |     |     |
| rev8 <i>rd, rs</i>          | Byte-reverse register           |     | ✓   |     |     |
| rol <i>rd, rs1, rs2</i>     | Rotate left (Register)          |     | ✓   |     |     |
| ror <i>rd, rs1, rs2</i>     | Rotate right (Register)         |     | ✓   |     |     |

| Mnemonic                         | Instruction              | zba | zbb | zbc | zbs |
|----------------------------------|--------------------------|-----|-----|-----|-----|
| <code>rori rd, rs1, shamt</code> | Rotate right (Immediate) |     | ✓   |     |     |
| <code>sext.b rd, rs</code>       | Sign-extend byte         |     | ✓   |     |     |
| <code>sext.h rd, rs</code>       | Sign-extend halfword     |     | ✓   |     |     |
| <code>sh1add rd, rs1, rs2</code> | Shift left by 1 and add  | ✓   |     |     |     |
| <code>sh2add rd, rs1, rs2</code> | Shift left by 2 and add  | ✓   |     |     |     |
| <code>sh3add rd, rs1, rs2</code> | Shift left by 3 and add  | ✓   |     |     |     |
| <code>xnor rd, rs1, rs2</code>   | Exclusive NOR            |     | ✓   |     |     |
| <code>zext.h rd, rs</code>       | Zero-extend halfword     |     | ✓   |     |     |

### 3.8.1. ANDN

#### Description

This instruction performs the bitwise logical AND operation between *rs1* and the bitwise inversion of *rs2*.

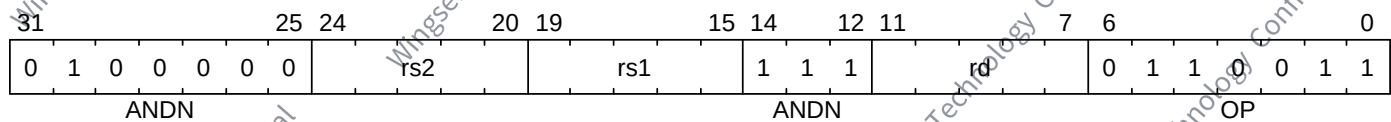
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

andn *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
uint<XLEN> result;

result = X[rs1] & ~X[rs2];
X[rd] = result;
```



### 3.8.2. BCLR

#### Description

This instruction returns *rs1* with a single bit cleared at the index specified in *rs2*. The index is read from the lower  $\log_2(XLEN)$  bits of *rs2*.

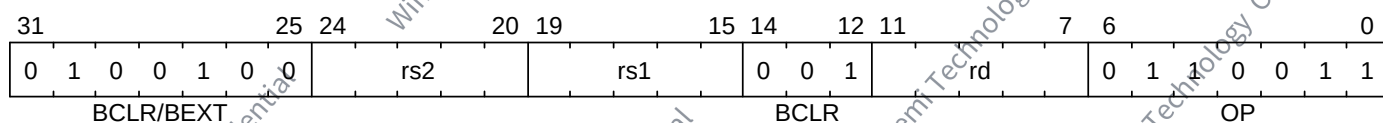
#### Configuration

This instruction is present only when zbs is configured.

#### Mnemonic

`bclr rd, rs1, rs2`

#### Encoding



#### Pseudo-code

```
uint<log2(XLEN)> index = X[rs2] & (XLEN-1);
X[rd] = X[rs1] & ~(1<<index);
```

### 3.8.3. BCLRI

#### Description

This instruction returns *rs1* with a single bit cleared at the index specified in *shamt*. The index is read from the lower  $\log_2(XLEN)$  bits of *shamt*.

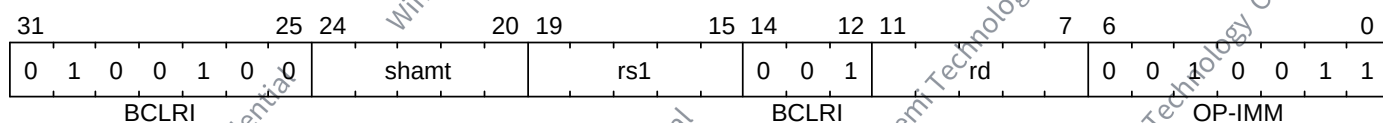
#### Configuration

This instruction is present only when zbs is configured.

#### Mnemonic

`bclri rd, rs1, shamt`

#### Encoding



#### Pseudo-code

```
uint<log2(XLEN)> index = shamt & (XLEN-1);
X[rd] = X[rs1] & ~(1<<index);
```

### 3.8.4. BEXT

#### Description

This instruction returns a single bit extracted from *rs1* at the index specified in *rs2*. The index is read from the lower  $\log_2(\text{XLEN})$  bits of *rs2*.

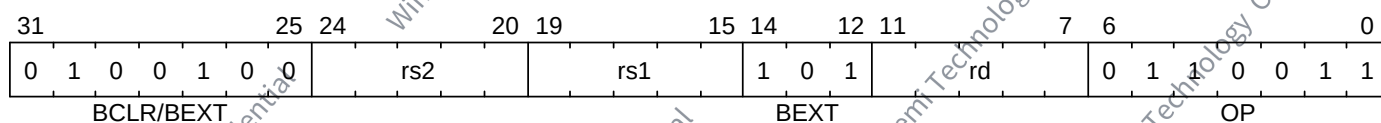
#### Configuration

This instruction is present only when zbs is configured.

#### Mnemonic

`bext rd, rs1, rs2`

#### Encoding



#### Pseudo-code

```
uint<log2(XLEN)> index = X[rs2] & (XLEN-1);
X[rd] = (X[rs1]>>index) & 1;
```

### 3.8.5. BEXTI

#### Description

This instruction returns a single bit extracted from *rs1* at the index specified in *shamt*. The index is read from the lower  $\log_2(XLEN)$  bits of *shamt*.

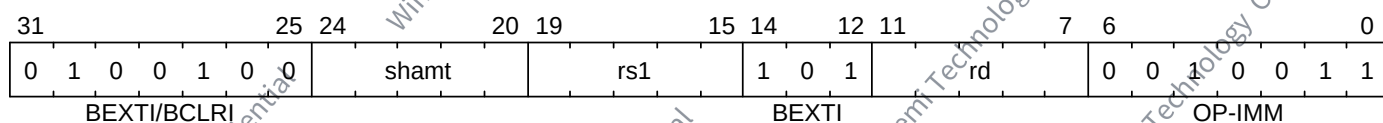
#### Configuration

This instruction is present only when zbs is configured.

#### Mnemonic

*bexti rd, rs1, shamt*

#### Encoding



#### Pseudo-code

```
uint<log2(XLEN)> index = shamt & (XLEN-1);
X[rd] = (X[rs1]>>index) & 1;
```

### 3.8.6. BINV

#### Description

This instruction returns *rs1* with a single bit inverted at the index specified in *rs2*. The index is read from the lower  $\log_2(XLEN)$  bits of *rs2*.

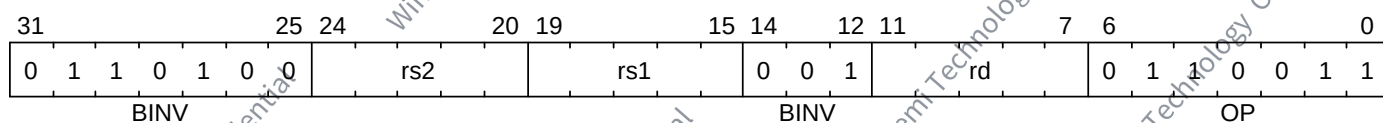
#### Configuration

This instruction is present only when zbs is configured.

#### Mnemonic

`binv rd, rs1, rs2`

#### Encoding



#### Pseudo-code

```
uint<log2(XLEN)> index = X[rs2] & (XLEN-1);
X[rd] = X[rs1] ^ (1<<index);
```

### 3.8.7. BINVI

#### Description

This instruction returns *rs1* with a single bit inverted at the index specified in *shamt*. The index is read from the lower  $\log_2(XLEN)$  bits of *shamt*.

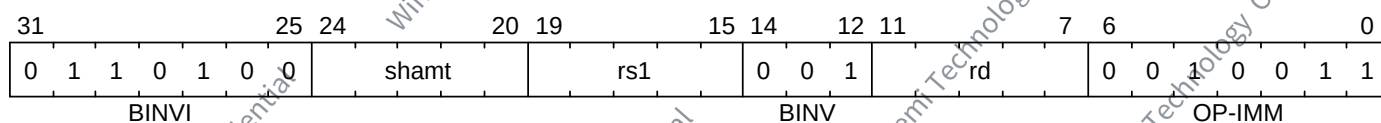
#### Configuration

This instruction is present only when zbs is configured.

#### Mnemonic

*binvi rd, rs1, shamt*

#### Encoding



#### Pseudo-code

```
uint<log2(XLEN)> index = shamt & (XLEN-1);
X[rd] = X[rs1] ^ (1<<index);
```

### 3.8.8. BSET

#### Description

This instruction returns *rs1* with a single bit set at the index specified in *rs2*. The index is read from the lower  $\log_2(\text{XLEN})$  bits of *rs2*.

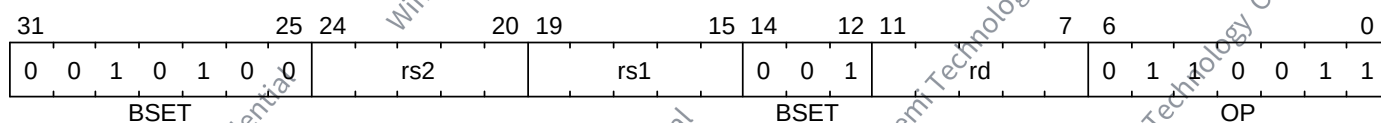
#### Configuration

This instruction is present only when zbs is configured.

#### Mnemonic

`bset rd, rs1, rs2`

#### Encoding



#### Pseudo-code

```
uint<log2(XLEN)> index = X(rs2) & (XLEN-1);
```

```
X[rd] = X[rs1] | (1<<index);
```

### 3.8.9. BSETI

#### Description

This instruction returns *rs1* with a single bit set at the index specified in *shamt*. The index is read from the lower  $\log_2(XLEN)$  bits of *shamt*.

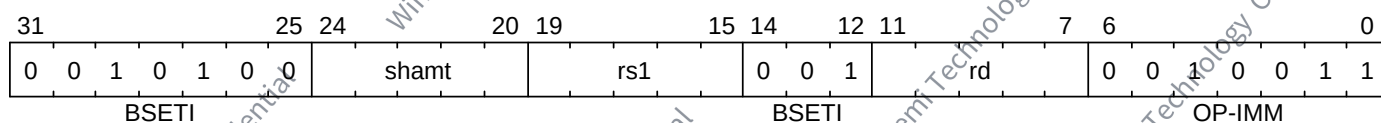
#### Configuration

This instruction is present only when zbs is configured.

#### Mnemonic

`bseti rd, rs1, shamt`

#### Encoding



#### Pseudo-code

```
uint<log2(XLEN)> index = shamt & (XLEN-1);
```

```
X[rd] = X[rs1] | (1<<index);
```



### 3.8.10. CLMUL

#### Description

clmul produces the lower half of the 2·XLEN carry-less product.

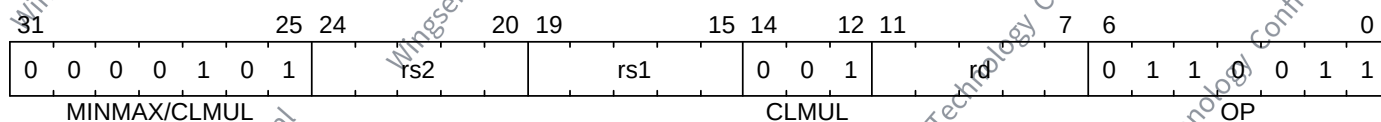
#### Configuration

This instruction is present only when zbc is configured.

#### Mnemonic

clmul rd, rs1, rs2

#### Encoding



#### Pseudo-code

```
uint32 rs1_val = X[rs1];
uint32 rs2_val = X[rs2];
uint32 result = 0;

for (int i=0; i<XLEN; i++) {
    if ((rs2_val>>i)&1) {
        result = result ^ (rs1_val<<i);
    }
}

X[rd] = result;
```

### 3.8.11. CLMULH

#### Description

clmulh produces the upper half of the 2·XLEN carry-less product.

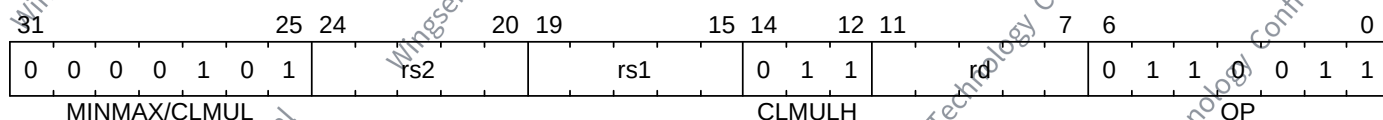
#### Configuration

This instruction is present only when zbc is configured.

#### Mnemonic

clmulh rd, rs1, rs2

#### Encoding



#### Pseudo-code

```
uint32 rs1_val = X[rs1];
uint32 rs2_val = X[rs2];
uint32 result = 0;

for (int i=1; i<XLEN; i++) {
    if ((rs2_val>>i)&1) {
        result = result ^ (rs1_val>>(XLEN-i));
    }
}

X[rd] = result;
```

### 3.8.12. CLMULR

#### Description

clmulr produces bits  $2 \cdot \text{XLEN} - 2 : \text{XLEN} - 1$  of the  $2 \cdot \text{XLEN}$  carry-less product.

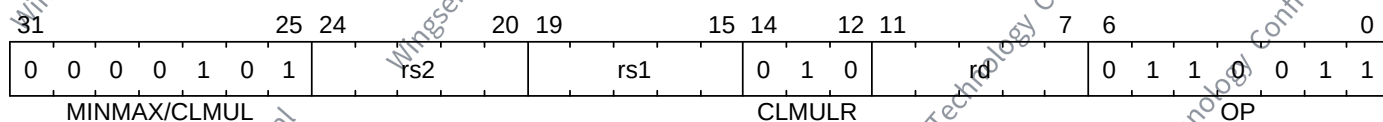
#### Configuration

This instruction is present only when zbc is configured.

#### Mnemonic

clmulr *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
uint32 rs1_val = X[rs1];
uint32 rs2_val = X[rs2];
uint32 result = 0;

for (int i=0; i<XLEN-1; i++) {
    if ((rs2_val>>i)&1) {
        result = result ^ (rs1_val>>(XLEN-i-1));
    }
}

X[rd] = result;
```

### 3.8.13. CLZ

#### Description

This instruction counts the number of 0's before the first 1, starting at the most-significant bit (i.e., XLEN-1) and progressing to bit 0. Accordingly, if the input is 0, the output is XLEN, and if the most-significant bit of the input is a 1, the output is 0.

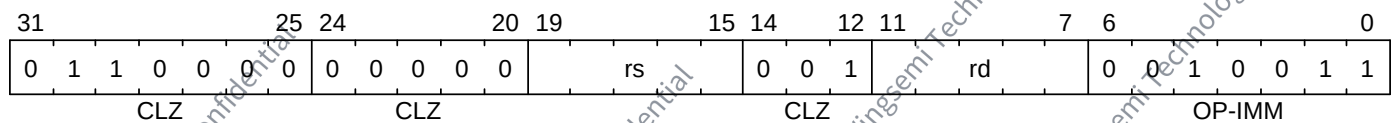
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

clz rd, rs

#### Encoding



#### Pseudo-code

```
uint32 result = 0;

for (int i=0; i<XLEN; i++) {
    if ((X[rs]>>(XLEN-i-1))==0b0) {
        result = result + 1;
    } else {
        break;
    }
}

X[rd] = result;
```

### 3.8.14. CPOP

#### Description

This instructions counts the number of 1's (i.e., set bits) in the source register.

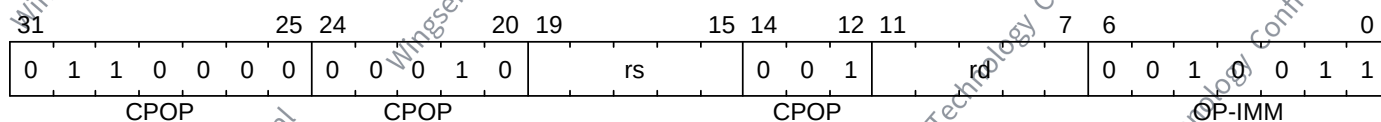
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

cpop rd, rs

#### Encoding



#### Pseudo-code

```
uint32 bitcount = 0;
uint32 rs_val = X[rs];

for (int i=0; i<XLEN; i++) {
    if ((rs_val>>i)&1) {
        bitcount = bitcount + 1;
    }
}

X[rd] = bitcount;
```

### 3.8.15. CTZ

#### Description

This instruction counts the number of 0's before the first 1, starting at the least-significant bit (i.e., 0) and progressing to the most-significant bit (i.e., XLEN-1). Accordingly, if the input is 0, the output is XLEN, and if the least-significant bit of the input is a 1, the output is 0.

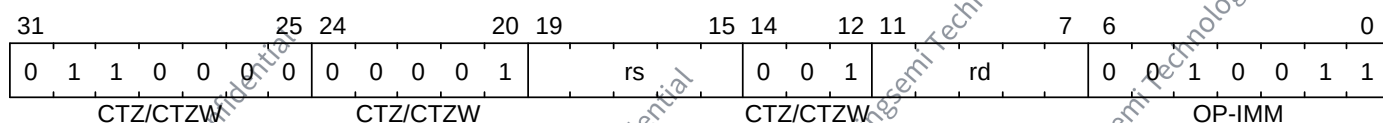
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

ctz rd, rs

#### Encoding



#### Pseudo-code

```
uint32 result = 0;

for (int i=0; i<XLEN; i++) {
    if (((X[rs]>>i) & 0b1) == 0b0) {
        result = result + 1;
    } else {
        break;
    }
}

X[rd] = result;
```

### 3.8.16. MAX

#### Description

This instruction returns the larger of two signed integers.

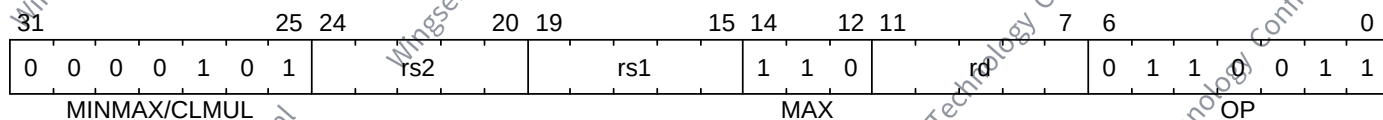
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

max rd, rs1, rs2

#### Encoding



#### Pseudo-code

```
int32 rs1_val = X[rs1];
int32 rs2_val = X[rs2];
int32 result;

result = rs1_val < rs2_val ? rs2_val : rs1_val;

X[rd] = result;
```

### 3.8.17. MAXU

#### Description

This instruction returns the larger of two unsigned integers.

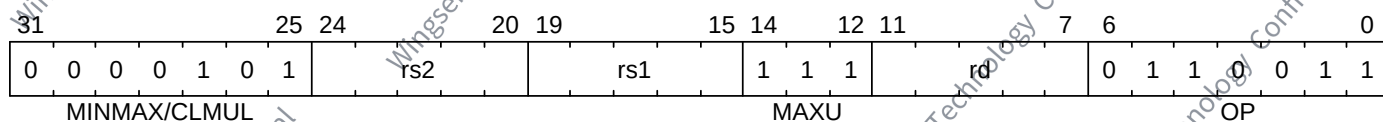
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

maxu rd, rs1, rs2

#### Encoding



#### Pseudo-code

```
uint32 rs1_val = X[rs1];
uint32 rs2_val = X[rs2];
uint32 result;

result = rs1_val < rs2_val ? rs2_val : rs1_val;

X[rd] = result;
```



### 3.8.18. MIN

#### Description

This instruction returns the smaller of two signed integers.

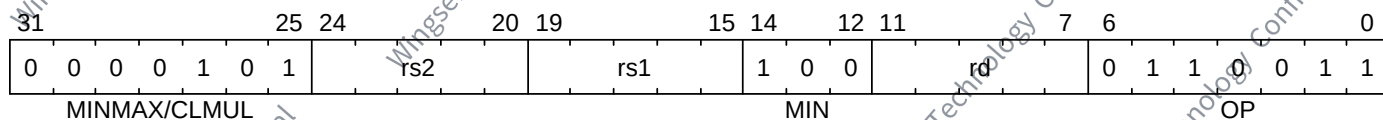
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

min rd, rs1, rs2

#### Encoding



#### Pseudo-code

```
int32 rs1_val = X[rs1];
int32 rs2_val = X[rs2];
int32 result;

result = rs1_val < rs2_val ? rs1_val : rs2_val;

X[rd] = result;
```

### 3.8.19. MINU

#### Description

This instruction returns the smaller of two unsigned integers.

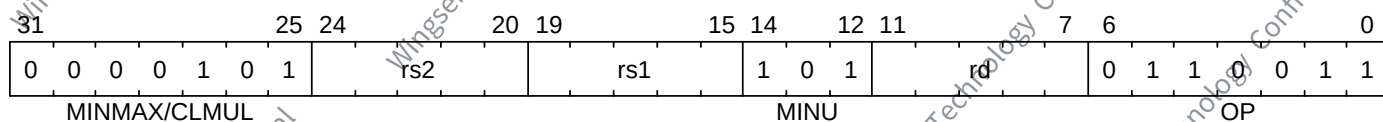
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

minu rd, rs1, rs2

#### Encoding



#### Pseudo-code

```
uint32 rs1_val = X[rs1];
uint32 rs2_val = X[rs2];
uint32 result;

result = rs1_val < rs2_val ? rs1_val : rs2_val;

X[rd] = result;
```

### 3.8.20. ORC.B

#### Description

Combines the bits within each byte using bitwise logical OR. This sets the bits of each byte in the result *rd* to all zeros if no bit within the respective byte of *rs* is set, or to all ones if any bit within the respective byte of *rs* is set.

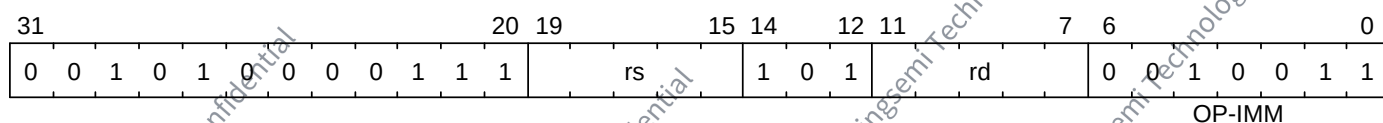
#### Configuration

This instruction is present only when *zbb* is configured.

#### Mnemonic

*orc.b rd, rs*

#### Encoding



#### Pseudo-code

```
X[rd][31:24] = X[rs][31:24] == 0 ? 8'b00000000 : 8'b11111111;
X[rd][23:16] = X[rs][23:16] == 0 ? 8'b00000000 : 8'b11111111;
X[rd][15: 8] = X[rs][15: 8] == 0 ? 8'b00000000 : 8'b11111111;
X[rd][ 7: 0] = X[rs][ 7: 0] == 0 ? 8'b00000000 : 8'b11111111;
```

### 3.8.21. ORN

#### Description

This instruction performs the bitwise logical OR operation between *rs1* and the bitwise inversion of *rs2*.

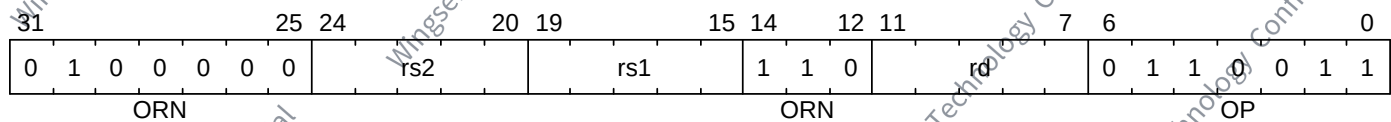
#### Configuration

This instruction is present only when *zbb* is configured.

#### Mnemonic

orn *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
uint<XLEN> result;  
  
result = X[rs1] | ~X[rs2];  
X[rd] = result;
```

### 3.8.22. REV8

#### Description

This instruction reverses the order of the bytes in *rs*.

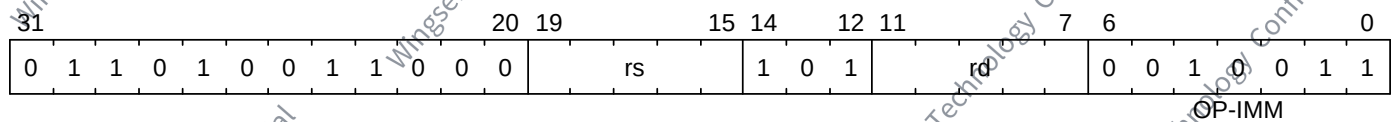
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

rev8 *rd, rs*

#### Encoding



#### Pseudo-code

```
X[rd][31:24] = X[rs][ 7: 0];
X[rd][23:16] = X[rs][15: 8];
X[rd][15: 8] = X[rs][23:16];
X[rd][ 7: 0] = X[rs][31:24];
```

### 3.8.23. ROL

#### Description

This instruction performs a rotate left of *rs1* by the amount in least-significant  $\log_2(\text{XLEN})$  bits of *rs2*.

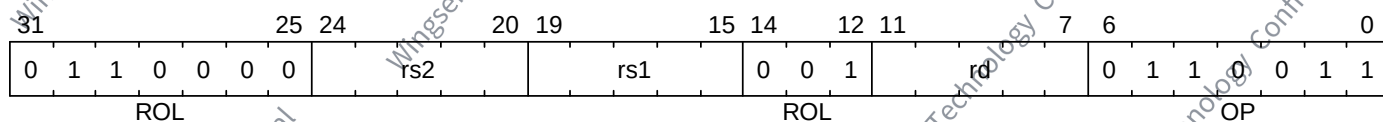
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

rol *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
uint5 shamt = X[rs2][4:0];
uint32 rs1_val;
uint32 result;

rs1_val = X[rs1];
result = (rs1_val<<shamt) | (rs1_val>>(XLEN-shamt));

X[rd] = result;
```

### 3.8.24. ROR

#### Description

This instruction performs a rotate right of *rs1* by the amount in least-significant  $\log_2(\text{XLEN})$  bits of *rs2*.

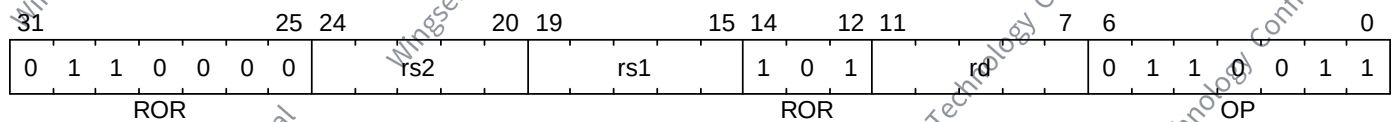
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

`ror rd, rs1, rs2`

#### Encoding



#### Pseudo-code

```
uint5 shamt = X[rs2][4:0];
uint32 rs1_val;
uint32 result;

rs1_val = X[rs1];
result = (rs1_val >> shamt) | (rs1_val << (XLEN - shamt));

X[rd] = result;
```

### 3.8.25. RORI

#### Description

This instruction performs a rotate right of *rs1* by the amount in the least-significant  $\log_2(\text{XLEN})$  bits of *shamt*.

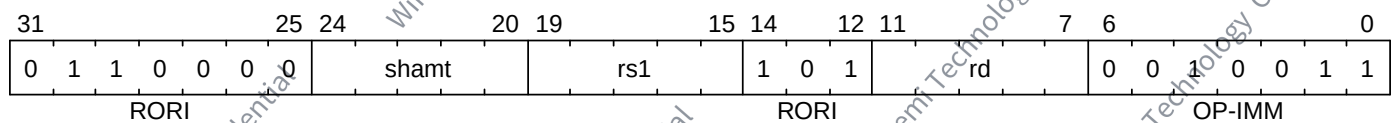
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

`rori rd, rs1, shamt`

#### Encoding



#### Pseudo-code

```
uint32 rs1_val;
uint32 result;

rs1_val = X[rs1];
result = (rs1_val >> shamt) | (rs1_val << (XLEN - shamt));

X[rd] = result;
```



### 3.8.26. SEXT.B

#### Description

This instruction sign-extends the least-significant byte in the source to XLEN by copying the most-significant bit in the byte (i.e., bit 7) to all of the more-significant bits.

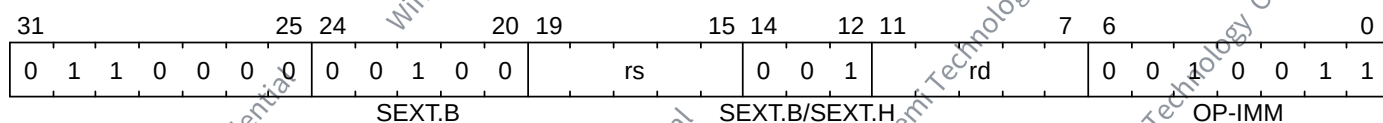
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

sext.b rd, rs

#### Encoding



#### Pseudo-code

```
int8 rs_val = X[rs][7:0];
X[rd] = SignExt(rs_val, XLEN);
```

### 3.8.27. SEXT.H

**Description**

This instruction sign-extends the least-significant halfword in *rs* to XLEN by copying the most-significant bit in the halfword (i.e., bit 15) to all of the more-significant bits.

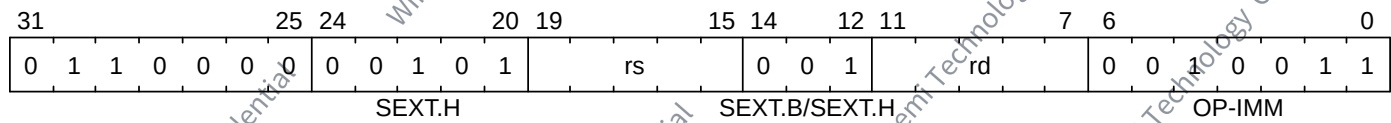
**Configuration**

This instruction is present only when zbb is configured.

**Mnemonic**

`sext.h rd, rs`

**Encoding**



**Pseudo-code**

```
int16 rs_val = X[rs][15:0];
X[rd] = SignExt(rs_val, XLEN);
```

### 3.8.28. SH1ADD

**Description**

This instruction shifts *rs1* to the left by 1 bit and adds it to *rs2*.

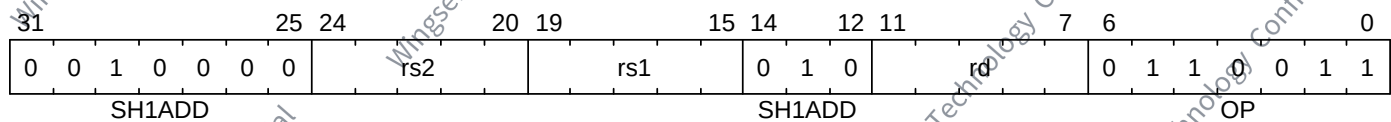
**Configuration**

This instruction is present only when zba is configured.

**Mnemonic**

sh1add *rd*, *rs1*, *rs2*

**Encoding**



**Pseudo-code**

```
X[rd] = X[rs2] + (X[rs1]<<1);
```

### 3.8.29. SH2ADD

#### Description

This instruction shifts *rs1* to the left by 2 places and adds it to *rs2*.

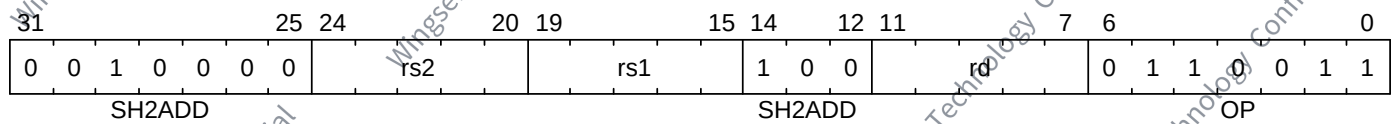
#### Configuration

This instruction is present only when *zba* is configured.

#### Mnemonic

sh2add *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
X[rd] = X[rs2] + (X[rs1]<<2);
```

### 3.8.30. SH3ADD

**Description**

This instruction shifts *rs1* to the left by 3 places and adds it to *rs2*.

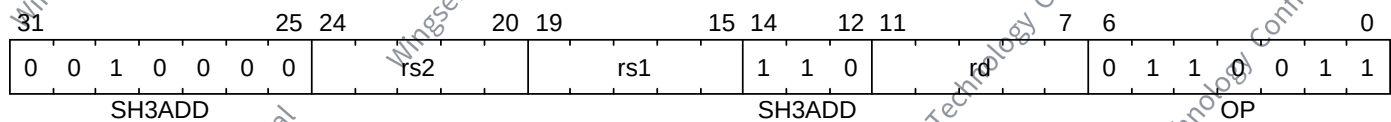
**Configuration**

This instruction is present only when *zba* is configured.

**Mnemonic**

sh3add *rd, rs1, rs2*

**Encoding**



**Pseudo-code**

```
X[rd] = X[rs2] + (X[rs1]<<3);
```

### 3.8.31. XNOR

#### Description

This instruction performs the bit-wise exclusive-NOR operation on *rs1* and *rs2*.

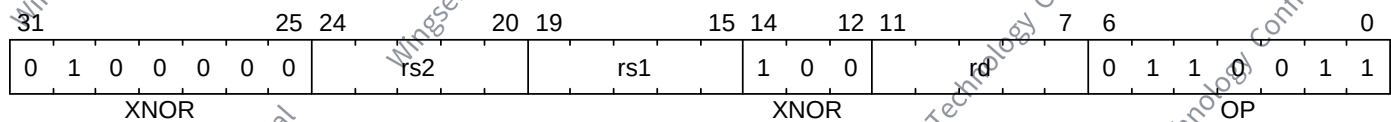
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

xnor *rd*, *rs1*, *rs2*

#### Encoding



#### Pseudo-code

```
uint<XLEN> result;

result = ~(X[rs1] ^ X[rs2]);
X[rd] = result;
```

### 3.8.32. ZEXT.H

#### Description

This instruction zero-extends the least-significant halfword of the source to XLEN by inserting 0's into all of the bits more significant than 15.

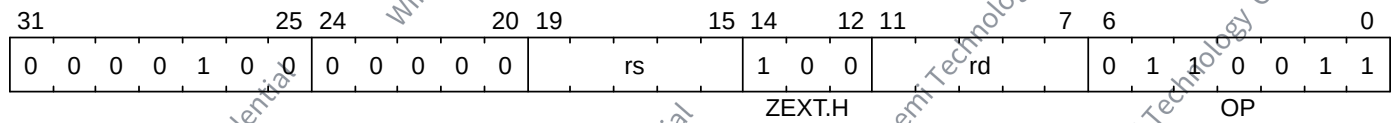
#### Configuration

This instruction is present only when zbb is configured.

#### Mnemonic

`zext.h rd, rs`

#### Encoding



#### Pseudo-code

```
uint16 rs_val = X[rs][15:0];
X[rd] = ZeroExt(rs_val, XLEN);
```

### 3.9. "Zc" code-size reduction

Zc\* is a group of extensions which define subsets of the existing C extension and new extensions which only contain 16-bit encodings.

- Simple operations for use on all architectures zcb
- PUSH/POP and double move zcmp
- Table jump zcmt

| Mnemonic                         | Instruction                                   | zcb | zcmp | zcmt |
|----------------------------------|-----------------------------------------------|-----|------|------|
| c.lbu rd', uimm2(rs1')           | Load unsigned halfword, 16-bit encoding       | ✓   |      |      |
| c.lhu rd', uimm2(rs1')           | Load unsigned halfword, 16-bit encoding       | ✓   |      |      |
| c.lh rd', uimm2(rs1')            | Load signed halfword, 16-bit encoding         | ✓   |      |      |
| cm.jalt index                    | Jump via table with optional link             |     |      | ✓    |
| cm.jt index                      | Jump via table                                |     |      | ✓    |
| cm.mva01s, r1sc, r2sc            | Move two s0-s7 registers into a0-a1           |     | ✓    |      |
| cm.mvsa01 r1sc, r2sc             | Move a0-a1 into two different s0-s7 registers |     | ✓    |      |
| cm.pop {reg_list}, stack_adj     | Pop registers                                 |     | ✓    |      |
| cm.popret {reg_list}, stack_adj  | Pop registers                                 |     | ✓    |      |
| cm.popretz {reg_list}, stack_adj | Pop registers                                 |     | ✓    |      |
| cm.push {reg_list}, -stack_adj   | Create stack frame: push registers            |     | ✓    |      |
| c.mul rd', rs2'                  | Multiply                                      | ✓   |      |      |
| c.not rd'                        | Bitwise not                                   | ✓   |      |      |
| c.sb rs2', uimm2(rs1')           | Store byte                                    | ✓   |      |      |
| c.sext.b rd'                     | Sign extend byte, 16-bit encoding             | ✓   |      |      |
| c.sext.h rd'                     | Sign extend halfword, 16-bit encoding         | ✓   |      |      |
| c.sh rs2', uimm2(rs1')           | Store halfword, 16-bit encoding               | ✓   |      |      |
| c.zext.b rd'                     | Zero extend byte                              | ✓   |      |      |
| c.zext.h rd'                     | Zero extend halfword                          | ✓   |      |      |

#### 3.9.1. PUSH/POP Register Instructions

PUSH, POP, POPREST are used to reduce the size of function prologues and epilogues.

1. The PUSH instruction
  - adjusts the stack pointer to create the stack frame
  - pushes (stores) the registers specified in the register list to the stack frame
2. The POP instruction
  - pops (loads) the registers in the register list from the stack frame
  - adjusts the stack pointer to destroy the stack frame
3. The POPRET instructions



- pop (load) the registers in the register list from the stack frame
- *cm.popretz* also moves zero into *a0* as the return value
- adjust the stack pointer to destroy the stack frame
- execute a *ret* instruction to return from the function

### 3.9.1.1. PUSH/POP Fault Handling

In Wing-M130 processor, the hart can discard the executed uops of PUSH/POP sequence to taken the interrupt/exception, and after returning from this interrupt/exception, the multiple instruction is replayed from the start of the instruction.

It is strongly recommended that PUSH/POP sequence is not used with memory location being Device memory. Otherwise the result could be UNDEFINED.

If a trap occurs during the sequence then *mepc* is updated with the PC of the instruction, *mtval* updated with the bad address if it was an access fault and *mcause* updated with the type of trap.

### 3.9.2. Table Jump Overview

*cm.jt* (Jump via table) and *cm.jalt* (Jump via table with optional link) are referred to as table jump.

Table jump uses a 256-entry XLEN wide table in instruction memory to contain function addresses. The table must be a minimum of 64-byte aligned.

Table entries follow the current data endianness. This is different from normal instruction fetch which is always little-endian.

*cm.jt* and *cm.jalt* encodings index the table, giving access to functions within the full XLEN wide address space.

This is used as a form of dictionary compression to reduce the code size of *jal* / *auipc+jalr* / *jr* / *auipc+jr* instructions.

Table jump allows the linker to replace the following instruction sequences with a *cm.jt* or *cm.jalt* encoding, and an entry in the table:

- 32-bit *j* calls
- 32-bit *jal* ra calls
- 64-bit *auipc+jr* calls to fixed locations
- 64-bit *auipc+jalr* ra calls to fixed locations
  - The *auipc+jr/jalr* sequence is used because the offset from the PC is out of the  $\pm 1\text{MB}$  range.

If a return address stack is implemented, then as *cm.jalt* is equivalent to *jal ra*, it pushes to the stack.

#### 3.9.2.1. Table Jump Fault Handling

For a table jump instruction, the table entry that the instruction selects is considered an extension of the instruction itself. Hence, the execution of a table jump instruction involves two instruction fetches, the first to read the instruction (*cm.jt/cm.jalt*) and the second to read from the jump vector table (JVT). Both instruction fetches are *implicit* reads, and both require execute permission; read permission is irrelevant. It is

recommended that the second fetch be ignored for hardware triggers and breakpoints.

Memory writes to the jump vector table require an instruction barrier (*fence.i*) to guarantee that they are visible to the instruction fetch.

Multiple contexts may have different jump vector tables. JVT may be switched between them without an instruction barrier if the tables have not been updated in memory since the last *fence.i*.

If an exception occurs on either instruction fetch, *mepc* is set to the PC of the table jump instruction, *mcause* is set as expected for the type of fault and *mtval* (if not set to zero) contains the fetch address which caused the fault.

### 3.9.3. C.LBU

#### Description

This instruction loads a byte from the memory address formed by adding  $rs1$  to the zero extended immediate  $uimm2$ . The resulting byte is zero extended to XLEN bits and is written to  $rd$ .

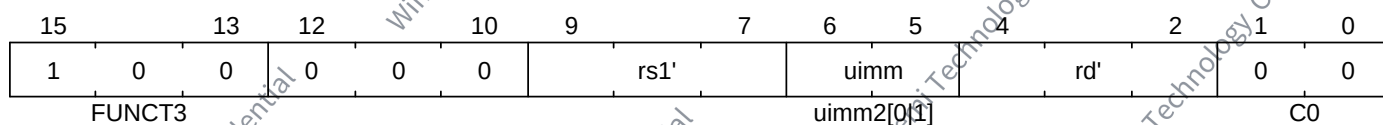
#### Configuration

This instruction is present only when zcb is configured.

#### Mnemonic

$c.lbu\ rd',\ uimm2(rs1')$

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;
uint8 data;

addr_t = X[rs1'+8] + ZeroExt(uimm[0]::uimm[1], XLEN);
data = MemLoad(addr_t, Byte, AccType_Normal);
X[rd'+8] = ZeroExt(data, XLEN);
```

### 3.9.4. C.LH

#### Description

This instruction loads a halfword from the memory address formed by adding  $rs1'$  to the zero extended immediate  $uimm2$ . The resulting halfword is sign extended to XLEN bits and is written to  $rd'$ .

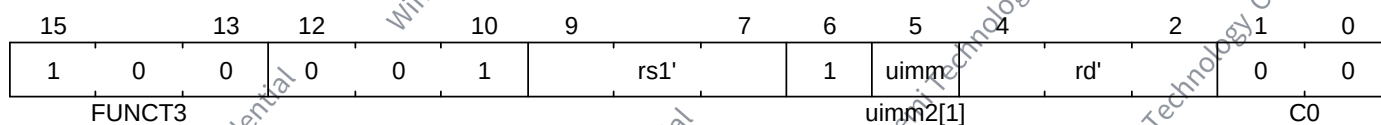
#### Configuration

This instruction is present only when zcb is configured.

#### Mnemonic

`c.lh  $rd'$ ,  $uimm2(rs1')$`

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;
int16 data;

addr_t = X[ $rs1'+8$ ] + ZeroExt( $uimm2::1'b0$ , XLEN);
data = MemLoad(addr_t, HalfWord, AccType_Normal);
X[ $rd'+8$ ] = SignExt(data, XLEN);
```

### 3.9.5. C.LHU

#### Description

This instruction loads a halfword from the memory address formed by adding  $rs1'$  to the zero extended immediate  $uimm2$ . The resulting halfword is zero extended to XLEN bits and is written to  $rd'$ .

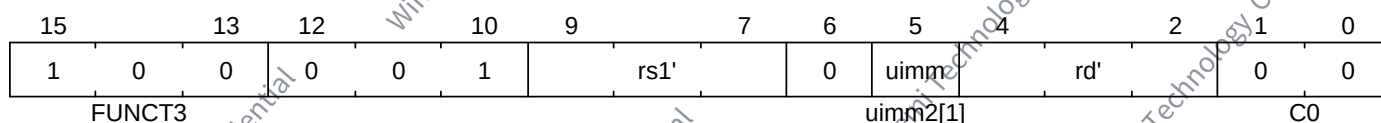
#### Configuration

This instruction is present only when zcb is configured.

#### Mnemonic

`c.lhu rd', uimm2(rs1')`

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;
uint16 data;

addr_t = X[rs1'+8] + ZeroExt(uimm::1'b0, XLEN);
data = MemLoad(addr_t, HalfWord, AccType_Normal);
X[rd'+8] = ZeroExt(data, XLEN);
```

### 3.9.6. C.SB

#### Description

This instruction stores the least significant byte of  $rs2'$  to the memory address formed by adding  $rs1'$  to the zero extended immediate  $uimm2$ .

#### Configuration

This instruction is present only when zcb is configured.

#### Mnemonic

`c.sb  $rs2'$ ,  $uimm2(rs1')$`

#### Encoding

|        |    |    |            |   |   |      |   |      |   |      |   |    |
|--------|----|----|------------|---|---|------|---|------|---|------|---|----|
| 15     | 13 | 12 | 10         | 9 | 7 | 6    | 5 | 4    | 2 | 1    | 0 |    |
| 1      | 0  | 0  | 0          | 1 | 0 | rs1' |   | uimm |   | rs2' |   |    |
| FUNCT3 |    |    | uimm2[0:1] |   |   |      |   |      |   |      |   | C0 |

#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[ $rs1'+8$ ] + ZeroExt( $uimm[0]::uimm[1]$ , XLEN);
MemStore(X[ $rs2'+8$ ][7:0], addr_t, Byte, AccType_Normal);
```

### 3.9.7. C.SH

#### Description

This instruction stores the least significant halfword of  $rs2'$  to the memory address formed by adding  $rs1'$  to the zero extended immediate  $uimm2$ .

#### Configuration

This instruction is present only when zcb is configured.

#### Mnemonic

`c.sh  $rs2'$ ,  $uimm2(rs1')$`

#### Encoding

|        |    |    |    |   |   |          |   |   |      |      |   |   |   |
|--------|----|----|----|---|---|----------|---|---|------|------|---|---|---|
| 15     | 13 | 12 | 10 | 9 | 7 | 6        | 5 | 4 | 2    | 1    | 0 |   |   |
| 1      | 0  | 0  | 0  | 1 | 1 | rs1'     |   | 0 | uimm | rs2' |   | 0 | 0 |
| FUNCT3 |    |    |    |   |   | uimm2[1] |   |   |      | C0   |   |   |   |

#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1'+8] + ZeroExt(uimm::1'b0, XLEN);
MemStore(X[rs2'+8][15:0], addr_t, HalfWord, AccType_Normal);
```

### 3.9.8. C.ZEXT.B

**Description**

This instruction zero-extends the least-significant byte in the operand to XLEN bits by inserting zeros into all of the bits more significant than 7.

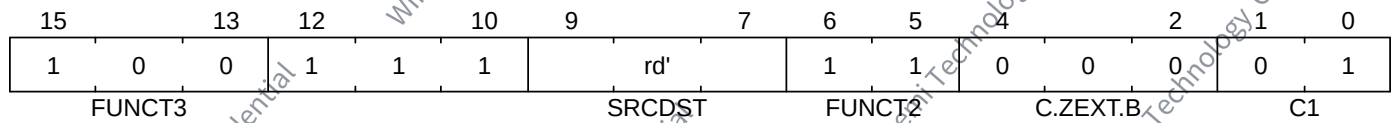
**Configuration**

This instruction is present only when zcb is configured.

**Mnemonic**

c.zext.b rd'

**Encoding**



**Pseudo-code**

```
X[rd'+8] = ZeroExt(X[rd'+8][7:0], XLEN);
```



### 3.9.9. C.SEXT.B

#### Description

This instruction sign-extends the least-significant byte in the operand to XLEN bits by copying the most-significant bit in the byte (i.e., bit 7) to all of the more-significant bits.

#### Configuration

This instruction is present only when zcb and zbb is configured.

#### Mnemonic

c.sext.b rd'

#### Encoding

|        |    |    |        |   |   |        |   |   |          |   |   |    |   |   |
|--------|----|----|--------|---|---|--------|---|---|----------|---|---|----|---|---|
| 15     | 13 | 12 | 10     | 9 | 7 | 6      | 5 | 4 | 2        | 1 | 0 |    |   |   |
| 1      | 0  | 0  | 1      | 1 | 1 | rd'    |   | 1 | 1        | 0 | 0 | 1  | 0 | 1 |
| FUNCT3 |    |    | SRCDST |   |   | FUNCT2 |   |   | C.SEXT.B |   |   | C1 |   |   |

#### Pseudo-code

```
X[rd'+8] = SignExt(X[rd'+8][7:0], XLEN);
```

### 3.9.10. C.ZEXT.H

#### Description

This instruction zero-extends the least-significant halfword of the operand to XLEN bits by inserting zeros into all of the bits more significant than 15.

#### Configuration

This instruction is present only when zcb and zbb is configured.

#### Mnemonic

c.zext.h rd'

#### Encoding

| 15     | 13 | 12 | 10     | 9 | 7 | 6      | 5 | 4        | 2 | 1 | 0  |
|--------|----|----|--------|---|---|--------|---|----------|---|---|----|
| 1      | 0  | 0  | 1      | 1 | 1 | rd'    |   | 1        | 1 | 0 | 1  |
| FUNCT3 |    |    | SRCDST |   |   | FUNCT2 |   | C.ZEXT.H |   |   | C1 |

#### Pseudo-code

```
X[rd'+8] = ZeroExt(X[rd'+8][15:0], XLEN);
```

### 3.9.11. C.SEXT.H

#### Description

This instruction sign-extends the least-significant halfword in the operand to XLEN bits by copying the most-significant bit in the halfword (i.e., bit 15) to all of the more-significant bits.

#### Configuration

This instruction is present only when zcb and zbb is configured.

#### Mnemonic

c.sext.h rd'

#### Encoding

|        |    |    |        |   |   |        |   |   |          |   |   |    |   |   |
|--------|----|----|--------|---|---|--------|---|---|----------|---|---|----|---|---|
| 15     | 13 | 12 | 10     | 9 | 7 | 6      | 5 | 4 | 2        | 1 | 0 |    |   |   |
| 1      | 0  | 0  | 1      | 1 | 1 | rd'    |   | 1 | 1        | 0 | 1 | 1  | 0 | 1 |
| FUNCT3 |    |    | SRCDST |   |   | FUNCT2 |   |   | C.SEXT.H |   |   | C1 |   |   |

#### Pseudo-code

```
X[rd'+8] = SignExt(X[rd'+8][15:0], XLEN);
```

### 3.9.12. C.NOT

#### Description

This instruction takes the one’s complement of *rd'* and writes the result to the same register.

#### Configuration

This instruction is present only when zcb is configured.

#### Mnemonic

c.not *rd'*

#### Encoding



#### Pseudo-code

```
X[rd'+8] = X[rd'+8] ^ -1;
```

### 3.9.13. C.MUL

#### Description

This instruction multiplies XLEN bits of the source operands from  $rd'$  and  $rs2'$  and writes the lowest XLEN bits of the result to  $rd'$ .

#### Configuration

This instruction is present only when zcb and either M-extension or zmmul is configured.

#### Mnemonic

c.mul  $rd', rs2'$

#### Encoding

|        |    |    |        |   |   |        |   |   |      |      |   |    |   |
|--------|----|----|--------|---|---|--------|---|---|------|------|---|----|---|
| 15     | 13 | 12 | 10     | 9 | 7 | 6      | 5 | 4 | 2    | 1    | 0 |    |   |
| 1      | 0  | 0  | 1      | 1 | 1 | rd'    |   | 1 | 0    | rs2' |   | 0  | 1 |
| FUNCT3 |    |    | SRCDST |   |   | FUNCT2 |   |   | SRC2 |      |   | C1 |   |

#### Pseudo-code

```
int32 rd_val = X[rd'+8];
int32 rs1_val = X[rs2'+8];
int64 result_wide = rd_val * rs2_val;
```

```
X[rd'+8] = result_wide[31:0];
```

### 3.9.14. CM.PUSH

#### Description

This instruction pushes (stores) the registers in *reg\_list* to the memory below the stack pointer, and then creates the stack frame by decrementing the stack pointer by *stack\_adj*, including any additional stack space requested by the value of *spimm*.

**NOTE** *rlist* values 0 to 3 are reserved for a future EABI variant called *cm.push.e*

#### Configuration

This instruction is present only when *zcmp* is configured.

#### Mnemonic

*cm.push {reg\_list}, -stack\_adj*

#### Encoding

| 15     | 13 | 12 | 8 | 7 | 4 | 3 | 2 | 1     | 0          |   |   |
|--------|----|----|---|---|---|---|---|-------|------------|---|---|
| 1      | 0  | 1  | 1 | 1 | 0 | 0 | 0 | rlist | spimm[5:4] | 1 | 0 |
| FUNCT3 |    |    |   |   |   |   |   | C2    |            |   |   |

#### Pseudo-code

```
#ifdef SUPPORT_RV32E
// When RV32E is configured as base ISA
uint<XLEN> addr_t;

switch (rlist) {
    case 4: {reg_list="ra";      xreg_list = {1};}
    case 5: {reg_list="ra, s0";  xreg_list = {1,8};}
    case 6: {reg_list="ra, s0-s1"; xreg_list = {1,8,9};}
    default: ExceptionTaken(ILLEGAL_INSTRUCTION);
}

stack_adj = 16 + spimm*16;

addr_t = X[2] - 4;
for (i in xreg_list) {
    MemStore(X[i], addr_t, Word, AccType_Normal);
    addr_t = addr_t - 4;
}

X[2] = X[2] - stack_adj;

#else
// When RV32I is configured as base ISA,
uint<XLEN> addr_t;

switch (rlist) {
    case 4: {
        stack_adj_base = 16;
```

```

    reg_list = "ra";
    xreg_list = {1};}
case 5: {
    stack_adj_base = 16;
    reg_list="ra, s0";
    xreg_list={1, 8};}
case 6: {
    stack_adj_base = 16;
    reg_list="ra, s0-s1";
    xreg_list={1, 8, 9};}
case 7: {
    stack_adj_base = 16;
    reg_list="ra, s0-s2";
    xreg_list={1, 8, 9, 18};}
case 8: {
    stack_adj_base = 32;
    reg_list="ra, s0-s3";
    xreg_list={1, 8, 9, 18, 19};}
case 9: {
    stack_adj_base = 32;
    reg_list="ra, s0-s4";
    xreg_list={1, 8, 9, 18, 19, 20};}
case 10: {
    stack_adj_base = 32;
    reg_list="ra, s0-s5";
    xreg_list={1, 8, 9, 18, 19, 20, 21};}
case 11: {
    stack_adj_base = 32;
    reg_list="ra, s0-s6";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22};}
case 12: {
    stack_adj_base = 48;
    reg_list="ra, s0-s7";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23};}
case 13: {
    stack_adj_base = 48;
    reg_list="ra, s0-s8";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23, 24};}
case 14: {
    stack_adj_base = 48;
    reg_list="ra, s0-s9";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23, 24, 25};}
case 15: {
    stack_adj_base = 64;
    reg_list="ra, s0-s11";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27};}
default:
    ExceptionTaken(ILLEGAL_INSTRUCTION);
}

stack_adj = stack_adj_base + spimm*16;

addr_t = X[2] - 4;

```

```
for (i in xreg_list) {  
    MemStore(X[i], addr_t, Word, AccType_Normal);  
    addr_t = addr_t - 4;  
}  
  
X[2] = X[2] - stack_adj;  
#endif
```



### 3.9.15. CM.POP

#### Description

This instruction pops (loads) the registers in *reg\_list* from stack memory, and then adjusts the stack pointer by *stack\_adj*.

**NOTE** *rlist* values 0 to 3 are reserved for a future EABI variant called *cm.pop.e*

#### Configuration

This instruction is present only when *zcmp* is configured.

#### Mnemonic

cm.pop {*reg\_list*}, *stack\_adj*

#### Encoding

|        |    |    |   |   |   |  |   |       |   |            |   |   |
|--------|----|----|---|---|---|--|---|-------|---|------------|---|---|
| 15     | 13 | 12 |   | 8 | 7 |  | 4 | 3     | 2 | 1          | 0 |   |
| 1      | 0  | 1  | 1 | 1 | 0 |  |   | rlist |   | spimm[5:4] | 1 | 0 |
| FUNCT3 |    |    |   |   |   |  |   |       |   | C2         |   |   |

#### Pseudo-code

```
#ifdef SUPPORT_RV32E
// When RV32E is configured as base ISA
uint<XLEN> addr_t;

switch (rlist) {
    case 4: {reg_list="ra";      xreg_list = {1};}
    case 5: {reg_list="ra, s0";  xreg_list = {1,8};}
    case 6: {reg_list="ra, s0-s1"; xreg_list = {1,8,9};}
    default: ExceptionTaken(ILLEGAL_INSTRUCTION);
}

stack_adj = 16 + spimm*16;

addr_t = X[2] + stack_adj - 4;
for (i in xreg_list) {
    X[i] = MemLoad(addr_t, Word, AccType_Normal);
    addr_t = addr_t - 4;
}

X[2] = X[2] + stack_adj;

#else
// When RV32I is configured as base ISA
uint<XLEN> addr_t;

switch (rlist) {
    case 4: {
        stack_adj_base = 16;
        reg_list = "ra";
        xreg_list = {1};
    }
}
```

```

case 5: {
    stack_adj_base = 16;
    reg_list="ra, s0";
    xreg_list={1, 8};}
case 6: {
    stack_adj_base = 16;
    reg_list="ra, s0-s1";
    xreg_list={1, 8, 9};}
case 7: {
    stack_adj_base = 16;
    reg_list="ra, s0-s2";
    xreg_list={1, 8, 9, 18};}
case 8: {
    stack_adj_base = 32;
    reg_list="ra, s0-s3";
    xreg_list={1, 8, 9, 18, 19};}
case 9: {
    stack_adj_base = 32;
    reg_list="ra, s0-s4";
    xreg_list={1, 8, 9, 18, 19, 20};}
case 10: {
    stack_adj_base = 32;
    reg_list="ra, s0-s5";
    xreg_list={1, 8, 9, 18, 19, 20, 21};}
case 11: {
    stack_adj_base = 32;
    reg_list="ra, s0-s6";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22};}
case 12: {
    stack_adj_base = 48;
    reg_list="ra, s0-s7";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23};}
case 13: {
    stack_adj_base = 48;
    reg_list="ra, s0-s8";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23, 24};}
case 14: {
    stack_adj_base = 48;
    reg_list="ra, s0-s9";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23, 24, 25};}
case 15: {
    stack_adj_base = 64;
    reg_list="ra, s0-s11";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27};}
default:
    ExceptionTaken(ILLEGAL_INSTRUCTION);
}

stack_adj = stack_adj_base + spimm*16;

addr_t = X[2] + stack_adj - 4;
for (i in xreg_list) {
    X[i] = MemLoad(addr_t, Word, AccType_Normal);

```

```
    addr_t = addr_t - 4;  
}  
  
X[2] = X[2] + stack_adj;  
#endif
```

### 3.9.16. CM.POPRET

#### Description

This instruction pops (loads) the registers in *reg\_list* from stack memory, adjusts the stack pointer by *stack\_adj* and then returns to *ra*.

**NOTE** *rlist* values 0 to 3 are reserved for a future EABI variant called *cm.popret.e*

#### Configuration

This instruction is present only when *zcmp* is configured.

#### Mnemonic

*cm.popret {reg\_list}, stack\_adj*

#### Encoding

|        |    |    |   |   |   |   |              |    |                    |   |   |
|--------|----|----|---|---|---|---|--------------|----|--------------------|---|---|
| 15     | 13 | 12 |   | 8 | 7 |   | 4            | 3  | 2                  | 1 | 0 |
| 1      | 0  | 1  | 1 | 1 | 1 | 0 | <i>rlist</i> |    | <i>spimm</i> [5:4] | 1 | 0 |
| FUNCT3 |    |    |   |   |   |   |              | C2 |                    |   |   |

#### Pseudo-code

```
#ifdef SUPPORT_RV32E
// When RV32E is configured as base ISA
uint<XLEN> addr_t;
uint<XLEN> target_pc;

switch (rlist) {
    case 4: {reg_list="ra";      xreg_list = {1};}
    case 5: {reg_list="ra, s0";  xreg_list = {1,8};}
    case 6: {reg_list="ra, s0-s1"; xreg_list = {1,8,9};}
    default: ExceptionTaken(ILLEGAL_INSTRUCTION);
}

stack_adj = 16 + spimm*16;

addr_t = X[2] + stack_adj - 4;
for (i in xreg_list) {
    X[i] = MemLoad(addr_t, Word, AccType_Normal);
    addr_t = addr_t - 4;
}

X[2] = X[2] + stack_adj;

target_pc = X[1][XLEN-1:1] :: 1'b0;
BranchTo(target_pc, BranchType_RET, Unconditional);

#else
// When RV32I is configured as base ISA
uint<XLEN> addr_t;
uint<XLEN> target_pc;
```

```

switch (rlist) {
  case 4: {
    stack_adj_base = 16;
    reg_list = "ra";
    xreg_list = {1};}
  case 5: {
    stack_adj_base = 16;
    reg_list="ra, s0";
    xreg_list={1, 8};}
  case 6: {
    stack_adj_base = 16;
    reg_list="ra, s0-s1";
    xreg_list={1, 8, 9};}
  case 7: {
    stack_adj_base = 16;
    reg_list="ra, s0-s2";
    xreg_list={1, 8, 9, 18};}
  case 8: {
    stack_adj_base = 32;
    reg_list="ra, s0-s3";
    xreg_list={1, 8, 9, 18, 19};}
  case 9: {
    stack_adj_base = 32;
    reg_list="ra, s0-s4";
    xreg_list={1, 8, 9, 18, 19, 20};}
  case 10: {
    stack_adj_base = 32;
    reg_list="ra, s0-s5";
    xreg_list={1, 8, 9, 18, 19, 20, 21};}
  case 11: {
    stack_adj_base = 32;
    reg_list="ra, s0-s6";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22};}
  case 12: {
    stack_adj_base = 48;
    reg_list="ra, s0-s7";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23};}
  case 13: {
    stack_adj_base = 48;
    reg_list="ra, s0-s8";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23, 24};}
  case 14: {
    stack_adj_base = 48;
    reg_list="ra, s0-s9";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23, 24, 25};}
  case 15: {
    stack_adj_base = 64;
    reg_list="ra, s0-s11";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27};}
  default:
    ExceptionTaken(ILLEGAL_INSTRUCTION);
}

```

```
stack_adj = stack_adj_base + spimm*16;

addr_t = X[2] + stack_adj - 4;
for (i in xreg_list) {
    X[i] = MemLoad(addr_t, Word, AccType_Normal);
    addr_t = addr_t - 4;
}

X[2] = X[2] + stack_adj;

target_pc = X[1][XLEN-1:1] :: 1'b0;
BranchTo(target_pc, BranchType_RET, Unconditional);
#endif
```

### 3.9.17. CM.POPRETZ

#### Description

This instruction pops (loads) the registers in *reg\_list* from stack memory, adjusts the stack pointer by *stack\_adj*, moves zero into *a0* and then returns to *ra*.

**NOTE** *rlist* values 0 to 3 are reserved for a future EABI variant called *cm.popret.e*

#### Configuration

This instruction is present only when *zcmp* is configured.

#### Mnemonic

*cm.popretz {reg\_list}, stack\_adj*

#### Encoding

|        |    |    |   |   |   |   |              |    |                    |   |   |
|--------|----|----|---|---|---|---|--------------|----|--------------------|---|---|
| 15     | 13 | 12 |   | 8 | 7 |   | 4            | 3  | 2                  | 1 | 0 |
| 1      | 0  | 1  | 1 | 1 | 0 | 0 | <i>rlist</i> |    | <i>spimm</i> [5:4] | 1 | 0 |
| FUNCT3 |    |    |   |   |   |   |              | C2 |                    |   |   |

#### Pseudo-code

```
#ifdef SUPPORT_RV32E
// When RV32E is configured as base ISA,
uint<XLEN> addr_t;
uint<XLEN> target_pc;

switch (rlist) {
    case 4: {reg_list="ra";      xreg_list = {1};}
    case 5: {reg_list="ra, s0";  xreg_list = {1,8};}
    case 6: {reg_list="ra, s0-s1"; xreg_list = {1,8,9};}
    default: ExceptionTaken(ILLEGAL_INSTRUCTION);
}

stack_adj = 16 + spimm*16;

addr_t = X[2] + stack_adj - 4;
for (i in xreg_list) {
    X[i] = MemLoad(addr_t, Word, AccType_Normal);
    addr_t = addr_t - 4;
}

X[2] = X[2] + stack_adj;

X[10] = 0;

target_pc = X[1][XLEN-1:1] :: 1'b0;
BranchTo(target_pc, BranchType_RET, Unconditional);

#else
// When RV32I is configured as base ISA,
uint<XLEN> addr_t;
```

```
uint<XLEN> target_pc;

switch (rlist) {
  case 4: {
    stack_adj_base = 16;
    reg_list = "ra";
    xreg_list = {1};}
  case 5: {
    stack_adj_base = 16;
    reg_list="ra, s0";
    xreg_list={1, 8};}
  case 6: {
    stack_adj_base = 16;
    reg_list="ra, s0-s1";
    xreg_list={1, 8, 9};}
  case 7: {
    stack_adj_base = 16;
    reg_list="ra, s0-s2";
    xreg_list={1, 8, 9, 18};}
  case 8: {
    stack_adj_base = 32;
    reg_list="ra, s0-s3";
    xreg_list={1, 8, 9, 18, 19};}
  case 9: {
    stack_adj_base = 32;
    reg_list="ra, s0-s4";
    xreg_list={1, 8, 9, 18, 19, 20};}
  case 10: {
    stack_adj_base = 32;
    reg_list="ra, s0-s5";
    xreg_list={1, 8, 9, 18, 19, 20, 21};}
  case 11: {
    stack_adj_base = 32;
    reg_list="ra, s0-s6";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22};}
  case 12: {
    stack_adj_base = 48;
    reg_list="ra, s0-s7";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23};}
  case 13: {
    stack_adj_base = 48;
    reg_list="ra, s0-s8";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23, 24};}
  case 14: {
    stack_adj_base = 48;
    reg_list="ra, s0-s9";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23, 24, 25};}
  case 15: {
    stack_adj_base = 64;
    reg_list="ra, s0-s11";
    xreg_list={1, 8, 9, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27};}
  default:
    ExceptionTaken(ILLEGAL_INSTRUCTION);
}
```



```
}

stack_adj = stack_adj_base + spimm*16;

addr_t = X[2] + stack_adj - 4;
for (i in xreg_list) {
    X[i] = MemLoad(addr_t, Word, AccType_Normal);
    addr_t = addr_t - 4;
}

X[2] = X[2] + stack_adj;

X[10] = 0;

target_pc = X[1][XLEN-1:1] :: 1'b0;
BranchTo(target_pc, BranchType_RET, Unconditional);
#endif
```

### 3.9.18. CM.MVA01S

#### Description

This instruction moves *r1sc* into *a0* and *r2sc* into *a1*. The execution is atomic, so it is not possible to observe state where only one of *a0* or *a1* have been updated.

#### NOTE

The s register mapping is taken from the UABI, and may not match the currently unratified EABI. *cm.mva01s* may be included in the future.

#### Configuration

This instruction is present only when *zcmp* is configured.

#### Mnemonic

*cm.mva01s r1sc, r2sc*

#### Encoding

|        |    |    |    |   |   |      |   |   |   |      |   |   |   |
|--------|----|----|----|---|---|------|---|---|---|------|---|---|---|
| 15     | 13 | 12 | 10 | 9 | 7 | 6    | 5 | 4 | 2 | 1    | 0 |   |   |
| 1      | 0  | 1  | 0  | 1 | 1 | r1sc |   | 1 | 1 | r2sc |   | 1 | 0 |
| FUNCT3 |    |    |    |   |   | C2   |   |   |   |      |   |   |   |

#### Pseudo-code

```
uint5 xreg1;
uint5 xreg2;

#ifdef SUPPORT_RV32E
    if ((r1sc>1) || (r2sc>1)) {
        ExceptionTaken(ILLEGAL_INSTRUCTION);
    }
#endif

xreg1 = r1sc[2:1]>0 :: r1sc[2:1]==0 :: r1sc[2:0];
xreg2 = r2sc[2:1]>0 :: r2sc[2:1]==0 :: r2sc[2:0];
X[10] = X[xreg1];
X[11] = X[xreg2];
```

### 3.9.19. CM.MVSA01

#### Description

This instruction moves *a0* into *r1sc* and *a1* into *r2sc*. *r1sc* and *r2sc* must be different. Otherwise an illegal instruction exception is issued. The execution is atomic, so it is not possible to observe state where only one of *r1sc* or *r2sc* has been updated.

#### NOTE

The s register mapping is taken from the UABI, and may not match the currently unrati ed EABI. *cm.mvsa01* may be included in the future.

#### Configuration

This instruction is present only when *zcmp* is configured.

#### Mnemonic

*cm.mvsa01 r1sc, r2sc*

#### Encoding

|        |    |    |    |   |   |      |   |   |      |    |   |
|--------|----|----|----|---|---|------|---|---|------|----|---|
| 15     | 13 | 12 | 10 | 9 | 7 | 6    | 5 | 4 | 2    | 1  | 0 |
| 1      | 0  | 1  | 0  | 1 | 1 | r1sc | 0 | 1 | r2sc | 1  | 0 |
| FUNCT3 |    |    |    |   |   |      |   |   |      | C2 |   |

#### Pseudo-code

```
uint5 xreg1;
uint5 xreg2;

if (r1sc==r2sc) {
    ExceptionTaken(ILLEGAL_INSTRUCTION);
}

#ifdef SUPPORT_RV32E
    if ((r1sc>1) || (r2sc>1)) {
        ExceptionTaken(ILLEGAL_INSTRUCTION);
    }
#endif

xreg1 = r1sc[2:1]>0 :: r1sc[2:1]==0 :: r1sc[2:0];
xreg2 = r2sc[2:1]>0 :: r2sc[2:1]==0 :: r2sc[2:0];
X[xreg1] = X[10];
X[xreg2] = X[11];
```

### 3.9.20. CM.JALT

#### Description

*cm.jalt* reads an entry from the jump vector table in memory and jumps to the address that was read, linking to *ra*.

#### Configuration

This instruction is present only when *zcmt* is configured.

#### Mnemonic

*cm.jalt* *index*

#### Encoding

| 15     | 13 | 12 | 10 | 9 | 2 | 1     | 0   |
|--------|----|----|----|---|---|-------|-----|
| 1      | 0  | 1  | 0  | 0 | 0 | index | 1 0 |
| FUNCT3 |    |    |    |   |   | C2    |     |

#### NOTE

It is decoded as CM.JALT under the condition  $\text{index} \geq 32$ .

#### Pseudo-code

```
uint<XLEN> table_address;
uint<XLEN> target_address;

if (jvt.mode!=0) {
    ExceptionTaken(ILLEGAL_INSTRUCTION);
}

table_address = JVT.base + (index<<2);
target_address = MemLoad(table_address, Word, AccType_Normal);

X[1] = PC + 2;
BranchTo(target_address & ~0x1, BranchType_DIRCALL, Unconditional);
```

### 3.9.21. CM.JT

#### Description

*cm.jt* reads an entry from the jump vector table in memory and jumps to the address that was read.

#### Configuration

This instruction is present only when *zcmt* is configured.

#### Mnemonic

*cm.jt* *index*

#### Encoding

| 15     | 13 | 12 | 10 | 9 |   | 2     | 1  | 0 |
|--------|----|----|----|---|---|-------|----|---|
| 1      | 0  | 1  | 0  | 0 | 0 | index | 1  | 0 |
| FUNCT3 |    |    |    |   |   |       | C2 |   |

#### NOTE

It is decoded as CM.JT under the condition  $\text{index} < 32$ .

#### Pseudo-code

```
uint<XLEN> table_address;
uint<XLEN> target_address;

if (jvt.mode!=0) {
    ExceptionTaken(ILLEGAL_INSTRUCTION);
}

table_address = JVT.base + (index<<2);
target_address = MemLoad(table_address, Word, AccType_Normal);

BranchTo(target_address & ~0x1, BranchType_DIR, Unconditional);
```

## 3.10. RISC-V Integer Conditional (Zicond) Operations Extension

The "Conditional" operations extension provides a simple solution that provides most of the benefit and all of the flexibility one would desire to support conditional arithmetic and conditional-select/move operations, while remaining true to the RISC-V design philosophy. The instructions follow the format for R-type instructions with 3 operands (i.e., 2 source operands and 1 destination operand). Using these instructions, branchless sequences can be implemented (typically in two instruction sequences) without the need for instruction fusion, special provisions during the decoding of architectural instructions, or other microarchitectural provisions.

The following instructions comprise the zicond extension:

| Mnemonic                            | Instruction                                     | zicond |
|-------------------------------------|-------------------------------------------------|--------|
| <code>czero.eqz rd, rs1, rs2</code> | Conditional zero, if condition is equal to zero | ✓      |
| <code>czero.nez rd, rs1, rs2</code> | Conditional zero, if condition is nonzero       | ✓      |

Based on these two instructions, synthetic instructions (i.e., short instruction sequences) for the following **conditional arithmetic** operations are supported:

- conditional add, if zero
- conditional add, if non-zero
- conditional subtract, if zero
- conditional subtract, if non-zero
- conditional bitwise-and, if zero
- conditional bitwise-and, if non-zero
- conditional bitwise-or, if zero
- conditional bitwise-or, if non-zero
- conditional bitwise-xor, if zero
- conditional bitwise-xor, if non-zero

Additionally, the following **conditional select** instructions are supported:

- conditional-select, if zero
- conditional-select, if non-zero

More complex conditions, such as comparisons against immediates, registers, single-bit tests, comparisons against ranges, etc. can be realized by composing these new instructions with existing instructions.

### 3.10.1. Usage Examples

The instructions from this extension can be used to construct sequences that perform conditional arithmetic, conditional-bitwise-logical, and conditional-select operations.

#### 3.10.1.1. Instruction Sequences

| Operation                                                                            | Instruction sequence                                                |                                                                                      | Length                            |
|--------------------------------------------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------|
| <b>Conditional add, if zero</b><br>$rd = (rc == 0) ? (rs1 + rs2) : rs1$              | <code>czero.nez</code><br><code>add</code>                          | <code>rd, rs2, rc</code><br><code>rd, rs1, rd</code>                                 | 2 insns                           |
| <b>Conditional add, if non-zero</b><br>$rd = (rc != 0) ? (rs1 + rs2) : rs1$          | <code>czero.eqz</code><br><code>add</code>                          | <code>rd, rs2, rc</code><br><code>rd, rs1, rd</code>                                 |                                   |
| <b>Conditional subtract, if zero</b><br>$rd = (rc == 0) ? (rs1 - rs2) : rs1$         | <code>czero.nez</code><br><code>sub</code>                          | <code>rd, rs2, rc</code><br><code>rd, rs1, rd</code>                                 |                                   |
| <b>Conditional subtract, if non-zero</b><br>$rd = (rc != 0) ? (rs1 - rs2) : rs1$     | <code>czero.eqz</code><br><code>sub</code>                          | <code>rd, rs2, rc</code><br><code>rd, rs1, rd</code>                                 |                                   |
| <b>Conditional bitwise-or, if zero</b><br>$rd = (rc == 0) ? (rs1   rs2) : rs1$       | <code>czero.nez</code><br><code>or</code>                           | <code>rd, rs2, rc</code><br><code>rd, rs1, rd</code>                                 |                                   |
| <b>Conditional bitwise-or, if non-zero</b><br>$rd = (rc != 0) ? (rs1   rs2) : rs1$   | <code>czero.eqz</code><br><code>or</code>                           | <code>rd, rs2, rc</code><br><code>rd, rs1, rd</code>                                 |                                   |
| <b>Conditional bitwise-xor, if zero</b><br>$rd = (rc == 0) ? (rs1 ^ rs2) : rs1$      | <code>czero.nez</code><br><code>xor</code>                          | <code>rd, rs2, rc</code><br><code>rd, rs1, rd</code>                                 |                                   |
| <b>Conditional bitwise-xor, if non-zero</b><br>$rd = (rc != 0) ? (rs1 ^ rs2) : rs1$  | <code>czero.eqz</code><br><code>xor</code>                          | <code>rd, rs2, rc</code><br><code>rd, rs1, rd</code>                                 |                                   |
| <b>Conditional bitwise-and, if zero</b><br>$rd = (rc == 0) ? (rs1 \& rs2) : rs1$     | <code>and</code><br><code>czero.eqz</code><br><code>or</code>       | <code>rd, rs1, rs2</code><br><code>rtmp, rs1, rc</code><br><code>rd, rd, rtmp</code> |                                   |
| <b>Conditional bitwise-and, if non-zero</b><br>$rd = (rc != 0) ? (rs1 \& rs2) : rs1$ | <code>and</code><br><code>czero.nez</code><br><code>or</code>       | <code>rd, rs1, rs2</code><br><code>rtmp, rs1, rc</code><br><code>rd, rd, rtmp</code> | 3 insns<br>(requires 1 temporary) |
| <b>Conditional select, if zero</b><br>$rd = (rc == 0) ? rs1 : rs2$                   | <code>czero.nez</code><br><code>czero.eqz</code><br><code>or</code> | <code>rd, rs1, rc</code><br><code>rtmp, rs2, rc</code><br><code>rd, rd, rtmp</code>  |                                   |
| <b>Conditional select, if non-zero</b><br>$rd = (rc != 0) ? rs1 : rs2$               | <code>czero.eqz</code><br><code>czero.nez</code><br><code>or</code> | <code>rd, rs1, rc</code><br><code>rtmp, rs2, rc</code><br><code>rd, rd, rtmp</code>  |                                   |

### Description

## Configuration

### Mnemonic

## Encoding



Page 192



### 3.10.3. CZERO.NEZ

#### Description

If *rs2* contains a nonzero value, this instruction writes the value zero to *rd*. Otherwise, this instruction copies the contents of *rs1* to *rd*.

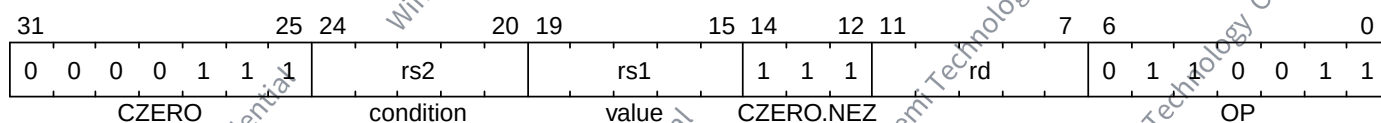
#### Configuration

This instruction is present only when *zicond* is configured.

#### Mnemonic

`czero.nez rd, rs1, rs2`

#### Encoding



#### Pseudo-code

```
X[rd] = (X[rs2] != 0) ? 0 : X[rs1];
```



## 3.12. Wingsemi Instruction Extensions

Wing-M130 processor implements additional instruction set extensions to enhance the base RISC-V architecture, which provides the combination of performance improvement, code-size reduction and also power consumption reduction. Therefore, applications, running on Wing-M130 processor, could benefit from these extensions and experiences a better performance, smaller code-size, lower power consumption than a processor with basic RISC-V architecture.

The groups of these extensions are \* Conditional Branch Compare with Register-Immediate xcci \* Register-Immediate/Register Load/Store with Post/Pre-Increment xcls \* Pre-shifted Arithmetic Instructions xcpa \* Byte Reverse Instructions xberv

| Mnemonic                                  | Instruction                                             | xcci | xcls | xcpa | xberv |
|-------------------------------------------|---------------------------------------------------------|------|------|------|-------|
| addshf rd, rs1, rs2 { < shtype >, shamt } | Add with a shifted register                             |      |      |      |       |
| andshf rd, rs1, rs2 { < shtype >, shamt } | Bitwise AND with a shifted register                     |      |      | ✓    |       |
| beqi rs1, cmpimm, <label>                 | Compare and Branch if equal                             | ✓    |      |      |       |
| bgei rs1, cmpimm, <label>                 | Signed Compare and Branch if greater than or equal to   | ✓    |      |      |       |
| bgeui rs1, cmpimm, <label>                | Unsigned Compare and Branch if greater than or equal to | ✓    |      |      |       |
| blti rs1, cmpimm, <label>                 | Signed Compare and Branch if less than                  | ✓    |      |      |       |
| bltui rs1, cmpimm, <label>                | Unsigned Compare and Branch if less than                | ✓    |      |      |       |
| bnei rs1, cmpimm, <label>                 | Compare and Branch if unequal                           | ✓    |      |      |       |
| lb.post rd, offset(rs1!)                  | Load byte, post-increment with imm-offset               |      | ✓    |      |       |
| lb.pre rd, [offset(rs1)]!                 | Load byte, pre-increment with imm-offset                |      | ✓    |      |       |
| lbr rd, rs2(rs1)                          | Load byte with register offset                          |      | ✓    |      |       |
| lbr.post rd, rs2(rs1!)                    | Load byte, post-increment with reg-offset               |      | ✓    |      |       |
| lbr.pre rd, [rs2(rs1)]!                   | Load byte, pre-increment with reg-offset                |      | ✓    |      |       |
| lbu.post rd, offset(rs1!)                 | Load byte, post-increment with imm-offset               |      | ✓    |      |       |
| lbu.pre rd, [offset(rs1)]!                | Load byte, pre-increment with imm-offset                |      | ✓    |      |       |
| lbur rd, rs2(rs1)                         | Load byte with register offset                          |      | ✓    |      |       |
| lbr.post rd, rs2(rs1!)                    | Load byte, post-increment with reg-offset               |      | ✓    |      |       |
| lbur.pre rd, [rs2(rs1)]!                  | Load byte, pre-increment with reg-offset                |      | ✓    |      |       |
| lh.post rd, offset(rs1!)                  | Load halfword, post-increment with imm-offset           |      | ✓    |      |       |

| Mnemonic                                 | Instruction                                    | xcci | xcls | xcpa | xberv |
|------------------------------------------|------------------------------------------------|------|------|------|-------|
| lh.pre rd, [offset(rs1)]!                | Load halfword, pre-increment with imm-offset   |      | ✓    |      |       |
| lhr rd, rs2(rs1)                         | Load halfword with register offset             |      | ✓    |      |       |
| lhr.post rd, rs2(rs1)!                   | Load halfword, post-increment with reg-offset  |      | ✓    |      |       |
| lhr.pre rd, [rs2(rs1)]!                  | Load halfword, pre-increment with reg-offset   |      | ✓    |      |       |
| lhu.post rd, offset(rs1!)                | Load halfword, post-increment with imm-offset  |      | ✓    |      |       |
| lhu.pre rd, [offset(rs1)]!               | Load halfword, pre-increment with imm-offset   |      | ✓    |      |       |
| lhur rd, rs2(rs1)                        | Load halfword with register offset             |      | ✓    |      |       |
| lhur.post rd, rs2(rs1)!                  | Load halfword, post-increment with reg-offset  |      | ✓    |      |       |
| lhur.pre rd, [rs2(rs1)]!                 | Load halfword, pre-increment with reg-offset   |      | ✓    |      |       |
| lw.post rd, offset(rs1!)                 | Load word, post-increment with imm-offset      |      | ✓    |      |       |
| lw.pre rd, [offset(rs1)]!                | Load word, pre-increment with imm-offset       |      | ✓    |      |       |
| lwr rd, rs2(rs1)                         | Load word with register offset                 |      | ✓    |      |       |
| lwr.post rd, rs2(rs1)!                   | Load word, post-increment with reg-offset      |      | ✓    |      |       |
| lwr.pre rd, [rs2(rs1)]!                  | Load word, pre-increment with reg-offset       |      | ✓    |      |       |
| orshf rd, rs1, rs2 { < shtype >, shamt } | Bitwise OR with a shifted register             |      | ✓    |      |       |
| rev8hw rd, rs1                           | Reverse bytes in two halfword                  |      |      |      | ✓     |
| rev8slhw rd, rs1                         | Reverse bytes in the lower halfword            |      |      |      | ✓     |
| sb.post rs2, offset(rs1!)                | Store byte, post-increment with imm-offset     |      | ✓    |      |       |
| sb.pre rs2, [offset(rs1)]!               | Store byte, pre-increment with imm-offset      |      | ✓    |      |       |
| sbr rd, rs2(rs1)                         | Store byte with register offset                |      | ✓    |      |       |
| sbr.post rd, rs2(rs1)!                   | Store byte, post-increment with reg-offset     |      | ✓    |      |       |
| sbr.pre rd, [rs2(rs1)]!                  | Store byte, pre-increment with reg-offset      |      | ✓    |      |       |
| sh.post rs2, offset(rs1!)                | Store halfword, post-increment with imm-offset |      | ✓    |      |       |
| sh.pre rs2, [offset(rs1)]!               | Store halfword, pre-increment with imm-offset  |      | ✓    |      |       |
| shr rd, rs2(rs1)                         | Store halfword with register offset            |      | ✓    |      |       |
| shr.post rd, rs2(rs1)!                   | Store halfword, post-increment with reg-offset |      | ✓    |      |       |

| Mnemonic                                                                       | Instruction                                   | xcci | xcls | xcpa | xberv |
|--------------------------------------------------------------------------------|-----------------------------------------------|------|------|------|-------|
| shr.pre <i>rd</i> , [ <i>rs2(rs1)</i> ]!                                       | Store halfword, pre-increment with reg-offset |      | ✓    |      |       |
| subshf <i>rd</i> , <i>rs1</i> , <i>rs2</i> { < <i>shtype</i> >, <i>shamt</i> } | Subtract with a shifted register              |      |      | ✓    |       |
| sw.post <i>rs2</i> , offset( <i>rs1</i> !)                                     | Store word, post-increment with imm-offset    |      | ✓    |      |       |
| sw.pre <i>rs2</i> , [offset( <i>rs1</i> )]!                                    | Store word, pre-increment with imm-offset     |      | ✓    |      |       |
| swr <i>rd</i> , <i>rs2(rs1)</i>                                                | Store word with register offset               |      | ✓    |      |       |
| swr.post <i>rd</i> , <i>rs2(rs1)</i>                                           | Store word, post-increment with reg-offset    |      | ✓    |      |       |
| swr.pre <i>rd</i> , [ <i>rs2(rs1)</i> ]!                                       | Store word, pre-increment with reg-offset     |      | ✓    |      |       |
| xorshf <i>rd</i> , <i>rs1</i> , <i>rs2</i> { < <i>shtype</i> >, <i>shamt</i> } | Bitwise XOR with a shifted register           |      |      | ✓    |       |

### 3.12.1. ADDSHF

#### Description

This instruction adds a register *rs2* with an optionally shifted register *rs1*, writes the result to the destination *rd*.

#### Configuration

This instruction is present only when xcpa is configured.

#### Mnemonic

`addshf rd, rs2, rs1 { < shtype >, shamt }`

#### Encoding

|        |    |    |  |  |  |       |    |  |  |  |    |     |  |  |  |    |    |     |  |  |    |    |  |   |   |   |   |  |    |  |   |   |  |  |   |   |   |   |   |   |
|--------|----|----|--|--|--|-------|----|--|--|--|----|-----|--|--|--|----|----|-----|--|--|----|----|--|---|---|---|---|--|----|--|---|---|--|--|---|---|---|---|---|---|
| 31     | 30 | 29 |  |  |  | 25    | 24 |  |  |  | 20 | 19  |  |  |  | 15 | 14 |     |  |  | 12 | 11 |  |   |   | 7 | 6 |  |    |  | 0 |   |  |  |   |   |   |   |   |   |
| shtype |    |    |  |  |  | shamt |    |  |  |  |    | rs2 |  |  |  |    |    | rs1 |  |  | 0  |    |  | 0 | 0 |   |   |  | rd |  |   | 0 |  |  | 1 | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```

int<XLEN> rs1_shft;

rs1_shft = shift_reg(X[rs1], shtype, shamt);

X[rd] = (int) X[rs2] + rs1_shft;

function shift_reg(uint<XLEN> val, uint2 shtype, uint5 shamt) {

    if (shamt==0) {
        ExceptionTaken(ILLEGAL_INSTRUCTION);
    }

    with (shtype) {
        case 'sll':
            // Logical shift left of a bitstring
            return SLL(uint<XLEN> val, uint5 shamt);
        case 'srl':
            // Logical shift right of a bitstring
            return SRL(uint<XLEN> val, uint5 shamt);
        case 'sra':
            // Arithmetical shift right of a bitstring
            return SRA(uint<XLEN> val, uint5 shamt);
        case 'ror':
            // Rotate right of a bitstring
            return ROR(uint<XLEN> val, uint5 shamt);
    }
}

```

### 3.12.2. ANDSHF

#### Description

This instruction bitwise ANDs a register *rs2* with an optionally shifted register *rs1*, writes the result to the destination *rd*.

#### Configuration

This instruction is present only when xcpa is configured.

#### Mnemonic

`andshf rd, rs2, rs1 { < shtype >, shamt }`

#### Encoding

|        |    |    |  |  |  |       |    |  |  |  |    |     |  |  |  |    |    |     |  |  |    |    |  |   |  |   |   |  |  |   |   |  |    |  |  |  |  |  |   |  |  |   |  |  |   |  |  |   |  |  |   |  |  |   |  |  |   |  |  |
|--------|----|----|--|--|--|-------|----|--|--|--|----|-----|--|--|--|----|----|-----|--|--|----|----|--|---|--|---|---|--|--|---|---|--|----|--|--|--|--|--|---|--|--|---|--|--|---|--|--|---|--|--|---|--|--|---|--|--|---|--|--|
| 31     | 30 | 29 |  |  |  | 25    | 24 |  |  |  | 20 | 19  |  |  |  | 15 | 14 |     |  |  | 12 | 11 |  |   |  | 7 | 6 |  |  |   | 0 |  |    |  |  |  |  |  |   |  |  |   |  |  |   |  |  |   |  |  |   |  |  |   |  |  |   |  |  |
| shtype |    |    |  |  |  | shamt |    |  |  |  |    | rs2 |  |  |  |    |    | rs1 |  |  |    |    |  | 1 |  |   | 0 |  |  | 0 |   |  | rd |  |  |  |  |  | 0 |  |  | 1 |  |  | 0 |  |  | 1 |  |  | 0 |  |  | 1 |  |  | 1 |  |  |

#### Pseudo-code

```
uint<XLEN> rs1_shft;

rs1_shft = shift_reg(X[rs1], shtype, shamt);

X[rd] = X[rs2] & rs1_shft;

function shift_reg(uint<XLEN> val, uint2 shtype, uint5 shamt) {

    if (shamt==0) {
        ExceptionTaken(ILLEGAL_INSTRUCTION);
    }

    with (shtype) {
        case 'sll':
            // Logical shift left of a bitstring
            return SLL(uint<XLEN> val, uint5 shamt);
        case 'srl':
            // Logical shift right of a bitstring
            return SRL(uint<XLEN> val, uint5 shamt);
        case 'sra':
            // Arithmetical shift right of a bitstring
            return SRA(uint<XLEN> val, uint5 shamt);
        case 'ror':
            // Rotate right of a bitstring
            return ROR(uint<XLEN> val, uint5 shamt);
    }
}
```



### 3.12.3. BEQI

#### Description

This instruction takes the branch if register *rs1* and the immediate *ci* are equal. The target address is calculated by adding sign-extended 9-bit immediate in multiples of 2 bytes to the address of the branch instruction.

#### Configuration

This instruction is present only when *xcc1* is configured.

#### Mnemonic

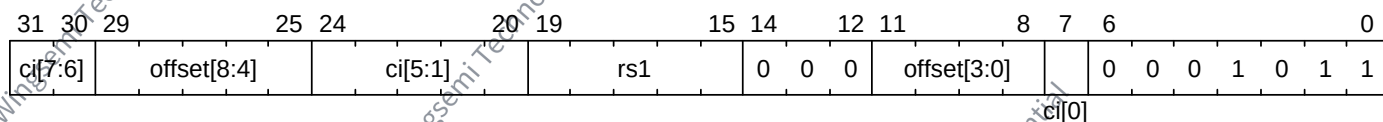
`beqi rs1, ci, <label>`

**<label>** - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int10 offset_t = offset :: 1'b0;
```

```
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Pseudo-code

```
int10 offset_t = offset :: 1'b0;
uint<XLEN> target_pc = PC + SignExt(offset_t, XLEN);

#ifdef SUPPORT_RV32C
    CheckInsnAddrMisalign(target_pc);
#endif

if (X[rs1] == SignExt(ci, XLEN)) {
    BranchTo(target_pc, BranchType_DIR, Conditional);
}
```



### 3.12.4. BGEI

#### Description

This instruction takes the branch if register *rs1* is greater than or equal to the immediate *ci*. Each is treated as signed value. The target address is calculated by adding sign-extended 9-bit immediate in multiples of 2 bytes to the address of the branch instruction.

#### Configuration

This instruction is present only when *xcc1* is configured.

#### Mnemonic

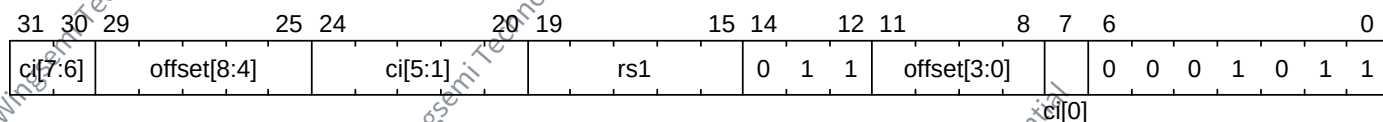
`bgei rs1, ci, <label>`

**<label>** - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int10 offset_t = offset :: 1'b0;
```

```
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Pseudo-code

```
int10 offset_t = offset :: 1'b0;
uint<XLEN> target_pc = PC + SignExt(offset_t, XLEN);

#ifdef SUPPORT_RV32C
    CheckInsnAddrMisalign(target_pc);
#endif

if ((int)X[rs1] >= SignExt(ci, XLEN)) {
    BranchTo(target_pc, BranchType_DIR, Conditional);
}
```

### 3.12.5. BGEUI

#### Description

This instruction takes the branch if register *rs1* is greater than or equal to the immediate *ci*. Each is treated as unsigned value. The target address is calculated by adding sign-extended 9-bit immediate in multiples of 2 bytes to the address of the branch instruction.

#### Configuration

This instruction is present only when *xcc1* is configured.

#### Mnemonic

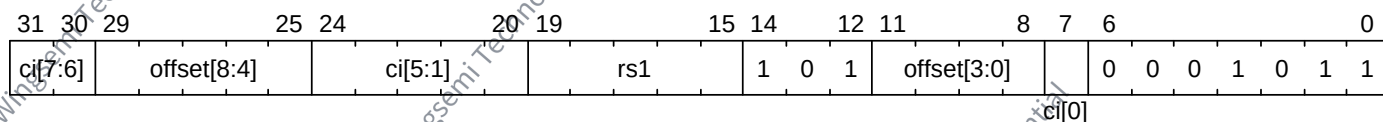
`bgeui rs1, ci, <label>`

**<label>** - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int10 offset_t = offset :: 1'b0;
```

```
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Pseudo-code

```
int10 offset_t = offset :: 1'b0;
uint<XLEN> target_pc = PC + SignExt(offset_t, XLEN);

#ifdef SUPPORT_RV32C
    CheckInsnAddrMisalign(target_pc);
#endif

if ((uint)X[rs1] >= ZeroExt(ci, XLEN)) {
    BranchTo(target_pc, BranchType_DIR, Conditional);
}
```

### 3.12.6. BLTI

#### Description

This instruction takes the branch if register *rs1* is less than the immediate *ci*. Each is treated as signed value. The target address is calculated by adding sign-extended 9-bit immediate in multiples of 2 bytes to the address of the branch instruction.

#### Configuration

This instruction is present only when *xcc1* is configured.

#### Mnemonic

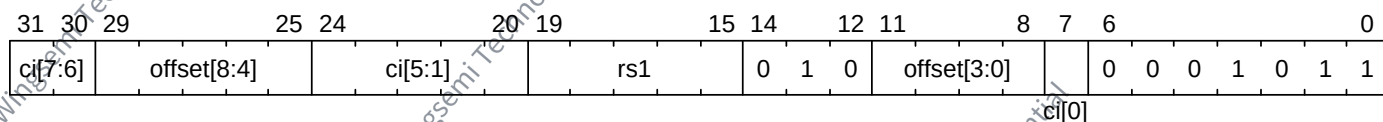
`blti rs1, ci, <label>`

<label> - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int10 offset_t = offset :: 1'b0;
```

```
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Pseudo-code

```
int10 offset_t = offset :: 1'b0;
uint<XLEN> target_pc = PC + SignExt(offset_t, XLEN);

#ifdef SUPPORT_RV32C
    CheckInsnAddrMisalign(target_pc);
#endif

if ((int)X[rs1] < SignExt(ci, XLEN)) {
    BranchTo(target_pc, BranchType_DIR, Conditional);
}
```

### 3.12.7. BLTUI

#### Description

This instruction takes the branch if register *rs1* is less than the immediate *ci*. Each is treated as unsigned value. The target address is calculated by adding sign-extended 9-bit immediate in multiples of 2 bytes to the address of the branch instruction.

#### Configuration

This instruction is present only when *xcc1* is configured.

#### Mnemonic

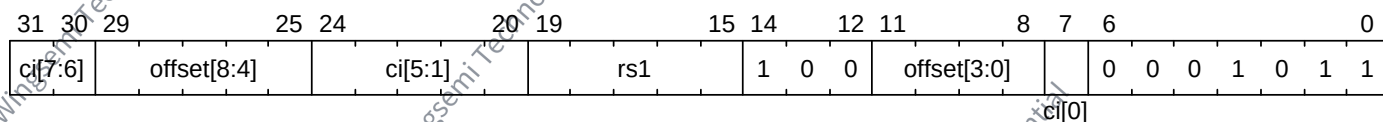
`bltui rs1, ci, <label>`

<*label*> - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int10 offset_t = offset :: 1'b0;
```

```
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Pseudo-code

```
int10 offset_t = offset :: 1'b0;
uint<XLEN> target_pc = PC + SignExt(offset_t, XLEN);

#ifdef SUPPORT_RV32C
    CheckInsnAddrMisalign(target_pc);
#endif

if ((uint)X[rs1] < ZeroExt(ci, XLEN)) {
    BranchTo(target_pc, BranchType_DIR, Conditional);
}
```

### 3.12.8. BNEI

#### Description

This instruction takes the branch if register *rs1* and the immediate *ci* are unequal. The target address is calculated by adding sign-extended 9-bit immediate in multiples of 2 bytes to the address of the branch instruction.

#### Configuration

This instruction is present only when *xcc1* is configured.

#### Mnemonic

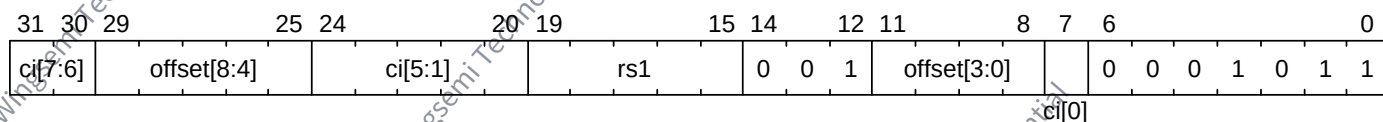
`bnei rs1, ci, <label>`

**<label>** - the label is encoded as a signed PC offset in multiples of 2 bytes, i.e.

```
int10 offset_t = offset :: 1'b0;
```

```
<label> = PC + SignExt(offset_t, XLEN);
```

#### Encoding



#### Pseudo-code

```
int10 offset_t = offset :: 1'b0;
uint<XLEN> target_pc = PC + SignExt(offset_t, XLEN);

#ifdef SUPPORT_RV32C
    CheckInsnAddrMisalign(target_pc);
#endif

if (X[rs1] != SignExt(ci, XLEN)) {
    BranchTo(target_pc, BranchType_DIR, Conditional);
}
```

### 3.12.9. LB.POST

#### Description

This instruction loads a 8-bit value from memory, then sign-extended to 32 bits before storing in *rd*. The address is obtained by *rs1*, and *rs1* is incremented by the sign-extended 8-bit offset.

#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

`lb.post rd, offset(rs1!)`

#### Encoding

| 31 | 28 | 27 |   | 20     | 19 |  | 15 | 14 | 12 | 11  |  | 7 | 6 |   | 0 |    |  |  |   |   |   |   |   |   |   |
|----|----|----|---|--------|----|--|----|----|----|-----|--|---|---|---|---|----|--|--|---|---|---|---|---|---|---|
| 0  | 0  | 0  | 0 | offset |    |  |    |    |    | rs1 |  |   | 1 | 1 | 1 | rd |  |  | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```
X[rd] = SignExt(MemLoad(X[rs1], byte, AccType_Normal), XLEN);
X[rs1] = X[rs1] + SignExt(offset, XLEN);
```

### 3.12.10. LB.PRE

#### Description

This instruction loads a 8-bit value from memory, then sign-extended to 32 bits before storing in *rd*. The address is obtained by adding register *rs1* to the sign-extended 8-bit offset, and is written back to *rs1*.

#### Configuration

This instruction is present only when `xc1s` is configured.

#### Mnemonic

`lb.pre rd, [offset(rs1)]!`

#### Encoding

| 31 | 28 | 27 |   |        |  |  | 20 | 19 |  |     |  |  | 15 | 14 |  | 12 | 11 |   |    |  |  | 7 | 6 |  |   |   |   | 0 |   |   |   |
|----|----|----|---|--------|--|--|----|----|--|-----|--|--|----|----|--|----|----|---|----|--|--|---|---|--|---|---|---|---|---|---|---|
| 1  | 0  | 0  | 0 | offset |  |  |    |    |  | rs1 |  |  |    |    |  | 1  | 1  | 1 | rd |  |  |   |   |  | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```
uint<XLEN> addr_t;
```

```
addr_t = X[rs1] + SignExt(offset, XLEN);
```

```
X[rd] = SignExt(MemLoad(addr_t, byte, AccType_Normal), XLEN);
```

```
X[rs1] = addr_t;
```

### 3.12.11. LBR

#### Description

This instruction loads a 8-bit value from memory, then sign-extended to 32 bits before storing in *rd*. The address is obtained by adding register *rs1* to the register *rs2*.

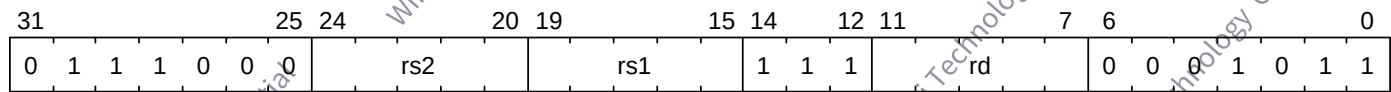
#### Configuration

This instruction is present only when xcl1s is configured.

#### Mnemonic

`lbr rd, rs2(rs1)`

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t = X[rs1] + X[rs2];  
  
X[rd] = SignExt(MemLoad(addr_t, Byte, AccType_Normal), XLEN);
```



### 3.12.12. LBR.POST

#### Description

This instruction loads a 8-bit value from memory, then sign-extended to 32 bits before storing in *rd*. The address is obtained by *rs1*, and *rs1* is incremented by *rs2*.

#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

`lbr.post rd, rs2(rs1!)`

#### Encoding

| 31 | 25 | 24 | 20 | 19 | 15 | 14 | 12  | 11 | 7   | 6 | 0 |   |   |    |  |   |   |   |   |   |   |   |
|----|----|----|----|----|----|----|-----|----|-----|---|---|---|---|----|--|---|---|---|---|---|---|---|
| 0  | 0  | 1  | 1  | 0  | 0  | 0  | rs2 |    | rs1 |   | 1 | 1 | 1 | rd |  | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```
X[rd] = SignExt(MemLoad(X[rs1], byte, AccType_Normal), XLEN);
```

```
X[rs1] = X[rs1] + X[rs2];
```

### 3.12.13. LBR.PRE

#### Description

This instruction loads a 8-bit value from memory, then sign-extended to 32 bits before storing in *rd*. The address is obtained by adding register *rs1* to the register *rs2*, and is written back to *rs1*.

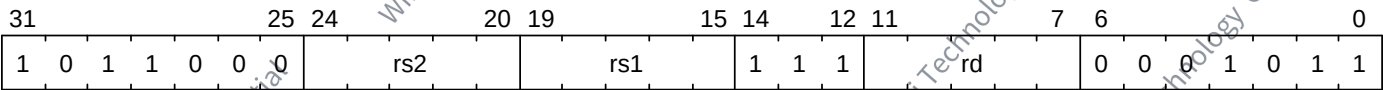
#### Configuration

This instruction is present only when xcl1s is configured.

#### Mnemonic

`lbr.pre rd, [rs2(rs1)]!`

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + X[rs2];
X[rd] = SignExt(MemLoad(addr_t, Byte, AccType_Normal), XLEN);
X[rs1] = addr_t;
```

### 3.12.14. LBU.POST

#### Description

This instruction loads a 8-bit value from memory, then zero-extended to 32 bits before storing in *rd*. The address is obtained by *rs1*, and *rs1* is incremented by the sign-extended 8-bit offset.

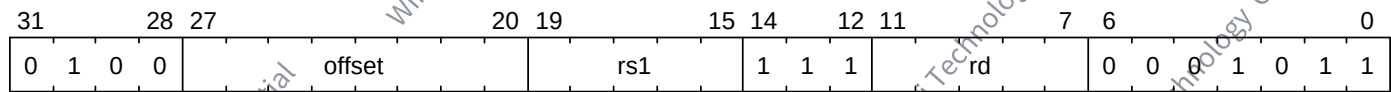
#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

*lbu.post rd, offset(rs1!)*

#### Encoding



#### Pseudo-code

```
X[rd] = ZeroExt(MemLoad(X[rs1], byte, AccType_Normal), XLEN);  
  
X[rs1] = X[rs1] + SignExt(offset, XLEN);
```

### 3.12.15. LBU.PRE

#### Description

This instruction loads a 8-bit value from memory, then zero-extended to 32 bits before storing in *rd*. The address is obtained by adding register *rs1* to the the sign-extended 8-bit offset, and is written back to *rs1*.

#### Configuration

This instruction is present only when xc15 is configured.

#### Mnemonic

`lbu.pre rd, [offset(rs1)]!`

#### Encoding

| 31 | 28 | 27 |   |        |  |  |  |  |  |  |  |  | 20 | 19  |  |  |  |   | 15 | 14 |    |  | 12 | 11 |  |  |   |   | 7 | 6 |   |   |  |  | 0 |
|----|----|----|---|--------|--|--|--|--|--|--|--|--|----|-----|--|--|--|---|----|----|----|--|----|----|--|--|---|---|---|---|---|---|--|--|---|
| 1  | 1  | 0  | 0 | offset |  |  |  |  |  |  |  |  |    | rs1 |  |  |  | 1 | 1  | 1  | rd |  | 0  |    |  |  | 0 | 0 | 1 | 0 | 1 | 1 |  |  |   |

#### Pseudo-code

```
uint<XLEN> addr_t;
```

```
addr_t = X[rs1] + SignExt(offset, XLEN);
```

```
X[rd] = ZeroExt(MemLoad(addr_t, byte, AccType_Normal), XLEN);
```

```
X[rs1] = addr_t;
```

### 3.12.16. LBUR

#### Description

This instruction loads a 8-bit value from memory, then zero-extended to 32 bits before storing in *rd*. The address is obtained by adding register *rs1* to the register *rs2*.

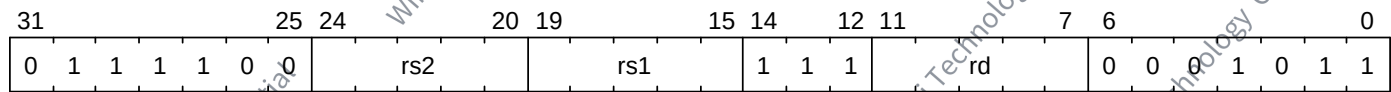
#### Configuration

This instruction is present only when xcl1s is configured.

#### Mnemonic

lbur *rd*, *rs2*(*rs1*)

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t = X[rs1] + X[rs2];  
  
X[rd] = ZeroExt(MemLoad(addr_t, Byte, AccType_Normal), XLEN);
```

### 3.12.17. LBUR.POST

#### Description

This instruction loads a 8-bit value from memory, then zero-extended to 32 bits before storing in *rd*. The address is obtained by *rs1*, and *rs1* is incremented by *rs2*.

#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

*lbur.post rd, rs2(rs1!)*

#### Encoding

| 31 | 25 | 24 | 20 | 19 | 15 | 14 | 12  | 11 | 7   | 6 | 0 |   |   |    |  |   |   |   |   |   |   |   |
|----|----|----|----|----|----|----|-----|----|-----|---|---|---|---|----|--|---|---|---|---|---|---|---|
| 0  | 0  | 1  | 1  | 1  | 0  | 0  | rs2 |    | rs1 |   | 1 | 1 | 1 | rd |  | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```
X[rd] = ZeroExt(MemLoad(X[rs1], byte, AccType_Normal), XLEN);
```

```
X[rs1] = X[rs1] + X[rs2];
```

### 3.12.18. LBUR.PRE

#### Description

This instruction loads a 8-bit value from memory, then zero-extended to 32 bits before storing in *rd*. The address is obtained by adding register *rs1* to the register *rs2*, and is written back to *rs1*.

#### Configuration

This instruction is present only when `xc1s` is configured.

#### Mnemonic

`lbur.pre rd, [rs2(rs1)]!`

#### Encoding

| 31 | 25 | 24 | 20 | 19 | 15 | 14 | 12  | 11 | 7   | 6 | 0 |   |   |    |  |   |   |   |   |   |   |   |
|----|----|----|----|----|----|----|-----|----|-----|---|---|---|---|----|--|---|---|---|---|---|---|---|
| 1  | 0  | 1  | 1  | 1  | 0  | 0  | rs2 |    | rs1 |   | 1 | 1 | 1 | rd |  | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + X[rs2];
X[rd] = ZeroExt(MemLoad(addr_t, byte, AccType_Normal), XLEN);
X[rs1] = addr_t;
```

### 3.12.19. LH.POST

#### Description

This instruction loads a 16-bit value from memory, then sign-extended to 32 bits before storing in *rd*. The address is obtained by *rs1*, and *rs1* is incremented by the sign-extended 8-bit offset in multiples of 2 bytes.

#### Configuration

This instruction is present only when xc15 is configured.

#### Mnemonic

`lh.post rd, offset(rs1!)`

#### Encoding

| 31 | 28 | 27 |   |             |  |  |  |  |  |  |  |  |  | 20 | 19  |  |  |  |   | 15 | 14 |    |  | 12 | 11 |   |   |   |   | 7 | 6 |  |  |  |  | 0 |
|----|----|----|---|-------------|--|--|--|--|--|--|--|--|--|----|-----|--|--|--|---|----|----|----|--|----|----|---|---|---|---|---|---|--|--|--|--|---|
| 0  | 0  | 0  | 1 | offset[8:1] |  |  |  |  |  |  |  |  |  |    | rs1 |  |  |  | 1 | 1  | 1  | rd |  | 0  | 0  | 0 | 1 | 0 | 1 | 1 |   |  |  |  |  |   |

#### Pseudo-code

```
X[rd] = SignExt(MemLoad(X[rs1], Halfword, AccType_Normal), XLEN);
```

```
X[rs1] = X[rs1] + SignExt(offset::1'b0, XLEN);
```



### 3.12.20. LH.PRE

#### Description

This instruction loads a 16-bit value from memory, then sign-extended to 32 bits before storing in *rd*. The address is obtained by adding register *rs1* to the sign-extended 8-bit offset in multiples of 2 bytes, and is written back to *rs1*.

#### Configuration

This instruction is present only when xc15 is configured.

#### Mnemonic

*lh.pre rd, [offset(rs1)]!*

#### Encoding

| 31 | 28 | 27 |   |             |  |  |  |  |  |  |  |  |  |  | 20 | 19  |  |  |  |   | 15 | 14 | 12 | 11 |  |  |   |   | 7 | 6 |   |   |   |  | 0 |
|----|----|----|---|-------------|--|--|--|--|--|--|--|--|--|--|----|-----|--|--|--|---|----|----|----|----|--|--|---|---|---|---|---|---|---|--|---|
| 1  | 0  | 0  | 1 | offset[8:1] |  |  |  |  |  |  |  |  |  |  |    | rs1 |  |  |  | 1 | 1  | 1  | rd |    |  |  | 0 | 0 | 0 | 1 | 0 | 1 | 1 |  |   |

#### Pseudo-code

```
uint<XLEN> addr_t;
```

```
addr_t = X[rs1] + SignExt(offset::1'b0, XLEN);
```

```
X[rd] = SignExt(MemLoad(addr_t, Halfword, AccType_Normal), XLEN);
```

```
X[rs1] = addr_t;
```

### 3.12.21. LHR

#### Description

This instruction loads a 16-bit value from memory, then sign-extended to 32 bits before storing in *rd*. The address is obtained by adding register *rs1* to the register *rs2*.

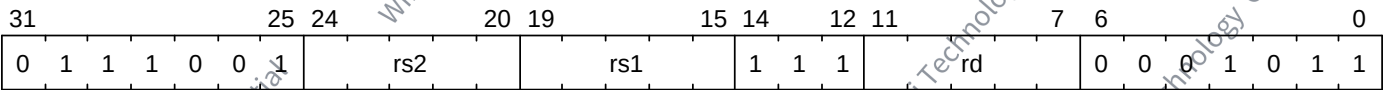
#### Configuration

This instruction is present only when xcl1s is configured.

#### Mnemonic

lhr *rd*, *rs2*(*rs1*)

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t = X[rs1] + X[rs2];  
  
X[rd] = SignExt(MemLoad(addr_t, Halfword, AccType_Normal), XLEN);
```

### 3.12.22. LHR.POST

#### Description

This instruction loads a 16-bit value from memory, then sign-extended to 32 bits before storing in *rd*. The address is obtained by *rs1*, and *rs1* is incremented by *rs2*.

#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

*lhr.post rd, rs2(rs1!)*

#### Encoding

| 31            |  | 25 |  | 24 |  | 20  |  | 19 |  | 15 |  | 14  |  | 12 |  | 11 |  | 7     |  | 6 |  | 0 |  |    |  |  |  |  |  |               |  |  |  |  |  |
|---------------|--|----|--|----|--|-----|--|----|--|----|--|-----|--|----|--|----|--|-------|--|---|--|---|--|----|--|--|--|--|--|---------------|--|--|--|--|--|
| 0 0 1 1 0 0 1 |  |    |  |    |  | rs2 |  |    |  |    |  | rs1 |  |    |  |    |  | 1 1 1 |  |   |  |   |  | rd |  |  |  |  |  | 0 0 0 1 0 1 1 |  |  |  |  |  |

#### Pseudo-code

```
X[rd] = SignExt(MemLoad(X[rs1], Halfword, AccType_Normal), XLEN);
```

```
X[rs1] = X[rs1] + X[rs2];
```

### 3.12.23. LHR.PRE

#### Description

This instruction loads a 16-bit value from memory, then sign-extended to 32 bits before storing in *rd*. The address is obtained by adding register *rs1* to the register *rs2*, and is written back to *rs1*.

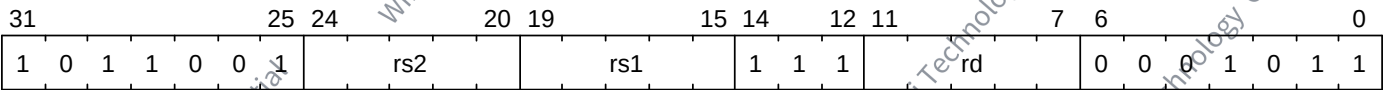
#### Configuration

This instruction is present only when `xc1s` is configured.

#### Mnemonic

`lhr.pre rd, [rs2(rs1)]!`

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + X[rs2];
X[rd] = SignExt(MemLoad(addr_t, Halfword, AccType_Normal), XLEN);
X[rs1] = addr_t;
```

### 3.12.24. LHU.POST

#### Description

This instruction loads a 16-bit value from memory, then zero-extended to 32 bits before storing in *rd*. The address is obtained by *rs1*, and *rs1* is incremented by the sign-extended 8-bit offset in multiples of 2 bytes.

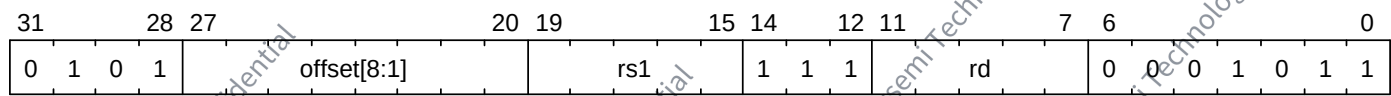
#### Configuration

This instruction is present only when xc15 is configured.

#### Mnemonic

`lhu.post rd, offset(rs1!)`

#### Encoding



#### Pseudo-code

```
X[rd] = ZeroExt(MemLoad(X[rs1], Halfword, AccType_Normal), XLEN);  
  
X[rs1] = X[rs1] + SignExt(offset::1'b0, XLEN);
```

### 3.12.25. LHU.PRE

#### Description

This instruction loads a 16-bit value from memory, then zero-extended to 32 bits before storing in *rd*. The address is obtained by adding register *rs1* to the the sign-extended 8-bit offset in multiples of 2 bytes, and is written back to *rs1*.

#### Configuration

This instruction is present only when xc15 is configured.

#### Mnemonic

`lhu.pre rd, [offset(rs1)]!`

#### Encoding

| 31 | 28 | 27 |   |             |  |  |  |  |  |  |  |  |  |  | 20 | 19  |  |  |  |   | 15 | 14 |    |  | 12 | 11 |   |   |   |   | 7 | 6 |  |  |  |  | 0 |
|----|----|----|---|-------------|--|--|--|--|--|--|--|--|--|--|----|-----|--|--|--|---|----|----|----|--|----|----|---|---|---|---|---|---|--|--|--|--|---|
| 1  | 1  | 0  | 1 | offset[8:1] |  |  |  |  |  |  |  |  |  |  |    | rs1 |  |  |  | 1 | 1  | 1  | rd |  | 0  | 0  | 0 | 1 | 0 | 1 | 1 |   |  |  |  |  |   |

#### Pseudo-code

```
uint<XLEN> addr_t;
```

```
addr_t = X[rs1] + SignExt(offset::1'b0, XLEN);
```

```
X[rd] = ZeroExt(MemLoad(addr_t, Halfword, AccType_Normal), XLEN);
```

```
X[rs1] = addr_t;
```

### 3.12.26. LHUR

#### Description

This instruction loads a 16-bit value from memory, then zero-extended to 32 bits before storing in *rd*. The address is obtained by adding register *rs1* to the register *rs2*.

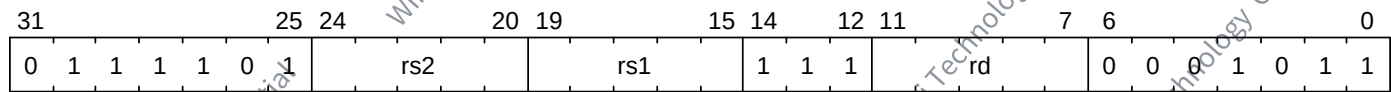
#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

*lhur rd, rs2(rs1)*

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t = X[rs1] + X[rs2];  
  
X[rd] = ZeroExt(MemLoad(addr_t, Halfword, AccType_Normal), XLEN);
```

### 3.12.27. LHUR.POST

#### Description

This instruction loads a 16-bit value from memory, then zero-extended to 32 bits before storing in *rd*. The address is obtained by *rs1*, and *rs1* is incremented by *rs2*.

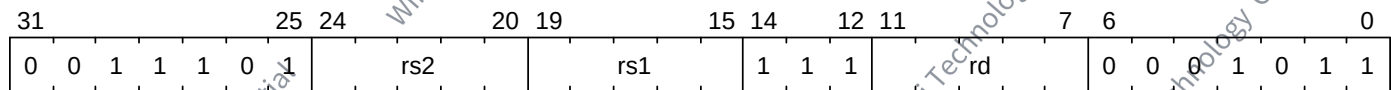
#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

*lhur.post rd, rs2(rs1!)*

#### Encoding



#### Pseudo-code

```
X[rd] = ZeroExt(MemLoad(X[rs1], Halfword, AccType_Normal), XLEN);  
  
X[rs1] = X[rs1] + X[rs2];
```



### 3.12.28. LHUR.PRE

#### Description

This instruction loads a 16-bit value from memory, then zero-extended to 32 bits before storing in *rd*. The address is obtained by adding register *rs1* to the register *rs2*, and is written back to *rs1*.

#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

*lhur.pre rd, [rs2(rs1)]!*

#### Encoding

| 31 | 25 | 24 | 20 | 19 | 15 | 14 | 12  | 11 | 7   | 6 | 0 |   |   |    |  |   |   |   |   |   |   |   |
|----|----|----|----|----|----|----|-----|----|-----|---|---|---|---|----|--|---|---|---|---|---|---|---|
| 1  | 0  | 1  | 1  | 1  | 0  | 1  | rs2 |    | rs1 |   | 1 | 1 | 1 | rd |  | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + X[rs2];
X[rd] = ZeroExt(MemLoad(addr_t, Halfword, AccType_Normal), XLEN);
X[rs1] = addr_t;
```

### 3.12.29. LW.POST

#### Description

This instruction loads a 32-bit value from memory and stores to *rd*. The address is obtained by *rs1*, and *rs1* is incremented by the sign-extended 8-bit offset in multiples of 4 bytes.

#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

`lw.post rd, offset(rs1!)`

#### Encoding

| 31 | 28 | 27 |   | 20          | 19 |  | 15 | 14 | 12  | 11 |  | 7 | 6 |   | 0  |
|----|----|----|---|-------------|----|--|----|----|-----|----|--|---|---|---|----|
| 0  | 0  | 1  | 0 | offset[9:2] |    |  |    |    | rs1 |    |  | 1 | 1 | 1 | rd |
|    |    |    |   |             |    |  |    |    |     |    |  |   | 0 | 0 | 0  |
|    |    |    |   |             |    |  |    |    |     |    |  |   | 1 | 0 | 1  |
|    |    |    |   |             |    |  |    |    |     |    |  |   |   |   | 1  |

#### Pseudo-code

```
X[rd] = MemLoad(X[rs1], Word, AccType_Normal);
```

```
X[rs1] = X[rs1] + SignExt(offset::2'b0, XLEN);
```

### 3.12.30. LW.PRE

#### Description

This instruction loads a 32-bit value from memory and stores to *rd*. The address is obtained by adding register *rs1* to the sign-extended 8-bit offset in multiples of 4 bytes, and is written back to *rs1*.

#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

`lw.pre rd, [offset(rs1)]!`

#### Encoding

| 31 | 28 | 27 |   | 20          | 19 |  | 15 | 14 | 12 | 11  |  | 7 | 6 |   | 0 |    |  |  |   |   |   |   |   |   |   |
|----|----|----|---|-------------|----|--|----|----|----|-----|--|---|---|---|---|----|--|--|---|---|---|---|---|---|---|
| 1  | 0  | 1  | 0 | offset[9:2] |    |  |    |    |    | rs1 |  |   | 1 | 1 | 1 | rd |  |  | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + SignExt(offset::2'b0, XLEN);
X[rd] = MemLoad(addr_t, Halfword, AccType_Normal);
X[rs1] = addr_t;
```

### 3.12.31. LWR

#### Description

This instruction loads a 32-bit value from memory, and stores to *rd*. The address is obtained by adding register *rs1* to the register *rs2*.

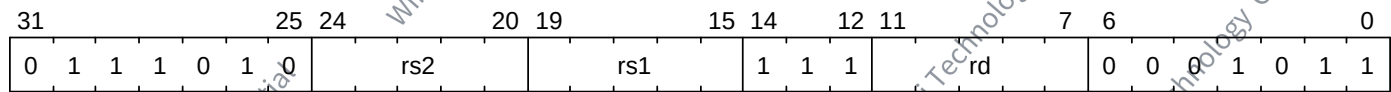
#### Configuration

This instruction is present only when xcl<sub>s</sub> is configured.

#### Mnemonic

`lwr rd, rs2(rs1)`

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t = X[rs1] + X[rs2];  
  
X[rd] = MemLoad(addr_t, Word, AccType_Normal);
```

### 3.12.32. LWR.POST

#### Description

This instruction loads a 32-bit value from memory and stores to *rd*. The address is obtained by *rs1*, and *rs1* is incremented by *rs2*.

#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

`lwr.post rd, rs2(rs1!)`

#### Encoding

| 31 | 25 | 24 | 20 | 19 | 15 | 14 | 12  | 11 | 7   | 6 | 0 |   |   |    |  |   |   |   |   |   |   |   |
|----|----|----|----|----|----|----|-----|----|-----|---|---|---|---|----|--|---|---|---|---|---|---|---|
| 0  | 0  | 1  | 1  | 0  | 1  | 0  | rs2 |    | rs1 |   | 1 | 1 | 1 | rd |  | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```
X[rd] = MemLoad(X[rs1], Word, AccType_Normal);
```

```
X[rs1] = X[rs1] + X[rs2];
```

### 3.12.33. LWR.PRE

#### Description

This instruction loads a 32-bit value from memory and stores to *rd*. The address is obtained by adding register *rs1* to the register *rs2*, and is written back to *rs1*.

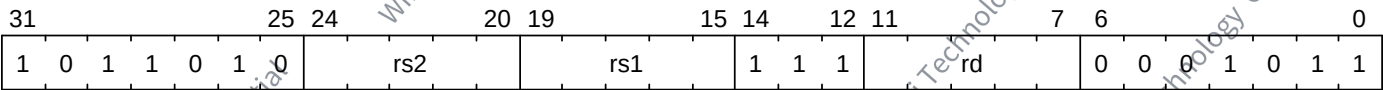
#### Configuration

This instruction is present only when xcl1s is configured.

#### Mnemonic

`lwr.pre rd, [rs2(rs1)]!`

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + X[rs2];
X[rd] = MemLoad(addr_t, Word, AccType_Normal);
X[rs1] = addr_t;
```

### 3.12.34. ORSHF

#### Description

This instruction bitwise ORs a register *rs2* with an optionally shifted register *rs1*, writes the result to the destination *rd*.

#### Configuration

This instruction is present only when xcpa is configured.

#### Mnemonic

orshf *rd*, *rs2*, *rs1* { < *shtype* >, *shamt* }

#### Encoding

|        |    |    |  |  |  |       |    |  |  |  |    |     |  |  |  |    |    |     |  |  |    |    |  |   |   |   |   |  |  |    |   |  |  |   |   |   |   |   |   |   |
|--------|----|----|--|--|--|-------|----|--|--|--|----|-----|--|--|--|----|----|-----|--|--|----|----|--|---|---|---|---|--|--|----|---|--|--|---|---|---|---|---|---|---|
| 31     | 30 | 29 |  |  |  | 25    | 24 |  |  |  | 20 | 19  |  |  |  | 15 | 14 |     |  |  | 12 | 11 |  |   |   | 7 | 6 |  |  |    | 0 |  |  |   |   |   |   |   |   |   |
| shtype |    |    |  |  |  | shamt |    |  |  |  |    | rs2 |  |  |  |    |    | rs1 |  |  |    |    |  | 0 | 1 | 0 |   |  |  | rd |   |  |  | 0 | 1 | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```
uint<XLEN> rs1_shft;

rs1_shft = shift_reg(X[rs1], shtype, shamt);

X[rd] = X[rs2] | rs1_shft;

function shift_reg(uint<XLEN> val, uint2 shtype, uint5 shamt) {

    if (shamt==0) {
        ExceptionTaken(ILLEGAL_INSTRUCTION);
    }

    switch (shtype) {
        case 'sll':
            // Logical shift left of a bitstring
            return SLL(uint<XLEN> val, uint5 shamt);
        case 'srl':
            // Logical shift right of a bitstring
            return SRL(uint<XLEN> val, uint5 shamt);
        case 'sra':
            // Arithmetical shift right of a bitstring
            return SRA(uint<XLEN> val, uint5 shamt);
        case 'ror':
            // Rotate right of a bitstring
            return ROR(uint<XLEN> val, uint5 shamt);
    }
}
```

### 3.12.35. SB.POST

#### Description

This instruction stores a 8-bit value into memory. The address is obtained by *rs1*, and *rs1* is incremented by the sign-extended 8-bit offset.

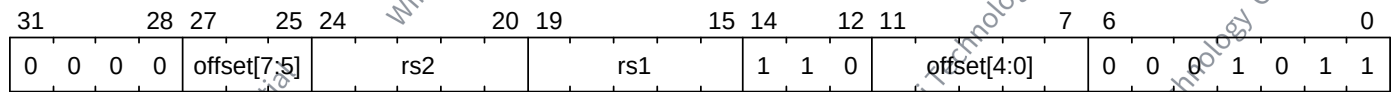
#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

*sb.post rs2, offset(rs1!)*

#### Encoding



#### Pseudo-code

```
MemStore(X[rs2][7:0], X[rs1], Byte, AccType_Normal);  
  
X[rs1] = X[rs1] + SignExt(offset, XLEN);
```



### 3.12.36. SB.PRE

#### Description

This instruction stores a 8-bit value into memory. The address is obtained by adding register *rs1* to the sign-extended 8-bit offset, and is written back to *rs1*.

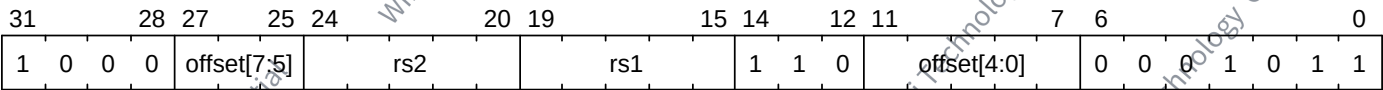
#### Configuration

This instruction is present only when xcl<sub>s</sub> is configured.

#### Mnemonic

sb.pre *rs2*, [*offset(rs1)*]

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + SignExt(offset, XLEN);
MemStore(X[rs2][7:0], addr_t, Byte, AccType_Normal);
X[rs1] = addr_t;
```

### 3.12.37. SBR

#### Description

This instruction stores a 8-bit value into memory. The address is obtained by adding register *rs1* to the register *rs2*.

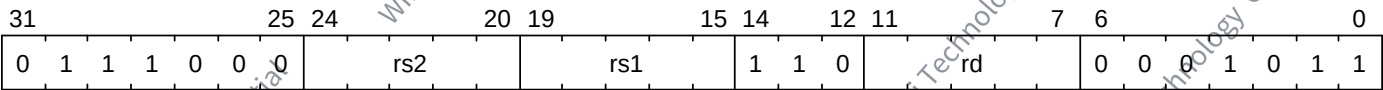
#### Configuration

This instruction is present only when xcl<sub>s</sub> is configured.

#### Mnemonic

sbr *rd*, *rs2*(*rs1*)

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t = X[rs1] + X[rs2];  
  
MemStore(X[rd][7:0], addr_t, Byte, AccType_Normal);
```

### 3.12.38. SBR.POST

**Description**

This instruction stores a 8-bit value into memory. The address is obtained by *rs1*, and *rs1* is incremented by *rs2*.

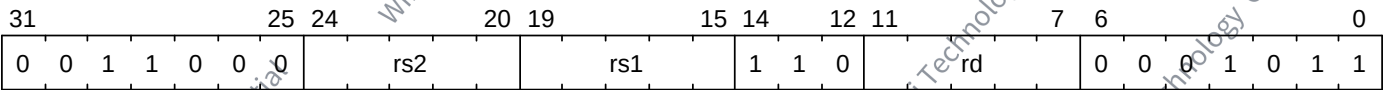
**Configuration**

This instruction is present only when *xc1s* is configured.

**Mnemonic**

sbr.post *rd*, *rs2(rs1!)*

**Encoding**



**Pseudo-code**

```
MemStore(X[rd][7:0], X[rs1], Byte, AccType_Normal);  
  
X[rs1] = X[rs1] + X[rs2];
```

### 3.12.39. SBR.PRE

#### Description

This instruction stores a 8-bit value into memory. The address is obtained by adding register *rs1* to the register *rs2*, and is written back to *rs1*.

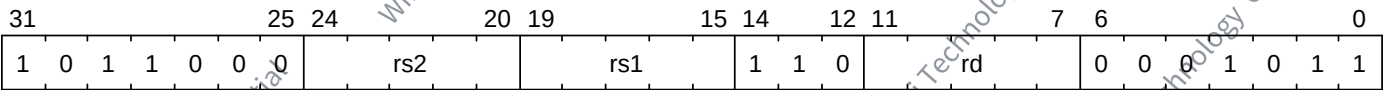
#### Configuration

This instruction is present only when `xc1s` is configured.

#### Mnemonic

`sbr.pre rd, [rs2(rs1)]!`

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + X[rs2];
MemStore(X[rd][7:0], addr_t, Byte, AccType_Normal);
X[rs1] = addr_t;
```

### 3.12.40. SH.POST

#### Description

This instruction stores a 16-bit value into memory. The address is obtained by *rs1*, and *rs1* is incremented by the sign-extended 8-bit offset in multiples of 2 bytes.

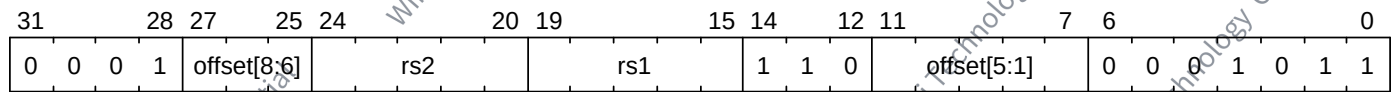
#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

*sh.post rs2, offset(rs1!)*

#### Encoding



#### Pseudo-code

```
MemStore(X[rs2][15:0], X[rs1], Halfword, AccType_Normal);  
  
X[rs1] = X[rs1] + SignExt(offset::1'b0, XLEN);
```

### 3.12.41. SH.PRE

#### Description

This instruction stores a 16-bit value into memory. The address is obtained by adding register *rs1* to the sign-extended 8-bit offset in multiples of 2 bytes, and is written back to *rs1*.

#### Configuration

This instruction is present only when `xc1s` is configured.

#### Mnemonic

`sh.pre rs2, [offset(rs1)]!`

#### Encoding

| 31 | 28 | 27 | 25 | 24          | 20  | 19  | 15 | 14 | 12 | 11          | 7 | 6 | 0 |   |   |   |   |
|----|----|----|----|-------------|-----|-----|----|----|----|-------------|---|---|---|---|---|---|---|
| 1  | 0  | 0  | 1  | offset[8:6] | rs2 | rs1 | 1  | 1  | 0  | offset[5:1] | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + SignExt(offset::1'b0, XLEN);
MemStore(X[rs2][15:0], addr_t, Halfword, AccType_Normal);
X[rs1] = addr_t;
```

### 3.12.42. SHR

**Description**

This instruction stores a 16-bit value into memory. The address is obtained by adding register *rs1* to the register *rs2*.

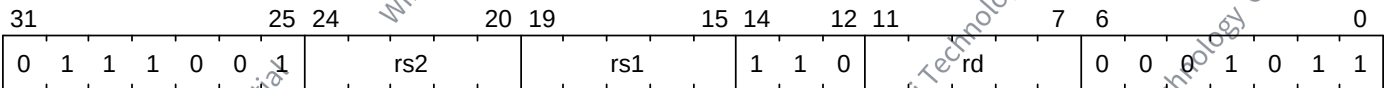
**Configuration**

This instruction is present only when xcl<sub>s</sub> is configured.

**Mnemonic**

shr *rd*, *rs2*(*rs1*)

**Encoding**



**Pseudo-code**

```
uint<XLEN> addr_t = X[rs1] + X[rs2];  
  
MemStore(X[rd][15:0], addr_t, Halfword, AccType_Normal);
```

### 3.12.43. SHR.POST

**Description**

This instruction stores a 16-bit value into memory. The address is obtained by *rs1*, and *rs1* is incremented by *rs2*.

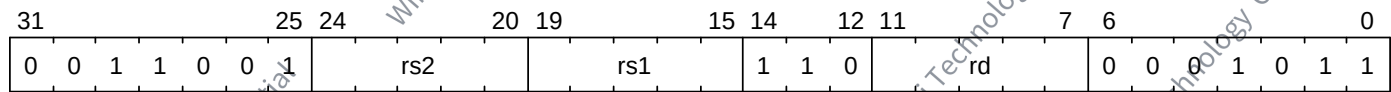
**Configuration**

This instruction is present only when xcl<sub>s</sub> is configured.

**Mnemonic**

shr.post *rd*, *rs2(rs1!)*

**Encoding**



**Pseudo-code**

```
MemStore(X[rd][15:0], X[rs1], Halfword, AccType_Normal);  
  
X[rs1] = X[rs1] + X[rs2];
```



### 3.12.44. SHR.PRE

#### Description

This instruction stores a 16-bit value into memory. The address is obtained by adding register *rs1* to the register *rs2*, and is written back to *rs1*.

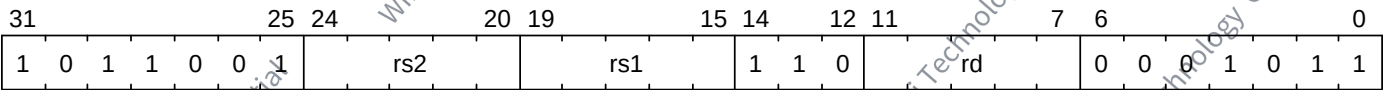
#### Configuration

This instruction is present only when `xc1s` is configured.

#### Mnemonic

`shr.pre rd, [rs2(rs1)]!`

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + X[rs2];
MemStore(X[rd][15:0], addr_t, Halfword, AccType_Normal);
X[rs1] = addr_t;
```

### 3.12.45. SUBSHF

#### Description

This instruction subtracts a register *rs2* with an optionally shifted register *rs1*, writes the result to the destination *rd*.

#### Configuration

This instruction is present only when xcpa is configured.

#### Mnemonic

subshf *rd, rs2, rs1* { < *shtype* >, *shamt* }

#### Encoding

|        |    |    |  |  |  |       |    |  |  |  |    |     |  |  |  |    |    |     |  |  |    |    |  |   |   |    |   |  |   |  |   |   |   |   |   |
|--------|----|----|--|--|--|-------|----|--|--|--|----|-----|--|--|--|----|----|-----|--|--|----|----|--|---|---|----|---|--|---|--|---|---|---|---|---|
| 31     | 30 | 29 |  |  |  | 25    | 24 |  |  |  | 20 | 19  |  |  |  | 15 | 14 |     |  |  | 12 | 11 |  |   |   | 7  | 6 |  |   |  | 0 |   |   |   |   |
| shtype |    |    |  |  |  | shamt |    |  |  |  |    | rs2 |  |  |  |    |    | rs1 |  |  | 0  |    |  | 0 | 1 | rd |   |  | 0 |  |   | 1 | 0 | 1 | 1 |

#### Pseudo-code

```

int<XLEN> rs1_shft;

rs1_shft = shift_reg(X[rs1], shtype, shamt);

X[rd] = (int) X[rs2] - rs1_shft;

function shift_reg(uint<XLEN> val, uint2 shtype, uint5 shamt) {

    if (shamt==0) {
        ExceptionTaken(ILLEGAL_INSTRUCTION);
    }

    switch (shtype) {
        case 'sll':
            // Logical shift left of a bitstring
            return SLL(uint<XLEN> val, uint5 shamt);
        case 'srl':
            // Logical shift right of a bitstring
            return SRL(uint<XLEN> val, uint5 shamt);
        case 'sra':
            // Arithmetical shift right of a bitstring
            return SRA(uint<XLEN> val, uint5 shamt);
        case 'ror':
            // Rotate right of a bitstring
            return ROR(uint<XLEN> val, uint5 shamt);
    }
}

```

### 3.12.46. SW.POST

#### Description

This instruction stores a 32-bit value into memory. The address is obtained by *rs1*, and *rs1* is incremented by the sign-extended 8-bit offset in multiples of 4 bytes.

#### Configuration

This instruction is present only when *xc1s* is configured.

#### Mnemonic

`sw.post rs2, offset(rs1!)`

#### Encoding

| 31 | 28 | 27 | 25 | 24          | 20  | 19 | 15  | 14 | 12 | 11 | 7 | 6           | 0 |   |   |   |   |   |   |
|----|----|----|----|-------------|-----|----|-----|----|----|----|---|-------------|---|---|---|---|---|---|---|
| 0  | 0  | 1  | 0  | offset[9:7] | rs2 |    | rs1 |    | 1  | 1  | 0 | offset[6:2] | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```
MemStore(X[rs2][31:0], X[rs1], Word, AccType_Normal);
```

```
X[rs1] = X[rs1] + SignExt(offset::2'b0, XLEN);
```

### 3.12.47. SW.PRE

#### Description

This instruction stores a 32-bit value into memory. The address is obtained by adding register *rs1* to the sign-extended 8-bit offset in multiples of 4 bytes, and is written back to *rs1*.

#### Configuration

This instruction is present only when `xc1s` is configured.

#### Mnemonic

`sw.pre rs2, [offset(rs1)]!`

#### Encoding

| 31 | 28 | 27 | 25 | 24          | 20  | 19 | 15  | 14 | 12 | 11 | 7 | 6           | 0 |   |   |   |   |   |   |
|----|----|----|----|-------------|-----|----|-----|----|----|----|---|-------------|---|---|---|---|---|---|---|
| 1  | 0  | 1  | 0  | offset[9:7] | rs2 |    | rs1 |    | 1  | 1  | 0 | offset[6:2] | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + SignExt(offset::2'b0, XLEN);
MemStore(X[rs2][31:0], addr_t, Word, AccType_Normal);
X[rs1] = addr_t;
```

### 3.12.48. SWR

#### Description

This instruction stores a 32-bit value into memory. The address is obtained by adding register *rs1* to the register *rs2*.

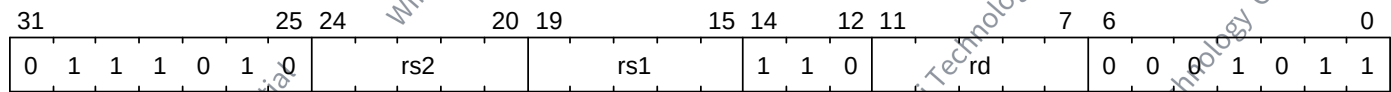
#### Configuration

This instruction is present only when xcl<sub>s</sub> is configured.

#### Mnemonic

swr *rd*, *rs2*(*rs1*)

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t = X[rs1] + X[rs2];  
  
MemStore(X[rd][31:0], addr_t, Word, AccType_Normal);
```

### 3.12.49. SWR.POST

**Description**

This instruction stores a 32-bit value into memory. The address is obtained by *rs1*, and *rs1* is incremented by *rs2*.

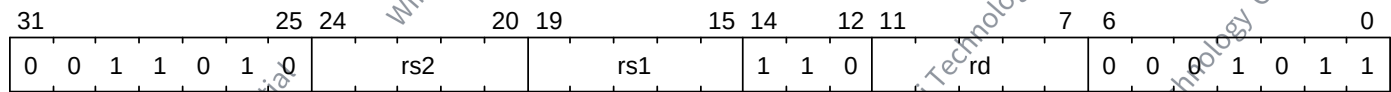
**Configuration**

This instruction is present only when *xc1s* is configured.

**Mnemonic**

*swr.post rd, rs2(rs1!)*

**Encoding**



**Pseudo-code**

```
MemStore(X[rd][31:0], X[rs1], Word, AccType_Normal);  
  
X[rs1] = X[rs1] + X[rs2];
```

### 3.12.50. SWR.PRE

#### Description

This instruction stores a 32-bit value into memory. The address is obtained by adding register *rs1* to the register *rs2*, and is written back to *rs1*.

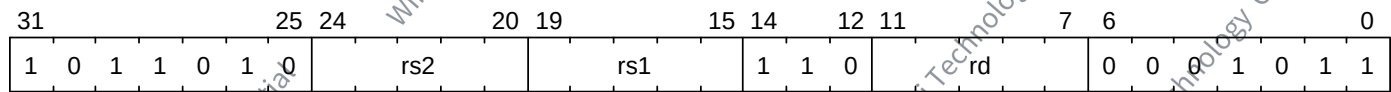
#### Configuration

This instruction is present only when `xc1s` is configured.

#### Mnemonic

`swr.pre rd, [rs2(rs1)]!`

#### Encoding



#### Pseudo-code

```
uint<XLEN> addr_t;

addr_t = X[rs1] + X[rs2];
MemStore(X[rd][31:0], addr_t, Word, AccType_Normal);
X[rs1] = addr_t;
```

### 3.12.51. REV8HW

#### Description

This instruction reverses the order of two bytes in each 16-bit halfword of a 32-bit register.

#### Configuration

This instruction is present only when xberv is configured.

#### Mnemonic

rev8hw rd, rs

#### Encoding

| 31  | 20 | 19 | 15 | 14 | 12 | 11 | 7 | 6 | 0 |
|-----|----|----|----|----|----|----|---|---|---|
| 1   | 1  | 1  | 1  | 0  | 0  | 0  | 0 | 0 | 0 |
| 0   | 0  | 0  | 0  | 0  | 0  | 0  | 0 | 0 | 0 |
| rs1 |    |    |    |    |    |    |   |   |   |
| 1   | 1  | 0  |    |    |    |    |   |   |   |
| rd  |    |    |    |    |    |    |   |   |   |
| 0   | 0  | 0  | 0  | 1  | 0  | 1  | 1 |   |   |

#### Pseudo-code

```

X[rd][31:24] = X[rs][23:16];
X[rd][23:16] = X[rs][31:24];
X[rd][15: 8] = X[rs][ 7: 0];
X[rd][ 7: 0] = X[rs][15: 8];

```



### 3.12.52. REV8SLHW

#### Description

This instruction reverses the order of two bytes in the lower 16-bit halfword of a 32-bit register, sign-extend the reversed halfword to a word.

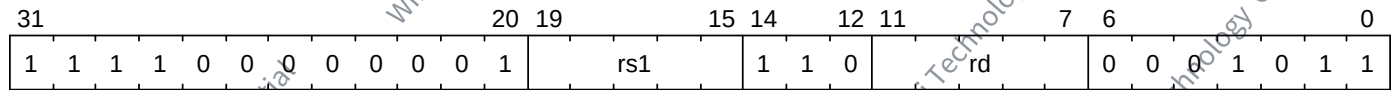
#### Configuration

This instruction is present only when xberv is configured.

#### Mnemonic

rev8slhw rd, rs

#### Encoding



#### Pseudo-code

```
X[rd][31: 8] = SignExt(X[rs][ 7: 0], 24);
X[rd][ 7: 0] = X[rs][15: 8];
```

### 3.12.53. XORSHF

#### Description

This instruction bitwise XORs a register *rs2* with an optionally shifted register *rs1*, writes the result to the destination *rd*.

#### Configuration

This instruction is present only when xcpa is configured.

#### Mnemonic

`xorshf rd, rs2, rs1 { < shtype >, shamt }`

#### Encoding

|        |    |    |  |  |  |       |    |  |  |  |    |     |  |  |  |    |    |     |  |  |    |    |  |   |   |   |    |  |  |   |   |   |   |   |
|--------|----|----|--|--|--|-------|----|--|--|--|----|-----|--|--|--|----|----|-----|--|--|----|----|--|---|---|---|----|--|--|---|---|---|---|---|
| 31     | 30 | 29 |  |  |  | 25    | 24 |  |  |  | 20 | 19  |  |  |  | 15 | 14 |     |  |  | 12 | 11 |  |   |   | 7 | 6  |  |  |   | 0 |   |   |   |
| shtype |    |    |  |  |  | shamt |    |  |  |  |    | rs2 |  |  |  |    |    | rs1 |  |  |    |    |  | 0 | 1 | 1 | rd |  |  | 0 | 1 | 0 | 1 | 1 |

#### Pseudo-code

```
uint<XLEN> rs1_shft;

rs1_shft = shift_reg(X[rs1], shtype, shamt);

X[rd] = X[rs2] ^ rs1_shft;

function shift_reg(uint<XLEN> val, uint2 shtype, uint5 shamt) {

    if (shamt==0) {
        ExceptionTaken(ILLEGAL_INSTRUCTION);
    }

    with (shtype) {
        case 'sll':
            // Logical shift left of a bitstring
            return SLL(uint<XLEN> val, uint5 shamt);
        case 'srl':
            // Logical shift right of a bitstring
            return SRL(uint<XLEN> val, uint5 shamt);
        case 'sra':
            // Arithmetical shift right of a bitstring
            return SRA(uint<XLEN> val, uint5 shamt);
        case 'ror':
            // Rotate right of a bitstring
            return ROR(uint<XLEN> val, uint5 shamt);
    }
}
```

## 4. Physical Memory Protection (PMP)

Wing-M130 processor optionally supports physical memory protection (PMP) that provides per-hart machine-mode control registers to allow physical memory access privileges (read, write, execute) to be specified for each physical memory region.

**NOTE** The config of PMP feature depends on whether U-mode is configured or not.

### 4.1. Features

- Configurable PMP regions from 0, 4, 8, 16, 32.
- Configurable minimum PMP granularity from 16, 32, 64, 128, 256, 512, 1024, 2048, 4096 in byte.
- Support memory access permission check for read, write and execute.
- Support address matching mode:
  - Naturally Aligned POver-of-Two region (NAPOT).
- Support PMP region lock. When it is locked, PMP checks additionally apply to M-mode access.
- Support the fixed priority when an access matches more than one region.

### 4.2. Memory Access Permission

PMP checks are applied to all accesses whose effective privilege mode is U-mode. When `pmp<n>cfg.L` is set PMP checks are also applied to all accesses in M-mode.

PMP violations are always trapped precisely at the processor. Attempting to fetch an instruction from a PMP region that does not have execute permissions raises an instruction access-fault exception. Attempting to execute a load or load-reserved instruction which accesses a physical address within a PMP region without read permissions raises a load access-fault exception. Attempting to execute a store, store-conditional, or AMO instruction which accesses a physical address within a PMP region without write permissions raises a store access-fault exception.

#### 4.2.1. Instruction fetches permission check

Instruction fetches permission is managed by `pmp<n>cfg.X`. When set, indicate that the PMP entry permits instruction execution in U-mode.

#### 4.2.2. Data accesses permission check

Data read and write permissions are managed by `pmp<n>cfg.R` and `pmp<n>cfg.W` respectively. When set, indicate that the PMP entry permits data load and write.

**NOTE**

Data read access includes a load or load-reserved instruction. Data write access includes a store, store-conditional or AMO instruction.

When `mstatus.mprv` is set and `mstatus.mpp` is U, `pmp<n>cfg.R` and `pmp<n>cfg.W` take effect on accesses in any modes, including U-mode and M-mode.

**NOTE**

pmp<n>cfg.R, W, X fields form a collective WARL field for which the combination with R=0 and W=1 are reserved. Write X=0/1, R=0 and W=1 to pmp<n>cfg will return X=0, R=0 and W=0.

### 4.3. Address Matching Mode

pmp<n>cfg.A encodes the address-matching mode of the associated PMP address register. When A=0, this PMP entry is disabled and matches no addresses. Wing-M130 supports naturally aligned power-of-two regions (NAPOT) only.

#### 4.3.1. NAPOT

NAPOT ranges make use of the low-order bits of the associated address register to encode the size of the range, as shown in Table 9.

| pmpaddr<n>  | pmp<n>cfg.A | Match type and size            |
|-------------|-------------|--------------------------------|
| yyyy...yyyy | NA4         | 4-byte NAPOT range             |
| yyyy...yyy0 | NAPOT       | 8-byte NAPOT range             |
| yyyy...yy01 | NAPOT       | 16-byte NAPOT range            |
| yyyy...y011 | NAPOT       | 32-byte NAPOT range            |
| ...         | ...         | ...                            |
| yy01...1111 | NAPOT       | $2^{XLEN}$ -byte NAPOT range   |
| y011...1111 | NAPOT       | $2^{XLEN+1}$ -byte NAPOT range |
| 0111...1111 | NAPOT       | $2^{XLEN+2}$ -byte NAPOT range |
| 1111...1111 | NAPOT       | $2^{XLEN+3}$ -byte NAPOT range |

Table 9. NAPOT range encoding in PMP address and configuration registers

### 4.4. PMP Locking

The pmp<n>cfg.L bit indicates that the PMP entry is locked, i.e., writes to the configuration register and associated address registers are ignored. Locked PMP entries remain locked until the hart is reset. If PMP entry *i* is locked, writes to pmp<i>cfg and pmpaddr<i> are ignored. Additionally, if PMP entry *i* is locked and pmp<i>cfg.A is set to TOR, writes to pmpaddr<i-1> are ignored.

**NOTE**

Setting the L bit locks the PMP entry even when the A field is set to OFF.

In addition to locking the PMP entry, the L bit indicates whether the R/W/X permissions are enforced on M-mode accesses. When the L bit is set, these permissions are enforced for all privilege modes. When the L bit is clear, any M-mode access matching the PMP entry will succeed; the R/W/X permissions apply only to U-mode.

### 4.5. Matching Logic

### 4.5.1. Address Matching Priority

PMP entries are statically prioritized. The lowest-numbered PMP entry that matches any byte of an access determines whether that access succeeds or fails. The matching PMP entry must match all bytes of an access, or the access fails.

### 4.5.2. Multiple accesses

A single instruction may generate multiple accesses, which may not be mutually atomic. An access-fault exception is generated if at least one access generated by an instruction fails, though other accesses generated by that instruction may succeed with visible side effects. Notably, instructions that reference virtual memory are decomposed into multiple accesses.

Wing-M130 supports misaligned loads and stores, and they may be decomposed into multiple accesses, some of which may succeed before an access-fault exception occurs. In particular, a portion of a misaligned store that passes the PMP check may become visible, even if another portion fails the PMP check.

## 4.6. CSR registers

PMP entries are described by an 8-bit configuration register  $\text{pmp}\langle n \rangle\text{cfg}$  and one 32-bit address register  $\text{pmpaddr}\langle n \rangle$ . The PMP configuration registers are densely packed into CSRs to minimize context-switch time. Four 8-bit configuration registers ( $\text{pmp}\langle 4*n \rangle\text{cfg}$ ,  $\text{pmp}\langle 4*n+1 \rangle\text{cfg}$ ,  $\text{pmp}\langle 4*n+2 \rangle\text{cfg}$  and  $\text{pmp}\langle 4*n+3 \rangle\text{cfg}$ ) are packed into one configuration register  $\text{pmpcfg}\langle n \rangle$ . The layout is shown in Figure 2.

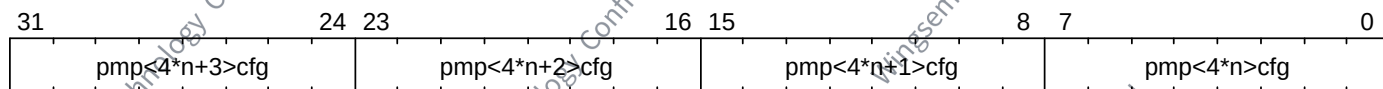


Figure 2. PMP configuration register  $\text{pmpcfg}\langle n \rangle$  layout

In Wing-M130, the number of PMP regions to be configured must be a multiple of four. And the maximum 64 regions can be configured.

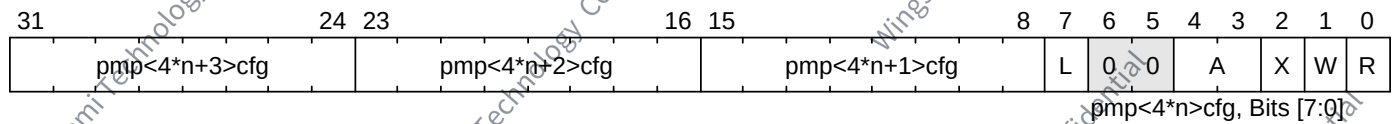
| CSR Address   | Privilege | Register                              | Description                                                           |
|---------------|-----------|---------------------------------------|-----------------------------------------------------------------------|
| $0x3A0+n$     | MRW       | $\text{pmpcfg}\langle n \rangle$      | Physical memory protection configuration register $\langle n \rangle$ |
| ...           | ...       | ...                                   | ...                                                                   |
| $0x3B0+4*n$   | MRW       | $\text{pmpaddr}\langle 4*n \rangle$   | Physical memory protection address register $\langle 4*n \rangle$     |
| $0x3B0+4*n+1$ | MRW       | $\text{pmpaddr}\langle 4*n+1 \rangle$ | Physical memory protection address register $\langle 4*n+1 \rangle$   |
| $0x3B0+4*n+2$ | MRW       | $\text{pmpaddr}\langle 4*n+2 \rangle$ | Physical memory protection address register $\langle 4*n+2 \rangle$   |
| $0x3B0+4*n+3$ | MRW       | $\text{pmpaddr}\langle 4*n+3 \rangle$ | Physical memory protection address register $\langle 4*n+3 \rangle$   |
| ...           | ...       | ...                                   | ...                                                                   |



### 4.6.1. PMP configuration register, *pmpcfg<n>*

#### Description

*pmpcfg<n>* register holds four 8-bit PMP configuration register *pmp<4\*n+3>cfg*, *pmp<4\*n+2>cfg*, *pmp<4\*n+1>cfg* and *pmp<4\*n>cfg* to minimize context-switch time. They occupy the bits [31:24], [23:16], [15:8] and [7:0] of *pmpcfg<n>*, respectively. The following shows the layout of *pmpcfg<n>* and details *pmp<4\*n>cfg*.



#### Configuration

This register is present only when pmp is implemented.

#### Field Description

##### *pmp<4\*n+3>cfg*, Bits [31:24]

PMP configuration register, for the PMP region with number  $4*n+3$ . For the detail fields of this register, refer to *pmp<4\*n>cfg*.

##### *pmp<4\*n+2>cfg*, Bits [23:16]

PMP configuration register, for the PMP region with number  $4*n+2$ . For the detail fields of this register, refer to *pmp<4\*n>cfg*.

##### *pmp<4\*n+1>cfg*, Bits [15:8]

PMP configuration register, for the PMP region with number  $4*n+1$ . For the detail fields of this register, refer to *pmp<4\*n>cfg*.

##### *pmp<4\*n>cfg*, Bits [7:0]

PMP configuration register, for the PMP region with number  $4*n$ . The bit fields of this register are shown in the following,

##### L, Bit [7]

Lock mode of *pmp<4\*n>cfg*. This bit also indicates whether the R/W/X permissions are enforced on M-mode accesses. When this bit is set, these permissions are enforced for all privilege modes.

**0b0** the PMP entry ( $4*n$ ) is not locked.

**0b1** the PMP entry ( $4*n$ ) is locked, i.e. writes to the configuration register and associated address register are ignored.

This field is reset to 0.

##### RES0, Bits [6:5]

Reserved, **WARL** field.

##### A, Bits [4:3]

The address matching mode of *pmp<4\*n>cfg*.

**0b00** OFF, Null region.

- 0b01** TOR, Top of range. If this option is configured off, write 0b01 to this field will read as 0b00.
- 0b10** **RES** (NA4, Naturally aligned four-byte region). Write 0b10 to this field will read as 0b00.
- 0b11** NAPOT, Naturally aligned power-of-two region.

This field is reset to 0.

#### **X, Bit [2]**

Instruction execution permission of `pmp<4*n>cfg`.

- 0b0** the PMP entry `<4*n>` doesn't permit instruction execution.
- 0b1** the PMP entry `<4*n>` permits instruction execution.

This field is reset to 0.

#### **W, Bit [1]**

Write permission of `pmp<4*n>cfg`.

- 0b0** the PMP entry `<4*n>` doesn't permit write.
- 0b1** the PMP entry `<4*n>` permits write.

This field is reset to 0.

#### **R, Bit [0]**

Read permission of `pmp<4*n>cfg`.

- 0b0** the PMP entry `<4*n>` doesn't permit read.
- 0b1** the PMP entry `<4*n>` permits read.

This field is reset to 0.

#### **NOTE**

`pmp<n>cfg.R`, `W`, `X` fields form a collective **WARL** field for which the combination with `R=0` and `W=1` are reserved. Write `X=0/1`, `R=0` and `W=1` to `pmp<n>cfg` will return `X=0`, `R=0` and `W=0`.

#### **CSR Address**

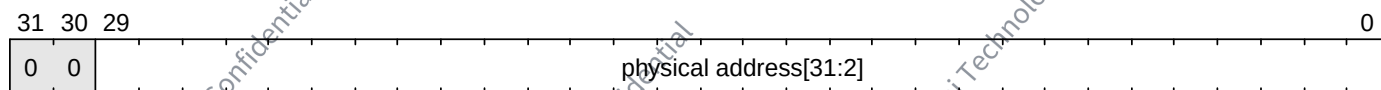
`0x3A0+n`



## 4.6.2. PMP address register, $pmpaddr<i>$ ( $i=4*n, 4*n+1, 4*n+2, 4*n+3$ )

### Description

$pmpaddr<i>$  encodes bits 31-2 of physical address for RV32.



### Configuration

This register is present only when pmp is implemented.

### Field Description

#### RES0, Bits [31:30]

Reserved. WARL field.

#### address[31:2], Bits [29:0]

The physical address [31:2].

This field is reset to an UNSPECIFIED value.

### CSR Address

0x3B0+i

## 5. Physical Memory Attribute (PMA)

The physical memory map for a complete system includes various address range, some corresponding to memory regions, some to memory-mapped control registers, and some to vacant holes in the address space. Some memory regions might not support reads, writes, or execution; some might not support subword or subblock accesses; some might not support atomic operations; and some might not support cache coherence or might have different memory models. Similarly, memory-mapped control registers vary in their supported access widths, support for atomic operations, and whether read and write accesses have associated side effects. In RISC-V systems, these properties and capabilities of each region of the machine's physical address space are termed physical memory attributes (PMAs).

Wing-M130 processor optionally support PMAs, which are inherent properties of the underlying hardware and rarely change during system operation. Unlike PMP described in [Chapter 4](#), PMAs do not vary by execution content. The PMAs of some memory regions are fixed at chip design time.

### 5.1. Features

- Configurable PMA regions.
- Support address matching mode:
  - Naturally Aligned POver-of-Two region (NAPOT).
- Support the specification of vacant holes in the address space.
- Support the specification of accessibility of the physical memories.
- Support the specification of memory attribute of the physical memories.
  - Support device memory and normal memory.
  - When the region is defined to be a normal memory, it can be specified to either cacheable or non-cacheable.

### 5.2. Address Match

Each region is specified by a range of address space. Same to the address matching mode of PMP, PMA supports NAPOT mode only. The minimum granularity of each region is 4KB.

### 5.3. Vacant holes

PMA supports 1-bit to indicate this region is a vacant hole. Any accesses, including instruction fetch, load and store accesses, hit this region will return the corresponding access fault.

### 5.4. Accessibility

Each region can be specified to allow the following accesses from instruction fetch, load access and/or store access, each represented by 1-bit, X, R, W, respectively.

- Instruction fetches to a PMA region with X-bit cleared will trigger an instruction access fault exception.
- Load accesses to a PMA region with R-bit cleared will trigger a load access fault exception.
- Store accesses to a PMA region with W-bit cleared will trigger a store access fault exception.

## 5.5. Memory Attribute

### 5.5.1. Idempotent

A PMA region can be classified to either a normal memory or a device memory by clearing/setting the bit `idempotency_n`.

When this bit is cleared,

- The memory location is identified as normal memory, where accesses can be repeated with no side effects.
- Speculative accesses can be made to Normal Memory.
- Unaligned accesses can be supported.
- Accesses can be merged before accessing the target memory system.
- The memory locations with Normal Memory attribute can be assigned to either cacheable or non-cacheable attribute.

When this bit is set,

- The memory location is identified as device memory, where accesses can have side-effect, and could be not repeated.
- Speculative accesses cannot be made to Device Memory.
- Unaligned accesses are not supported.
- Two write accesses can be merged before they reach the endpoint unless there is an ordered-before relationship between two writes.
- The memory locations with Device Memory attribute are always non-cacheable.

### 5.5.2. Cacheable

When the region is specified with idempotent attribute (normal memory), an additional attribute could be assigned to this region, specifying whether it is cacheable (set) or non-cacheable (clear).

When this bit is set, data from the external memories could be cached into the internal caches, if configured.

## 5.6. Configuration of PMA Regions

Since PMA reflects a specific hardware platform and rarely changes during system operation. The configuration of PMA regions is statically defined at RTL level. The dynamic configuration of PMA regions is not supported.

The configuration of PMA regions shall ensure that each valid address only hits one PMA region.

## 6. Memory Subsystem (MSS)

Memory Subsystem transfers internal accesses for the core to the interconnect. Wing-M130 processor provides flexibility for configuring, deploying memory subsystem on demand.

### 6.1. Features

- Support unalignment accesses to Normal Memory
- Support little-endian by default
- Optionally support big-endian, statically configured.
- Support configurable external interfaces from I-BUS, D-BUS (optional), M-BUS and P-BUS (optional).

### 6.2. Memory Model

#### 6.2.1. Memory Type

Wing-M130 processor classifies memory regions into two kinds of types. They are mutually exclusive.

- Normal Memory
- Device Memory

#### 6.2.2. Normal Memory

Normal Memory is idempotent, and has the following properties

- Read accesses can be repeated with no side effects.
- Speculative accesses can be made to Normal Memory.
- Unaligned accesses can be supported.
- Accesses can be merged before accessing the target memory system.
- The memory locations with Normal Memory attribute can be assigned to either cacheable or non-cacheable attribute.

#### 6.2.3. Device Memory

Device Memory is non-idempotent, and has the following properties

- Read accesses made to Device Memory regions will have side effects, and could not be repeated.
- Speculative accesses cannot be made to Device Memory.
- Unaligned accesses are not supported.
- Two write accesses can be merged before they reach the endpoint unless there is an ordered-before relationship between two writes.
- The memory locations with Device Memory attribute are always non-cacheable.

### 6.3. Alignment

All instruction fetches are halfword-aligned unless otherwise specified.

Data accesses to Normal Memory are permitted to be unaligned, except for

- Atomic Instructions: `lr.w`, `sc.w`, `amoadd.w`, `amoand.w`, `amomaxu.w`, `amomax.w`, `amominu.w`, `amomin.w`, `amoswap.w`, `amoxor.w`

Otherwise, an alignment access fault shall be raised.

Data accesses to Device memory are made always aligned. An unaligned access will raise an alignment access fault.

## 6.4. Endianness

By default Wing-M130 processor supports little-endian. Additionally, Wing-M130 processor could supports big-endian memory system with limitations

- Instruction fetches are always treated as little-endian.
- All accesses to P-Bus, if configured, are always treated as little-endian.
- All accesses to System Memory-mapped Register Space are always treated as little endian.

The endianness of other data accesses could be configured to either little endian or big endian.

### 6.4.1. Endianness Format

In a little-endian memory system, the mapping between bytes from memory and the interpreted value in a register is illustrated in [Figure 3](#).

- A byte or halfword at address A is the least significant byte or halfword within the word at that address.
- A byte at a halfword address A is the least significant bytes within the halfword at that address.

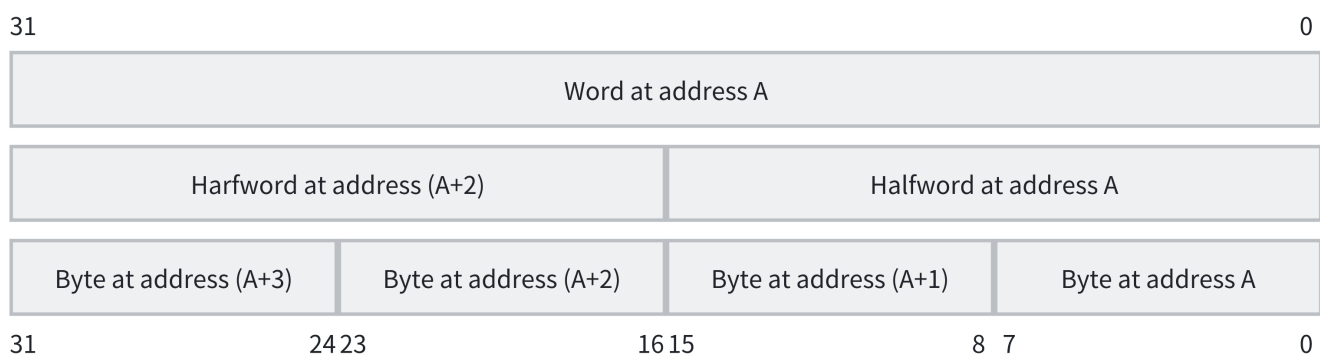


Figure 3. Little Endian

In a big-endian memory system the mapping between bytes from memory and the interpreted value in a register is illustrated in [Figure 4](#).

- A byte or halfword at address A is the most significant byte or halfword within the word at that address.
- A byte at a halfword address A is the most significant byte within the halfword at that address.

The Wing-M130 processor supports byte-invariant big-endian (**BE8**), which is compatible with AMBA AHB bus. For details, please refer to *AMBA AHB Protocol Specification*.

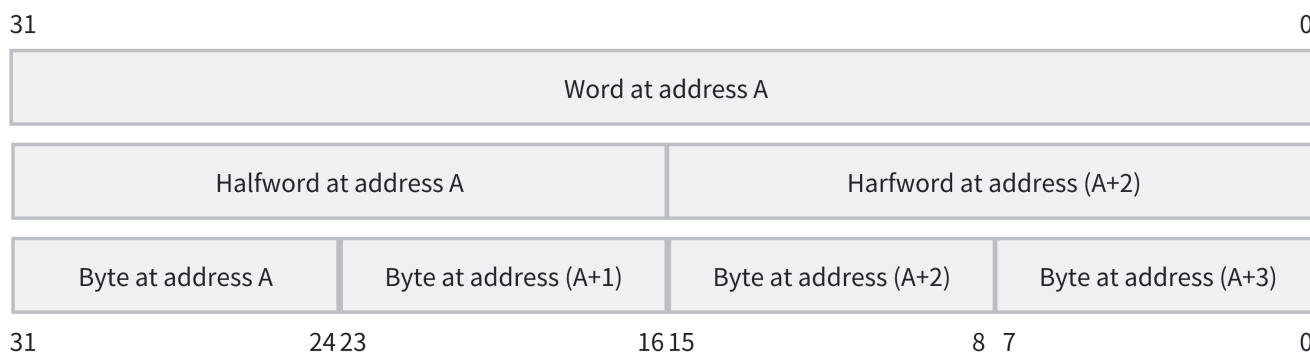


Figure 4. Big Endian

## 6.4.2. Endianness Configuration

Wing-M130 processor provides an external signal to SoC for the endianness configuration. It is either

- Implemented with a static value.
- Configured by a hardware input on reset.

## 6.4.3. Instructions to Perform Byte Reverse

### 6.4.3.1. Load/Store Instructions

For data accesses with load/store instructions, when bit-endian is configured, the loaded or pre-store data is converted based on the data element size that endianness applied to.

| Instruction class          | Instructions                                                                                                                                                                            |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Load/Store <b>Byte</b>     | lb, lbu, c.lbu, sb, c.sb                                                                                                                                                                |
| Load/Store <b>Halfword</b> | lh, lhu, c.lh, c.lhu, sh, c.sh                                                                                                                                                          |
| Load/Store <b>Word</b>     | lw, c.lw, c.lwsp, c.sw, c.swsp, sw, c.sw cm.push, cm.pop, cm.popret, cm.popretz, lr.w, sc.w, amoadd.w, amoand.w, amomaxu.w, amomax.w, amominu.w, amomin.w, amoor.w, amoswap.w, amoxor.w |

Table 10. Instructions performs endianness conversion

### 6.4.3.2. Byte Reverse Instruction

Wing-M130 processor provides instructions that reverse bytes in a register.

| Instruction | Description                                                                  |
|-------------|------------------------------------------------------------------------------|
| rev8        | Reverse the order of four bytes in a 32-bit register.                        |
| rev8hw      | Reverse the order of two bytes in each 16-bit halfword of a 32-bit register. |



| Instruction | Description                                                                                                                    |
|-------------|--------------------------------------------------------------------------------------------------------------------------------|
| rev8slhw    | Reverse the order of two bytes in the lower 16-bit halfword of a 32-bit register, sign-extend the reversed halfword to a word. |

*Table 11. Instructions reverse bytes in a register*

## 6.5. Memory-Map

### 6.5.1. System Memory-mapped Register Space

The System Memory-mapped Register Space (SMRS) is a 256KB memory-mapped space for accesses of memory-mapped registers of the following units. The base address is indicated by SMRS\_BASE.

- CLINT, refer to [Section 11.5](#).
- CLIC, refer to [Section 10.6](#).
- RAS, refer to [Section 12.6](#).

The memory type of this region shall be defined as Device Memory, with little endian only.

Generally, SMRS accesses support word accesses only unless otherwise noted. Accesses with the width of Byte and Halfword will return an access fault.

Particularly,

- The bitwidth of some registers defined in CLIC is byte. These registers are allowed to be accessed with Byte, Halfword, or Word. It is noted that the accesses shall be aligned to their data size.
- The registers defined in RAS are all 64-bit wide. These registers could be treated as two independent higher 32-bit and lower 32-bit part, and are also limited by word access only.

### 6.5.2. External Interfaces

Wing-M130 processor supports configuration of four external master interfaces, and one external slave interface.

- Instruction Fetch Bus (I-BUS). Instruction fetches are performed over this bus.
- Optional Data access Bus (D-BUS). Data accesses are performed over this bus.
- System Main Bus for both instruction and data accesses (M-BUS)
- Optional Peripheral Bus (P-BUS). Data accesses are performed over this bus. Instruction fetches are prohibited.
- Optional Subordinate Bus (S-BUS). External accesses are performed over this bus to access internal TCMs and MMRs.

[Figure 5](#) illustrates an reference configuration of memory-map table on Wing-M130 processor.

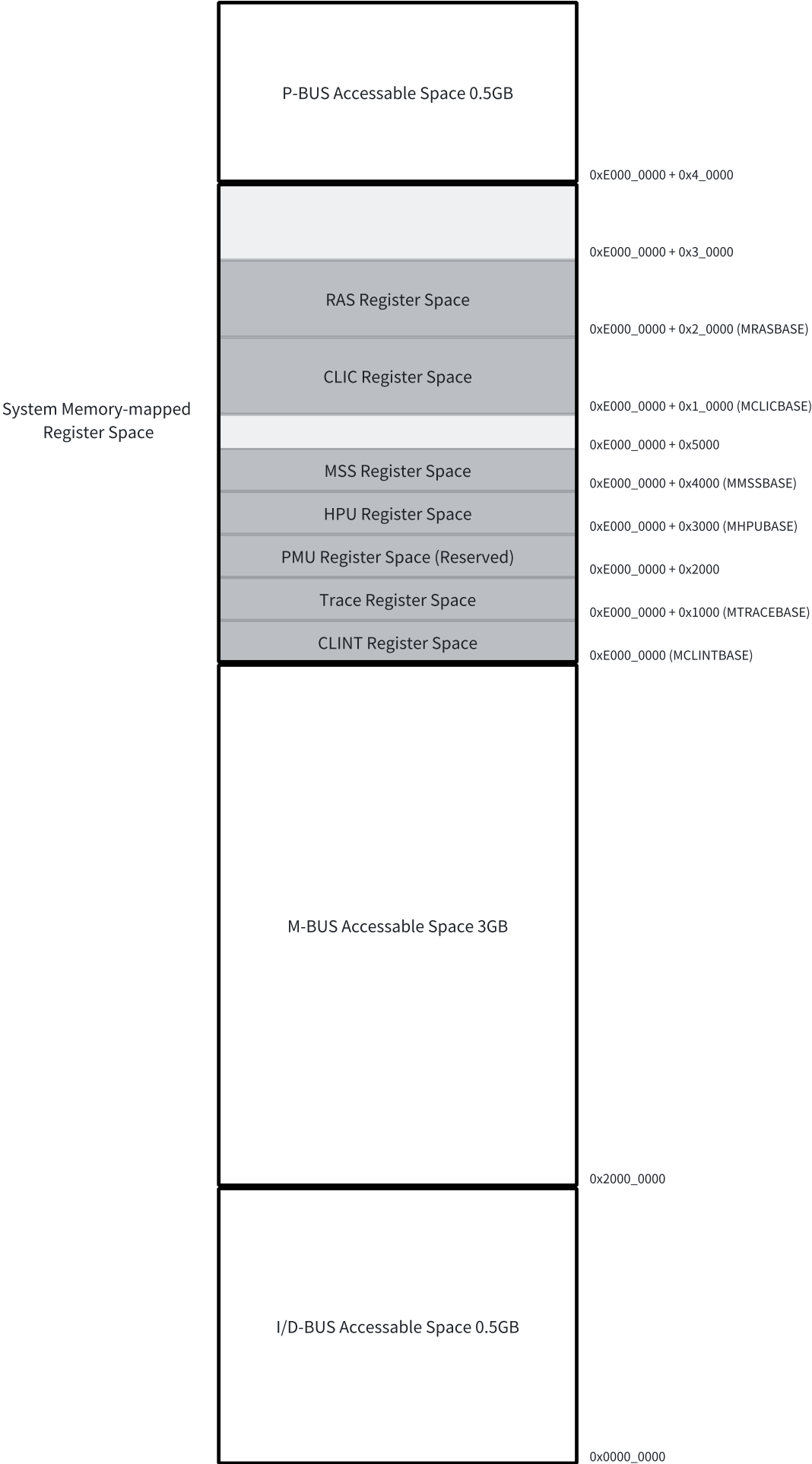


Figure 5. Memory Map of Wing-M130 processor



## 7. Debug System

### 7.1. Overview

Figure 6 shows the overview of Debug System. The software user interacts with the Debug Host (e.g. laptop), which is running a debugger (e.g. gdb). The debugger communicates with a Debug Translator (e.g. OpenOCD, which may include a hardware driver) to communicate with Debug Transport Hardware (e.g. Olimax USB-JTAG adapter). The Debug Transport Hardware connects the Debug Hosts to the hardware platform's Debug Transport Module (DTM) through JTAG (as shown in Figure 6). The DTM provides access to Debug Module (DM) using the Debug Module Interface (DMI).

DMs provide run control of their harts in the hardware platform. Abstract commands provide access to GPRs. Additional registers are accessible through abstract commands or by writing programs to the Program Buffer.

The Program Buffer allows the debugger to execute arbitrary instructions on a hart. This mechanism can also be used to access memory.

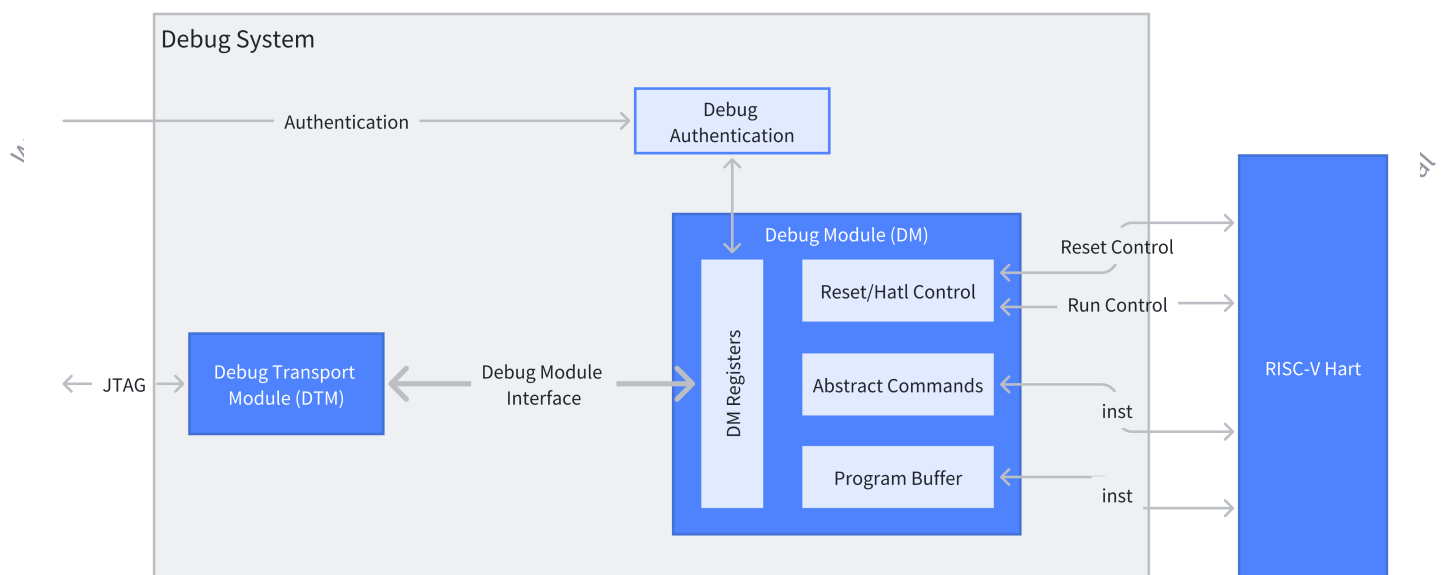


Figure 6. Overview of Debug System

### 7.2. Debug Module

The Debug Module implements a translation interface between abstract debug operations and their specific implementation.

#### 7.2.1. Features

- Provide the necessary information about the implementation to the debugger.
- Allow an individual hart to be halted and resumed.
- Provide status if the hart is halted.
- Provide abstract read and write access to a halted hart's GPRs.

- Provide access to a reset signal that allows debugging from the very first instruction after reset.
- Provide a Program Buffer to force the hart to execute arbitrary instructions.
- Provide abstract access to non-GPR hart registers.
- Allow memory access from a hart's point of view.
- Provide Debug Authentication

## 7.2.2. Debug Authentication

Wing-M130 provides an external signal for IP integrator to control the access to the Debug Module. It could allow access during a manufacturing process, and disable it permanently by wiring this signal to zero.

The status of debug authentication is reported on `dmstatus.authenticated`. When this bit is clear, the DM must not interact with the rest of the hardware platform, nor expose details about the harts connected to the DM. All DM registers should read 0, while writes should be ignored, with the following mandatory exceptions

- The bits `dmstatus.{authenticated,authbusy,version}` are readable.
- The bit `dmcontrol.dmactive` is readable and writeable.

### NOTE

It is recommended to keep the authentication signal clear/assert after reset. Otherwise, the behaviour when changing between clear and assert after reset is UNDEFINED.

## 7.2.3. Reset Control

### 7.2.3.1. Reset Harts

The Debug Module provides two methods to reset harts.

- `dmcontrol.ndmreset` - `ndmreset` resets the majority of the hardware platform except for the Debug System (including the Debug Module, Debug Transport Modules, and Debug Module Interface).
- `dmcontrol.hartreset` - `hartreset` resets the hart only. The Debug System, and the Trace Unit are excluded.

To perform either of these resets, the debugger first asserts the bit, and then clears it. The actual reset may start as soon as the bit is asserted, but may start an arbitrarily long time after the bit is deasserted. The reset itself may also take an arbitrarily long time. While the reset is on-going, harts are either in the running state, indicating it's possible to perform some abstract commands during this time, or in the unavailable state, indicating it's not possible to perform any abstract commands during this time.

Due to clock and power domain crossing issues, it might not be possible to perform arbitrary DMI accesses across hardware platform reset. While `ndmreset` or any external reset is asserted, the only supported DM operations are reading and writing `dmcontrol`. The behavior of other accesses is undefined.

When harts have been reset, they must set a sticky `havereset` state bit. The conceptual `havereset` state bits can be read for selected harts in `anyhavereset` and `allhavereset` in `dmstatus`. These bits must be set regardless of the cause of the reset. The `havereset` bits for the selected harts can be cleared by writing 1 to `ackhavereset` in `dmcontrol`. The `havereset` bits might or might not be cleared when `dmactive` is low.

#### 7.2.3.1.1. Halt-on-Reset

Once a hart's reset is complete, the bit `dmstatus.allhavereset` and the bit `dmstatus.anyhavereset` are set. When the bit `dmcontrol.setresethaltreq` is set, the hart will immediately enter Debug Mode (halted state), unless the bit `dmcontrol.clrresethaltreq` is also set.

Otherwise, if the hart was initially running it will execute normally (running state) and if the hart was initially halted it should now be running but may be halted.

#### 7.2.3.2. Reset Debug Module

The Debug Module's own state and registers should only be reset in two cases

- power-up reset - also known as Cold Reset. It resets not only the Debug Module but also the entire processor.
- while `dmcontrol.dmactive=0`.

`dmcontrol.dmactive` serves as a reset signal for the Debug Module itself. After changing the value of this bit, the debugger must poll `dmcontrol` until `dmactive` has taken the requested value before performing any action that assumes the requested `dmactive` state change has completed. Hardware may take an arbitrarily long time to complete activation or deactivation and will indicate completion by setting `dmactive` to the requested value.

#### 7.2.4. Hart DM States

Hart is in exactly one of the following four DM states: non-existent, unavailable, running, or halted. Which state the hart is in is reflected by `allnonexistent`, `anynonexistent`, `allunavail`, `anyunavail`, `allrunning`, `anyrunning`, `allhalted`, and `anyhalted` in `dmstatus`.

Harts are non-existent if they will never be part of this hardware platform, no matter how long a user waits. E.g. in a simple single-hart hardware platform only one hart exists, and all others are non-existent. Debuggers may assume that a hardware platform has no harts with indexes higher than the first non-existent one. Wing-M130 processor supports a single currently selected hart, and the hart is always present. The bit `dmstatus.allnonexistent` and the bit `dmstatus.anynonexistent` have the same encoding with the value 0, namely the hart exists in this hardware platform.

Harts are unavailable if they might exist/become available at a later time, or if there are other harts with higher indexes than this one. Harts may be unavailable for a variety of reasons including being reset, temporarily powered down, and not being plugged into the hardware platform. That means harts might become available or unavailable at any time, although these events should be rare in hardware platforms built to be easily debugged. There are no guarantees about the state of the hart when it becomes available. Wing-M130 processor supports a single currently selected hart. The bit `dmstatus.allunavail` and the bit `dmstatus.anyunavail` have the same meaning.

Harts are running when they are executing normally, as if no debugger was attached. This includes being in a low power mode or waiting for an interrupt, as long as a halt request will result in the hart being halted. Wing-M130 processor supports a single currently selected hart. The bit `dmstatus.allrunning` and the bit `dmstatus.anyrunning` have the same meaning.

Harts are halted when they are in Debug Mode, only performing tasks on behalf of the debugger. Wing-

M130 processor supports a single currently selected hart. The bit `dmstatus.allhalted` and the bit `dmstatus.anyhalted` have the same meaning.

### 7.2.5. Run Control

For every hart, the Debug Module tracks 4 conceptual bits of state: halt request, resume ack, halt-on-reset request, and hart reset. (The hart reset and halt-on-reset request bits are optional.) These 4 bits reset to 0, except for resume ack, which may reset to either 0 or 1. The DM receives halted, running, and havereset signals from each hart. The debugger can observe the state of resume ack in `allresumeack` and `anyresumeack`, and the state of halted, running, and havereset signals in `allhalted`, `anyhalted`, `allrunning`, `anyrunning`, `allhavereset`, and `anyhavereset`. The state of the other bits cannot be observed directly.

When a debugger writes 1 to `dmcontrol.haltreq`, each selected hart's halt request bit is set. When a running hart, or a hart just coming out of reset, sees its halt request bit high, it responds by halting, deasserting its running signal, and asserting its halted signal. Halted harts ignore their halt request bit.

When a debugger writes 1 to `dmcontrol.resumereq`, each selected hart's resume ack bit is cleared and each selected, halted hart is sent a resume request. Harts respond by resuming, clearing their halted signal, and asserting their running signal. At the end of this process the resume ack bit is set. These status signals of all selected harts are reflected in `allresumeack`, `anyresumeack`, `allrunning`, and `anyrunning`. Resume requests are ignored by running harts.

When halt or resume is requested, a hart must respond in finite time.

If the DM is reset while a hart is halted, it is UNSPECIFIED whether that hart resumes. Debuggers should use `resumereq` to explicitly resume harts before clearing `dmactive` and disconnecting.

### 7.2.6. Program Buffer

To support executing arbitrary instructions on a halted hart, a Debug Module can include a Program Buffer that a debugger can write small programs to. DMs that support all necessary functionality using abstract commands only may choose to omit the Program Buffer.

A debugger can write a small program to the Program Buffer, and then execute it exactly once with the Access Register Abstract Command, setting the `postexec` bit in command. The debugger can write whatever program it likes (including jumps out of the Program Buffer), but the program must end with `ebreak` or `c.ebreak`. An implementation may support an implicit `ebreak` that is executed when a hart runs off the end of the Program Buffer. This is indicated by `dmstatus.impebreak`. With this feature, a Program Buffer of just 2 32-bit words can offer efficient debugging.

While these programs are executed, the hart does not leave Debug Mode. If an exception is encountered during execution of the Program Buffer, no more instructions are executed, the hart remains in Debug Mode, and `cmderr` is set to 3 (exception error). If the debugger executes a program that doesn't terminate with an `ebreak` instruction, the hart will remain in Debug Mode and the debugger will lose control of the hart.

### 7.2.7. Abstract Commands

The DM supports a set of abstract commands. Depending on the implementation, the debugger may be able to perform some abstract commands even when the selected hart is not halted. Debuggers can only determine which abstract commands are supported by a given hart in a given state (running, halted, or held

in reset) by attempting them and then looking at `abstractcs.cmderr` to see if they were successful. Commands may be supported with some options set, but not with other options set. If a command has unsupported options set or if bits that are defined as 0 aren't 0, then the DM must set `abstractcs.cmderr` to 2 (not supported).

Debuggers execute abstract commands by writing them to `command`. They can determine whether an abstract command is complete by reading `abstractcs.busy`. If the debugger starts a new command while `abstractcs.busy` is set, `abstractcs.cmderr` becomes 1 (busy), the currently executing command still gets to run to completion, but any error generated by the currently executing command is lost. After completion, `abstractcs.cmderr` indicates whether the command was successful or not. Commands may fail because a hart is not halted, not running, unavailable, or because they encounter an error during execution.

If the command takes arguments, the debugger must write them to the `data<n>` registers before writing to `command`. If a command returns results, the Debug Module must ensure they are placed in the data registers before `abstractcs.busy` is cleared. Which data registers are used for the arguments is described in Table 12. In all cases the least-significant word is placed in the lowest-numbered data register. The argument width depends on the command being executed, and is always 32-bit wide.

| Argument Width | arg0/return value | arg1  |
|----------------|-------------------|-------|
| 32             | data0             | data1 |

Table 12. Use of `data<n>` registers

Before starting an abstract command, a debugger must ensure that `dmcontrol.{haltreq, resumereq, ackhavereset}` are all 0.

While an abstract command is executing (`abstractcs.busy` is high), a debugger must not change `dmcontrol.hartsel` (if support), and must not write 1 to `dmcontrol.{haltreq, resumereq, ackhavereset, setresethaltreq, clrresethaltreq}`.

If an abstract command does not complete in the expected time and appears to be hung, the debugger can try to reset the hart (using `dmcontrol.hartreset` or `dmcontrol.ndmreset`). If that doesn't clear busy, then it can try resetting the Debug Module (using `dmcontrol.dmactive`).

If an abstract command is started while the selected hart is unavailable or if a hart becomes unavailable while executing an abstract command, then the Debug Module may terminate the abstract command, setting `abstractcs.busy` low, and `abstractcs.cmderr` to 4 (halt/resume). Alternatively, the command could just appear to be hung (`abstractcs.busy` never goes low).

### 7.2.7.1 Supported Abstract Commands

Each abstract command is a 32-bit value. The top 8 bits contain `cmdtype` which determines the kind of command. Table 13 lists all commands.

| Value of <code>cmdtype</code> | Command                 |
|-------------------------------|-------------------------|
| 0                             | Access Register Command |
| 1                             | Quick Access            |



| Value of cmdtype | Command               |
|------------------|-----------------------|
| 2                | Access Memory Command |

Table 13. Supported Abstract Commands

### 7.2.7.2. Access Register

This command gives the debugger access to CPU registers and allows it to execute the Program Buffer. It performs the following sequence of operations

1. If `command.write` is clear and `command.transfer` is set, then copy data from the register specified by `command.regno` into the `arg0` region of data, and perform any side effects that occur when this register is read from M-mode.
2. If `command.write` is set and `command.transfer` is set, then copy data from the `arg0` region of data into the register specified by `command.regno`, and perform any side effects that occur when this register is written from M-mode.
3. If `command.aarpostincrement` and `command.transfer` are set, increment `command.regno`. `command.regno` may also be incremented if `command.aarpostincrement` is set and `command.transfer` is clear.
4. Execute the Program Buffer, if `command.postexec` is set.

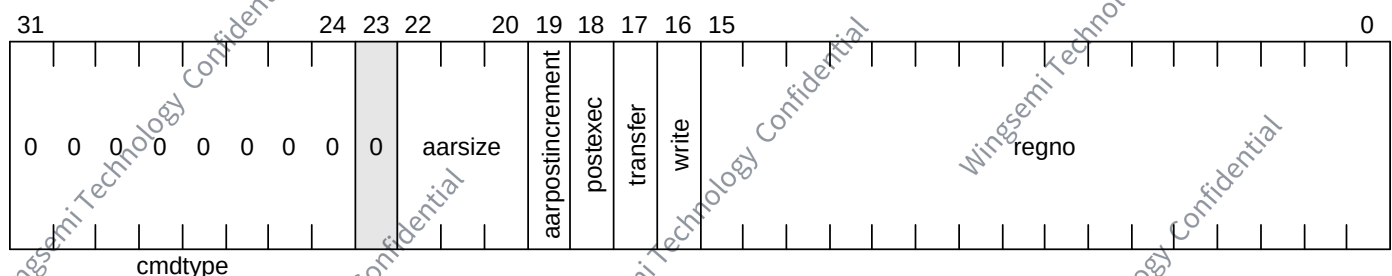
If any of these operations fail, `abstractcs.cmderr` is set and none of the remaining steps are executed. An implementation may detect an upcoming failure early, and fail the overall command before it reaches the step that would cause failure. If the failure is that the requested register does not exist in the hart, `abstractcs.cmderr` must be set to 3 (exception).

Debug Module implements this command and supports read and write access to all GPRs and CSRs.

Registers might not be accessible if they wouldn't be accessible by M mode code currently running. If this is the case, the debugger is responsible for changing state to make the registers accessible. The Core Debug Registers (Section 7.4) should be accessible if abstract CSR access is implemented.

This command modifies `arg0` only when a register is read. The other data registers are not changed.

#### 7.2.7.2.1. The Form of Abstract Command When `command.cmdtype=0`



#### Field Description

##### **cmdtype, Bits [31:24]**

`cmdtype`. Indicate Command Type. Write with the following encoding causes the corresponding Abstract Command to happen.

- 0** Access Register Command.

**RES0, Bit [23]**

Reserved, **WARL** field.

**aarsize, Bits [22:20]**

Access Register Size. This field is **WARZ** field

**Writing 2** 32-bit. Access the lowest 32 bits of the register.

**Writing other values** Write with other values has no effect. The `abstractcs.cmderr` is set to 2 (not supported).

**aarpostincrement, Bit [19]**

Access Register Post Increment. This field is **WARZ** field.

**Writing 0** disabled. No effect.

**Writing 1** enabled. After a successful register access, `regno` is incremented. Incrementing past the highest supported value causes `regno` to become unspecified. It is undefined whether the increment happens when transfer is 0.

**postexec, Bit [18]**

Post Execution. This field is **WARZ** field.

**Writing 0** disabled. No effect.

**Writing 1** enabled. Execute the program in the Program Buffer exactly once after performing the transfer, if any.

**transfer, Bit [17]**

transfer. This field is **WARZ** field.

**Writing 0** disabled. Don't do the operation specified by write.

**Writing 1** enabled. Do the operation specified by write. This bit can be used to just execute the Program Buffer without having to worry about placing valid values into `aarsize` or `regno`.

**write, Bit [16]**

write. This field is **WARZ** field. When `command.transfer` is set

**Writing 0** `arg0`. Copy data from the specified register into `arg0` portion of data.

**Writing 1** register. Copy data from `arg0` portion of data into the specified register.

**regno, Bit [15:0]**

`regno`. Number of the register to access

**Writing 0x0000-0x0fff** CSRs. The pc can be accessed here through `dpc`.

**Writing 0x1000-0x101f** GPRs

**Writing other values** Write with other values has no effect. The `abstractcs.cmderr` is set to 2 (not supported).

Perform the following sequence of operations:

- #### 7.2.7.3.1. The Form of Abstract Command When `command.cmdtype=1`

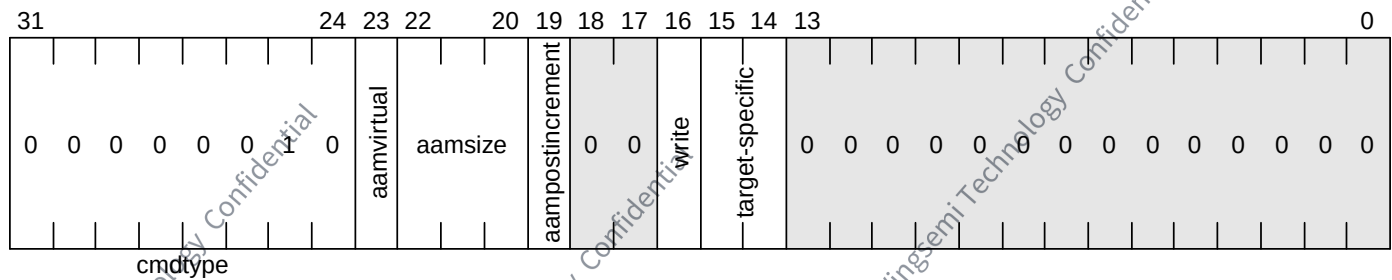


cmdtype. Indicate Command Type. Write with the following encoding causes the corresponding Abstract Command to happen.

- ### RES0, Bits [23:0]

This command modifies `arg0` only when memory is read. It modifies `arg1` only if `command.aampostincrement` is set. The other data registers are not changed.



7.2.7.4.1. The Form of Abstract Command When `command.cmdtype=2`**Field Description****cmdtype, Bits [31:24]**

cmdtype. Indicate Command Type. Write with the following encoding causes the corresponding Abstract Command to happen.

**2** Access Memory Command.

**aamvirtual, Bits [23]**

Access Memory Address Virtual. This field is **WARZ** field.

**Writing 0** Addresses are physical (to the hart they are performed on).

**Writing 1** Not supported. The `abstractcs.cmderr` is set to 2 (not supported).

**aamsize, Bits [22:20]**

Access Memory Size. This field is **WARZ** field.

**Writing 0** 8-bit. Access the lowest 8 bits of the memory location.

**Writing 1** 16-bit. Access the lowest 16 bits of the memory location.

**Writing 2** 32-bit. Access the lowest 32 bits of the memory location.

**Writing other vales** Write with other values has no effect. The `abstractcs.cmderr` is set to 2 (not supported).

**aampostincrement, Bit [19]**

Access Memory Post Increment. This field is **WARZ** field.

**Writing 0** disabled. No effect.

**Writing 1** enabled. After a memory access has completed, increment `arg1` (which contains the address used) by the number of bytes encoded in `aamsize`.

**RES0, Bits [18:17]**

Reserved, **WARL** field.

**write, Bit [16]**

write. This field is **WARZ** field.

**Writing 0** `arg0`. Copy data from the memory location specified in `arg1` into the low bits of `arg0`. The value of the remaining bits of `arg0` are UNSPECIFIED.

**Writing 1** memory. Copy data from the low bits of `arg0` into the memory location specified in `arg1`.

#### target-specific, Bits [15:14]

**RES0**, Reserved, **WARL** field.

#### RES0, Bits [13:0]

Reserved, **WARL** field.

## 7.2.8. Debug Module Interface

Debug Module is subordinate on a bus called the Debug Module Interface (DMI). The bus manager is the Debug Transport Module (DTM). DMI uses a full-featured AMBA Advanced Peripheral Bus (AMBA APB).

The DMI uses 7 address bits. Each address points at a single 32-bit register that can be read or written. The bottom of the address space is used for the DM.

The Debug Module is controlled via register accesses to its DMI address space.

## 7.2.9. Debug Module Registers

The DM Registers are accessed over the DMI bus. Each BM has a base address (which is 0 for the first DM).

DM Registers that are unimplemented or not mentioned in the table below return 0 when read. Writing them has no effect.

Each address points at a single 32-bit register that can be read or written.

| DMI Offset Address | Register                  | Description                 |
|--------------------|---------------------------|-----------------------------|
| 0x0004             | <code>data0</code>        | Abstract Data 0             |
| 0x0005             | <code>data1</code>        | Abstract Data 1             |
| 0x0010             | <code>dmcontrol</code>    | Debug module control        |
| 0x0011             | <code>dmstatus</code>     | Debug status                |
| 0x0016             | <code>abstractcs</code>   | Abstract control and status |
| 0x0017             | <code>command</code>      | Abstract command            |
| 0x0018             | <code>abstractauto</code> | Abstract command autoexec   |
| 0x0020             | <code>progbuf0</code>     | Program Buffer 0            |
| 0x0021             | <code>progbuf1</code>     | Program Buffer 1            |
| 0x0022             | <code>progbuf2</code>     | Program Buffer 2            |
| 0x0023             | <code>progbuf3</code>     | Program Buffer 3            |

### 7.2.9.1. Abstract Data, data<n>, n=0, . . . ,1

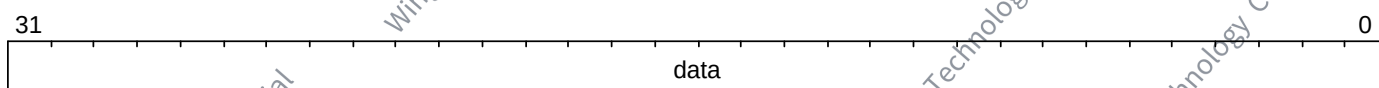
#### Description

The data<n> registers are read or changed by abstract command.

Accessing these registers while an abstract command is executing causes cmderr to be set to 1 (busy) if it is 0.

Attempts to write them while busy is set does not change their value.

The values in these registers might not be preserved after an abstract command is executed. The only guarantees on their contents are the ones offered by the command in question. If the command fails, no assumptions can be made about the contents of these registers.



#### Configuration

This register is present only when sdxext is implemented.

#### Field Description

data. Abstract data.

This field is reset to an UNSPECIFIED value.

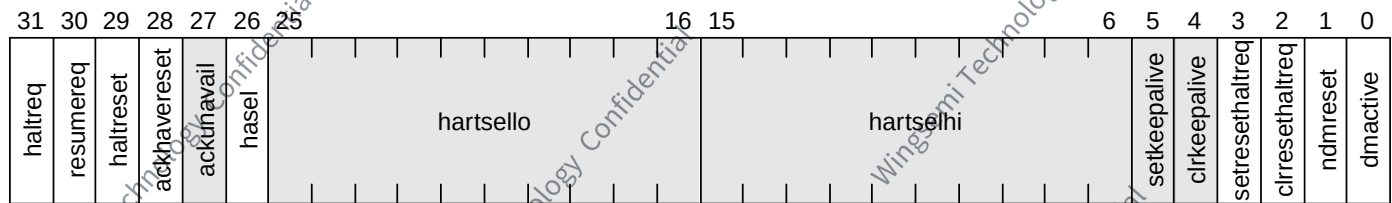
#### Offset Address

0x0004+n

### 7.2.9.2. Debug Module Control Register, dmcontrol

#### Description

The dmcontrol register controls the overall Debug Module as well as the corresponding hart.



#### Configuration

This register is present only when sdxext is implemented.

#### Field Description

##### haltreq, Bit [31]

Halt Request. This field is **WARZ** field.

**Writing 0** Clear the hart request bit for the hart. This may cancel outstanding halt request for the hart.

**Writing 1** Set the halt request bit for the hart. Running hart will halt whenever the halt request bit is set.

##### resumereq, Bit [30]

Writing 1 causes the hart to resume once, if it is halted when the write occurs. It also clears the resume ack bit for the hart. resumereq is ignored if haltreq is set. This field is **W1** field.

##### hartreset, Bit [29]

This field writes the reset bit for the hart. To perform a reset the debugger writes 1, and then writes 0 to deassert the reset signal.

**0** Deassert the hart reset.

**1** Assert the hart reset.

This field is reset to 0.

##### ackhavereset, Bit [28]

Writing 1 clears the havereset bit of the hart. This field is **W1** field.

##### ackunavail, Bit [27]

**RES0**, Reserved, **WARL** field.

##### hasel, Bit [26]

Select the definition of currently selected harts. This field is **WARL** field.

**0** single, there is a single currently selected hart.

##### hartsello, Bit [25:16]

**RES0**, Reserved, **WARL** field.

**hartselhi, Bit [15:6]**

**RES0**, Reserved, **WARL** field.

**setkeepalive, Bit [5]**

**RES0**, Reserved, **WARL** field.

**clrkeepalive, Bit [4]**

**RES0**, Reserved, **WARL** field.

**setresethaltreq, Bit [3]**

This field writes the halt-on-reset request bit for the hart, unless **clrresethaltreq** is simultaneously set to 1. When set to 1, the hart will halt upon the next deassertion of its reset. The halt-on-reset request bit is not automatically cleared. The debugger must write to **clrresethaltreq** to clear it. This field is **W1** field.

**clrresethaltreq, Bit [2]**

This field clears the halt-on-reset request bit for the hart. This field is **W1** field.

**ndmreset, Bit [1]**

This bit controls the reset signal from the DM to the rest of the hardware platform. The signal should reset every part of the hardware platform, including the hart, except for the DM and any logic required to access the DM. To perform a hardware platform reset, the debugger writes 1, and then writes 0 to deassert the reset.

- 0** Deassert the hardware platform reset.
- 1** Assert the hardware platform reset.

This field is reset to 0.

**dmactive, Bit [0]**

This bit serves as a reset signal for the Debug Module itself. To place the Debug Module into a known state, a debugger may write 0 to **dmactive**, pull until **dmactive** is observed 0, write 1 to **dmactive**, and poll until **dmactive** is observed 1.

- 0** inactive, the module's state, including authentication mechanism, takes its reset values (**dmactive** bit is the only bit which can be written to something other than its reset value). Any accesses to the module may fail.
- 1** active, the module functions normally. No other mechanism should exist that may result in resetting the Debug Module after power up.

This field is reset to 0.

**DMI Offset Address**

0x0010





- 1 The hart has been reset and reset has not been acknowledged for that hart.

**allresumeack, Bit [17]**

All currently selected harts have their resume ack bit set. Wing-M130 processor supports a single currently selected hart. This field has the same meaning with anyresumeack. This field is **R** field.

- 0 The hart does not have its resume ack bit set.
- 1 The hart has its resume ack bit set.

**anyresumeack, Bit [16]**

Any currently selected hart has its resume ack bit set. Wing-M130 processor supports a single currently selected hart. This field has the same meaning with allresumeack. This field is **R** field.

- 0 The hart does not have its resume ack bit set.
- 1 The hart has its resume ack bit set.

**allnonexistent, Bit [15]**

All currently selected harts do not exist. Wing-M130 processor supports a single currently selected hart, and the hart is always present. This field has the same meaning with anynonexistent. This field is **R** field, with the following encoding

- 0 The hart exists in this hardware platform.

**anynonexistent, Bit [14]**

Any currently selected hart does not exist. Wing-M130 processor supports a single currently selected hart, and the hart is always present. This field has the same meaning with allnonexistent. This field is **R** field, with the following encoding

- 0 The hart exists in this hardware platform.

**allunavail, Bit [13]**

All currently selected harts are unavailable. Wing-M130 processor supports a single currently selected hart. This field has the same meaning with anyunavail. This field is **R** field.

- 0 The hart is available.
- 1 The hart is unavailable.

**anyunavail, Bit [12]**

Any currently selected hart is unavailable. Wing-M130 processor supports a single currently selected hart. This field has the same meaning with allunavail. This field is **R** field.

- 0 The hart is available.
- 1 The hart is unavailable.

**allrunning, Bit [11]**

All currently selected harts are running. Wing-M130 processor supports a single currently selected hart. This field has the same meaning with anyrunning. This field is **R** field.

**0** The hart is not running.

**1** The hart is running.

#### **anyrunning, Bit [10]**

Any currently selected hart is running. Wing-M130 processor supports a single currently selected hart. This field has the same meaning with allrunning. This field is **R** field.

**0** The hart is not running.

**1** The hart is running.

#### **allhalted, Bit [9]**

All currently selected harts are halted. Wing-M130 processor supports a single currently selected hart. This field has the same meaning with anyhalted. This field is **R** field.

**0** The hart is not halted.

**1** The hart is halted.

#### **anyhalted, Bit [8]**

Any currently selected hart is halted. Wing-M130 processor supports a single currently selected hart. This field has the same meaning with allhalted. This field is **R** field.

**0** The hart is not halted.

**1** The hart is halted.

#### **authenticated, Bit [7]**

Indicate whether the debug is authenticated or not. This field is **R** field. Note that on components that don't implement authentication, this bit must be preset as 1.

**0** false. Authentication is required before using the DM.

**1** true. The authentication check has passed.

#### **authbusy, Bit [6]**

Indicate an ongoing authentication process. authbusy only becomes set in immediate response to an access to authdata. This field is **R** field.

**0** ready. The authentication module is ready to process the next read/write to authdata.

**1** busy. The authentication module is busy. Accessing authdata results in unspecified behavior.

This field is reset to 0.

#### **hasresethaltreq, Bit [5]**

Indicate whether Debug Module supports halt-on-reset functionality. This field is **R** field, with the following encoding

**1** Debug Module supports halt-on-reset functionality controllable by setresethaltreq and clrresethaltreq bits.



**confstrptrvalid, Bit [4]**

**RES0**, Reserved, **WARL** field.

**version, Bit [3:0]**

Debug Module version. This field is **R** field, and has the following encoding.

- 3** There is a Debug Module and it conforms to version 1.0 of RISC-V Debug Specification.

**Offset Address**

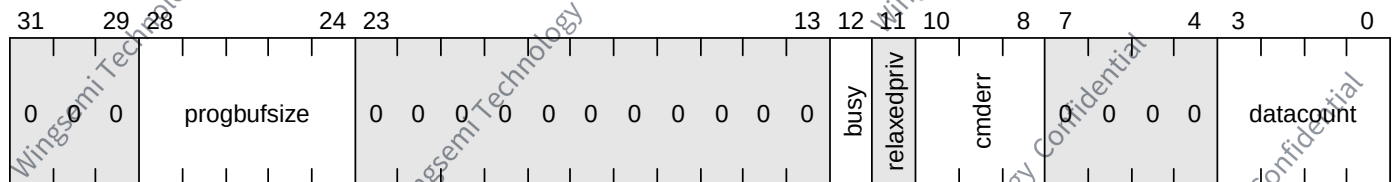
0x0011

#### 7.2.9.4. Abstract Control and Status, abstractcs

##### Description

The abstractcs register is a 32-bit read/write register that controls and reflects status of abstract command.

Writing this register while an abstract command is executing causes cmderr to become 1 (busy) once the command completes (busy becomes 0).



##### Configuration

This register is present only when sdxext is implemented.

##### Field Description

###### RES0, Bits [31:29]

Reserved, **WARL** field.

###### progbuFSIZE, Bits [28:24]

Program Buffer Size. This field is **WARL** field and has the following encoding

- 4 The size of the Program Buffer is 4.

###### RES0, Bits [23:13]

Reserved, **WARL** field.

###### busy, Bit [12]

busy. This bit is set as soon as command is written and is not cleared until that command has completed. This field is **R** field.

- 0 ready. There is no abstract command currently being executed.
- 1 busy. An abstract command is currently being executed.

This field is reset to 0.

###### relaxedpriv, Bit [11]

RES0, Reserved, **WARL** field.

###### cmderr, Bits [10:8]

command error. Gets set if an abstract command fails. The bits in this field remain set until they are cleared by writing 1 to them. No abstract command is started until the value is reset to 0. This field only contains a valid value if busy is 0. This field is **R/W1C** field.

- 0 none. No error.

- 1 busy. An abstract command was executing while `command`, `abstractcs`, or `abstractauto` was written, or when one of the data or `progbuf` registers was read or written. This status is only written if `cmderr` contains 0.
- 2 not supported. The command in `command` is not supported.
- 3 exception. An exception occurred while executing the command.
- 4 halt/resume. The abstract command couldn't execute because the hart wasn't in the required state (running/halted), or unavailable.
- 5 bus. The abstract command failed due to a bus error
- 6 reserved. Reserved for future use.
- 7 other. The command failed for another reason.

This field is reset to 0.

#### **RES0, Bits [7:4]**

Reserved, **WARL** field.

#### **datacount, Bits [3:0]**

`datacount`. Number of data register that are implemented as part of the abstract command interface. This field is **WARL** field and has the following encoding

- 2 Number of data register is 2.

#### **Offset Address**

0x0016

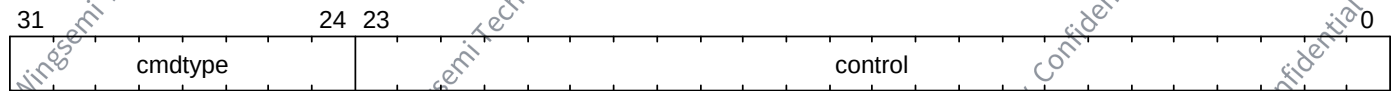
### 7.2.9.5. Abstract Command, command

#### Description

The command register is a 32-bit read/write register. Write to this register cause the corresponding abstract command to be executed.

Writing this register while an abstract command is executing causes cmderr to become 1 (busy) once the command completes (busy becomes 0).

If cmderr is non-zero, writes to this register are ignored.



#### Configuration

This register is present only when sdxtr is implemented.

#### Field Description

- cmdtype, Bits [31:24]**

cmdtype. The type determines the overall functionality of this abstract command. This field is **WARZ** field.

|                             |                                                                                           |
|-----------------------------|-------------------------------------------------------------------------------------------|
| <b>Writing 0</b>            | Access Register Command                                                                   |
| <b>Writing 1</b>            | Quick Access                                                                              |
| <b>Writing 2</b>            | Access Memory Command                                                                     |
| <b>Writing other values</b> | Write with other values has no effect. The abstractcs.cmderr is set to 2 (not supported). |
- control, Bits [23:0]**

This field is interpreted in a command-specific manner, described for each abstract command. Refer to [Section 7.2.7](#) for details.

#### Offset Address

0x0017

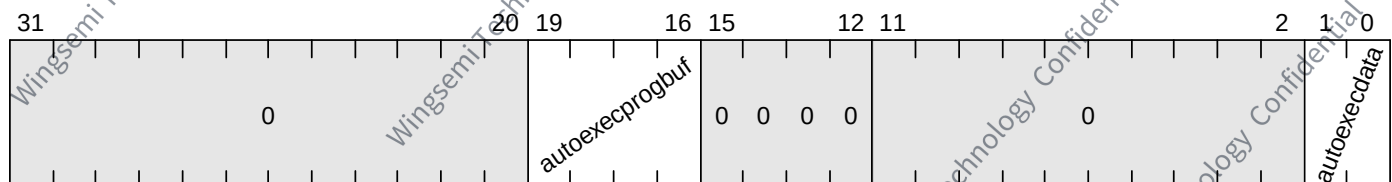
### 7.2.9.6. Abstract Command Autoexec, abstractauto

#### Description

The abstractauto register is a 32-bit read/write register to allow more efficient burst accesses.

If this register is implemented then bits corresponding to implemented progbuf and data registers must be writable. Other bits must be hard-wired to 0.

If this register is written while an abstract command is executing then the write is ignored and cmderr becomes 1 (busy) once the command completes (busy becomes 0).



#### Configuration

This register is present only when sdxext is implemented.

#### Field Description

##### autoexecprogbuf<15..4>, Bits [31:20]

RES0, Reserved, WARL field.

##### autoexecprogbuf<3..0>, Bits [19:16]

When a bit in this field is 1, read or write accesses to the corresponding progbuf word cause the DM to act as if the current value in command was written there again after the access to progbuf completes.

##### RES0, Bits [15:12]

Reserved, WARL field.

##### autoexecdata<11..2>, Bits [11:2]

RES0, Reserved, WARL field.

##### autoexecdata<1..0>, Bits [1:0]

When a bit in this field is 1, read or write accesses to the corresponding data word cause the DM to act as if the current value in command was written there again after the access to data completes.

#### Offset Address

0x0018

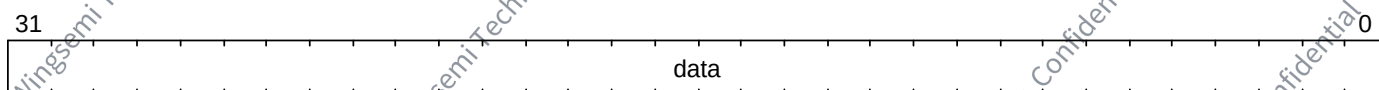
### 7.2.9.7. Program Buffer, `progbuf<n>`, $n=0, \dots, 3$

#### Description

The `progbuf<n>` registers provide write access to the program buffer. It may also be possible for the debugger to read from the program buffer through these registers.

Accessing these registers while an abstract command is executing causes `cmderr` to be set to 1 if it is 0.

Attempts to write them while busy is set does not change their value.



#### Configuration

This register is present only when `sdx` is implemented.

#### Field Description

data, Program buffer data.

This field is reset to an UNSPECIFIED value.

#### Offset Address

$0x0020+n$

## 7.3. Debug Transport Module

Debug Transport Module provides access to the DM over one or more transports (e.g. JTAG or USB).

Currently, JTAG DTM is defined in this specification. Additional DTMs may be added in future versions of this specification.

### 7.3.1. JTAG Debug Transport Module

This Debug Transport Module is based around a normal JTAG Test Access Port (TAP). The JTAG TAP allows access to arbitrary JTAG registers by first selecting one using the JTAG instruction register (IR), and then accessing it through the JTAG data register (DR).

### 7.3.2. JTAG DTM Registers

JTAG TAPs used as a DTM must have an IR of at least 5 bits. When the TAP is reset, IR must default to 0b00001, selecting the IDCODE instruction. A full list of JTAG registers along with their encoding is in the following

| IR Address | Register | Description                         |
|------------|----------|-------------------------------------|
| 0x01       | idcode   | Identify a specific silicon version |
| 0x10       | dtmcs    | DTM Control and Status Register     |
| 0x11       | dmi      | Debug Module Interface Access       |
| 0x1f       | bypass   | Bypass                              |

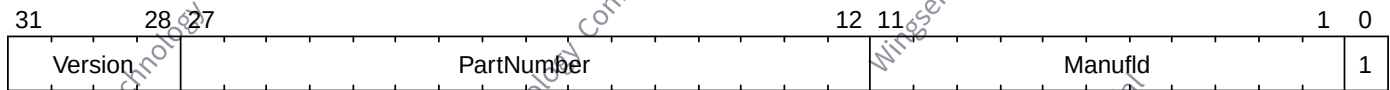


### 7.3.2.1. IDCODE Register, idcode

#### Description

This register is selected (in IR) when the TAP state machine is reset. Its definition is exactly as defined in IEEE Std 1149.1-2023.

This register is read-only register.



#### Configuration

This register is present only when sdxext is implemented.

#### Field Description

##### Version, Bits [31:28]

Identifies the release version of this part. This field is **R** field and is encoded as 0x1.

##### PartNumber, Bits [27:12]

Identifies the designer's part number of this part. This field is **R** field and is encoded as

| JTAG DTM | PartNumber |
|----------|------------|
| JTAG     | 0xDA00     |
| cJTAG    | 0xDA01     |

##### Manufld, Bits [11:1]

Identifies the designer/manufacture of this part. Bits 6:0 must be bits 6:0 of the designer/manufacture's Identification Code as assigned by JEDEC Standard JEP106. Bits 10:7 contain the modulo-16 count of the number of continuation characters (0x7f) in that same Identification Code.

This field is encoded as 0x77B.

##### Bit [0]

This field is encoded as 0b1.

#### IR Address

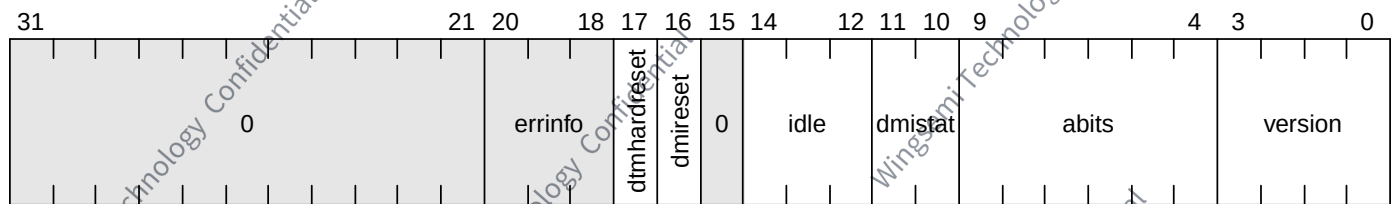
0x01



### 7.3.2.2. DTM Control and Status Register, dtmcs

#### Description

This register keeps track of DTM status



#### Configuration

This register is present only when sdxext is implemented.

#### Field Description

##### RES0, Bits [31:21]

Reserved, **WARL** field.

##### errinfo, Bits [20:18]

**RES0**, Reserved, **WARL** field.

##### dtmhardreset, Bit [17]

Writing 1 to this bit does a hard reset of the DTM, causing the DTM to forget about any outstanding DMI transactions, and returning all registers and internal state to their reset value. In general this should only be used when the Debugger has reason to expect that the outstanding DMI transaction will never complete.

This field is **W1** field.

##### dmireset, Bit [16]

Writing 1 to this bit clears the sticky error state, but does not affect outstanding DMI transactions.

This field is **W1** field.

##### Bit [15]

**RES0**, Reserved, **WARL** field.

##### idle, Bits [14:12]

This is a hint to the debugger of the minimum number of cycles a debugger should spend in Run-Test/Idle after every DMI scan to avoid a busy return code (dmistat of 3). A debugger must still check dmistat when necessary. This field is **R** field that is encoded as

**0** It is not necessary to enter Run-Test/Idle at all.

##### dmistat, Bits [11:10]

This field is read-only alias of dmi.op. This field is **R** field.

##### abits, Bits [9:4]

The size of address in dmi. This field is **R** field that is encoded as

**7** The size of DMI address is 7 bits.

**version, Bit [3:0]**

Debug Specification Version. This field is **R** field with the following encoding

- 1
- Version described in spec versions 0.13 and 1.0.

**IR Address**

0x10



- 1 (reserved)** Reserved.
- 2 (failed)** A previous operation failed. The data scanned into dmi in this access will be ignored. This status is sticky and can be cleared by writing dmireset in dtmcs.
- This indicates that the DM itself or the DMI responded with an error. There are no specified cases in which the DM would respond with an error, and DMI is not required to support returning errors.
- 3 (busy)** An operation was attempted while a DMI request is still in progress. The data scanned into dmi in this access will be ignored. This status is sticky and can be cleared by writing dmireset in dtmcs. If a debugger sees this status, it needs to give the target more TCK edges between Update-DR and Capture-DR. The simplest way to do that is to add extra transitions in Run-Test/Idle.

**IR Address**

0x11

#### 7.3.2.4. Bypass, bypass

##### Description

This 1-bit register has no effect. It is used when a debugger does not want to communicate with this TAP.

This register is read-only register.



##### Configuration

This register is present only when sdext is implemented.

##### Field Description

###### Bit [0]

This field is encoded as 0.

##### IR Address

0x1f

## 7.4. CSR Registers

| CSR Address | Privilege | Register  | Description                       |
|-------------|-----------|-----------|-----------------------------------|
| 0x7B0       | DRW       | dcsr      | Debug control and status register |
| 0x7B1       | DRW       | dpc       | Debug program counter             |
| 0x7B2       | DRW       | dscratch0 | Debug scratch register 0          |
| 0x7C0       | DRW       | dscratch2 | Debug scratch register 2          |
| 0x7C1       | DRW       | dscratch3 | Debug scratch register 3          |

### Description

|          |    |    |   |   |   |   |   |   |   |    |      |          |          |         |    |         |         |        |           |          |       |   |        |      |      |     |   |   |   |
|----------|----|----|---|---|---|---|---|---|---|----|------|----------|----------|---------|----|---------|---------|--------|-----------|----------|-------|---|--------|------|------|-----|---|---|---|
| 31       | 28 | 27 |   |   |   |   |   |   |   | 19 | 18   | 17       | 16       | 15      | 14 | 13      | 12      | 11     | 10        | 9        | 8     |   | 6      | 5    | 4    | 3   | 2 | 1 | 0 |
| debugver |    | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | pelp | ebreakvs | ebreakvu | ebreakm | 0  | ebreaks | ebreaku | stepie | stopcount | stoptime | cause | v | mprven | nmip | step | prv |   |   |   |

This register is present only when `sdx` is implemented.

**debugver, Bits [31:28]**

The Debug Version. This field is **WARL** field.

**4** The Debug Module is compliant with RISC-V Debug Specification 1.0.

Reserved, **WARL** field.

### When `zicfilp` is implemented

Previous Expected Landing Pad.

**0b0** The previous e1p is NO LP EXPECTED

**0b1** The previous elp is LP EXPECTED

**Otherwise**

**RES0**, Reserved, **WPRI** field.

## RES0. Reserved. WARL field.

**RES0**, Reserved, **WARL** field.

ebreak mode in M-mode.

**0b0** ebreak instruction in M-mode triggers a breakpoint exception.

**0b1** ebreak instruction in M-mode enters Debug Mode.

This field is reset to 0.

Reserved, **WARL** field.



**ebreaks, Bit [13]**

**RES0**, Reserved, **WARL** field.

**ebreaku, Bit [12]**

ebreak mode in U-mode.

**0b0** ebreak instruction in U-mode triggers a breakpoint exception.

**0b1** ebreak instruction in U-mode enters Debug Mode.

This field is reset to 0.

**stepie, Bit [11]**

Interrupt Enable during single step. The debugger must not change the value of this bit while the hart is running.

**0b0** Interrupts (including NMI) are disabled during single stepping with step set.

**0b1** Interrupts (including NMI) are enabled during single stepping with step set.

**stopcount, Bit [10]**

Control whether the local counters continue incrementing when Debug Mode was entered.

**0b0** cycle and instret increment as usual.

**0b1** Don't increment any hart-local counters while in Debug Mode or on ebreak instructions that cause entry into Debug Mode.

This field is reset to 0.

**stoptime, Bit[9]**

Control whether time continues to reflect mtime when Debug Mode was entered.

**0b0** time continues to reflect mtime.

**0b1** time is frozen at the time that Debug Mode was entered. When leaving Debug Mode, time will reflect the latest value of mtime again.

This field is reset to 0.

**cause, Bits [8:6]**

Explain why Debug Mode was entered. This field is read-only and write-with-no-effect field.

**0** Not applicable.

**1** An ebreak instruction was executed.

**2** A trigger module trigger fired with action=1.

**3** The debugger requested entry to Debug Mode using haltreq

**4** The hart single stepped because step was set.



**5** The hart halted directly out of reset due to `resethaltreq`.

**6** The hart halted because it's part of a halt group.

This field is reset to 0.

#### v, Bit [5]

**RES0**, Reserved, **WARL** field.

#### mprven, Bit [4]

Enable `mstatus.mprv` in Debug Mode.

**0b0** Disabled, `mstatus.mprv` is ignored in Debug Mode.

**0b1** Enabled, `mstatus.mprv` takes effect in Debug Mode.

This field is reset to 0.

#### nmip, Bit [3]

Indicate a NMI pending for the hart. This field is read-only and write-with-no-effect field.

**0b0** There is no pending NMI for the hart.

**0b1** There is a pending NMI for the hart.

This field is reset to 0.

#### step, Bit [2]

When set and not in Debug Mode, the hart will only execute a single instruction and then enter Debug Mode. The debugger must not change the value of this bit while the hart is running.

**0b0** When returned from Debug Mode, the hart will continue executing instructions.

**0b1** When returned from Debug Mode, the hart will only execute a single instruction and then enter Debug Mode.

This field is reset to 0.

#### prv, Bits [1:0]

##### **When U-mode is implemented**

This field holds the privilege mode the hart was operating in when Debug Mode was entered. This field is **WARL**. Write any value other than `0b00` will read as `0b11`.

**0b00** The privilege mode the hart was operating in when Debug Mode was entered is U-mode.

**0b11** The privilege mode the hart was operating in when Debug Mode was entered is M-mode.

##### **Otherwise**

This field holds the privilege mode the hart was operating in when Debug Mode was entered. This field is **WARL** with the following encoding.

**0b11** The privilege mode the hart was operating in when Debug Mode was entered is M-mode.

This field is reset to 3.

**CSR Address**

0x7B0

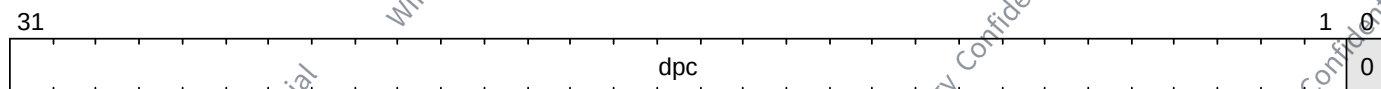
### Description

The dpc register is a 32-bit read/write register that is updated with the address of the next instruction to be executed upon entry to Debug Mode. Refer to [Table 14](#).

| Cause          | Address in dpc                                                                                                                                                                                           |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ebreak         | Address of the ebreak instruction.                                                                                                                                                                       |
| single step    | Address of the instruction that would be executed next if no debugging was going on, i.e., pc+4 for 32-bit instructions that don't change program flow, the destination pc on taken jumps/branches, etc. |
| trigger module | The address of the next instruction to be executed at the time that Debug Mode was entered.                                                                                                              |
| halt request   | The address of the next instruction to be executed at the time that Debug Mode was entered.                                                                                                              |

*Table 14. The Address in dpc upon Debug Mode Entry*

Executing the Program Buffer may cause the value of `dpc` to become unspecified. If that is the case, it must be possible to read/write `dpc` using an abstract command with `postexec` not set. The debugger must attempt to save `dpc` between halting and executing a Program Buffer, and then restore `dpc` before leaving Debug Mode.



## Configuration

This register is present only when `sdx` is implemented.

### Field Description

**dpc[31:1], Bits [31:1]**

The address where the debugger return to.

### RES0, Bit [0]

Reserved, **WARL** field.

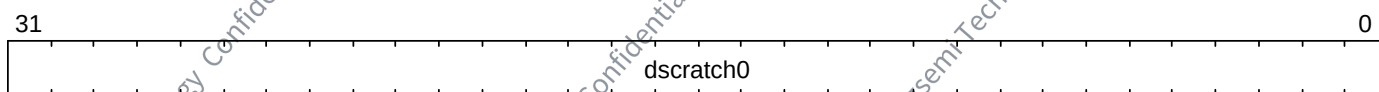
## CSR Address

0x7B1

### 7.4.3. Debug Scratch Register 0, dscratch0

#### Description

The dscratch0 is a 32-bit read/write register that can be used by implementations that need it. The Debug Module may use this register internally.



#### Configuration

This register is present only when sdxext is implemented.

#### Field Description

##### dscratch0, Bits [31:0]

Debug Scratch Register 0. It is used by implementations that need it.

This field is reset to an UNSPECIFIED value.

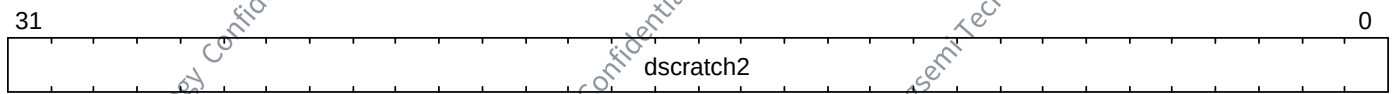
#### CSR Address

0x7B2

## 7.4.4. Debug Scratch Register 2, dscratch2

### Description

The dscratch2 is a 32-bit read/write register that can be used by implementations that need it. The Debug Module may use this register internally.



### Configuration

This register is present only when sdxext is implemented.

### Field Description

#### dscratch2, Bits [31:0]

Debug Scratch Register 2. It is used by implementations that need it.

This field is reset to an UNSPECIFIED value.

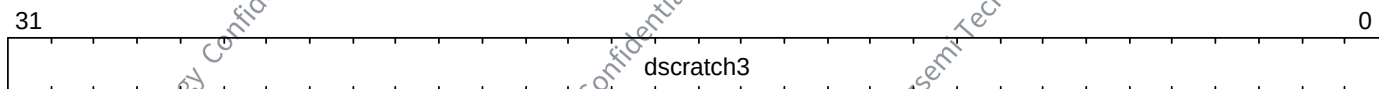
### CSR Address

0x7C0

### 7.4.5. Debug Scratch Register 3, dscratch3

#### Description

The dscratch3 is a 32-bit read/write register that can be used by implementations that need it. The Debug Module may use this register internally.



#### Configuration

This register is present only when sdxext is implemented.

#### Field Description

##### **dscratch3, Bits [31:0]**

Debug Scratch Register 3. It is used by implementations that need it.

This field is reset to an UNSPECIFIED value.

#### CSR Address

0x7C1

## 8. Trigger Module

Wing-M130 processor could optionally implement a Trigger Module which is compatible with RISC-V `sdtrig` extension.

Triggers can cause a breakpoint exception, entry into Debug Mode, or a trace action without having to execute a special instruction. This makes them invaluable when debugging code from ROM. They can trigger on execution of instructions at a given memory address, or on the address in loads/stores.

If `sdtrig` is implemented, the Trigger Module must support at least one trigger. Accessing trigger CSRs that are not used by any of the implemented triggers must result in an illegal instruction exception. M-Mode and Debug Mode accesses to trigger CSRs that are used by any of the implemented triggers must succeed, regardless of the current type of the currently selected trigger.

A trigger matches when the conditions that it specifies (e.g. a load from a specific address) are met. A trigger fires when a trigger that matches performs the action configured for that trigger.

Triggers do not fire while in Debug Mode.

### 8.1. Features

- Support trigger to Native Debug and External Debug (if `sdext` is implemented).
  - Support two-level stacking of trigger enable in Native Debug Mode
- Support Memory Access triggers
  - Configurable number of memory access triggers.
  - Match the address of instruction fetch, load and/or store access.
  - Support configurable address match size.
  - Support full address match only.
  - Can be configured to match in M-mode and/or in U-mode, if U-mode is implemented.

### 8.2. Trigger Type

The supported triggers on Wing-M130 processor is Memory Access Trigger which match the address of instruction fetch, load and/or store access.

#### 8.2.1. Memory Access Trigger

The memory access trigger, namely `mcontrol16`, matches the address of instruction fetch, load and/or store access only (`mcontrol16.select=0`), which are controlled by `mcontrol16.{execute, load, store}` respectively.

For data access with load or store, the address match size can be configured to

- Match any size (`mcontrol16.size=0`);
- Match 8-bit only (`mcontrol16.size=1`);
- Match 16-bit only (`mcontrol16.size=2`);



- Match 32-bit only (`mcontrol6.size=3`);

For instruction fetch, the address match size can be configured to

- Match any size (`mcontrol6.size=0`);
- Match 16-bit instructions (`mcontrol6.size=2`);
- Match 32-bit instructions (`mcontrol6.size=3`).

The configuration with `mcontrol6.execute=1` and `mcontrol6.size=1` match no instruction addresses.

### 8.2.1.1. Address Match Mode

The only supported address match mode is equal (`mcontrol6.match=0`), which performs full address matching that is specified in `tdata2`.

When the trigger matches an address and fires, `mcontrol6.hit0` is set.

### 8.2.1.2. Trigger Match Enable in Different Privilege Levels

The trigger performs address matching on the condition when this trigger is enabled in the corresponding privilege level.

- Whether the trigger is enabled or not in M-mode is controlled by `mcontrol6.m`.
- Whether the trigger is enabled or not in U-mode is controlled by `mcontrol6.u`, if U-mode is implemented.

### 8.2.1.3. Trigger Match and Fire Priority

If multiple triggers matches simultaneously, instruction address match has the higher priority than load/store address match. Load address match and store address match have the same priority. Only the trigger with a higher priority fires and an action is taken.

If multiple triggers with the same priority match, `mcontrol6.hit0` is set for all of them. If at least one of these triggers has the `mcontrol6.action=1` (enter Debug Mode), the hart enters Debug Mode. It is the responsibility of an external debugger to check the status of the `hit0` bit of those triggers and decide whether those triggers with the `mcontrol6.action=0` would raise the breakpoint exception after exiting Debug Mode. If none of these triggers has the `mcontrol6.action=1`, `mtval` is updated in accordance with them.

### 8.2.1.4. Trigger Timing

The triggers on Wing-M130 processor are always fired before the retirement of instructions, including the execute address, load address and store address. (Refer to [Table 15](#))

| Match Type      | Trigger Timing |
|-----------------|----------------|
| Execute Address | Before         |
| Load Address    | Before         |
| Store Address   | Before         |

Table 15. Trigger Timings



### 8.2.1.5. Trigger Fire Action

Triggers can be configured to take one of several actions when they fire.

- `mcontrol6.action=0`, raise a breakpoint exception. This action is used in Native Debug, where the debugger is hosted in a more privileged mode. Refer to [Section 8.3](#).
- `mcontrol6.action=1`, enter Debug Mode. This action is used in External Debug. Refer to [Chapter 7](#).
- `mcontrol6.action=2`, trace on
- `mcontrol6.action=3`, trace off
- `mcontrol6.action=4`, trace notify

## 8.3. Native Debug Triggers

The Wing-M130 processor supports both Native Debug and External Debug, Where

- Native Debug supports debugging through the generation and handling of debug exceptions. The triggers are programmed to cause a breakpoint exception to trap to a more privileged mode. And the hart itself hosts a debugger in this privilege level. (`mcontrol6.action=0`)
- External Debug supports debugging by an external debugger. The external debugger programs hart to generate debug events that causes the hart to enter Debug Mode. The triggers are programmed to enter Debug Mode. (`mcontrol6.action=1`)

When a trigger fires it causes a breakpoint exception to trap to M-mode. When a trigger is programmed to have `mcontrol6.m=0`. The trigger only fires in U-mode. However, when `mcontrol6.m=1`, it is possible for the trigger to fire in M-mode that the resulting exception will be handled in.

In this case such a trigger may cause a breakpoint exception while already in a trap handler. This might have the hart unable to resume normal execution because the context information with `mcause` and `mepc` would be overwritten.

To solve the reentrancy, Wing-M130 processor implements two-level stacking of trigger enable in M-mode (`tcontrol.{mpte, mte}`).

When any trap into M-mode is taken, `tcontrol.mpte` is set the value of `tcontrol.mte` and `tcontrol.mte` is set to 0 synchronously, temporarily disabling a new trap caused by trigger taken into M-mode. After the context saving, the trigger enable bit `tcontrol.mte` could be enabled by software as the context related CSRs `mcause`, `mepc` have already been saved.

## 8.4. Trigger Access Concept

The triggers are accessed by CSR opcodes, which are shared by the same CSR address 0x7A1. Each trigger is accessed by setting the corresponding index in `tselect`. If the trigger `tselect.index` points to does not exist, a read from `tdata1.type` will return 0, which means there is no trigger at this `tselect.index`. Only an exist trigger returns the valid `tdata1.type`, which, on Wing-M130 processor, is `mcontrol6`. The set of accessible triggers must start at 0, and be contiguous. It is used to discover the triggers.

## 8.5. CSR Registers

The following lists CSR registers.

| CSR Address | Privilege | Register  | Description          |
|-------------|-----------|-----------|----------------------|
| 0x7A0       | MRW       | tselect   | Trigger select       |
| 0x7A1       | MRW       | tdata1    | Trigger data 1       |
| 0x7A2       | MRW       | tdata2    | Trigger data 2       |
| 0x7A4       | MRW       | tinfo     | Trigger info         |
| 0x7A5       | MRW       | tcontrol  | Trigger control      |
| 0x7A1       | MRW       | mcontrol6 | Match control type 6 |

### 8.5.1. Trigger Select, tselect

#### Description

The tselect register is 32-bit read/write register which determines which trigger is accessible through the other trigger registers. The set of accessible triggers must start at 0, and be contiguous.

This register is **WARL**. Writes of values greater than or equal to the number of supported triggers may result in a different value in this register than what was written, or may point to a trigger where tdata1.type=0. To verify that what they wrote is a valid index, debuggers can read back the value and check that tselect holds what they wrote and read tdata1 to see that type is non-zero.

Since triggers can be used both by Debug Mode and M-mode, the external debugger must restore this register if it modifies it.



#### Configuration

This register is present only when sdtrig is implemented.

#### Field Description

##### index, Bits [31:0]

Select a valid trigger. This field is **WARL** field. Write with other values will return 0. The valid value depends on how many triggers are configured. For example, if there are totally 4 mcontrol6 configured, the range of index is from 0 to 3.

When there are no triggers configured, write this field results in an illegal instruction exception.

This field is reset to an UNSPECIFIED value.

#### CSR Address

0x7A0

### Description

Writing 0 to this register must result in a trigger that is disabled.



This register is present only when `sdtrig` is implemented.

**type, Bits [31:28]**

0 none, there is no trigger at this tselect.

**6** mcontrol16, the selected trigger is an address match trigger.

**15** disabled, the selected trigger is present, but is disabled. data field is UNSPECIFIED.

This field is reset to an UNSPECIFIED value.

Disable tdata writable at M-mode. This bit is only writable from Debug Mode. Writes from M-mode are ignored.

**0** Both Debug and M-mode can write the tdata registers at the selected trigger.

1 Only Debug Mode can write the tdata registers at the selected trigger. Writes from other modes are ignored.

This field is reset to 0.

Trigger-specific data. Refer to the corresponding triggers. If the field type is 0, then this field is hard-wired to 0.

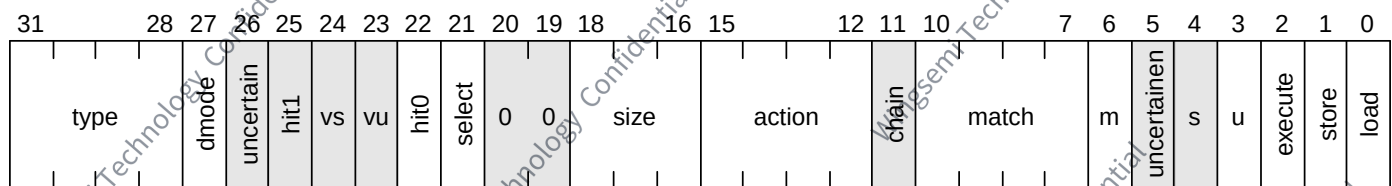
This field is reset to an UNSPECIFIED value.

## 0x7A1

### 8.5.3. Match Control Type 6, mcontrol6

#### Description

The mcontrol6 register is a 32-bit read/write register that is accessible as `tdata1` when `tdata1.type` is 6.



#### Configuration

This register is present only when `sdtrig` is implemented.

#### Field Description

##### type, Bits [31:28]

Trigger Type. Refer to `tdata1.type`. This field is **WARL** field. Write values other than 6 will read as 15 (disabled).

##### dmode, Bits [27]

Disable `tdata` writable at M-mode. Refer to `tdata1.dmode`.

##### uncertain, Bit [26]

**RES0**, Reserved, **WARL** field.

##### hit1, Bit [25]

**RES0**, Reserved, **WARL** field.

##### vs, Bit [24]

**RES0**, Reserved, **WARL** field.

##### vu, Bit [23]

**RES0**, Reserved, **WARL** field.

##### hit0, Bit [22]

Whether this trigger is fired or not. The TM updates this field when the trigger fires.

**0** false, the trigger did not fire.

**1** before, the trigger fired before the instruction that matched it was retired, but after all preceding instructions are retired. This explicitly allows for instructions to be partially executed. `xepc` or `dpc` (depending on action) must be set to the address of the instruction that matched.

This field is reset to 0.

##### select, Bit [21]

This bit determines the contents of the 32-bit compare values. This value is **WARL** field.

- 0** address, There is at least one compare value and it contains the lowest address of the access. In addition, it is recommended that there are additional compare values for the other accessed addresses match. (E.g. on a 32-bit read from 0x4000, the lowest address is 0x4000 and the other addresses are 0x4001, 0x4002, and 0x4003)

### RES0, Bits [20:19]

Reserved, **WARL** field.

### size, Bits [18:16]

The address match size. This field is **WARL** field. Write with other values will read as 0.

- 0** any, the trigger will attempt to match against an access of any size.
- 1** 8-bit, the trigger will only match against 8-bit memory accesses.
- 2** 16-bit, the trigger will only match against 16-bit memory accesses.
- 3** 32-bit, the trigger will only match against 32-bit memory accesses.

This field is reset to 0.

### NOTE

In the case of address matching of an instruction, the size configuration with the value 1 matches no instructions.

### action, Bits [15:12]

The action to taken when the trigger fires. This field is **WARL** field. Write with other values will read as 0.

- 0** Raise a breakpoint exception. (Native Debug Mode)
- 1** Enter Debug Mode. (External Debug Mode) This action is only legal when the trigger's mcontrol6.dmode is 1. Otherwise, write this field to 1 will read as 0.

This field is reset to 0.

### chain, Bit [11]

**RES0**, Reserved, **WARL** field.

### match, Bit [10:7]

The address match mode. This field is **WARL** field and has the following encoding.

- 0** Match when any compare value equals tdata2.

### m, Bit [6]

Enable the trigger in M-mode.

- 0b0** The trigger is disabled in M-mode.
- 0b1** The trigger is enabled in M-mode.

This field is reset to 0.



**uncertainen, Bit [5]**

**RES0**, Reserved, **WARL** field.

**s, Bit [4]**

**RES0**, Reserved, **WARL** field.

**u, Bit [3]**

**When U-mode is implemented**

Enable the trigger in U-mode.

**0b0** The trigger is disabled in U-mode.

**0b1** The trigger is enabled in U-mode.

This field is reset to 0.

**Otherwise**

**RES0**, Reserved, **WARL** field.

**execute, Bit [2]**

Control whether the trigger fires on the address of an instruction.

**0b0** The trigger does not fire on the address of an instruction.

**0b1** The trigger fires on the address of an instruction.

This field is reset to 0.

**store, Bit [1]**

Control whether the trigger fires on store address.

**0b0** The trigger does not fire on the address of any store.

**0b1** The trigger fires on the address of any store.

This field is reset to 0.

**load, Bit [0]**

Control whether the trigger fires on load address.

**0b0** The trigger does not fire on the address of any load.

**0b1** The trigger fires on the address of any load.

This field is reset to 0.

**CSR Address**

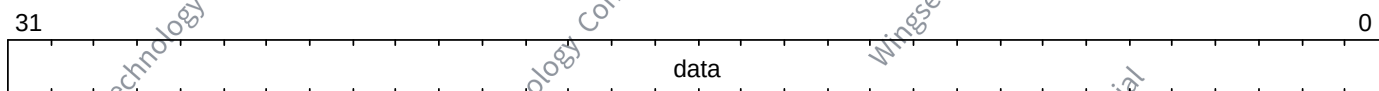
0x7A1

## 8.5.4. Trigger Data 2, tdata2

### Description

The tdata2 register is a 32-bit read/write register that contains trigger-specific data.

If the trigger is disabled, then this register can be written with any value supported by any of the trigger types supported by this trigger.



### Configuration

This register is present only when sdtrig is implemented.

### Field Description

#### data, Bits [31:0]

Trigger-specific data.

This field is reset to an UNSPECIFIED value.

### CSR Address

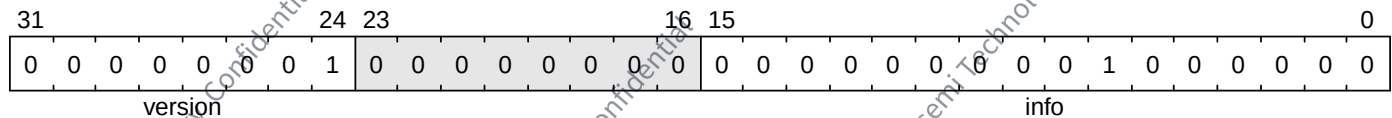
0x7A2



### 8.5.5. Trigger Info, tinfo

#### Description

The tinfo register is a 32-bit read/write register that provides information about triggers.



#### Configuration

This register is present only when `sdtrig` is implemented.

#### Field Description

##### version, Bits [31:24]

The version of the `sdtrig` extension implemented. This field is **WARL** field.

- 1 Support the ratified version 1.0 of RISC-V Debug Specification.

##### RES0, Bits [23:16]

Reserved, **WARL** field.

##### info, Bits [15:0]

One bit for each possible `tdata1.type` enumerated in `tdata1`. Bit N corresponds to type N. If the bit is set, then that type is supported by the currently selected trigger. This field is **WARL** field.

- 0x40 Bitmap for each type of triggers. Bit N corresponds to type N.

#### CSR Address

0x7A4

### Description

[illegible]

This register is present only when `sdtrig` is implemented.

**RES0, Bits [31:8]**

**mpte, Bit [7]**

This field is reset to 0.

Reserved, **WARL** field.

M-mode trigger enable field. When any trap into M-mode is taken, `mte` is set to 0. When `mret` is executed, `mte` is set to the value of `mpte`.

**0b0** disabled, triggers with action=0 do not match/fire while the hart is in M-mode.

**0b1** enabled, triggers do match/fire while the hart is in M-mode.

This field is reset to 0.

Reserved, **WARL** field.

0x7A5

## 9. Performance Monitor Unit (PMU)

Wing-M130 processor provides basic counters and timers which comprise cycle count, real-time clock, and instruction retired.

### 9.1. Features

- Support 32-bit counters (xcntbw32)
- Provide the control of the availability of the counters to the next lower privileged mode.
- Provide the control of counters' increment.
- Counters are accessible by CSR registers.

### 9.2. Counters Availability to the Next-Lower Privileged Mode

There is a CSR register `mcounteren` that controls the availability of the counters to the next-lower privileged mode.

The settings in this register only control accessibility. The act of reading or writing this register does not affect the underlying counters, which continue to increment even when not accessible.

When the corresponding bit in the `mcounteren` register is clear, attempts to read the corresponding counter register while executing in U-mode will cause an illegal-instruction exception. When one of these bits is set, access to the corresponding register is permitted in the next implemented privilege mode.

The cycle and instret CSRs are read-only shadows of `mcycle` and `minstret`, respectively. The time CSR is a read-only shadow of the memory-mapped `mtime` register. Analogously, on RV32I the `cycleh` and `instreth` CSRs are read-only shadows of `mcycleh` and `minstreth`, respectively. On RV32I the `timeh` CSR is a read-only shadow of the upper 32 bits of the memory-mapped `mtime` register, while `time` shadows only the lower 32 bits of `mtime`.

### 9.3. Counters' Increment Control

The CSR register `mcountinhibit` controls which of the counters increment.

The settings in this register only control whether the counters increment; their accessibility is not affected by the setting of this register.

When the corresponding bit in the `mcountinhibit` is clear, the counter register increment as usual. When the corresponding bit is set, the corresponding counter does not increment.

### 9.4. Basic Counters and Timers

The basic counters and timers comprise the first three of these counters (`cycle`, `time`, and `instret`), which have dedicated functions (cycle count, real-time clock, and instructions retired, respectively).

By default, counters are 64-bit width. It could be configured to 32-bit

The cycle counter provides a count of the number of clock cycles executed by the processor core on which

the hart is running from an arbitrary start time in the past.

The real-time clock counter increments by one with each tick of the real-time clock. Refer to [Chapter 11](#) for more details.

The instruction retired counter counts the number of instructions retired by this hart from some arbitrary start point in the past.

**NOTE**

Instructions that cause synchronous exceptions, including ECALL and EBREAK, are not considered to retire and hence do not increment the instret CSR.

## 9.5. CSR Registers

| CSR Address | Privilege | Register      | Description                                            |
|-------------|-----------|---------------|--------------------------------------------------------|
| 0x306       | MRW       | mcounteren    | Machine counter enable                                 |
| 0x320       | MRW       | mcountinhibit | Machine counter-inhibit register                       |
| 0xB00       | MRW       | mcycle        | Machine cycle counter                                  |
| 0xB02       | MRW       | minstret      | Machine instruction retired counter                    |
| 0xC00       | URO       | cycle         | Cycle counter for RDCYCLE instruction                  |
| 0XC01       | URO       | time          | Timer for RDTIME instruction                           |
| 0xC02       | URO       | instret       | Instructions-retired counter for RDINSTRET instruction |

### Description

[illegible]

This register is present only when U-mode is implemented.

### RES0, Bits [31:3]

**IR, Bit [2]**

Instruction retired counter enable in lower privilege-level.

**0b0** Access instret register in U-mode causes an illegal instruction exception.

**0b1** Access instret register in U-mode is permitted.

**TM, Bit [1]**

Timer enable in lower privilege-level.

**0b0** Access time register in U-mode causes an illegal instruction exception.

**0b1** Access time register in U-mode is permitted.

**CY, Bit [0]**

Cycle counter enable in lower privilege-level.

**0b0** Access cycle register in U-mode causes an illegal instruction exception.

**0b1** Access cycle register in U-mode is permitted.

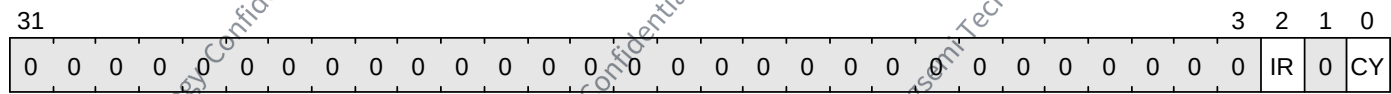
## CSR Address

0x306

### 9.5.2. Machine Counter-Inhibit Register, mcountinhibit

#### Description

The mcountinhibit is a 32-bit read/write register that controls which of the hardware performance-monitoring counters increment.



#### Configuration

This register is present only when zicntr and/or zihpm are/is implemented.

#### Field Description

##### RES0, Bits [31:3]

Reserved, WARL field.

##### IR, Bit [2]

Instruction retired counter increments.

0b0 minstret register increments.

0b1 minstret register does not increment.

This field is reset to 0.

##### RES0, Bit [1]

Reserved, WARL field.

##### CY, Bit [0]

Cycle counter increments.

0b0 mcycle register increments.

0b1 mcycle register does not increment.

This field is reset to 0.

#### CSR Address

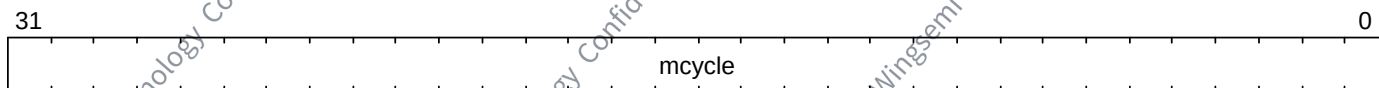
0x320



### 9.5.3. Machine Cycle Counter, mcycle

#### Description

The mcycle register is a 64-bit read/write register counts the number of clock cycles executed by the processor core on which the hart is running. Read/Write of this register only returns bits 31..0 of the corresponding counter. The higher bits 63..32 are read-only-zero.



#### Configuration

This register is present only when zicntr is implemented.

#### Field Description

##### mcycle, Bits [31:0]

Contain the lower 32-bit of mcycle counter.

This field is reset to zero.

#### CSR Address

0xB00



### 9.5.4. Machine Instruction Retired Counter, minstret

#### Description

The minstret register is a 64-bit read/write register counts the number of instructions the hart has retired. Read/Write of this register only returns bits 31..0 of the corresponding counter. The higher bits 63..32 are read-only-zero.



#### Configuration

This register is present only when zicntr is implemented.

#### Field Description

##### minstret, Bits [31:0]

Contain the lower 32-bit of minstret counter.

This field is reset to zero.

#### CSR Address

0xB02

### 9.5.5. Cycle Counter, cycle

#### Description

The cycle register is a 64-bit read-only register which is shadow of `mcycle`. Read of this register only returns bits 31..0 of the corresponding counter. The higher bits 63..32 are read-only-zero.



#### Configuration

This register is present only when `zicntr` and U-mode are implemented.

#### Field Description

##### cycle, Bits [31:0]

Contain the lower 32-bit of `cycle` counter.

This field is reset to zero.

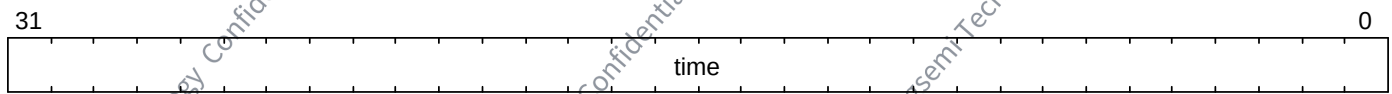
#### CSR Address

0xC00

## 9.5.6. Timer, time

### Description

The time register is a 64-bit read-only register which is shadow of MMR mtimer. Read of this register only returns bits 31:0 of the corresponding counter. The higher bits 63..32 are read-only-zero.



### Configuration

This register is present only when zicntr and U-mode are implemented.

### Field Description

#### time, Bits [31:0]

Contain the lower 32-bit of time counter.

This field is reset to zero.

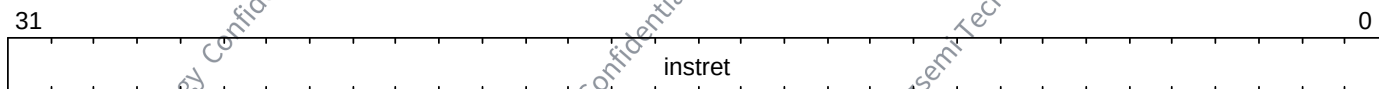
### CSR Address

0xC01

### 9.5.7. Instruction Retired Counter, instret

#### Description

The instret register is a 64-bit read-only register which is shadow of minstret. Read of this register only returns bits 31:0 of the corresponding counter. The higher bits 63..32 are read-only-zero.



#### Configuration

This register is present only when zicntr and U-mode are implemented.

#### Field Description

##### instret, Bits [31:0]

Contain the lower 32-bit of instret counter.

This field is reset to zero.

#### CSR Address

0xC02

## 10. Interrupt Controller

Wing-M130 processor supports Core-Local Interrupt Controller (CLIC, `mtvec.mode=0b11`).

### 10.1. Features

- Support up to 4096 interrupt inputs per hart.
- Configurable vector mode, trigger type and priority for each interrupt.
- Support the selective hardware vectoring extension (`smclicshv`).
- Support a global configuration register `mcliccfg` to define how the `clicintctl[i]` are subdivided into level and priority fields. (`smclicconfig`)
- Support horizontal interrupt preemption.
- Support resumable NMI (`smrnmi`)

### 10.2. Core-Local Interrupt Controller (CLIC)

The Core-Local Interrupt Controller (CLIC) Privileged Architecture Extensions (`smclic`) are designed to provide low-latency, vectored, pre-emptive interrupts for RISC-V systems.

The CLIC supports up to 4096 interrupt inputs per hart. Each interrupt input  $i$  has four 8-bit memory-mapped control registers:

- An interrupt-pending bit (`clicintip[i]`),
- An interrupt-enable bit (`clicintie[i]`),
- Interrupt attributes (`clicintattr[i]`) to specify privilege mode, trigger type and vector mode (if `smclicshv` is supported),
- And interrupt control bits to specify level and priority (`clicintctl[i]`)

When the first 32 interrupt inputs are reserved for the CLINT mode interrupts present in the `mip` and `mie` registers, up to 4064 local interrupts can be added.

#### 10.2.1. CLIC Interrupt ID Mapping

For forward compatibility, the first 32 interrupt IDs are reserved, of which the allocated IDs are illustrated in [Table 16](#).

| Interrupt | Exception Code | Description                                           |
|-----------|----------------|-------------------------------------------------------|
| 1         | -1             | NMI (if configured, refer to <a href="#">[unmi]</a> ) |
| 1         | 3              | Machine software interrupt                            |
| 1         | 7              | Machine timer interrupt                               |
| 1         | 11             | Machine external interrupt                            |
| 1         | 31             | Machine asynchronous RAS fault                        |

Table 16. Interrupt Code

CLIC interrupt inputs are allocated IDs beginning at interrupt ID 32.

### 10.2.2. Interrupt Enable and Pending

Each interrupt input  $i$  has a dedicated interrupt-enable bit in `clicintie[i]`. This control bit is read-write to enable/disable the corresponding interrupt.

- Writing 1 to `clicintie[i]` enables the corresponding interrupt.
- Writing 0 to `clicintie[i]` disables the corresponding interrupt.

Meanwhile, each interrupt input  $i$  has another dedicated interrupt-pending bit indicating the corresponding interrupt is in pending state.

When the interrupt input is configured for level-sensitive input, the `clicintip[i]` bit reflects the value of an input signal to the interrupt controller after any conditional inversion specified by the `clicintattr[i]` field, and software writes to the bit are ignored. Software clears the interrupt at the source device.

When the interrupt input is configured for edge-sensitive input, `clicintip[i]` is a read-write register that can be updated both by hardware interrupt inputs and by software. The bit is set by hardware after an edge of the appropriate polarity is observed on the interrupt input, as determined by the `clicintattr[i]` field. Software writes can set or clear edge-triggered pending bits directly by writes to the memory-mapped register. Edge-triggered pending bits can also be cleared when a CSR instruction that accesses `mnxti` includes a write.

#### NOTE

When the input is configured for edge-sensitive input, hardware clears the associated interrupt pending bit when an interrupt is serviced in vectored mode (`clicintattr[i].shv==0b1`).

#### NOTE

The value in the `clicintip[i]` is undefined when switching from level-sensitive mode to edge-triggered mode in `clicintattr[i]`.

### 10.2.3. Interrupt Trigger Type and Polarity

The interrupt trigger type and polarity of each interrupt input  $i$  can be specified by `clicintattr[i].trig`.

- Bit 1, `trig[0]`, is defined as "edge-triggered" (0: level-triggered, 1: edge-triggered).
- Bit 2: `trig[1]` is defined as "negative-edge" (0: positive-edge, 1: negative-edge).

More specifically, there can be four possible combinations: positive level-triggered, negative level-triggered, positive edge-triggered, and negative edge-triggered.

#### NOTE

The implementations may support only certain trigger type and polarity. In this case, these bits become hardwired to fixed values.

- As for the timer interrupt, positive edge-triggered and positive level-triggered are supported. (Refer to [Section 11.3](#))
- For the software interrupt, only positive edge-triggered is supported. (Refer to [Section 11.5.4](#))

## 10.2.4. Interrupt Level and Interrupt Priority

CLIC extends interrupt preemption to support up to 256 interrupt levels for each privilege mode, where higher-numbered interrupt levels can preempt lower-numbered interrupt levels. Interrupt level 0 corresponds to regular execution outside of an interrupt handler. Levels 1-255 correspond to interrupt handler levels.

### 10.2.4.1. Specifying Interrupt Level

At a privilege level, interrupt level and interrupt priority are assigned to an interrupt, which is specified by an 8-bit memory-mapped register `clicintctl[i]`, where the `CLICINTCLTBITS` parameter specifies how many hardware bits are actually implemented in the `clicintctl[i]` registers, with the range from 0 to 8. The implemented bits are kept left-justified in the most-significant bits of each 8-bit `clicintctl[i]` register, with the lower unimplemented bits treated as hardwired to 1.

The 4-bit `mcliccfg.mnlbits` fields indicate how many upper bits in `clicintctl[i]` are assigned to encode the interrupt level in M-mode, while the lower bits are used to encode the interrupt priority.

Although the interrupt level is an 8-bit unsigned integer, the number of bits actually assigned or implemented can be fewer than 8-bit. The number of bits actually implemented can be derived from `mcliccfg.mnlbits` and the parameter `CLICINTCLTBITS` which specifies bits

For example, if `mcliccfg.mnlbits > CLICINTCLTBITS`, then the lower bits of the 8-bit interrupt level are assumed to be all 1s. Similarly, if `mcliccfg.mnlbits < 8`, then the lower bits of the 8-bit interrupt level are assumed to be all 1s;

If `mcliccfg.mnlbits=0`, then all interrupts are treated as level 255.

Example of `mcliccfg` settings

| CLICINTCLTBITS | mnlbits | clicintctl[i] | interrupt levels                    |
|----------------|---------|---------------|-------------------------------------|
| 0              | 2       | .....         | 255                                 |
| 1              | 2       | 1.....        | 127, 255                            |
| 2              | 2       | 11.....       | 63, 127, 191, 255                   |
| 3              | 3       | 111.....      | 31, 63, 95, 127, 159, 191, 223, 255 |
| 4              | 1       | 1ppp....      | 127, 255                            |

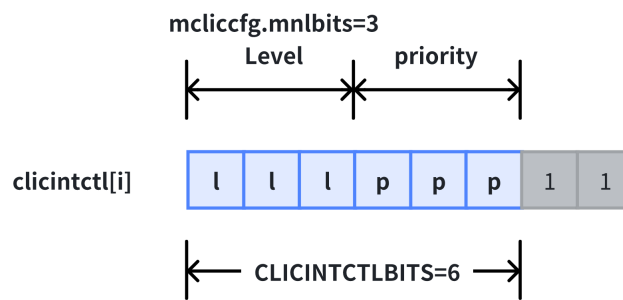
"." bits are non-existent bits for level encoding, assumed to be 1

"1" bits are available variable bits in level specification

"p" bits are available variable bits in priority specification

Example of `clicintctl[i]` is shown in [Figure 7](#), with hardware implemented bits being 6, of which the upper 3 bits are configured to encode the interrupt level and the lower 3 bits are assigned to encode the interrupt priority.



Figure 7. Example of `clicintctl[i]`

#### 10.2.4.2. Specifying Interrupt Priority

The least-significant bits in `clicintctl[i]` that are not configured to be part of the interrupt level are interrupt priority, which are used to prioritize among interrupts pending-and-enabled at the same privilege mode and interrupt level. The highest-priority interrupt at a given privilege mode and interrupt level is taken first. In case there are multiple pending-and-enabled interrupts at the same highest priority, the highest-numbered interrupt is taken first.

##### NOTE

The highest numbered interrupt wins in a tie (when privilege mode, level and priority are all identical).

Notice that the 8-bit interrupt level is used to determine preemption (for nesting interrupts). In contrast, the 8-bit interrupt priority does not affect preemption but is only used as a tie-breaker when there are multiple pending interrupts with the same interrupt level.

Any implemented priority bits are treated as the most-significant bits of a 8-bit unsigned integer with lower unimplemented bits set to 1. For example, with one priority bit (`p111_1111`), interrupts can be set to have priorities 127 or 255, and with two priority bits (`pp11_1111`), interrupts can be set to have priorities 63, 127, 191, or 255.

#### 10.2.4.3. Interrupt-Level Threshold

CLIC provides a threshold CSR `mintthresh` for the threshold level of M-mode.

A typical usage of the interrupt-level threshold is for implementing critical sections. The current handler can temporarily raise its effective interrupt level to implement a critical section among a subset of levels, while still allowing higher interrupt levels to preempt.

The current hart's effective interrupt level would then be:  $\text{effective\_level} = \max(\text{mintstatus.mil}, \text{mintthresh.th})$

The `max` is used to prevent a hart from dropping below its original level which would break assumptions in design, and also makes it simple for software to remove threshold without knowing its own level by simply setting `mintthresh` to the lowest supported `mintthresh` value.

The interrupt-level threshold is only valid when running in associated privilege mode and not in other modes. This is because interrupts for lower privilege modes are always disabled, whereas interrupts for higher privilege modes are always enabled.



If the hart is currently running at M-mode, an MRET instruction that changes the privilege mode to U-mode also sets `mintthresh` to the lowest supported `mintthresh` value. This helps software avoid a higher privilege mode from having a non-minimum threshold while a lower privilege mode is running.

**NOTE**

The anticipated use of threshold is to provide critical sections within code running at one privilege level, not to selectively mask interrupts before running lower-privilege code. If desired, higher-privilege-mode interrupts can be selectively disabled using local interrupt enables before switching to a lower privilege mode.

**NOTE**

This behavior significantly reduces the hardware cost because it only needs to select one global maximum interrupt and compare with the threshold of the associated privilege mode. If higher-privilege modes could have non-minimum thresholds, hardware would have to select multiple maximum interrupts (one for the current mode and one for each higher-privilege mode) qualified by the per-mode threshold, then pick a qualified maximum interrupt with the highest privilege mode.

If the number of bits actually implemented in the `th` field is less than 8 (e.g. an implementation option when `CLICINTCTLBITS` is less than 8), the number of implemented bits `INTTHRESHBITS` must be greater than `CLICINTCTLBITS` and the implemented bits should be kept left-justified in the most-significant bits of the 8-bit field, with the lower unimplemented bits treated as hardwired to 1. For example, if `CLICINTCTLBITS` is 1 and `INTTHRESHBITS` is 2, interrupts can be set to level 127 or 255 and `mintthresh.th` can be set to 63, 127, 191, or 255.

### 10.2.5. Interrupt Arbitration

To select an interrupt to present to the core, the CLIC hardware combines the valid bits in `clicintattr.mode` and `clicintctl` to form an unsigned integer, then picks the global maximum across all pending-and-enabled interrupts based on this value. Next, `mcliccfg.mnlbits` defines how to split the `clicintctl` value into interrupt level and interrupt priority. Finally, the interrupt level of this selected interrupt is compared with the interrupt-level threshold of the associated privilege mode to determine whether it is qualified or masked by the threshold (and thus no interrupt is presented).

- An interrupt is a candidate in interrupt arbitration when it is a pending-and-enabled interrupt with `clicintie[i] & clicintip[i]`.
- Pick the global maximum across all pending-and-enabled interrupts based on `clicintctl` and their ID numbers.
- Check whether the interrupt level of the picked interrupt is greater than `max(mintstatus.mil, mintthresh.th)`. Otherwise, it is masked. And there is no interrupts provided to processors.
- The interrupt will trap to interrupt handler when either the current privilege mode is M-mode and the bit `mstatus.mie` is set, or the current privilege mode is U-mode, if U-mode is configured.

### 10.2.6. Vectored Mode and None-Vectored Mode

Wing-M130 processor supports the selective hardware vectoring extension (`smclicshv`), where each interrupt input can be configured in either vectored mode or non-vectored mode.

This feature gives users the flexibility to select the behavior for each interrupt: either hardware vectoring or non-vectoring. As a result, it allows users to optimize each interrupt and enjoy the benefits of both

behaviors. More specifically, hardware vectoring has the advantage of faster interrupt response at the price of slightly increasing the code size (to save/restore contexts). On the other hand, non-vectoring has the advantage of smaller code size (by sharing and reusing one copy of common code to save/restore contexts) at the price of slightly slower interrupt response.

- When `clicintattr[i].shv` is 0b0, this interrupt input is non-vectorized and when triggered, it jumps to the common base address held at `mtvec`.
- When `clicintattr[i].shv` is 0b1, this interrupt input is hardware-vectorized, it jumps to the trap-handler function pointer specified in `mtvt`.

Refer to [Section 10.2.7.2](#) for more details.

## 10.2.7. Enter Interrupt handler

### 10.2.7.1. Update CSR Registers

#### 10.2.7.1.1. Update mepc

The register `mepc` holds the return address when the hart returns from interrupt handler. When a trap is taken into M-mode, `mepc` is written with the address of the instruction that comes after that committed instruction.

#### 10.2.7.1.2. Update mstatus

`mstatus.mpie` is set to the value of `mstatus.mie`, and `mstatus.mie` is set to 0.

`mstatus.mpp` is set to the previous privilege mode. Therefore, when a trap is taken from privilege mode `y` into privilege mode `x`, `mstatus.mpp` is set to `y`.

#### 10.2.7.1.3. Update mcause

The register `mcause` is written with a code indicating the event that caused the trap. [Table 16](#) lists the mandatory interrupt related codes in Wing-M130 processor. `mcause` register adds `mpil` field to record previous interrupt level. When an interrupt is taken, `mcause.mpil` is updated with `mintstatus.mil`. Meanwhile, `mintstatus.mil` is written with the interrupt level of the interrupt input `i`.

If the interrupt input `i` works in vectored mode (`clicintattr[i].shv==1`) and the trigger type is edge-triggered, the corresponding pending bit in `clicintip[i]` is cleared.

### 10.2.7.2. Redirect the Program Counter

The PC will be redirected to the entry of an interrupt handler. When the corresponding interrupt is in non-vectorized mode, harts jumps to the trap handler address held in the upper 26 bits of `mtvec` (NBASE) for all exceptions and interrupts.

When the interrupt is in vectored mode, the hart fetches a 32-bit handler address with permissions corresponding to the handler's mode from the in-memory table whose base address (TBASE) is in `mtvt`. The trap handler function address is fetched from `TBASE+4*exccode`. If the fetch is successful, the hart clears the low bit of the handler address and sets the PC to this handler address. If the trap handler function address fetch is unsuccessful and an exception occurs, the `mcause.minhv` is set indicating that

mepc of the exception handler contains a trap handler function address instead of the address of an instruction.

Note that synchronous exception traps are always taken to NBASE.

`clicintattr[i].shv` Action on the redirect PC

```
1'b0      pc = {mtvec[31:6], 6'b0};           // CLIC non-vectorized mode
1'b1      pc = MEM[{mtvt[31:6], 6'b0}+4*exccode] & ~1; // CLIC vectored mode
```

Example of vector table layout is

```
# Vector table layout for RV32 (4-byte function pointer)
mtvt -> 0x800000 # Interrupt 0 handler function pointer
        0x800004 # Interrupt 1 handler function pointer
        0x800008 # Interrupt 2 handler function pointer
        0x80000c # Interrupt 3 handler function pointer
```

## 10.2.8. Return from an Interrupt

When executing an `mret` instruction, the hart is returned from the interrupt handler.

`mstatus.mie` is set to `mstatus.mpie`, and `mstatus.mpie` is set to 1.

The privilege mode trap returned is set to the mode indicated by `mstatus.mpp`. `mstatus.mpp` is set to 0b00, if U-mode is configured. Otherwise, it is set to 0b11.

If U-mode is supported, and `mret` instruction that changes the privilege mode to U-mode also sets `mstatus.mprv` to 0.

The redirect PC is set to the address offered by mepc.

### 10.2.8.1. Case When `mcause.minhv` is set

Setting `mcause.minhv` indicates that `mepc` contains a trap handler function address instead of the address of an instruction (interrupt return address).

If the `mcause.minhv` bit is set, the hart resumes the trap handler memory access to retrieve the function pointer for vectoring with permissions corresponding to M-mode. The trap handler function address is obtained from `mepc` with the low bits of the address cleared to force the access to be naturally aligned to 4-byte table entry. If the fetch is successful, the hart clears the low bit of the handler address and sets the PC to this handler address.

For permission-checking purposes, the memory access to retrieve the function pointer for hardware vectoring is an *implicit* fetch at M-mode, and requires execute permission; read permission is irrelevant.

If there is an access exception on the table fetch, `mepc` is written with the faulting address. `mtval` is either set to zero or written with the faulting address.

Memory writes to the vector table require an instruction barrier (`fence.i`) to guarantee that they are visible to the instruction fetch.

## 10.2.9. Interrupt Preemption and Nested Interrupts

CLIC supports both horizontal and vertical interrupt preemption.

- Incoming interrupts with a higher interrupt level can preempt an active interrupt handler running at a lower interrupt level in the same privilege level, provide interrupts are globally enabled in this privilege mode.
- Incoming interrupts for a higher privilege mode can preempt an active interrupt handler running in a lower privilege mode, regardless of global interrupt enable in lower privilege mode.

### NOTE

In Wing-M130 processor, interrupts are handled in M-mode only. So only horizontal interrupt preemption happens.

When an interrupt input is taken, the global interrupt enable `mstatus.mie` is automatically disabled. It allows the interrupt handler to save registers from the interrupted context and other interrupted context information like `mepc` and `mstatus`. To enable nested interrupts, software in the interrupt handler shall firstly enable `mstatus.mie`. If there are pending interrupts, hardware will follow the flow in [Section 10.2.5](#) to pick out an interrupt with a higher interrupt level to provide to processors.

## 10.2.10. Interrupt Tail-Biting

When there is an interrupt, of which the interrupt level is greater than the saved interrupt context (held in `mcause.pil`) and greater than the interrupt threshold of the corresponding privilege mode, the interrupt will be taken after current interrupt is returned. [Figure 8](#) illustrates a back-to-back interrupt handling flow.

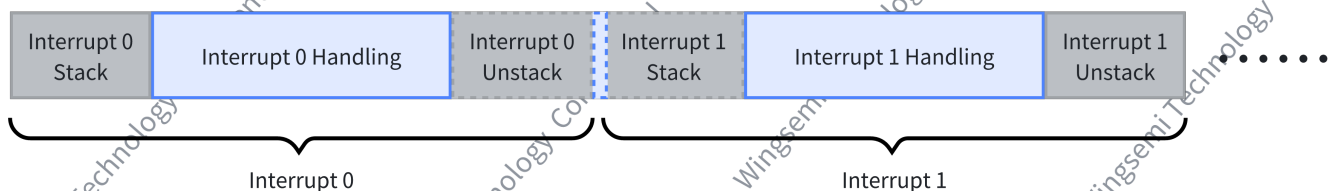


Figure 8. The Conventional Back-to-Back Interrupt Handling

Before Interrupt 0 returns from interrupt handler, the handler restores the saved context, executes `mret` instruction and returns back to the previous context. As there is a valid pending interrupt 1, the hart will trap to the interrupt handler again, and perform context stacking operations. Obviously, there is a redundant back-to-back context unstacking and stacking process between two interrupts, as shown in the dashed boxes.

Tail-Biting optimizes unstacking and stacking operation, without incurring the full cost of an interrupt pipeline flush and context save/restore, thus reducing the number of back-to-back interrupt handling cycles.

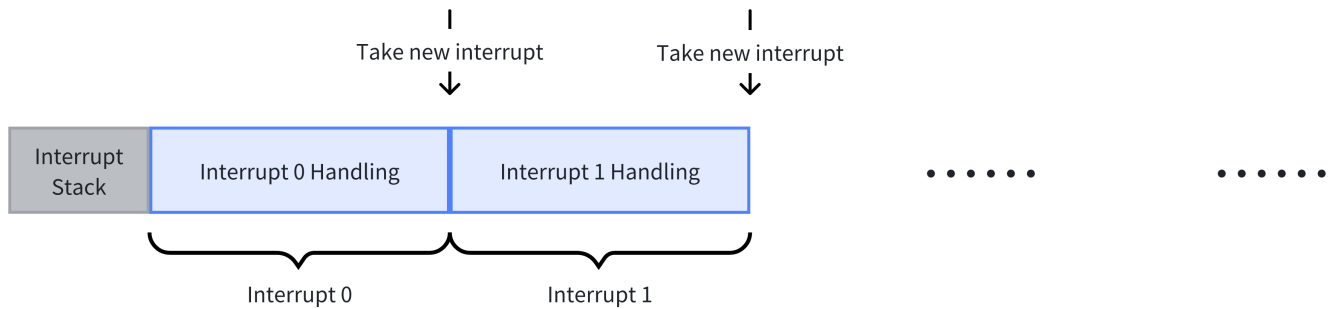


Figure 9. The Back-to-Back Tail-Bite Interrupt Handling

### 10.2.10.1. mnxti Registers

CLIC implements a CSR named `mnxti` that is used by software to service the next horizontal interrupt. The `mnxti` CSR is designed to be accessed using `CSRRSI`/`CSRRCI` instructions, where the value read is a pointer to an entry in the trap handler table and the write back updates the interrupt-enable status. In addition, writes to the `mnxti` have side-effects that update the interrupt context state.

The `mnxti` CSR is intended to be used inside an interrupt handler after an initial interrupt has been taken and `mcause` and `mepc` registers updated with the interrupted context and the id of the interrupt.

### 10.2.10.2. Accessible Instructions

The CSR is only designed to be used with the `CSRR` (`csrrs rd, csr, x0`), `CSRRSI`, and `CSRRCI` instructions. Accessing the `mnxti` CSR using any other CSR instruction form (`CSRWR` / `CSRWS`, `rs1!=x0` / `CSRRC`/`CSRRI`) is reserved. Use of `mnxti` with `CSRRSI` with non-zero `uimm` values for bits, 0, 1, 2, and 4 are reserved. Executing these reserved CSR operations on `mnxti` will be treated as NOP.

CSR read of `mnxti` will return

- Zero value, when one of the followings is true
  - There is no available interrupt
  - There is an available interrupt but it a selectively hardware vectored interrupt
  - It works in non-CLIC mode.
- Otherwise, return the pointer to trap handler entry.

CSR write of `mnxti` will

- Perform read-modify-write operation on `mstatus.mie`.
- Have side-effects on other CSR register, where the `mcause.exccode` and `mintstatus.mil` can be updated with the new interrupt id and level. If the interrupt is edge-triggered, then the pending bit is also zeroed.

These instructions have the equivalent behavior with read operation only

- `csrr rd, mnxti`
- `csrrsi rd, mnxti, 0`



- `csrrci rd, mnxti, 0`

These two instructions perform read-modify-write on `mstatus.mie` and have side-effects.

- `csrrsi rd, mnxti, mie (uimm5==8)`
- `csrrci rd, mnxti, mie (uimm5==8)`

**NOTE** Instructions that access `mnxti` in debug mode will be treated as nop.

### 10.2.10.3. Pseudo-Code

The following illustrates pseudo-code for CSRRSI. The behavior of CSRR, and CSRRCI could be deduced accordingly.

```
// Pseudo-code for csrrsi rd, mnxti, uimm[4:0] in M-mode
// clic.priv, clic.level, clic.id represent the highest-ranked interrupt
// currently present in the CLIC
// For csrrci rd, mnxti, uimm[4:0]
// mstatus.mie &= ~uimm[3];
mstatus.mie |= uimm[3]; // Performed regardless of interrupt readiness.
if (clic.priv==M &&
    clic.level > mcause.pil &&
    clic.level > mintthresh.th &&
    clicintattr[clic.id].shv==0) {
    // There is an available interrupt, non-hardware-vectorized interrupt
    if (uimm[4:0] != 0) { // Side-effects should occur.
        // Commit to servicing the available interrupt.
        mintstatus.mil = clic.level; // Update hart's interrupt level.
        mcause.exccode = clic.id;    // Update interrupt id in mcause.
        mcause.interrupt = 1;        // Set interrupt bit in mcause.
        // If edge interrupt,
        if (clicintattr[clic.id][1] == 1) {
            clicintip[clic.id] = 0; // clear edge interrupt
        }
    }
    rd = TBASE + XLEN/8 * clic.id; // Return pointer to trap handler entry.
} else {
    // No interrupt or in non-CLIC mode.
    rd = 0;
}
```

### 10.2.11. Conditional Swap mscratch on Privilege Mode Change

To accelerate interrupt handling with multiple privilege modes, a new CSR `mscratchcsw` is defined for all but the lowest privilege mode supported in a given implementation to support conditional swapping of the `mscratch` register when transitioning between privilege modes. The CSR instruction is used once at the entry to a handler routine and once at handler exit, so only adds two instructions to the interrupt code path.

`mscratchcsw` is only designed to be used with CSRRW with neither `rd` nor `rs1` set to `x0`. Accessing the `mscratchcsw` register with the CSRRW with either `rd` or `rs1` set to `x0`, or using any other CSR instruction

forms, are reserved. Executing these reserved CSR operations on `mscratchcsw` will be treated as NOP.

When using CSRRW to access `mscratchcsw`, the value written into `rd` is either `mscratch` if `mpp` is different than the current privilege mode, or `rs1` if `mpp` is the same as the current privilege mode. The `mscratch` register is only written with the original value of `rs1` if there is a privilege mode difference.

The formal description of this operation is

```
// csrrw rd, mscratchcsw, rs1
// In normal use, the instruction sets both rs1=sp, and rd=sp
uint<XLEN> t;

if (mstatus.mpp==PRIV_M) {
    X[rd] = X[rs1];
} else {
    t = X[rs1];
    X[rd] = mscratch;
    mscratch = t;
}
```

## 10.2.12. Conditional Swap `mscratch` on Interrupt Level Change

A new `mscratchcsw1` CSR is added to support faster swapping of the stack pointer between interrupt and non-interrupt code running in the same privilege mode. The swap condition is true when the interrupter and interruptee are not both application tasks or not both interrupt handlers.

`mscratchcsw1` is only designed to be used with CSRRW with neither `rd` nor `rs1` set to `x0`. Accessing the `mscratchcsw1` register with the CSRRW with either `rd` or `rs1` set to `x0`, or using any other CSR instruction forms, are reserved. Executing these reserved CSR operations on `mscratchcsw1` will be treated as NOP.

The formal description of this operation is

```
// csrrw rd, mscratchcsw1, rs1
// In normal use, the instruction sets both rs1=sp, and rd=sp
uint<XLEN> t;

if ((mcause.pil==0) != (mintstatus.mil==0)) {
    t = X[rs1];
    X[rd] = mscratch;
    mscratch = t;
} else {
    X[rd] = X[rs1];
}
```

## 10.3. Cases on Interrupt Entry

In Wing-M130 processor, interrupt is sampled at WB-stage. Once detected, the oldest instruction in WB-stage is to be committed conventionally. However, there are some special cases to be handled.

- When there is an in-flight load instruction with access to device locations, the interrupt wouldn't be taken until the instruction is committed.

- When there is an in-flight multiple instruction, the uops of instruction can be interrupted or committed on conditions. (Refer to [Section 10.3.1](#))
- When there are on-going operations caused by fence instructions, the interrupt wouldn't be taken until those operations are finished and therefore fence instructions are also committed.
- When there is a ECALL/EBREAK instruction, the interrupt wouldn't be taken. The corresponding exception is taken instead.
- The other cases are IMPLEMENTATION DEFINED.

### 10.3.1. Multiple Instruction on Interrupt Entry

Multiple Instruction is defined as those instructions that are split into multiple uops to execute and write multiple registers. In Wing-M130 processor, the instructions are

- CM.PUSH
- CM.POP
- CM.POPRET
- CM.POPRETZ

When an interrupt input is sampled on an in-flight multiple instructions, some uops have already be executed, while some are not.

The hart can discard the executed uops of multiple instruction to take the interrupt, and after returning from this interrupt, the multiple instruction is replayed from the start of the instruction.

#### NOTE

It is strongly recommended that Multiple Instruction is not used with memory location being Device memory. Otherwise, the result could be UNDEFINED.

## 10.4. Non-Maskable Interrupts (NMI)

Non-maskable interrupts (NMIs) are only used for hardware error conditions, and cause an immediate jump to the NMI vector running in M-mode regardless of the state of a hart's interrupt enable bits.

### 10.4.1. Resumable Non-Maskable Interrupts (RNMI)

The standard extension for resumable non-maskable interrupts (`smrnm`) adds support for resumable non-maskable interrupts (RNMIs) to RISC-V. The extension adds for new CSRs (`mnepc`, `mncause`, `mnstatus`, and `mnsratch`) to hold the interrupted state, and one new instruction `MNRET` to resume from the RNMI handler.

#### 10.4.1.1. RNMI Enable

Although NMI interrupts are defined to be nonmaskable. `smrnm` still provides a global-interrupt-enable such that NMIs are masked out of reset to give software the opportunity to initialize data structures and devices for subsequent NMI handling.

Software can set `mnstatus.nmie` to 1, but attempts to clear it have no effect.

#### NOTE

Normally, only reset sequences will explicitly set the NMIE bit.



**NOTE**

When `mnstatus.nmie=0`, the hart behaves as though `mstatus.mprv` were clear, regardless of the current setting of `mstatus.mprv`.

**10.4.1.2. Enter RNMI Interrupt Handler**

When an RNMI interrupt is detected, the interrupted PC is written to the `mnepc` CSR, the type of RNMI to the `mncause` CSR (if has), and the privilege mode of the interrupted context to the `mnstatus` CSR. The `mnstatus.nmie` bit is cleared, masking all interrupts.

The hart then enters machine-mode and jumps to the RNMI trap handler address.

**NOTE**

If the hart encounters an exception while executing in M-mode with the `mnstatus.nmie` bit clear, the actions taken are the same as if the execution had occurred while `mnstatus.nmie` were set, except that the program counter is set to the RNMI exception trap handler address specified by `mnvec`.

**10.4.1.3. Return from an Interrupt**

`rmrnmi` provides an M-mode only instruction `MNRET` that uses the values in `mnepc` and `mnstatus` to return to the program counter, privilege mode of the interrupted context. This instruction also sets `mnstatus.nmie`. If `MNRET` changes the privilege mode to a mode less privileged than M-mode, it also sets `mstatus.mprv` to 0.

**NOTE**

The `smrnmi` extension does not change the behavior of the `MRET` instruction. In particular, `MRET` is unaffected by the `mnstatus.nmie` bit, and their execution does not alter the `mnstatus.nmie` bit.

## 10.5. CSR Registers

The following lists CSR registers.

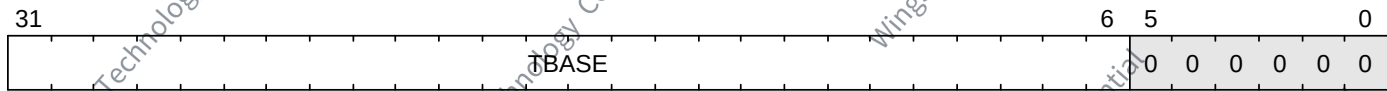
| CSR Address | Privilege | Register     | Description                                                          |
|-------------|-----------|--------------|----------------------------------------------------------------------|
| 0x307       | MRW       | mtvt         | Machine Trap Vector Table register                                   |
| 0x345       | MRW       | mnxti        | Machine Next Interrupt Handler Address and Interrupt-Enable Register |
| 0xFB1       | MRO       | mintstatus   | Machine Interrupt Status Register                                    |
| 0x347       | MRW       | mintthresh   | Machine Interrupt Threshold Register                                 |
| 0x348       | MRW       | mscratchcsw  | Conditional Scratch Swap on Privilege Mode Change                    |
| 0x349       | MRW       | mscratchcsw1 | Conditional Scratch Swap on Interrupt Level Change                   |
| 0x740       | MRW       | mnscratch    | Resumable NMI scratch register                                       |
| 0x741       | MRW       | mnepc        | Resumable NMI program counter                                        |
| 0x742       | MRW       | mncause      | Resumable NMI cause                                                  |
| 0x744       | MRW       | mnstatus     | Resumable NMI status                                                 |
| 0xFC0       | MRO       | mnrvec       | NMI trap handler address                                             |
| 0x7C2       | MRW       | mcontextsw   | Context Swap                                                         |

### 10.5.1. CLIC CSRs

#### 10.5.1.1. Machine Trap Vector Table register, mtvt

**Description**

The `mtvt` CSR holds the base address of the trap vector table, aligned on a 64-byte boundary. Values other than 0 in the low 6 bits of `mtvt` are reserved.



**Configuration**

This register is present only when `smclic` is implemented.

**Field Description**

**TBASE, Bits [31:6]**

Hold the base address of the trap vector table, aligned on a 64-byte boundary.

This field is reset to an UNSPECIFIED value.

**RES0, Bits [5:0]**

Reserved, **WARL** field.

**CSR Address**

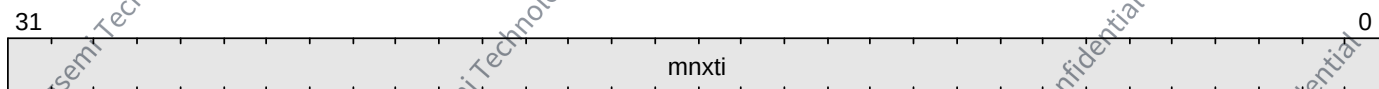
0x307

### 10.5.1.2. Machine Next Interrupt Handler Address and Interrupt-Enable Register, `mnxti`

#### Description

The `mnxti` CSR can be used by software to service the next horizontal interrupt for the same privilege mode when it has greater level than the saved interrupt context and greater level than the interrupt threshold of the corresponding privilege mode, without incurring the full cost of an interrupt pipeline flush and context save/restore.

This register is applied for interrupt tail-biting. Refer to [Section 10.2.10](#) for details.



#### Configuration

This register is present only when `smclic` is implemented.

#### Field Description

##### `mnxti`, Bits [31:0]

It is a **shadow** field, which contains no physical bits.

#### CSR Address

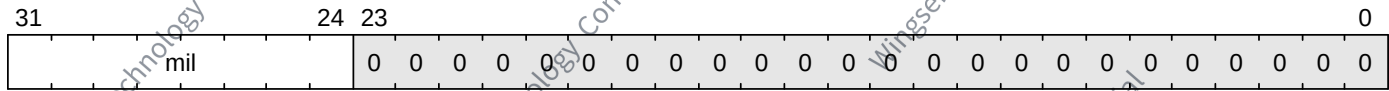
0x345

10.5.1.3. Machine Interrupt Status Register, `mintstatus`

Description

The `mintstatus` holds the active interrupt level for each supported privilege mode. This register is exposed to support debug.

This register is a Read-Only register.



Configuration

This register is present only when `smclic` is implemented.

Field Description

**mil, Bits [31:24]**  
Current interrupt level in M-mode.  
  
This field is reset to 0.

**RES0, Bits [23:0]**  
Reserved, **WARL** field.

**CSR Address**  
0xFB1

10.5.1.4. Machine Interrupt Threshold Register, mintthresh

Description

The mintthresh holds the threshold level of the associated privilege mode. Refer to [Section 10.2.4.3](#) for more details.



Configuration

This register is present only when smclic is implemented.

Field Description

**RES0, Bits [31:8]**  
Reserved, **WARL** field.

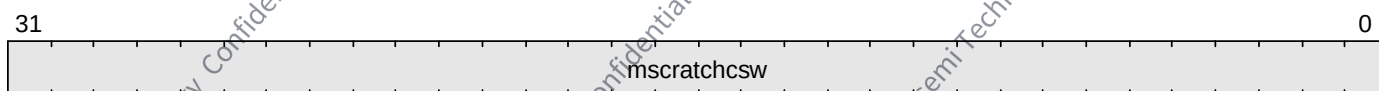
**th, Bits [7:0]**  
The threshold level of M-mode.  
  
This field is reset to 0.

**CSR Address**  
0x347

### 10.5.1.5. Conditional Scratch Swap on Privilege Mode Change, mscratchcsw

#### Description

The mscratchcsw supports conditional swapping of the mscratch register when transitioning between privilege modes. Refer to [Section 10.2.11](#) for more details.



#### Configuration

This register is present only when smclic is implemented.

#### Field Description

##### mscratchcsw, Bits [31:0]

It is a **shadow** field, which contains no physical bits.

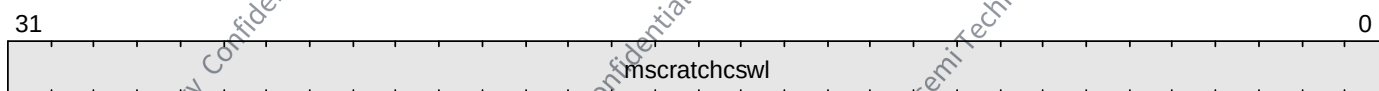
#### CSR Address

0x348

### 10.5.1.6. Conditional Scratch Swap on Interrupt Level Change, mscratchcsw1

#### Description

The mscratchcsw1 supports faster swapping of the stack pointer between interrupt and non-interrupt code running in the same privilege mode. Refer to [Section 10.2.12](#) for more details.



#### Configuration

This register is present only when smclic is implemented.

#### Field Description

##### mscratchcsw1, Bits [31:0]

It is a **shadow** field, which contains no physical bits.

#### CSR Address

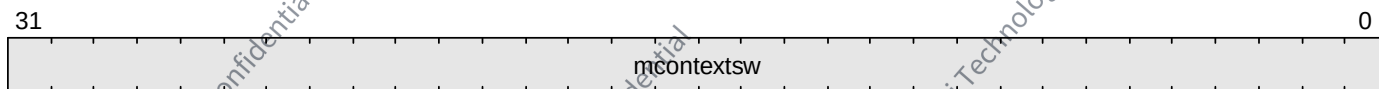
0x349



### 10.5.1.7. Machine Context Swap Register, mcontextsw

#### Description

The mcontextsw supports faster swapping of the data between mcause/mepc and the stack.



#### Configuration

This register is mandatory implemented.

#### Field Description

##### mcontextsw, Bits [31:0]

It is a **shadow** field, which contains no physical bits.

#### CSR Address

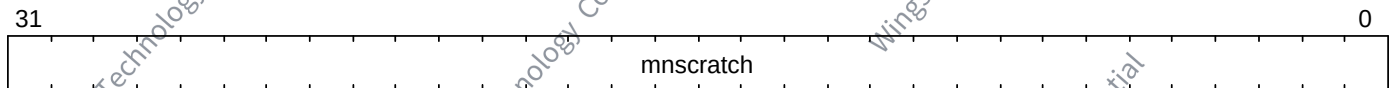
0x7C2

## 10.5.2. NMI CSRs

### 10.5.2.1. Resumable NMI Scratch Register, mnscratch

#### Description

The mnscratch CSR holds an 32-bit read-write register which enables the NMI trap handler to save and restore the context that was interrupted.



#### Configuration

This register is present only when smrnmi is implemented.

#### Field Description

##### mnscratch, Bits [31:0]

Software dedicated content.

This field is reset to an UNSPECIFIED value.

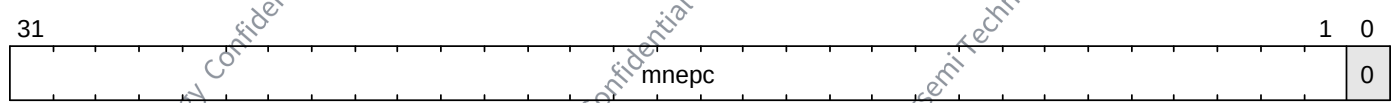
#### CSR Address

0x740

10.5.2.2. Resumable NMI Program Counter, mnepc

Description

The mnepc CSR is a 32-bit read-write register which on entry to the NMI trap handler holds the PC of the instruction that took the interrupt.



Configuration

This register is present only when smrnmi is implemented.

Field Description

mnepc, Bits [31:1]

Hold the PC of the instruction that took the NMI interrupt.

This field is reset to an UNSPECIFIED value.

RES0, Bit [0]

Reserved, WARL field.

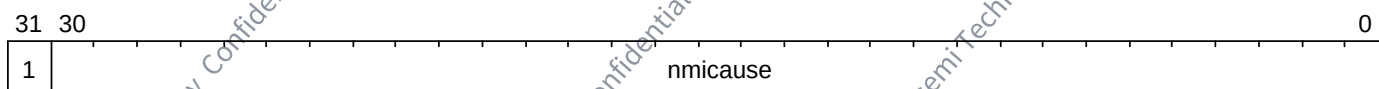
CSR Address

0x741

### 10.5.2.3. Resumable NMI Cause, mncause

#### Description

The mncause CSR holds the reason for the NMI, with bit 31 set to 1, and the NMI cause encoded in the least-significant bits or zero if NMI causes are not supported.



#### Configuration

This register is present only when smrnmi is implemented.

#### Field Description

##### Interrupt, Bit [31]

This field is a **WARL** field with the value 1.

**0b1** This is always labeled as an interrupt.

##### nmicause, Bits [30:0]

It encodes NMI cause in the least-significant bits. If NMI causes are not supported, this field is a **RES0** field.

Wing-M130 processor provides external pins for SoC to configure this field. Software should treat this field as read-only, write with no effect.

#### CSR Address

0x742

#### 10.5.2.4. Resumable NMI Status, mnstatus

##### Description

The mnstatus CSR is a read/write register which holds the status of the hart's NMI state.



##### Configuration

This register is present only when smrnmi is implemented.

##### Field Description

###### RES0, Bits [31:13]

Reserved, **WARL** field.

###### MNPP, Bits [12:11]

Hold the privilege mode of the interrupted context, encoded in the same manner as mstatus.mpp.

The MPP bits hold the previous privilege mode in M-mode. This field is **WARL**. Write any value other than 0b00 will read as 0b11.

**0b00** The previous privilege mode is U-mode.

**0b11** The previous privilege mode is M-mode.

This field is reset to 0b11.

###### RES0, Bit [10]

Reserved, **WARL** field.

###### MNPELP, Bit [9]

RES0, Reserved, **WPRI** field.

###### RES0, Bit [8]

Reserved, **WARL** field.

###### MNPV, Bit [7]

RES0, Reserved, **WARL** field.

###### RES0, Bits [6:4]

Reserved, **WARL** field.

###### NMIE, Bit [3]

Non-maskable interrupts enable. Refer to [Section 10.4.1.1](#) for more details.

**0b0** All interrupts are disabled.

**0b1** Nonmaskable interrupts are enabled. The enable/disable status of other interrupts depends on mstatus.mie and their own independent enable/disable control.

This field is reset to 0.

**RES0, Bits [6:4]**  
Reserved, **WARL** field.

**CSR Address**  
0x744

### Description

This register is present only when either `smunmi` or `smrnmi` is implemented.

**mnvec, Bits [31:2]****RES0, Bits [1:0]**

## CSR Address

0xFC0

## 10.6. Memory-Mapped Registers

CLIC memory-mapped registers must be made accessible to the M-mode software running on the hart. The system components which have access to CLIC memory-mapped registers shall be made accessible to M-mode only and other privilege levels are prevented by using system's standard memory protection mechanisms. This can be done either using PMPs in microcontroller systems, or page tables (and/or PMPs) in harts with virtual memory support.

CLIC memory-mapped registers occupy 20KB. The base address is specified via the parameter MCLICBASE, where since the CLIC memory map must be aligned at a 4KiB boundary, the MCLICBASE paramter has its 12 least-significant bits hardwired to zero. It is used to inform software about the location of CLIC M-mode memory mapped registers.

| Offset Address    | Register                | Description                                    |
|-------------------|-------------------------|------------------------------------------------|
| 0x0000            | mcliccfg                | Machine CLIC configuration register            |
| ...               | ...                     | ...                                            |
| 0x1000+4 <i>i</i> | clicintip< <i>i</i> >   | CLIC interrupt pending register <i>i</i>       |
| 0x1001+4 <i>i</i> | clicintie< <i>i</i> >   | CLIC interrupt enable register <i>i</i>        |
| 0x1002+4 <i>i</i> | clicintattr< <i>i</i> > | CLIC interrupt attribute register <i>i</i>     |
| 0x1003+4 <i>i</i> | clicintctl< <i>i</i> >  | CLIC interrupt input control register <i>i</i> |
| ...               | ...                     | ...                                            |

whether *n* is in the range from 0 to 31 (with the corresponding address from 0x0040 to 0x00BC), and *i* is in the range from 0 to 4095 (with the corresponding address from 0x1000 to 0x4FFF).

- If an input *n* is not present in the hardware, the corresponding clicinttrig<*n*> appears hardwired to zero.
- If an input *i* is not present in the hardware, the corresponding clicintip<*i*>, clicintie<*i*>, clicintattr<*i*>, clicintctl<*i*> memory locations appear hardwired to zero.



### Description

[illegible]

## Configuration

## Field Description

Reserved, **WPRI** field.

This field specifies how many bits are physically implemented in clicintattn<n>.mode. In Wing-M130, this field is **WARL** and is hardwired to 0.

**mnbits, Bits[3:0]**

This field indicates how many upper bits in `clicintctl<n>` are assigned to encode the interrupt level at M-mode. This field is **WARL** field. Refer to [Section 10.2.4](#) for more details.

## Offset Address

0x0000

## 10.6.2. CLIC Interrupt Pending Register, clicintip<n>

### Description

The clicintip<n> register holds the state whether the interrupt n is pending or not. The pending bit is located in bit 0 of the byte. Software should assume clicintip<n>=0 means no interrupt pending, and clicintip<n>!=0 indicates an interrupt is pending to accommodate possible future expansion of the clicintip field.

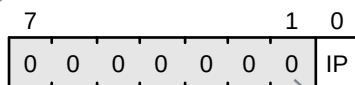
When the input is configured for level-sensitive input, the clicintip<n> reflects the value of an input signal to the interrupt controller after any conditional inversion specified by the clicintattr<i> field, and software writes to the bit are ignored. Software clears the interrupt at the source device.

When the input is configured for edge-sensitive input, this register is a read-write register that can be updated both by hardware interrupt inputs and by software. The bit is set by hardware after an edge of the appropriate polarity is observed on the interrupt input. Software writes can set or clear edge-triggered pending bits directly by writes to the memory-mapped register.

The value in the clicintip<n> is UNDEFINED when switching from level-sensitive mode to edge-sensitive mode in clicintattr<n>.

#### NOTE

Software cannot rely on the underlying clicintip<n> register bits used in edge-triggered mode to hold state while in level-sensitive mode.



### Configuration

This register is present only when smclic is implemented.

### Field Description

#### RES0, Bits 7:6

Reserved, WARL field.

#### IP, Bit [0]

Interrupt Pending.

**0b0** This interrupt is not in pending state.

**0b1** This interrupt is in pending state.

This field is reset to 0.

### Offset Address

0x1000+4\*n

### 10.6.3. CLIC Interrupt Enable Register, clicintie<n>

#### Description

The clicintie<n> register enables/disables the corresponding interrupt. The enable bit is located in bit 0 of the byte. Software should assume clicintie<n>=0 means no interrupt enabled, and clicintie<n>!=0 indicates an interrupt is enabled to accommodate possible future expansion of the clicintie field.

**NOTE** clicintie<n> is the individual enable bit while xstatus.xie is the global enable bit for the current privilege mode. Therefore, for an interrupt i to be enabled in the current privilege mode, both clicintie<i> and xstatus.xie have to be set.

**NOTE** In contrast, since xstatus.xie only takes effect in the current privilege mode according to RISC-V convention, an interrupt i from a higher privilege mode is enabled as long as clicintie<i> is set (regardless of the setting of xstatus.xie in the higher privilege modes).



#### Configuration

This register is present only when smclic is implemented.

#### Field Description

**RES0, Bits [7:6]**  
Reserved, WARL field.

**IE, Bit [0]**  
Interrupt Enable.

**0b0** This interrupt is disabled.

**0b1** This interrupt is enabled.

This field is reset to 0.

#### Offset Address

0x1001+4\*n

## 10.6.4. CLIC Interrupt Attribute Register, clicintattr<n>

### Description

The clicintattr<n> register specifies various attributes for interrupt n.

| 7    | 6 | 5 | 3 | 2 | 1    | 0   |
|------|---|---|---|---|------|-----|
| 1    | 1 | 0 | 0 | 0 | trig | shv |
| mode |   |   |   |   |      |     |

### Configuration

This register is present only when smclic is implemented.

### Field Description

#### mode, Bits [7:6]

This field specifies which privilege mode this interrupt operates in. It is a **WARL** field, and support the following encoding

**0b11** Machine.

#### RES0, Bits [5:3]

Reserved, **WPRI** field.

#### trig, Bits [2:1]

Specify the trigger type and polarity for this interrupt. Some implementations may want to save these bits so only certain trigger types are supported. In this case, these bits become hard-wired to fixed values (**WARL**).

**0b00** positive level-triggered.

**0b01** positive edge-triggered.

**0b10** negative level-triggered.

**0b11** negative edge-triggered.

This field is reset to an IMPLEMENTATION SPECIFIED value.

#### shv, Bit [0]

##### When smclicshv is implemented

**0b0** this interrupt is non-vectored and when triggered, it jumps to the common code at mtvec.

**0b1** this interrupt is hardware-vectored and when it is triggered, it jumps to the trap-handler specified in mtvt.

This field is reset to 0.

##### Otherwise

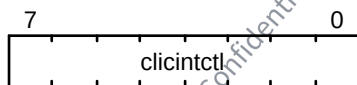
RES0, Reserved, **WARL** field.

$$0x1002 + 4 \cdot n$$
[illegible]

### 10.6.5. CLIC Interrupt Input Control Register, clicintctl<n>

#### Description

The clicintctl<n> register specifies interrupt level and interrupt priority.



#### Configuration

This register is present only when smclic is implemented.

#### Field Description

##### clicintctl

Specify interrupt level and interrupt priority. Refer to [Section 10.2.4](#) for more details.

This field is reset to 0.

#### Offset Address

0x1003+4\*n

## 11. Timer

Wing-M130 processor supports a real-time counter for generating timer interrupt for the system.

### 11.1. Features

- Support a 32-bit real-time counter (xcntbw32)
- The counter is a real counter which counts an external reference clock (default)
- The real-time counter is an incrementing, configurable wrap-on-ceil counter.
- Optionally provides timer calibration value (xtimecalib)

### 11.2. Timer Control

The counter is enabled and continues counting when setting `mtimectl.en` to 1. Otherwise, the counter is disabled.

When enabled, the counter incrementally counts until the value reaches `mtimecmp`. When `mtimectl.clr` is set, `mtime` will be cleared to zero on the next clock edge. Otherwise, it continues incrementally counting.

### 11.3. Timer Interrupt

A machine timer interrupt becomes pending whenever `mtime` contains a value greater than or equal to `mtimecmp`, treating the values as unsigned integers. The interrupt remains posted until `mtimecmp` becomes greater than `mtime` (typically as a result of writing `mtimecmp`).

When `mtimectl.clr` is set and the counter value reaches `mtimecmp`, `mtime` will become zero on the next clock edge. Therefore, the timer interrupt attribute shall be set as edge triggered.

When `mtimectl.clr` is cleared, the counter value continues counting and the timer value will be greater than `mtimecmp`. The interrupt remains posted. Therefore, the timer interrupt attribute shall be set as level triggered. The interrupt pending state shall be cleared by setting `mtimecmp` greater than `mtime`.

### 11.4. Timer Calibration

Wing-M130 processor optionally provides `mtimecalib` and `mtimectl.ine` for calibration.

`mtimecalib` provides a 10ms reference timing for the timer. It can be used and loaded directly into the `mtimecmp`. If 10ms cannot be presented exactly, `mtimectl.ine` is set, indicating `mtimecalib` is inexact and has been rounded to the nearest integer. For example,

```
// For a 12.5MHz clock
mtimecalib = (12.5MHz * 10ms) - 1
            = 124999
            = 32'h0001e847

// 10ms can be represented exactly on this clock
mtimectl.ine = 1'b0
```

```
// For a 8.125KHz clock
mtimecalib = (8.125KHz * 10ms) - 1
             = 80.25
             = 32'h00000050 // 80.25 with rounding to 80
// 10ms cannot be represented exactly on this clock
mtimectl.line = 1'b1
```



## 11.5. Memory-Mapped Registers

Timer memory-mapped registers must be made accessible to the M-mode software running on the hart. The system components which have access to Timer memory-mapped registers shall be made accessible to M-mode only and other privilege levels are prevented by using system's standard memory protection mechanisms. This can be done either using PMPs in microcontroller systems, or page tables (and/or PMPs) in harts with virtual memory support.

Timer memory-mapped registers occupy 4KB. The base address is specified via the parameter MCLINTBASE. The unused memory locations appear hardwired to zero.

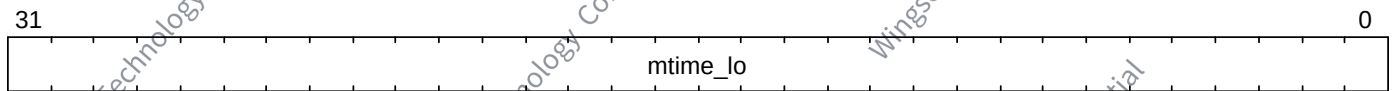
| Offset Address | Register      | Description                                          |
|----------------|---------------|------------------------------------------------------|
| 0x0000         | mtime_lo      | Machine timer register, lower part                   |
| 0x0008         | mtimecmp_lo   | Machine timer compare register, lower part           |
| 0x0010         | mtimectl      | Machine timer control register                       |
| 0x0020         | msip          | Machine software interrupt pending                   |
| 0x0028         | mtimecalib_lo | Machine timer calibration value register, lower part |
| 0x0030         | srcr          | Software Reset Control Register                      |

### 11.5.1. Machine Timer Register, lower part, mtime\_lo

#### Description

The mtime register is a M-mode read-write register with 64-bit. It increments at constant frequency. The mtime will wrap around if the count overflows.

This register mtime\_lo stores least significant 32-bit of mtime.



#### Configuration

This register is mandatory implemented.

#### Field Description

##### mtime\_lo, Bits [31:0]

Contain least significant 32-bit of mtime.

This field is reset to UNSPECIFIED value.

#### Offset Address

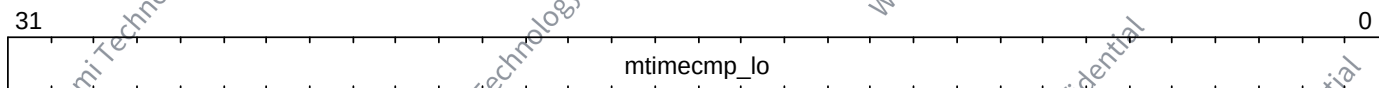
0x0000

## 11.5.2. Machine Timer Compare Register, lower part, mtimecmp\_lo

### Description

The mtimecmp register is a M-mode read-write register with 64-bit. It is compared to mtime register. Whenever mtime contains a value greater than or equal to mtimecmp. The timer interrupt remains posted until mtimecmp becomes greater than mtime.

This register mtimecmp\_lo stores least significant 32-bit of mtimecmp.



### Configuration

This register is mandatory implemented.

### Field Description

#### mtimecmp\_lo, Bits [31:0]

Contain least significant 32-bit of mtimecmp.

This field is reset to UNSPECIFIED value.

### Offset Address

0x0008

### 11.5.3. Machine Timer Control Register, `mtimectl`

#### Description

The `mtimectl` register is a 32-bit M-mode read-write register.



#### Configuration

This register is mandatory implemented.

#### Field Description

##### ine, Bit [31]

Inexact. Indicate whether the `mtimecalib` value is approximate. This field is **R** field.

- 0** The calibration value `mtimecalib` is exact.
- 1** The calibration value `mtimecalib` is inexact.

This field is reset to an UNSPECIFIED value.

##### rc1kn, Bit [30]

Indicate there is no reference clock for the timer. This field is **R** field.

- 0** There is an external reference clock provided for the timer.
- 1** There is no external reference clock provided for the timer.

This field is reset to an UNSPECIFIED value.

##### RES0, Bits [29:2]

Reserved, **WARL** field.

##### clr, Bit [1]

Wrap-on-ceil. Indicate whether the counter value continues counting or is cleared to zero, when it reaches `mtimecmp`.

- 0b0** When the counter value reaches `mtimecmp`, it continues counting.
- 0b1** When the counter value reaches `mtimecmp`, it is cleared to zero.

This field is reset to 0.

##### en, Bit [0]

Time Counter Enable. Indicate the enabled status of the Time Counter.

- 0b0** Counter is disabled, and stop counting.
- 0b1** Counter is enabled, and continue incremental counting.

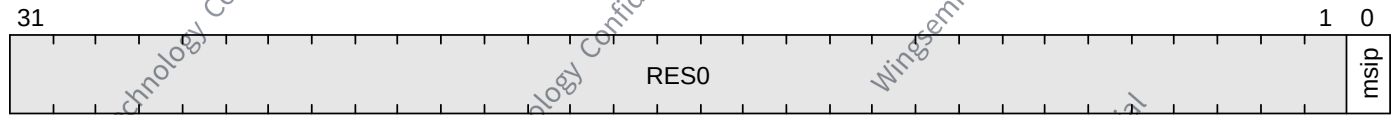
This field is reset to 0.

## 0x0010

### 11.5.4. Machine Software Interrupt Pending, msip

#### Description

The msip register is a 32-bit read-write register. Set this register presents the level-triggered software interrupt to the hart, which records into mip.msip. Clear this register clears the pending state of the level-triggered software interrupt.



#### Configuration

This register is mandatory implemented.

#### Field Description

- RES0, Bits [31:1]**  
Reserved, WARL field.
- msip, Bit [0]**  
Generate the software interrupt to the hart.
  - 0b0** No software interrupt.
  - 0b1** Present the software interrupt.

This field is reset to 0.

#### Offset Address

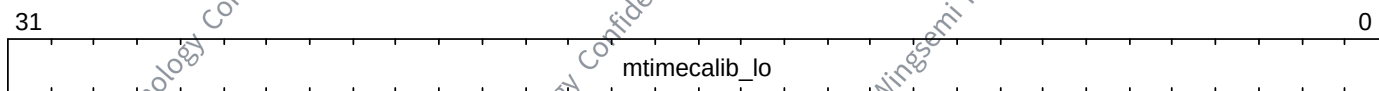
0x0020

### 11.5.5. Machine Timer Calibration Value Register, lower part, `mtimecalib_lo`

#### Description

The `mtimecalib` register is a 64-bit read-write register which provides calibration value for the timer.

This register `mtimecalib_lo` stores least significant 32-bit of `mtimecalib_lo`.



#### Configuration

This register is present only when `xtimecalib` is configured.

#### Field Description

##### `mtimecalib_lo`, Bits [31:0]

`mtimecalib_lo`. Timer calibration value. This field is **R** field.

This field is reset to an UNSPECIFIED value.

#### Offset Address

0x0028

### 11.5.6. Software Reset Control Register, srcr

#### Description

The srcr register is a 32-bit read-write register which allows software to request resets.



#### Configuration

This register is mandatory implemented.

#### Field Description

##### RES0, Bits [31:1]

Reserved, **WARL** field.

##### rreq, Bit [0]

Reset Request. This bit allows software to request a software reset.

**0** Do not request a software reset.

**1** Request a software reset.

This field is reset to 0.

#### Offset Address

0x0030



## 12. Reliability, Availability, and Serviceability (RAS)

Wing-M130 Processor implements the RAS Error Record Register Interface (RERI) specifications which augments Reliability, Availability, and Serviceability (RAS) features in the SoC with a standard mechanism for reporting errors by means of a memory-mapped register interface to enable error reporting, provide the facility to log the detected errors (including their severity, nature, and location), and configuring means to signal the error to a RAS handler component.

### 12.1. Feature

- Provide Error Record to record error information to a RAS handler.
- Provide Error Record Priority and Error Overwrite Rules.
- Support Corrected Error Counting.
- Support Error Report and Signaling.

### 12.2. Error Information Record

When an error is detected, the information of this error is collected and recorded in a error bank. Components in a system that support error detection may implement one or more banks of error records. Each error bank may implement one or more error records. Each error record corresponds to one or more hardware units of the component and reports errors detected by those hardware units.

Wing-M130 processor implements an error record to report errors detected by the hart.

#### 12.2.1. Error Detection and Correction

##### 12.2.1.1. Error Type

Errors that are reported on the hart are classified into two classes,

- CE, Corrected Error, is detected and corrected by the hart, and is not silently propagated.
- UUE, Uncorrected Urgent Error, is detected but cannot be corrected by a hardware component, and a RAS handler is invoked as immediate remedial actions are required.

#### NOTE

RISC-V RERI Architecture Specification also specifies Uncorrected Deferred Error, UDE, which is detected and could be propagated to the requester of the data. In Wing-M130 processor, UDE error is not supported.

##### 12.2.1.2. RAS Feature Implementation in Wing-M130 processor

Wing-M130 processor reports the following RAS events

- Interconnect UUE error, reported by error responses from I-BUS, D-BUS, M-BUS and P-BUS, if the corresponding interfaces are configured. (BUS\_UUE)

### 12.2.2. Error Information

Errors are caused by different reasons. Once an error occurs, information related to the error is collected and recorded in an error record. The following Table 17 concludes errors that could occur in Wing-M130 processor and the corresponding information that should be recorded in status\_0 and info\_0.

| Error   | pri | c | tt | scrub | ec | aec | tag | Error Source      |
|---------|-----|---|----|-------|----|-----|-----|-------------------|
| BUS_UUE | 0   | 0 | 4  | 0     | 14 | 0   | -   | instruction fetch |
| BUS_UUE | 0   | 0 | 4  | 0     | 14 | 1   | -   | load operations   |
| BUS_UUE | 0   | 0 | 5  | 0     | 14 | 1   | -   | store operations  |

Table 17. The Recorded Error Information of Errors

### 12.2.3. Error Record Enable and Disable

#### 12.2.3.1. Error Logging and Signaling Enable

The Error Logging and Signaling Enable control\_0.else bit enables/ disables error reporting functionality in the error record. When control\_0.else is 1, the hardware unit logs and signals errors in the error record. When control\_0.else is 0, the hardware unit continues detecting and correcting errors, but errors would not be logged and signaled.

#### 12.2.3.2. Error Signaling Enable

Both CE and UUE have an enable bit, control\_0.{ces, uues}, to enable signaling of CE and UUE respectively when they are logged (i.e. when control\_0.else==1). When set, a platform specific RAS signal is signaled to the hart. Otherwise, the signaling is disabled.

### 12.2.4. Error Record Invalidate

When the RAS handler is already handled the signaled error, the error record could be invalidated.

The status-register-invalidate bit control\_0.sinv, when written with a value of 1, causes the status\_0.v to be cleared. The control\_0.sinv always returns 0 on read. The control\_0.sinv bit enables software to read out and invalidate an error record without needing to explicitly write the status\_0 register.

## 12.3. Error Priority and Error Overwrite Rules

### 12.3.1. The Severity of Errors

The severity of errors is determined by the type of error. Specifically, the severity of errors from the highest to the lowest is: UUE > UDE (not supported in Wing-M130 processor) > CE.

### 12.3.2. Overwrite Rules

When a hardware unit detects an error it may find its error record still valid due to an earlier detected error that has not yet been consumed by software. The overwrite rules follow

- A higher severity error to overwrite a lower severity error.
- When the two errors have same severity, the priority of the errors (status\_0.pri) is used to determine

if the error record is overwritten. The multiple-occurrence status\_0.mo is set to 1 to indicate occurrence of second error of same severity.

- When an error record is overwritten by a higher severity error, the status bits indicating the severity of the older errors are retained.

When multiple errors occur simultaneously, the highest severity error with the highest priority is only recorded.

**NOTE** When the less severity error is CE, the CE counter could increment.

## 12.4. Error Report and Signaling

Depending on different cases, errors report as either machine synchronous RAS fault or machine asynchronous RAS fault. The following cases report as machine synchronous RAS fault, while the others report as machine asynchronous RAS fault.

- Errors that caused by Load requests are always reported as machine synchronous RAS fault.
- Errors that caused by instruction fetches that are not in a speculative path.
- Errors that caused by device store requests and atomic operations.

## 12.5. Corrected Error Counting

RAS module provides a corrected-error-counter (status\_0.cec), which holds an unsigned integer value. When it is enabled by control\_0.cece, the counter is incremented on each valid CE in addition to logging details of the error in the error record. If an unsigned integer overflow occurs on a status\_0.cec increment then the field status\_0.ceco is set to 1. The counter continues to count following an overflow.

**NOTE** A CE signal configured in control\_0.ces occurs only on a status\_0.cec overflow. If CE counting is disabled, the corrected-error-counter stops counting and the overflow will not triggered.

As CE counting is defined across logged errors. Therefore, the status\_0.{cec, ceco} fields of an error record are not modified and retain their values in the following cases

- When an UUE error is logged in an error record
- When the counter is disabled by clearing control\_0.cece
- When software writes 1 to control\_0.sinv or explicitly writes 0 to status\_0.v to invalidate the recorded error.

## 12.6. Memory-Mapped Registers

RAS memory-mapped registers must be made accessible to the M-mode software running on the hart. The system components which have access to RAS memory-mapped registers shall be made accessible to M-mode only and other privilege levels are prevented by using system's standard memory protection mechanisms. This can be done either using PMPs in microcontroller system, or page tables (and/or PMPs) in harts with virtual memory support.

RAS memory-mapped registers occupy 4KB. The base address is specified via the parameter MRASBASE. The unused memory locations appear hardwired to zero.

| Offset Address | Register        | Description                            |
|----------------|-----------------|----------------------------------------|
| 0x0000         | vendor_n_imp_id | Vendor and implementation ID           |
| 0x0008         | bank_info       | Error bank information                 |
| 0x0010         | valid_summary   | Summary of Valid Error Records         |
| 0x0040         | control_0       | Control register of error record 0     |
| 0x0048         | status_0        | Status register of error record 0      |
| 0x0050         | addr_0          | Address register of error record 0     |
| 0x0058         | info_0          | Information register of error record 0 |

### Description

The diagram illustrates the structure of the 64-bit fields `imp_id` and `vendor_id`. The `imp_id` field is 48 bits long, with the last 16 bits (bits 32-47) reserved for the vendor. The `vendor_id` field is 31 bits long, with the last 16 bits (bits 16-31) reserved for the vendor. The remaining bits are for the vendor's internal use.

This register is mandatory implemented.

**RES0, Bits [63:48]**

**imp\_id, Bits [47:32]**

**vendor\_id, Bits [31:0]**

## Address

0x0000

### Description

The diagram illustrates the 64-bit error record structure. It is divided into two main sections: 'version' (bits 0-55) and 'inst\_id' (bits 16-63). The 'version' section includes a 1-bit 'n\_err' field at bit 55 and a 1-bit 'recs' field at bit 56. The 'inst\_id' section is 48 bits long, starting from bit 16 to bit 63.

This register is mandatory implemented.

**version, Bits [63:56]****RES0, Bits [55:32]**

n\_err\_recs, Bits [31:16]

**inst id, Bits [15:0]**

Wing-M130 processor offers external pins to SoC for the configuration of each instance

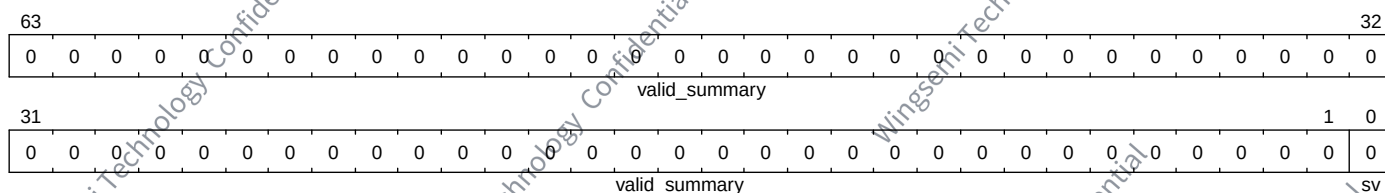
## 0x0008



### 12.6.3. Summary of Valid Error Records, valid\_summary

#### Description

The valid\_summary is a read-only register indicates a summary of the valid bits from the status registers of this error bank.



#### Configuration

This register is mandatory implemented.

#### Field Description

##### valid\_bitmap, Bits [63:1]

This field contains the valid bits from the status registers of this error bank. This field is **WARL** field and read as zero.

##### sv, Bit [0]

This bit indicates the valid\_bitmap provides a summary of the valid bits from the status registers of this error bank. This field is **WARL** field.

**0b0** The error bank does not provide a summary of valid bits and the valid\_bitmap is 0.

#### Offset Address

0x0010





**sinv, Bit [2]**

The Status Register Invalidate bit. This field is **W1** field. When written with a value of 1, causes the status\_0.v register to be cleared. The sinv field always returns 0 on read. The sinv bit enables software to read out and invalidate an error record without needing to explicitly write the status\_0 register.

**cece, Bit [1]**

The Corrected Error Counting Enable bit.

**0b0** The CE counter is disabled and does not count and retains its value.

**0b1** The CE counter is enabled and continues counting corrected errors.

This field is reset to 0.

**else, Bit [0]**

Enable error reporting functionality in the error record. When the functionality is disabled, the hardware continues performing error correction.

**0b0** The hardware unit does not log or signal errors in the error record.

**0b1** The hardware unit logs and signals errors in the error record.

This field is reset to 0.

**Offset Address**

0x0040



**0b0** The storage location that held the data value could not be updated with the corrected value. Or the error is not happened on memory locations.

**0b1** The storage location that held the data value has been updated with the corrected value.

This field is reset to 0.

#### **RES0, Bits [19:18]**

Reserved, **WPRI** field.

#### **tsv, Bit [17]**

**RES0**, Reserved, **WPRI** field.

#### **siv Bit [16]**

**RES0**, Reserved, **WPRI** field.

#### **at, Bits [15:12]**

The Address Type, a **WARL** field indicates the type of address reported in the `addr_0` register. It has the following encoding.

**1** Physical Address.

#### **iv, Bit [11]**

Information valid.

**0b0** The detected error does not reports additional information in `info_0` register.

**0b1** The detected error reports additional information in `info_0` register.

This field is reset to 0.

#### **tt, Bits [10:8]**

The transaction-type is a **WARL** field to report the type of transaction that detected. Write a invalid value would read as 0.

**0** Unspecified or not applicable.

**4** Explicit read.

**5** Explicit write.

**6** Implicit read.

**7** Implicit write.

This field is reset to 0.

#### **c, Bit [7]**

This field indicates the error has not propagated beyond the boundaries of the hardware unit that detected the error and thus may be containable through recovery actions. This field is **WARL** with the following encoding.

**0b0** The error is uncontainable.

**mo, Bit [6]**

This field indicates the multiple occurrence of errors of the same severity.

- 0b0** One error of the same severity occurred.
- 0b1** More than one error of the same severity occurred.

This field is reset to 0.

**pri, Bits [5:4]**

The pri field in the error record indicates the priority of the currently logged error in the error record.

- 0b00** Errors from the interconnect.
- 0b01** Errors from the source of L1 instruction cache.
- 0b10** Errors from the source of ITCM.
- 0b11** Errors from the source of DTCM.

This field is reset to an UNSPECIFIED value.

**uee, Bit [3]**

This field indicates whether the valid error record is an uncorrected urgent error.

- 0b0** When this is a valid error record, the detected error is not a UUE.
- 0b1** When this is a valid error record, the detected error is a UUE.

This field is reset to 0.

**ude, Bit [2]**

**RES0**, Reserved, **WPRI** field.

**ce, Bit [1]**

This field indicates whether the valid error record is a corrected error.

- 0b0** When this is a valid error record, the detected error is not a CE.
- 0b1** When this is a valid error record, the detected error is a CE.

This field is reset to 0.

**v, Bit [0]**

The valid error record.

- 0b0** The error record does not hold a valid error log.
- 0b1** The error record holds a valid error log.

This field is reset to 0.

**Offset Address**

0x0048

### Description

Diagram illustrating the 64-bit address field structure. The address is represented as a 64-bit vector, with bit positions 63 down to 0. The address value is 0, as indicated by the '0' in the bit 31 position and the '0' in the bit 0 position.

This field is mandatory implemented.

**RES0, Bits [63:32]****addr, Bits [31:0]**

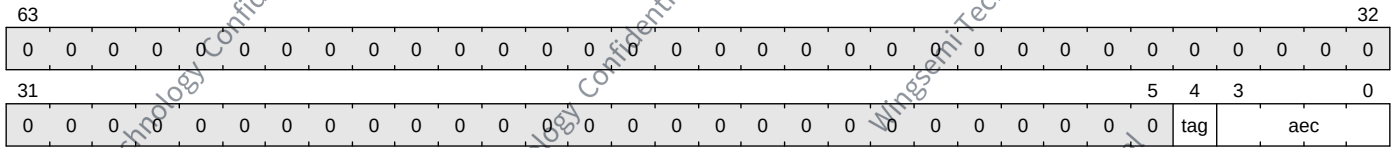
This field is reset to an UNSPECIFIED value.

## 0x0050

### 12.6.7. Information Register, info\_0

#### Description

The info\_0 is a read/write **WARL** register that provides additional information about the error when status\_0.iv is 1.



#### Configuration

This field is mandatory implemented.

#### Field Description

##### RES0, Bits [63:5]

Reserved, **WARL** field.

##### tag, Bit [4]

**RES0**, Reserved, **WARL** field.

##### aec, Bit [3:0]

Auxiliary Error Code. Provides additional information about this error. This field is **WARL** field. Write with other values will read as 0.

- 0 The error is caused by instruction fetch.
- 1 The error is caused by load or store operations issued by the hart.
- 3 The error is caused by accesses from S-Bus.

This field is reset to an UNSPECIFIED value.

#### Offset Address

0x0058